
LinuxKernelBook

Yayım 1.0.0

Kaan Aslan

26 May 2026

1	Giriş	3
1.1	Gereksinimler	3
1.2	Kaynak Listesi	3
1.3	Çekirdek Kodları Üzerinde Gezinme	5
1.4	İşletim Sistemlerine Giriş	5
1.5	İşletim Sisteminin Tanımı ve Yapısı	5
1.6	İşletim Sistemlerinin Sınıflandırılması	6
1.7	İşletim Sistemlerinin Tarihsel Gelişimi	8
1.8	UNIX Türevi Sistemlerin Tarihsel Gelişimi	9
1.9	Mac OS X Türevi Sistemler	10
1.10	GNU Projesi ve Özgür Yazılım	11
1.11	Linux'un Tarihsel Gelişimi	12
1.12	Linux Dağıtımları	13
1.13	POSIX Standartları	14
1.14	Linux Çekirdeğinin Kaynak Kod Yapısı	15
1.15	Linux Sistemlerinin Boot Edilmesi	16
2	Çekirdeğin Derlenmesi	21
2.1	Çekirdek Davranışını Değiştirme Yöntemleri	21
2.2	Çekirdek Komut Satırı Parametreleri	22
2.3	Çekirdeği Yeniden Derlemenin Gerekli Durumlar	22
2.4	Çekirdek Derlemesi Hakkında	22
2.5	Çekirdek Versiyon Numaralandırması	25
2.6	Derleme Araçları	26
2.7	Çekirdek Derleme Adımları	26
2.8	make modules_install Komutu ile İlgili Ayrıntılar	33
2.9	Manuel Kurulum	35
2.10	GRUB Yapılandırması	36
2.11	Çekirdek Derleme Süreci Özeti	37
2.12	Çekirdeğin Sistemden Kaldırılması	37
2.13	Çekirdek Kodları Üzerinde Değişiklik Yapma Yöntemleri	37
2.14	KBuild Sistemi ve GNU Make	37
2.15	Makefile Dosyalarının Güncellenmesi	38
2.16	Kconfig Dosyaları	39
2.17	Konfigürasyon Seçeneklerinin Kaynak Kodlara Yansıtılması	39
2.18	Konfigürasyon Seçeneklerinin Makefile Dosyalarına Yansıtılması	40
2.19	Yeni Dizin için Makefile ve Kconfig Dosyaları	41
2.20	Örnek: Çekirdeğe Yeni Modül Ekleme	42
2.21	Yeniden Derleme Sonrası Güncellemeler	43
2.22	Çekirdek ve Modül İmzalama	44
2.23	Çekirdek İmzasının İmzalanması	45
2.24	Kök Dosya Sisteminin Oluşturulması	45
2.25	Çekirdek Başlık Dosyalarının Kurulumu	47

3	Bağlı Listelerin ve Hash Tablolarının Çekirdek Gerçekleştirmeleri	49
3.1	Bağlı Listeler	49
3.2	Bağlı Listelere Neden Gerekseim Duyulur?	50
3.3	Linux Çekirdeğindeki Bağlı Liste Gerçekleştirmesi	50
3.4	Tam Örnek: Kullanıcı Alanında Bağlı Liste Kullanımı	55
3.5	RCU Desteği	58
3.6	Eski Çekirdek Sürümlerindeki list.h	58
3.7	Arama Algoritmaları ve Sözlük Tarzı Veri Yapıları	59
3.8	Hash Tabloları	60
3.9	Linux Çekirdeğinde Hash Tablosu Gerçekleştirmesi	62
3.10	RCU’lu Hash Tablosu Fonksiyonları	69
4	Proses Kontrol Bloğu ve task_struct Yapısı	71
4.1	Proses Kavramı	71
4.2	Proses Kontrol Bloğu	71
4.3	task_struct Yapısı	72
4.4	Proseslerin ID Değerleri (PID)	73
4.5	Thread Kavramı	73
4.6	Prosesler Arasındaki Altlık-Üstlük İlişkisi	73
4.7	task_struct Yapısının Dalı Budaklı Tasarımı	73
4.8	current Makrosu	74
4.9	fork ve pthread_create Çağrı Zincirleri	75
4.10	task_struct Nesneleri Arasındaki İlişkiler	75
4.11	task_struct Yapısına ilişkin Bağlı Listeler Üzerinde Gezinme İşlemleri	80
4.12	Tüm task_struct Nesnelerini Dolaşma	93
4.13	PID Tahsisatı	95
4.14	PID’den task_struct Nesnesine Hızlı Erişim	97
5	Linux Çekirdeğinde Dosya Sistemi - I. Bölüm	107
5.1	Giriş	107
5.2	Disk Aktarımına İlişkin Temel Bilgiler	107
5.3	Dosyalardan Okuma ve Yazma İşlemleri	110
5.4	Çekirdekteki Dosya İşlemi Veri Yapıları	116
5.5	task_struct İçerisindeki Dosya Sistemine İlişkin Veri Yapıları	117
5.6	files_struct Yapısı	118
5.7	Dosya Nesnesi (struct file)	119
5.8	Dosya Betimleyici Tablosu	119
5.9	2.6 Çekirdeğinde files_struct ve fdtable	122
5.10	Güncel Çekirdeklerdeki Dosya Veri Yapıları	123
5.11	Thread’ler, Fork ve Dosya Betimleyicileri	124
5.12	Dosya Sistemi Temel Nesneleri	124
5.13	inode ve dentry Önbellekleri	126
5.14	open() Fonksiyonunun İşleyişi	126
5.15	Prosesin Kök ve Çalışma Dizini	126
5.16	Dosya Sistemi Veri Yapılarına İlişkin Testler	127

Bu kitap Kaan ASLAN tarafından *C ve Sistem Programcıları Derneğindeki Linux Kernel İşletim Sistemlerinin Tasarımı ve Gerçekleştirilmesi* kursundaki kurs notları temel alınarak oluşturulmuştur.

Kitaptaki içerik belli bir kaynak referans alınarak oluşturulmamıştır ve içeriğin oluşturulmasında yapay zeka araçlarından faydalanılmamıştır. Kitap ile benzer içeriklere sahip kaynakların listesi *Giriş* bölümünde listelenmiştir.

Ön Gereksinimler

Okuyucunun C programlama diline hâkim olduğu ve temel Linux komut satırını kullanabildiği varsayılmaktadır. Çekirdeği derleyip denemek için sanallaştırma ortamına kurulu bir Debian türevi Linux dağıtımı önerilmektedir.

1.1 Gereksinimler

Kursumuz için bir sanal makineye herhangi bir Linux dağıtımının kurulması gerekmektedir. Dağıtım minimalist biçimde kurulabilir. Ancak Linux içerisinde C derleyicisinin (gcc ya da clang) ve binary utility araçlarının bulunuyor olması gerekir. Kursumuzda Linux kaynak kodları üzerinde değişiklikler yapıp çekirdeği yeniden derleyerek çeşitli denemeler yapacağız. Bu nedenle kurduğunuz sistemin bir kopyasını da saklamanızı salık veriyoruz.

Kursumuzda Debian türevi bir Linux dağıtımının kurulmuş olduğu varsayılacaktır. Biz bazı konularda açıklamalar yaparken Debian dağıtımının *apt* paket sistemini kullanacağız.

1.2 Kaynak Listesi

Linux çekirdeği konusunda yararlanabileceğiniz ek kaynakların listesini aşağıda veriyoruz. Bunların arasında en ayrıntılı olanı *Understanding the Linux Kernel* kitabıdır. Bu kitabın üç baskısı yapılmıştır. Birincisi çekirdeğin 2.2'li versiyonlarına, ikincisi 2.4 versiyonlarına ve üçüncüsü de 2.6'lı versiyonlarına ilişkindir. Ancak 2.6'dan sonra çekirdekte çeşitli alt sistemlerde değişiklikler yapıldığı için bu kitap Linux çekirdeğinin güncel versiyonlarını tam olarak yansıtmamaktadır.

Biz kursumuzda belli bir kitabın programını izlemeyeceğiz. Kurs konuları Kaan ASLAN'ın kendi deneyimlerinden ve bilgi birikiminden damıtılarak oluşturulmuştur.

1.2.1 Understanding the Linux Kernel

- [1] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2000. ISBN: 978-0-596-00002-5.
- [2] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2002. ISBN: 978-0-596-00213-5.
- [3] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005. ISBN: 978-0-596-00565-5.

Not

3. baskı Linux 2.6.11 çekirdeğini kapsar ve serinin son baskısıdır. Linux 5.x / 6.x için kernel.org resmi dokümantasyonu ve LWN.net makaleleri tamamlayıcı kaynak olarak kullanılmalıdır.

1.2.2 Linux Device Drivers

1. [4] A. Rubini and J. Corbet, *Linux Device Drivers*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 1998.
2. [5] A. Rubini and J. Corbet, *Linux Device Drivers*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2001.
3. [6] J. Corbet, A. Rubini, and G. Kroah-Hartman, *Linux Device Drivers*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005. ISBN: 978-0-596-00590-7. Çevrimiçi (ücretsiz): <https://lwn.net/Kernel/LDD3/>

Not

3. baskı Linux 2.6 çekirdeğini hedefler ve O'Reilly tarafından ücretsiz olarak çevrimiçi yayınlanmaktadır. 4. baskı henüz yayınlanmamıştır.

1.2.3 Kernel Internals ve Genel Mimari

1. [7] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010. ISBN: 978-0-672-32946-3.
2. [8] M. Gorman, *Understanding the Linux Virtual Memory Manager*. Upper Saddle River, NJ, USA: Prentice Hall, 2004. ISBN: 978-0-131-45348-5. Çevrimiçi (ücretsiz): <https://www.kernel.org/doc/gorman/>
3. [9] W. Mauerer, *Professional Linux Kernel Architecture*. Indianapolis, IN, USA: Wrox Press, 2008. ISBN: 978-0-470-34343-2.
4. [10] **G. Kroah-Hartman**, *Linux Kernel in a Nutshell*. Sebastopol, CA, USA: O'Reilly Media, 2006. Çevrimiçi (ücretsiz): <http://www.kroah.com/lkn/>

1.2.4 Linux Kernel Programming

1. [11] **K. N. Billimoria**, *Linux Kernel Programming: A Comprehensive Guide to Kernel Internals, Writing Kernel Modules, and Kernel Synchronization*, 1st ed. Birmingham, UK: Packt Publishing, 2021.
2. [12] **K. N. Billimoria**, *Linux Kernel Programming Part 2: Writing Character Device Drivers: Learn to Work with User-Kernel Interfaces, Handle Peripheral I/O and Hardware Interrupts*, 1st ed. Birmingham, UK: Packt Publishing, 2021.

1.2.5 Linux Kernel Debugging

1. [13] **K. N. Billimoria**, *Linux Kernel Debugging: Leverage Proven Tools and Advanced Techniques to Effectively Debug Linux Kernels and Kernel Modules*, 1st ed. Birmingham, UK: Packt Publishing, 2023.

Not

Billimoria'nın üç kitabı ([11], [12], [13]) birbirini tamamlar niteliktedir: kernel programlama → karakter sürücüler → kernel hata ayıklama şeklinde doğal bir okuma sırası oluşturur. [13] numaralı *Linux Kernel Debugging* kitabı ftrace, kprobes, KASAN, lockdep ve kdump araçlarını kaynak kod düzeyinde ele alır.

1.2.6 Sistem Programlama ve Kullanıcı Alanı Kaynakları

1. [14] **M. Kerrisk**, *The Linux Programming Interface*. San Francisco, CA, USA: No Starch Press, 2010. ISBN: 978-1-593-27220-3.
2. [15] **W. R. Stevens and S. A. Rago**, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013. ISBN: 978-0-321-63773-4.

3. [16] **B. Ward**, *How Linux Works: What Every Superuser Should Know*, 3rd ed. San Francisco, CA, USA: No Starch Press, 2021.
4. [17] **J. Levine**, *Linkers and Loaders*. San Francisco, CA, USA: Morgan Kaufmann, 2000.

1.2.7 Çevrimiçi Kaynaklar

1. [18] **The Linux Kernel Organization**, “The Linux Kernel Documentation,” [Çevrimiçi]. Erişim: <https://www.kernel.org/doc/html/latest/> [Erişim tarihi: Mayıs 2026].
2. [19] **M. Gorman**, “Understanding the Linux Virtual Memory Manager,” [Çevrimiçi]. Erişim: <https://www.kernel.org/doc/gorman/> [Erişim tarihi: Mayıs 2026].
3. [20] **LWN.net**, “Kernel development news and articles,” [Çevrimiçi]. Erişim: <https://lwn.net> [Erişim tarihi: Mayıs 2026].
4. [21] **LWN.net**, “Kernel Index — Memory Management,” [Çevrimiçi]. Erişim: https://lwn.net/Kernel/Index/#Memory_management [Erişim tarihi: Mayıs 2026].
5. [22] **Bootlin**, “Elixir Cross Referencer — Linux Kernel Source,” [Çevrimiçi]. Erişim: <https://elixir.bootlin.com/linux/latest/source> [Erişim tarihi: Mayıs 2026].
6. [23] **KernelNewbies**, “Linux Kernel Newbies,” [Çevrimiçi]. Erişim: <https://kernelnewbies.org> [Erişim tarihi: Mayıs 2026].

1.3 Çekirdek Kodları Üzerinde Gezinme

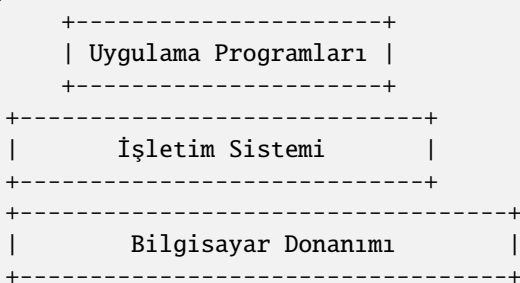
Kursumuzda Linux işletim sisteminin kaynak kodlarında gezinmek için <https://elixir.bootlin.com/linux/> sitesini kullanacağız. Bu sitede Linux’un 0.01 versiyonundan günümüzdeki en son versiyonuna kadar bütün resmi versiyonlarının kaynak kodları bulunmaktadır ve bu kaynak kodlar üzerinde gezintiler (navigation) yapılabilmektedir.

1.4 İşletim Sistemlerine Giriş

Bu bölümde işletim sistemleri hakkında genel bilgiler vereceğiz.

1.5 İşletim Sisteminin Tanımı ve Yapısı

İşletim sistemleri bilgisayar donanımının kaynaklarını yöneten, bilgisayar donanımı ile kullanıcı arasında arayüz oluşturan sistem programlarıdır. Bilgisayar bilimlerinin akademik öncülerinin çoğu işletim sistemlerini bir kaynak yöneticisi (resource manager) olarak tanımlamıştır.



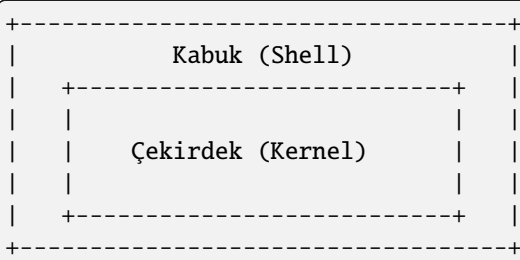
İşletim sistemlerinin yönettiği kaynakların en önemlileri şunlardır:

- **CPU:** İşletim sistemi hangi programın ne zaman, ne kadar süre için CPU’ya atanacağına karar verip bu işlemleri gerçekleştirmektedir.
- **Ana Bellek (Main Memory / RAM):** İşletim sistemi programların ana belleğin neresine yükleneceğine karar verir ve ana bellek kullanımını düzenler.

- **İkincil Bellekler:** İşletim sistemi bir dosya sistemi (file system) oluşturarak dosyaların parçalarını ikincil belleklerde etkin bir biçimde tutar ve kullanıcılara bir dosya kavramıyla sunar.
- **Çevre Birimleri (klavye, fare, yazıcı vb.):** İşletim sistemi fare, klavye, yazıcı gibi çevre birimlerini yöneterek onları kullanıma hazır hale getirir. Yardımcı işlemcileri (denetleyicileri) programlayarak onların işlev görmesini sağlamaktadır.
- **Ağ İşlemleri:** İşletim sistemi ağa ilişkin donanım birimlerini yöneterek dışarıdan gelen bilgileri onları talep eden programlara iletir.

İşletim sistemleri kaynak yönetimine göre alt sistemlere ayrılarak da incelenebilmektedir. Örneğin işletim sisteminin *çizelgeleyici (scheduler)* alt sistemi demekle CPU yönetimini sağlayan alt sistemi kastedilmektedir. Ana bellek yönetimi (memory management) yine soyutlanarak incelenen önemli alt sistemlerden biridir. İşletim sistemlerinin ikincil bellek yönetimine *dosya sistemi (file system)* da denilmektedir. Tabii bütün bu sistemler birbirinden kopuk olarak değil birbirleriyle ilişkili bir biçimde işlev görmektedir. Bu durumu insanın *solunum sistemi, dolaşım sistemi, sinir sistemi, boşaltım sistemi* gibi alt sistemlerine benzetebiliriz. Bu alt sistemlerin birinde bile çalışma bozukluğu oluşsa insan yaşamını yitirebilmektedir.

İşletim sistemleri yapı olarak iki kısımdan oluşmaktadır: Çekirdek (kernel) ve kabuk (shell). Çekirdek işletim sisteminin donanımı kontrol eden ve kaynakları yöneten motor kısmıdır. Aslında işletim sistemi denildiğinde akla çekirdek gelmektedir. Kabuk ise işletim sisteminin kullanıcı ile arayüz oluşturan önyüzüdür. Örneğin UNIX/Linux sistemlerinde bash gibi komut satırı, GNOME, KDE gibi pencere yöneticileri, Windows'taki masaüstü (Explorer), macOS'teki masaüstü (Aqua) bu işletim sistemlerinin kabuk kısımlarını oluşturmaktadır.



Peki işletim sistemi bu kadar temel donanım yönetimini sağlıyorsa işletim sistemi olmadan programlama yapılabilir mi? İşletim sistemi olmadan programlama faaliyetine halk arasında *bare metal programlama* denilmektedir. Bare metal programlama tipik olarak gömülü sistemlerde, mikrodenetleyicilerin kullanıldığı uygulamalarda kullanılmaktadır. Bare metal programlama genellikle özel bir amaca hizmet edecek biçimde yapılmaktadır. Amaçlar fazlaştığı zaman ve sistem karmaşıklaştığı zaman artık işletim sistemlerine gereksinim duyulmaktadır.

Bazı kontrol yazılımları işletim sistemlerinin bazı etkinliklerini de sağlamaktadır. Bir kontrol yazılımının işletim sistemi olarak isimlendirilmesi için yukarıda açıkladığımız kaynak yönetimlerinin önemli bir bölümünü sağlıyor olması gerekir. Bu kaynak yönetimlerinin çoğunu sağlamayan kontrol yazılımlarına genel olarak *firmware* de denilmektedir.

1.6 İşletim Sistemlerinin Sınıflandırılması

İşletim sistemleri çeşitli biçimlerde sınıflandırılabilir.

1.6.1 Proses Yönetimine Göre

Aynı anda tek bir programı çalıştıran işletim sistemlerine *tek prosesli (single processing)*, aynı anda birden fazla programı çalıştırabilen işletim sistemlerine ise *çok prosesli (multiprocessing) işletim sistemleri* denilmektedir. Örneğin DOS işletim sistemi tek prosesli bir sistemdi. Biz bu işletim sisteminde bir programı çalıştırdık ancak çalıştırdığımız program sonlanınca başka bir programı çalıştırabilirdik. Halbuki Windows, UNIX/Linux, macOS gibi işletim sistemleri çok prosesli işletim sistemleridir.

1.6.2 Kullanıcı Sayısına Göre

Birden fazla farklı kullanıcının çalışabildiği sistemlere *çok kullanıcı (multiuser)*, tek bir kullanıcının çalışabildiği sistemlere *tek kullanıcı (single user)* sistemler denilmektedir. Genellikle çok prosesli işletim sistemleri aynı zamanda çok kullanıcı sistemlerdir. Birden fazla kullanıcının söz konusu olduğu sistemlerde kullanıcıların yetkilerinin ayarlanması, kullanıcıların birbirlerinin alanlarına erişmesinin engellenmesi, sistem kaynaklarını belli oranlarda bölüşmesi gerekebilmektedir. Örneğin DOS tek kullanıcı bir sistemdi. Halbuki Windows, UNIX/Linux ve macOS sistemleri çok kullanıcı sistemleridir.

1.6.3 Çekirdek Yapısına Göre

İşletim sistemleri çekirdek yapısına göre *tek parçalı çekirdekli (monolithic kernel)* ve *mikro çekirdekli (microkernel)* olmak üzere ikiye ayrılmaktadır. Tek parçalı çekirdekli işletim sisteminin büyük kısmı çekirdek modunda çalışır. Mikro çekirdekli sistemlerde ise çekirdek modunda çalışan kısım minimize edilmeye çalışılmıştır. Aslında tek parçalı ve mikro çekirdekli tasarımları bir spektrum olarak düşünebiliriz. (Örneğin bu spektrumda bazı çekirdekler tek parçalı tarafa yakın, bazıları ise mikro tarafa yakın olabilmektedir.)

1.6.4 Dışsal Olaylarla Yanıt Verebilme Özelliğine Göre

İşletim sistemleri dışsal olaylara yanıt verme bakımından gerçek zamanlı olan (real-time) ve gerçek zamanlı olmayan (non-real-time) sistemler olmak üzere ikiye ayrılabilir. Dışsal olaylara hızlı bir biçimde yanıt verebilecek çekirdek yapısına sahip olan işletim sistemlerine *gerçek zamanlı (real-time) işletim sistemleri* denilmektedir. Gerçek zamanlı işletim sistemleri de kendi aralarında *katı (hard real-time)* ve *gevşek (soft real-time)* işletim sistemleri olmak üzere ikiye ayrılabilir. Katı gerçek zamanlı sistemler dışsal olaylara yanıt verme bakımından çok güvenilir olma iddiasındadır. Gevşek gerçek zamanlı sistemler ise bu konuda daha toleranslıdır.

1.6.5 Dağıtıklık Durumuna Göre

İşletim sistemleri dağıtıklık durumuna göre *dağıtık olan (distributed)* ve *dağıtık olmayan (non-distributed)* sistemler biçiminde ikiye ayrılabilir. Dağıtık işletim sistemlerinde sistem birden fazla bilgisayardan oluşan tek bir sistem gibi davranmaktadır. Örneğin 10 tane makineyi tek bir sistem olarak düşünebilirsiniz. Bu durumda bu bilgisayarların kaynakları (örneğin diskleri ve CPU'ları) bu 10 makine tarafından paylaşılmaktadır. Windows, UNIX/Linux ve macOS dağıtık işletim sistemleri değildir. Ancak bu sistemlerde dağıtık uygulamalar yapılabilir.

1.6.6 Donanım Özelliğine Göre

Neredeyse her yaygın masaüstü işletim sisteminin bir mobil versiyonu da oluşturulmuştur. iOS (iPhone Operating System) ve iPadOS Apple firmasının (yani macOS sistemlerinin) mobil işletim sistemleridir. Android bir çeşit mobil Linux sistemi olarak değerlendirilebilir. Android projesinde Linux çekirdeği alınmış, biraz özelleştirilmiş, bazı parçaları atılmış, buna bir mobil arayüz giydirilmiş ve sistem akıllı telefonlara ve tabletlere uygun hale getirilmiştir. Nokia eskiden Symbian sistemlerinde büyük bir pazar payına sahipti. Ancak bu firma akıllı telefon geçişini iyi yönetemedi. MeeGo ve Maemo gibi işletim sistemlerini denedi. Sonra ekonomik sıkıntılar sonucunda büyük ölçüde Microsoft tarafından satın alındı. Windows'un mobil versiyonuna genel olarak Windows CE denilmektedir. Windows CE'nin akıllı telefonlar ve tabletler için özelleştirilmiş biçimine ise Windows Mobile ve Windows Phone denilmektedir. Ancak Microsoft 2010 yılında Windows Mobile işletim sistemini, 2017'de de Windows Phone işletim sistemini sonlandırmıştır ve bu alandaki rekabetten tamamen çekilmiştir. Windows CE ise Windows IoT Core ismiyle farklı bir tasarımla evrimleşerek devam ettirilmektedir.

1.6.7 Kaynak Kod Lisansına Göre

Kaynak kod lisansına göre işletim sistemlerini kabaca açık kaynak kodlu (open source) ve mülkiyete bağlı (proprietary) olmak üzere ikiye ayırabiliriz. Açık kaynak kodlu işletim sistemleri değişik açık kaynak kod lisanslarına sahip olabilmektedir. Bunların kaynak kodları indirilip üzerinde değişiklikler yapılabilir. Örneğin Windows işletim sistemi mülkiyete sahiptir. Oysa Linux, BSD sistemleri, Solaris, Android gibi sistemler açık kaynak kodludur. macOS sistemlerinin ise çekirdeği açık, diğer kısımları (örneğin kabuk kısmı ve diğer katmanları) kapalıdır.

1.6.8 Kaynak Kodun Özgünlüğüne Göre

Bazı işletim sistemleri bazı işletim sistemlerinin kodları alınıp değiştirilerek oluşturulmuştur (örneğin Android ve macOS'ta olduğu gibi). Bazı işletim sistemlerinin kodları ise sıfırdan yazılmıştır. Kodları sıfırdan yazılan, yani orijinal kod temeline dayanan işletim sistemlerinden bazıları şunlardır:

- AT&T UNIX
- DOS
- Windows
- Linux
- BSD'ler (belli bir yıldan sonra)
- Solaris
- XENIX
- VMS

Burada orijinal mimari ile orijinal kod tabanını birbirine karıştırmamak gerekiyor. Linux UNIX işletim sisteminin mimarisini temel almıştır. Ancak tüm kodları sıfırdan yazılmıştır. Yani orijinal AT&T UNIX sistemindeki kaynak kodların bir bölümü kopyalanarak kullanılmamıştır.

1.6.9 GUI Çalışma Desteğine Göre

Bazı işletim sistemleri GUI çalışma modelini doğrudan desteklerken bazıları desteklememektedir. Örneğin Windows sistemleri çekirdekle entegre edilmiş bir GUI çalışma modeli sunmaktadır. UNIX/Linux sistemleri de X Window (ya da X11) ve Wayland katmanlarıyla benzer bir model sunmaktadır. Fakat örneğin DOS işletim sisteminin böyle bir doğal GUI desteği yoktu.

1.6.10 Ağ Üzerinde Hizmet Alıp Verme Rollerine Göre

İşletim sistemlerini ağ altında hizmet alıp verme rollerine göre *istemci (client)* ve *sunucu (server)* biçiminde de iki gruba ayırabiliriz. Bazı işletim sistemlerinin istemci versiyonları birbirlerinden ayrılmıştır. Bazılarında ise bu ayrım yapılmamıştır. Örneğin Windows 7, 8, 10, 11 sistemleri bu bakımdan istemci (client) sistemleridir. Halbuki Windows Server 2016, 2019 sunucu sistemleri olarak piyasaya sürülmüştür. Eskiden Mac OS X'in istemci ve sunucu versiyonları farklıydı. Fakat Mac OS X 10.7 (Lion) ile birlikte istemci ve sunucu versiyonları birleştirildi. Linux dağıtımlarının çoğu da hem istemci hem de sunucu olarak kullanılabilir. Ancak bazı dağıtımların istemci ve sunucu versiyonları farklıdır.

Peki işletim sistemlerinin istemci ve sunucu versiyonları arasındaki farklılıklar nelerdir? Kabaca iki tür farklılığın olduğunu söyleyebiliriz. Birincisi çekirdekle ilgili farklılıklardır. Genellikle sunucu sistemlerinde çizelgeleyici alt sistemde istemci sistemlerine göre farklılıklar bulunmaktadır. İkincisi ise barındırdıkları yardımcı yazılımlardır. İşletim sistemlerinin sunucu versiyonları hazır bazı sunucu programlarını da içermektedir.

1.7 İşletim Sistemlerinin Tarihsel Gelişimi

1940'lı yıllarda ilk elektronik bilgisayarlar yapıldığında henüz bir işletim sistemi kavramı yoktu. Bu bilgisayarlara program yazacak olanlar işletim sistemi faaliyetlerini de kendileri yapmak zorunda kalıyordu. (Yani şimdi mikrodenetleyicilere bare metal kod yazanlarda olduğu gibi.) Transistör bulunduktan sonra 1950'li yıllarda artık elektronik bilgisayarlar yavaş yavaş transistörlerle yapılmaya başlandı. Transistörlerin ortaya çıkması hem bilgisayarların kapasitelerini ve güvenilirliklerini artırmış, hem de güç harcamalarını düşürmüştür.

1950'li yıllarda IBM gibi pek çok bilgisayar üreticisi firma yalnızca donanım satıyordu. İşletim sistemi gibi programları yazmak kullanıcıların yapması gereken bir işti. Böylece donanımı satın alan her kurum işletim sistemine benzeyen programları da kendisi yazıyordu. Bu anlamda standart bir işletim sistemi yoktu. Bugünkü anlamda ilk işletim sisteminin General Motors'un 1956 yılında IBM'in 701 sistemi için yazdığı NAA IO (North American Aviation Input Output System) olduğu söylenebilir.

1960'lara gelindiğinde IBM, System/360 isminde yeni bir bilgisayar donanımı geliştirme işine girişti ve artık donanım ile işletim sistemini birlikte satma fikrini benimsedi. Bu donanım 1964 yılında duyuruldu ve 1965 yılında gerçekleştirildi. İlk System/360 Model 30 bilgisayarları o zamanın *Solid Logic Technology (SLD)* teknolojisiyle üretilmişti. Hem öncekilerden daha güçlüydü hem de daha az yer kaplıyordu. Saniyede 34500 işlem yapabiliyordu ve 8K ile 64K ana belleğe sahipti. 1967 yılında System/360'ın Model 60'ı piyasaya sürüldü. Bu model saniyede 16.6 milyon komut çalıştırabiliyordu ve ana belleği tipik olarak 512K, 768K ve 1 MB idi. IBM Sistem 360 donanımları için 1964 yılında ilk kez OS/360 işletim sistemini geliştirdi. IBM daha sonra 1967 yılında OS/360 Model 67 için OS/360'ın TSS 360 isminde zaman paylaşımı (time sharing system) bir versiyonunu daha geliştirmiştir. IBM'in System/360 makineleri ve işletim sistemleri önemli ticari başarı kazandı. System/360'ı System/370 izledi. System/360 ve System/370 için başka kurumlar da işletim sistemleri geliştirmiştir. Michigan Terminal System (MTS) ve MUSIC/SP bunlar arasında önemli olanlardır.

1960'lı yıllarda başka firmalar da işletim sistemleri geliştirmiştir. Örneğin Control Data Corporation firmasının SCOPE işletim sistemi batch işlemler yapabiliyordu. Aynı firma MACE isminde bu işletim sisteminin zaman paylaşımı bir versiyonunu da yazmıştır. Firma bu çalışmalarını 1970'li yıllarda Kronos işletim sistemiyle devam ettirmiştir. Burroughs firması 1961 yılında MCP işletim sistemi ile B5000 bilgisayarlarını, GE firması da 1962 yılında GECOS işletim sistemiyle GE-600 serisi bilgisayarlarını piyasaya sürdü. UNIVAC dünyanın ilk ticari bilgisayarlarını üreten firmadır. Bu firma da 1962 yılında UNIVAC 1107 için EXEC I işletim sistemini yazdı. Bu işletim sistemini sırasıyla Exec 2 ve Exec 8 izledi.

DEC (Digital Equipment Corporation) eskilerin en önemli bilgisayar üretici firmalarından biriydi. (DEC 1998 yılında Compaq firması tarafından, Compaq firması da 2002 yılında HP firması tarafından satın alındı.) Firmanın en önemli ürünleri PDP (Programmed Data Processor) isimli bilgisayarlarıdır. Firma PDP-1'den (1959) başlayarak PDP-16'ya (1971-1972) kadar PDP makinelerinin 16 versiyonunu piyasaya sürmüştür. DEC'in PDP-8'inin mini bilgisayar devrimini başlattığı söylenebilir. Bu model 50.000'in üzerinde satışa ulaşmıştır. UNIX işletim sistemi 1969 yılında ilk kez DEC'in PDP-7 modeli üzerinde yazılmıştır. 1965 yılında piyasaya sürülen DEC PDP-7 18 bitlik bir makineydi. Makine DECsys denilen işletim sistemi benzeri bir yönetici programla beraber satılıyordu. DEC'in 1966 yılında çıkardığı PDP-10 26 bitlik bir makineydi. DEC bu modelle birlikte işletim sistemi olarak TOPS-10 isimli bir sisteme geçti.

1960'lı yılların sonuna kadar işletim sistemleri ağırlıklı olarak sembolik makine diliyle yazılıyordu. 1960'lı yılların sonlarında AT&T Bell Lab. tarafından UNIX işletim sistemi geliştirildiğinde önemli bir devrim yaşandı. UNIX işletim sistemi 1973 yılında C ile yeniden

yazılmıştır. Böylece artık işletim sistemlerinin yüksek seviyeli dillerle de yazılabildiği görülmüştür. PDP-11'i 16 bitlik PDP-12 izledi. PDP-12 Intel'in x86 ve Motorola'nın 6800 işlemcileri için ilham kaynağı olmuştur.

1970'li yılların ikinci yarısında entegre devrelerin de geliştirilmesiyle *ev bilgisayarları (home computer)* ortaya çıkmaya başladı. Bunlarda genellikle BASIC yorumlayıcıları ile iç içe geçmiş CP/M ya da GEOS işletim sistemleri kullanılıyordu. 1970'li yıllarda pek çok firma farklı ev bilgisayarları üretmiştir. BBC Micro, Commodore 64, Apple II, Atari, Amstrad, ZX Spectrum dönemin en ünlü ev bilgisayarlarındandı. Bu makinelerde kullanılan işlemciler Intel'in 8080'i, Zilog'un Z80'i, Motorola'nın 6800'ü gibi 8 bitlik işlemcilerdi.

DEC firması 1977 yılında VAX isimli bilgisayarı ve 32 bitlik işlemci birimini piyasaya sürdü. VAX ailesi makineler o yıllarda önemli bir ticari başarı kazanmıştır. DEC VAX makineleri için VAX/VMS isimli bir işletim sistemi yazmıştı. DEC bu işletim sisteminin ismini 1992 yılında OpenVMS olarak değiştirdi. DEC 1992 yılında 64 bitlik RISC tasarımı olan Alpha işlemcilerini piyasaya sürdü ve OpenVMS Alpha işlemcilerine port edildi. OpenVMS hala kullanılmaya devam etmektedir. Itanium ve X86-64 portları da vardır.

Apple firması 1976 yılında kuruldu. Apple'ın ilk bilgisayarı Apple I idi. Bunu 1977'de Apple II, 1980'de de Apple III izledi. Bu ilk Apple bilgisayarlarında AppleDOS isimli işletim sistemleri kullanılıyordu. Daha sonra Apple 1983'te Lisa modelini piyasaya sürdü. 1983'ün sonlarında da ilk Macintosh bilgisayarını çıkardı. Lisa ile birlikte Apple grafik tabanlı işletim sistemlerine geçiş yaptı. Lisa ve sonraki Apple bilgisayarlarının hepsi grafik bir arayüze sahiptir. Macintosh markası daha sonra Mac olarak telaffuz edilmeye başlandı. Lisa bilgisayarlarında kullanılan işletim sistemi LisaOS ismindeydi. Apple daha sonra Macintosh bilgisayarlarının değişik versiyonlarını piyasaya sürdü. Bunlardaki işletim sistemini *System Software 1 (1984)*, *System Software 2 (1985)*, *System Software 3 (1986)*, *System Software 4 (1987)*, *System Software 5 (1987)*, *System Software 6 (1988)* ve *System Software 7 (1991)* olarak isimlendirdi. Apple *System Software 7.5*'ten sonra işletim sisteminin ismini *System Software* yerine Mac OS olarak değiştirdi ve System Software 7.6 versiyonu Mac OS 7.6 ismiyle çıktı. Daha sonra Apple 1997 yılında Mac OS 8'i, 1999 yılında da Mac OS 9'u çıkarmıştır.

1980'li yıllarda Mac bilgisayarlarının fiyatı çok yüksekti ve satışları da iyi gitmiyordu. Çünkü Steve Jobs bilgisayarların program yazmak için değil kullanmak için satın alınması gerektiğini düşünüyordu. Nihayet Apple'daki çalkantılar sonucunda Steve Jobs 1985 yılında Apple'dan ayrılmak zorunda kaldı (kovuldu da denebilir) ve NeXT firmasını kurdu. NeXT firması NeXT isimli bilgisayarları geliştirdi. Bu bilgisayarlarda NeXTSTEP isimli işletim sistemi kullanılıyordu. Daha sonra bu sistem açık hale getirildi ve OPENSTEP ismini aldı. Dünyanın ilk Web tarayıcısı Tim Berners Lee tarafından CERN'de NeXT bilgisayarları üzerinde gerçekleştirilmiştir.

Steve Jobs 1997 yılında Apple'a geri döndü. Apple da NeXT firmasını 200 milyon dolara satın aldı. Sonra piyasaya iMac ve Power Mac serileri çıktı. Daha sonra Steve Jobs Mac'lerin çekirdeklerini tamamen değiştirme kararı aldı. Mac'ler Mac OS'un 10 versiyonu ile birlikte yeni bir çekirdeğe geçtiler. Mac OS işletim sistemlerinin 10'lu versiyonları Roma rakamıyla Mac OS X biçiminde isimlendirilmiştir. Apple Mac OS X ismini 2012 yılında Mountain Lion (10.8) sürümü ile OS X olarak, 2016 yılında da Sierra (10.12) sürümüyle birlikte de macOS olarak değiştirmiştir.

DOS işletim sistemi text ekranda çalışıyordu. Microsoft da geleceğin grafik tabanlı işletim sistemlerinde olduğunu gördü ve yavaş yavaş DOS'u bırakarak grafik tabanlı bir sisteme geçmeyi planladı. Bunun için Windows isimli grafik arayüzün birinci versiyonunu 1985'te çıkardı. Bunu 1987'de Windows 2, 1990'da Windows 3.0 ve 1992'de de Windows 3.1 izledi. Bu 16 bit Windows sistemleri işletim sistemi değildi; DOS üzerinden çalıştırılan birer grafik arayüz gibiydi. Microsoft daha sonra Windows'u Windows NT 3.1 ile bağımsız bir işletim sistemi haline getirdi. Microsoft bundan sonra sırasıyla 1994 yılında Windows NT 3.5'i, 1995 yılında Windows NT 3.51'i ve Windows 95'i, 1998 yılında Windows 98'i, 2000 yılında Windows 2000 ve Windows ME'yi, 2001 yılında Windows XP'yi, 2006 yılında Windows Vista'yı, 2012 yılında Windows 8'i, 2015 yılında Windows 10'u ve nihayet 2021 yılında da Windows 11'i çıkarmıştır.

Linux işletim sistemi 1992 yılında bir dağıtım biçiminde piyasaya çıkmıştır. Linux işletim sisteminin hikayesi daha geniş olarak izleyen derste ele alınmaktadır.

1.8 UNIX Türevi Sistemlerin Tarihsel Gelişimi

UNIX işletim sistemi AT&T Bell Laboratuvarlarında 1969-1971 yılları arasında geliştirildi. Proje ekibinin lideri Ken Thompson'du. Çalışma ekibinde Dennis Ritchie, Brian Kernighan gibi önemli isimler de vardı. Ekip daha önce General Electric'in GE-645 mainframe bilgisayarı için Multics işletim sistemi üzerinde çalışıyordu. (Multics işletim sisteminin geliştirilmesine 1964 yılında başlandı. Projede General Electric, MIT ve Bell Lab birlikte çalışıyordu. Sonra proje Honeywell şirketi tarafından devralınmıştır.)

AT&T 1969 yılında bu projeden çekilerek kendi işletim sistemini geliştirmek istemiştir. Geliştirme çalışmasına DEC'in PDP-7 makinelerinde başlamıştır. UNIX ismi 1970 yılında Brian Kernighan tarafından Multics'ten kelime oyunu yapılarak uydurulmuştur. Proje ekibi AT&T'yi DEC PDP-11 almaya ikna etti ve böylece geliştirme çalışmaları PDP-11 ile devam etti. UNIX'in resmi olarak ilk sürümü Ekim 1971'de, ikinci sürümü Aralık 1972'de, üçüncü ve dördüncü sürümleri de 1973 yılında yayınlanmıştır.

UNIX işletim sistemi büyük ölçüde PDP'nin sembolik makine dili ve Ken Thompson'ın B isimli programlama diliyle geliştirilmiştir. B programlama dili fonksiyonları alıp DEC'in makine diline dönüştürüyordu. Bu bakımdan B bir yorumlayıcı değil derleyiciydi. İşte 1972 yılında Dennis Ritchie, Ken Thompson'ın B programlama dilinden hareketle C Programlama Dilini geliştirmiştir. UNIX işletim sisteminin dördüncü sürümü 1973 yılında yeniden C Programlama Dili ile yazılmıştır.

1974 yılında UNIX'in beşinci sürümü oluşturuldu. Bu sürümlerin hepsi araştırma amaçlıydı ve *educational license* ismiyle lisanslanmıştı. UNIX işletim sistemi bir araştırma projesi olarak organize edilmişti. Bu nedenle AT&T kaynak kodlarını araştırma kuruluşlarına ücretsiz

dağıtmıştır. 1975 yılında UNIX'in altıncı sürümü şirketlere yönelik hazırlandı. UNIX'in altıncı versiyonunun kaynak kodları 20.000 dolara (bugünün yaklaşık 120.000 doları) şirketlere sunuldu. 1977 yılında Bell Lab, UNIX'i Interdata 7/32 isimli 32 bit mimariye port etti. Bunu 1978'de VAX portu izledi.

1974 yılında California Üniversitesi (Berkeley) işletim sisteminin kopyasını Bell Lab'tan aldı. 1978 yılında *Berkeley Software Distribution (IBSD)* ismiyle AT&T dışındaki ilk UNIX dağıtımını gerçekleştirdi. Bu dağıtım hayatını hala FreeBSD, OpenBSD ve NetBSD olarak devam ettirmektedir. 1979'da BSD'nin ikinci versiyonu (2BSD) ve 1979'un sonlarına doğru da üçüncü versiyonu (3BSD) piyasaya sürüldü. Bunu 1980 yılında versiyon 4 (4BSD) izlemiştir. 1991 yılında BSD UNIX'ten AT&T kodları tamamen arındırılmış ve kod bakımından özgün hale getirilmiştir. BSD'nin son versiyonu 1995'te 4.4BSD Lite Release 2 olarak çıkmıştır.

1980'li yıllarda pek çok kurum ve ticari firma UNIX kodlarını lisans ücreti ödeyerek AT&T'den satın alıp kendilerine yönelik UNIX sistemleri oluşturmuştur. Bunların önemli olanları şunlardır:

AIX

IBM tarafından geliştirilmiş olan UNIX türevi sistemlerdir. İlk kez 1986 yılında piyasaya sürülmüştür. IBM AIX'i System/370, RS/6000 ve PS2 bilgisayarlarında kullanıyordu. Bu sistemler AT&T UNIX System 5 kodları temel alınarak geliştirilmiştir. AIX hala kullanılmaktadır. Son sürümü 2021 yılında 7.3 olarak piyasaya sürülmüştür. AIX PowerPC ve x86 işlemcileri için de port edilmiştir.

IRIX

SGI firması tarafından AT&T ve BSD kodları değiştirilerek 1988'de oluşturulmuştur. 2006'da bırakılmıştır.

HP-UX

HP 9000 bilgisayarları için 1982'de oluşturulmuştur. Motorola 68000 ve Itanium işlemcileri için yazılmıştır. Hala devam ettirilmektedir.

ULTRIX

DEC firmasının PDP-7, PDP-11 ve VAX donanımları için geliştirdiği UNIX sistemiydi. İlk versiyonu 1984 yılında çıktı. 1995 yılında piyasadan çekildi.

XENIX

Microsoft tarafından 1980 yılında geliştirilmeye başlanmıştır. İlk versiyonu 1980'in sonlarına doğru çıkmıştır. Daha sonra SCO firması Microsoft'la bu konuda iş birliği yapmış, 1987 yılında da Microsoft sistemi tamamen SCO'ya devretmiştir.

SCO-UNIX

SCO firması XENIX'i Microsoft'tan alınca bunu SCO-UNIX olarak devam ettirdi. SCO-UNIX'in ilk versiyonu 1989 yılında çıktı. SCO sonra bunu OpenServer ismiyle devam ettirmiştir.

FreeBSD, NetBSD ve OpenBSD

4.3BSD sistemleri temel alınarak geliştirilmiştir. FreeBSD ve NetBSD 1993 yılında, OpenBSD ise 1996 yılında piyasaya çıkmıştır. Sürdürülmeye devam etmektedir. FreeBSD genel amaçlı client ve server işletim sistemi olma niyetindedir. NetBSD daha taşınabilir ve geniş bir port'a sahiptir; daha çok bilimsel çalışmalarda tercih edilmektedir. OpenBSD güvenliğinin önemli olduğu alanlarda tercih edilmektedir.

SunOS / Solaris

Sun firmasının BSD kodlarıyla oluşturduğu UNIX türevi işletim sistemiydi. İlk versiyonu 1982 yılında çıktı. SunOS işletim sistemi 5.2 versiyonundan sonra (1992) Solaris ismiyle pazarlanmaya başlamıştır. Solaris daha sonra OpenSolaris biçiminde açık kaynak kodlu olarak bir süre varlığını devam ettirdi. Oracle firmasının Sun firmasını 2010'da satın almasından sonra bu proje de durduruldu. Bu proje Illumos ismiyle başka bir ekip tarafından devam ettirilmektedir.

Linux

Linus Torvalds'ın öncülüğünde geliştirilmiş en yaygın UNIX türevi işletim sistemidir. İlk versiyonu 1991 yılında çıkmıştır. Hala devam ettirilmektedir. Linux'un tarihsel gelişimi aşağıdaki bölümde ayrıntılı bir biçimde açıklanmaktadır.

Mac OS X / OS X / macOS

Carnegie Mellon üniversitesinin Mach isimli çekirdeği ile BSD UNIX sisteminin bir araya getirilmesiyle oluşturulmuştur. İlk versiyonu 2001 yılında piyasaya sürülmüştür.

1.9 Mac OS X Türevi Sistemler

İşletim sistemlerinin tarihsel gelişimini ele aldığımız önceki bölümde de belirttiğimiz gibi Apple firmasının Mac bilgisayarları Mac OS'un 10 versiyonu ile birlikte yeni bir çekirdeğe geçtiler. Mac OS işletim sistemlerinin 10'lu versiyonları Roma rakamıyla Mac OS X biçiminde isimlendirildi. Apple Mac OS X ismini 2012 yılında Mountain Lion (10.8) sürümü ile OS X olarak, 2016 yılında da Sierra (10.12) sürümüyle birlikte de macOS olarak değiştirdi. Biz Mac OS X, OS X ve macOS sistemlerine bu bölümde *Mac OS X türevi sistemler* de diyeceğiz.

Mac OS X türevi işletim sistemleri UNIX türevi sistemlerdir. Bu işletim sistemlerinin çekirdeğine Darwin denilmektedir. Darwin açık kaynak kodlu bir işletim sistemidir. Ancak Mac OS X türevi sistemler tam anlamıyla açık sistemler değildir. Bu sistemlerin çekirdeği açık olsa da geri kalan kısımları mülkiyete sahip (proprietary) biçimdedir.

Darwin'in hikayesi 1989 yılında NeXT'in NeXTSTEP işletim sistemiyle başladı. NeXTSTEP daha sonra OpenStep ismiyle API düzeyinde standart hale getirildi. 1996'nın sonunda, 1997'nin başında Steve Jobs Apple'a dönerken Apple da NeXT firmasını satın aldı ve sonraki işletim sistemini OpenStep üzerine kuracağını açıkladı. Bundan sonra Apple 1997'de OpenStep üzerine kurulu olan Rhapsody'yi çıkardı. 1998'de de yeni işletim sisteminin Mac OS X olacağını açıkladı. Daha sonra 2000 yılında Apple Rhapsody'den Darwin projesini türetti. Darwin her ne kadar Mac sistemlerinin çekirdeği olarak tasarlanmışsa da ayrı bir işletim sistemi olarak da yüklenebilmektedir. Ancak Darwin grafik arayüzü olmadığı için Mac programlarını çalıştıramamaktadır. Daha sonra Darwin'i bağımsız bir işletim sistemi haline getirmek amacıyla Darwin'den de çeşitli projeler türetilmiştir. Bunlardan biri Apple tarafından 2002'de başlatılan OpenDarwin'dir. Bu proje 2006'da sonlandırılmıştır. 2007'de PureDarwin projesi başlatılmıştır.

Darwin'in çekirdeği XNU üzerine oturtulmuştur. XNU, NeXT firması tarafından NEXTSTEP işletim sisteminde kullanılmak üzere geliştirilmiş bir çekirdektir. XNU, Carnegie Mellon (*Karnegi* diye okunuyor) üniversitesinin Mach 3 mikrokern çekirdeği ile 4.3BSD karışımı hibrit bir sistemdir.

Mac OS X türevi sistemlerin versiyonları şunlardır:

- Mac OS X 10.0 (Cheetah, 2001)
- Mac OS X 10.1 (Puma, 2001)
- Mac OS X 10.2 (Jaguar, 2002)
- Mac OS X 10.3 (Panther, 2003)
- Mac OS X 10.4 (Tiger, 2005)
- Mac OS X 10.5 (Leopard, 2007)
- Mac OS X 10.6 (Snow Leopard, 2009)
- Mac OS X 10.7 (Lion, 2011)
- OS X 10.8 (Mountain Lion, 2012)
- OS X 10.9 (Mavericks, 2013)
- OS X 10.10 (Yosemite, 2014)
- OS X 10.11 (El Capitan, 2015)
- macOS 10.12 (Sierra, 2017)
- macOS 10.13 (High Sierra, 2017)
- macOS 10.14 (Mojave, 2018)
- macOS 10.15 (Catalina, 2019)
- macOS 11 (Big Sur, 2020)
- macOS 12 (Monterey, 2021)
- macOS 13 (Ventura, 2022)
- macOS 14 (Sonoma, 2023)
- macOS 15 (Sequoia, 2024)

macOS büyük ölçüde POSIX uyumlu bir sistemdir.

1.10 GNU Projesi ve Özgür Yazılım

UNIX/Linux dünyasında önemli bir yeri olan GNU Projesi, özgür yazılım ve açık kaynak kod akımları üzerinde durmak istiyoruz.

1970'lerdeki mikro bilgisayarlar devrimine kadar yazılımda bir telif anlayışı yoktu. Yani yazılımın dağıtılması konusunda sözleşmeler ve hukuki yaptırımlara gerek duyulmamıştı. Yazılım zaten donanım ile birlikte satılıyordu ya da kuruma özel yapılıyordu. 1969 yılında IBM yazılımı donanım ile birlikte verdiği için rekabet kurallarına uymadığı gerekçesiyle mahkemeye verilmiştir ve cezaya çarptırılmıştır. 1970'li yıllarda yazılım maliyetleri artmış, yazılım sektörü genişlemiş ve lisanslama politikaları da uygulamaya sokulmuştur. Pek çok yazılım bu yıllarda özel lisanslarla piyasaya sürülmeye başlanmıştır. 1980'li yıllarda bu lisanslama faaliyetleri hız kazanmıştır.

1980’li yıllarda tüm UNIX türevi sistemler çeşitli biçimlerde sınırlandırıcı lisanslara sahipti. Bu nedenle bedava ve sınırlamasız UNIX türevi bir işletim sistemine gereksinim duyulmaya başlandı. İşte durumdan vazife çıkaran ünlü Emacs editörünün yazarı Richard Stallman 1983 yılının sonlarına doğru GNU projesini başlattı ve özgür yazılım (free software) fikrini ortaya attı. GNU projesinin amacı açık kaynak kodlu UNIX benzeri bir işletim sistemini ve geliştirme araçlarını yazmaktır. Proje fiilen 1984 yılında başlamıştır.

Stallman 1985 yılında özgür yazılım kavramını yaygınlaştırmak amacıyla Free Software Foundation (<https://www.fsf.org>) isimli kurumu kurdu ve artık GNU projesi bu kurum tarafından yürütülmeye başlandı. FSF özgür yazılım modeli için GPL (GNU Public License) denilen bir lisans da geliştirdi. Özgür yazılım akımında oluşturulan bir yazılım istenildiği gibi çalıştırılabilir, kopyalanabilir, incelenebilir, dağıtılabilir, değiştirilebilir ve iyileştirilebilir. Özgür yazılım tipik olarak aşağıdaki dört özgürlükle tanımlanmıştır:

Özgürlük 0

Programı her türlü amaç için çalıştırma özgürlüğü.

Özgürlük 1

Programın kaynak kodunu inceleme ve değiştirebilme özgürlüğü.

Özgürlük 2

Programın kopyalarını çıkartabilme ve yeniden dağıtabilme özgürlüğü.

Özgürlük 3

Programı iyileştirebilme ve iyileştirilmiş programı yayınlama özgürlüğü.

GNU projesi bağlamında pek çok temel araç (gcc derleyici, ld bağlayıcı vb.) geliştirilmiştir. Fakat hedeflenen çekirdek bir türlü oluşturulamamıştır.

Aslında özgür yazılım (free software) ile açık kaynak kod (open source) akımları arasında bazı farklar olmakla birlikte her iki akımın da hedefleri benzerdir. Özgür yazılım bir sosyal harekete benzetilirken açık kaynak kod akımı bir geliştirme metodolojisine benzetilmektedir. Biz kursumuzda tüm bu akımları *açık kaynak kod (open source)* olarak nitelendireceğiz. Özgür yazılım akımının temel lisansı GPL’dir (GNU Public Licence). Bunun yumuşatılmış LGPL (Lesser GPL) biçiminde bir versiyonu da oluşturulmuştur. Ayrıca Apache, MIT, BSD gibi açık kaynak kodlu başka lisanslar da vardır.

1.11 Linux’un Tarihsel Gelişimi

Linus Torvalds Helsinki Üniversitesinde öğrenciyken bir işletim sistemi yazmaya niyetlenmiştir. O zamanlarda telif uygulanmayan UNIX türevi bir işletim sistemi kalmamıştı ve GNU projesinin işletim sistemi de (GNU Hurd) bitirilememişti. MINIX isimli bir işletim sistemi Andrew Tanenbaum tarafından yazılmıştı ancak bu sisteminin lisansı yalnızca akademik kullanımlara izin verecek biçimde sınırlandırılmıştı. Linus Torvalds projesini USENET haber gruplarında duyurdu ve zamanla kendisine gönüllü yardım edecek sistem programcıları buldu. Yazılım dünyasında bu tür girişimlerle sık karşılaşıldığı halde başarı olasılığı nispeten düşük olmaktadır. Linus Torvalds’ın bu girişimi başarıya ulaşmıştır.

1991 yılında Linux’un 0.01 sürümü oluşturuldu. 1994 yılında stabil bir biçimde Linux 1.0 versiyonu dağıtılmaya başlandı. Bunu 1996 yılında Linux 2.0, 1999 yılında 2.2, 2000 yılında 2.4 ve 2003 yılında 2.6 izledi. Daha sonra Linux versiyon numaralandırma sistemi değiştirilmiştir. 2011 yılında 3.0, 2015 yılında 4.0, 2019 yılında 5.0, 2022 yılında da 6.0 versiyonu çıkmıştır.

Linux çekirdeklerinin versiyonları ve bu versiyonlarda eklenen önemli özellikler aşağıdaki tabloda verilmiştir.

Tablo 1: Linux Çekirdeği Sürüm Geçmişi

Sürüm	Tarih	Önemli Yenilikler
0.01	Ağustos 1991	Linus Torvalds tarafından duyurulan ilk sürüm; sadece temel fonksiyonlara sahipti.
1.0	Mart 1994	İlk resmi sürüm; çoklu işlemci desteği yoktu. Ağ üzerinden TCP/IP desteği sağlandı.
1.2	Mart 1995	x86 dışı mimarilere (Alpha, MIPS) ilk destekler geldi.
2.0	Haziran 1996	SMP (Simetrik Çoklu İşlemci) desteği eklendi. Daha fazla mimari desteği sunuldu.
2.2	Ocak 1999	Gelişmiş ağ yığını, IPv6 desteği, daha fazla SMP ölçeklenebilirliği.
2.4	Ocak 2001	USB, PCMCIA ve Bluetooth desteği; 64 GB RAM'e kadar bellek desteği (PAE ile).
2.6	Aralık 2003	Yeni scheduler (O(1)), udev ile dinamik aygıt yönetimi, sysfs, Native POSIX Thread Library (NPTL), preemptive kernel.
3.0	Temmuz 2011	Büyük bir teknik değişiklik yok; sadece sürüm numarası sadeleştirildi (2.6.x'lerin devamı).
4.0	Nisan 2015	Canlı kernel güncelleme (live patching) özelliği eklendi.
5.0	Mart 2019	Yeni donanım desteği, enerji verimliliği iyileştirmeleri, Adiantum şifreleme algoritması.
5.4	Kasım 2019	Lockdown modu, fs-verity desteği.
5.10	Aralık 2020	EXT4 ve Btrfs iyileştirmeleri, AMD GPU desteği.
5.15	Kasım 2021	NTFS3 dosya sistemi, yeni I/O kontrolcüsü.
6.0	Ekim 2022	Rust diline ilk çekirdek içi destek, Scheduler ve TCP performans iyileştirmeleri.
6.1	Aralık 2022	Rust desteği genişletildi, Intel AMX desteği.
6.6	Kasım 2023	Apple Silicon (M1/M2) desteği, yeni enerji yönetimi özellikleri.

Linux monolithic bir çekirdek yapısına sahiptir. Büyük ölçüde POSIX uyumu bulunmaktadır.

1.12 Linux Dağıtımları

Açık kaynak kodlu yazılımlar bir araya getirilip paketlenerek istenildiği gibi dağıtılabilmektedir. Dağıtım (distribution) bu anlamda kullanılan genel bir terimdir ve her türlü açık kaynak kodlu yazılım için dağıtım söz konusu olabilir. Ancak biz burada Linux dağıtımları üzerinde duracağız.

Linux temel olarak bir çekirdek geliştirme projesidir. Linux kaynak kodlarına baktığınızda tüm kodların çekirdekle ilgili olduğunu görürsünüz. Çekirdeğin dışındaki tüm yazılımlar (örneğin init prosesinden başlayarak, kabuk yazılımları, paket yöneticileri, pencere yöneticileri vb.) hep başka proje grupları tarafından gerçekleştirilmiş açık kaynak kodlu yazılımlardır. İşte tüm bu açık kaynak kodlu yazılımların Linux çekirdeği temelinde bir araya getirilmesi ve doğrudan kullanıcının install edip çalıştırabileceği biçimde paketlenmesine Linux dağıtımları denilmektedir. Linux dağıtımları pencere yöneticileri (KDE, GNOME gibi), paket yöneticileri (APT, RPM, YUM, DPKG, PACMAN, ZYPPE gibi) ve diğer yararlı uygulama programları bakımından farklılıklar gösterebilmektedir.

Toplamda iki yüzün üzerinde Linux dağıtımının olduğu söylenebilir. Ancak bunlar arasında az sayıda dağıtım çok popüler olmuştur. Bazı dağıtımlar bazı dağıtımlardan fork edilerek oluşturulmuştur. Aşağıda en çok kullanılan dağıtımlara ilişkin dağıtım ağacı verilmektedir:

```

Linux
├── Debian
│   ├── Ubuntu
│   │   ├── Linux Mint
│   │   ├── Pop!_OS
│   │   ├── elementary OS
│   │   └── Zorin OS
│   ├── Devuan # Systemd olmayan Debian
│   └── Kali Linux # Güvenlik test amaçlı
├── Red Hat Linux (eski)
│   ├── Fedora # Topluluk temelli, RHEL'in test yatağı
│   │   └── RHEL (Red Hat Enterprise Linux)
│   │       ├── CentOS (→ 2021 sonrası CentOS Stream)
│   │       ├── AlmaLinux
│   │       └── Rocky Linux
├── Slackware
│   └── Slax # Hafif sürüm
├── Arch Linux
│   ├── Manjaro
│   └── EndeavourOS

```

(sonraki sayfaya devam)

```

├── Gentoo
│   └── Calculate Linux
├── SUSE Linux
│   ├── openSUSE Leap
│   └── openSUSE Tumbleweed
├── Android # Mobil, Linux çekirdeğine dayalı
├── Alpine Linux # Minimal, güvenli, konteyner dostu
├── Chrome OS
│   └── Chromium OS # Açık kaynak tabanlı

```

En çok kullanılan Linux dağıtımları aşağıda özetlenmiştir.

Debian: En önemli ve en eski Linux dağıtımlarından biridir. Knoppix, Mint ve Ubuntu dağıtımları Debian türevi dağıtımlardır.

Fedora: Red Hat firması tarafından çıkarılmış olan dağıtımdır. İlk kez 2003 yılında oluşturulmuştur. RPM paket yöneticisini kullanır. CentOS ve Scientific Linux en önemli Fedora türevi dağıtımlardır. 2000 yılında ilk sürümü yapılan Red Hat Enterprise Linux (RHEL) en önemli Fedora türevidir. Ondan da CentOS, Scientific Linux gibi dağıtımlar türetilmiştir. CentOS server makinelerde en yaygın kullanılan Linux versiyonudur.

OpenSUSE: Alman SUSE firmasının desteklediği dağıtımdır. SUSE Linux Enterprise isminde ticari bir versiyonu da vardır. ZYpp, YaST ve RPM paket yöneticilerini kullanmaktadır.

Slackware: En eski Linux dağıtımdır. 1993 yılında oluşturulmuştur. Sürdürümü yavaş olmakla birlikte hala devam etmektedir.

1.13 POSIX Standartları

1980'li yıllarda AT&T ya da BSD kodlarından türetilmiş olan ve çoğunluğu şirketlere ait olan pek çok UNIX türevi işletim sistemi oluşturuldu. Bu işletim sistemleri birbirlerine çok benzemekle birlikte aralarında bazı farklılıklara da sahipti. İşte IEEE durumdan vazife çıkartarak bu UNIX türevi sistemleri standardize etmek için kolları sıvadı ve bunun sonucu olarak da POSIX standartlarını oluşturdu.

POSIX sözcüğü Richard Stallman tarafından önerilmiştir. *Portable Operating System Interface for UNIX* sözcüklerinden kısaltılarak uydurulmuştur ve *poziks* biçiminde okunmaktadır. POSIX standartları üzerinde çalışmalar 1985 yılında başlamıştır ve ilk standartlar 1988 yılında *IEEE Std 1003.1-1988* kod numarasıyla oluşturulmuştur. POSIX her ne kadar UNIX türevi sistemler için düşünülmüşse de UNIX türevi mimariye sahip olmayan sistemler için de kullanılabilir bir standarttır. (Örneğin Windows sistemleri Interix denilen alt sistemle POSIX uyumlu olarak da kullanılabilirliydi. Interix alt sistemi daha sonra Windows 8 ile birlikte Windows'tan kaldırılmıştır.)

POSIX standartları 4 bölümden oluşmaktadır:

1. **Base Definitions:** Bu bölümde temel tanımlamalar bulunmaktadır.
2. **Shell & Utilities:** Bu bölümde kabuk komutları ve standart utility programlar ele alınmaktadır.
3. **System Interfaces:** Bu bölümde C programcıları için hazır bulunan POSIX fonksiyonları açıklanmaktadır.
4. **Rationale:** Çeşitli kuralların ve özelliklerin gerekçeleri bu bölümde açıklanmaktadır.

POSIX standartlarının zaman içerisinde çeşitli versiyonları çıkartılmıştır. Bu versiyonlarda hem yeni POSIX fonksiyonları kütüphaneye eklenmiş hem de standartlardaki bazı bozukluklar ve uyumsuzluklar düzeltilmiştir. Standartın önemli versiyonları şu senelerde yayınlanmıştır: 1992, 1993, 1995, 1997, 2001, 2004, 2008, 2017. Standartlardaki en önemli değişim 1993 yılında *POSIX 1.b* diye de isimlendirilen *Realtime-extensions* ile 1995 yılında *POSIX 1.c* diye isimlendirilen *Thread-extensions* isimli eklemelerdir. Bu eklemelerle POSIX'e gerçek zamanlı işlemler için çeşitli özelliklerle thread kullanımı eklenmiştir.

Single UNIX Specification, UNIX türevi sistemler için oluşturulmuş diğer önemli standarttır. Bir sistemin UNIX olarak değerlendirilebilmesi için bu standartlara uygun olması gerekmektedir. Standartlar Austin Group isimli toplulukla Open Group isimli dernek tarafından geliştirilmiştir. Sürdürümü Open Group tarafından yapılmaktadır. Open Group hali hazırda UNIX sistemlerinin isim haklarını elinde bulundurmaktadır.

POSIX standartları ile Single UNIX Specification standartları arasında eskiden daha fazla farklılıklar vardı. Ancak bugün itibarıyla bu iki standart birbirlerine yaklaştırılmış ve son versiyonlarla aynı hale getirilmiştir. Single UNIX Specification dokümanlarına İnternet'ten Open Group'un web sitesinden erişilebilir:

<https://pubs.opengroup.org/onlinepubs/9799919799/>

1.14 Linux Çekirdeğinin Kaynak Kod Yapısı

Linux çekirdeğinin kaynak kod ağacı üzerinde temel dizinler aynı kalmak üzere zaman içerisinde çeşitli değişiklikler yapılmıştır. Biz burada çekirdeğin son versiyonlarını dikkate alarak açıklamalar yapacağız. Linux kaynak kod ağacındaki temel dizinler şunlardır:

```
linux-6.x/
├── arch/           → Mimarilere özel kodlar (x86, ARM, RISC-V, vb.)
├── block/          → Blok aygıt altyapısı
├── certs/          → Kernel modül imzalama için sertifikalar
├── crypto/         → Kriptografik algoritmalar ve API'ler
├── Documentation/ → Belgeler (API, özellikler, davranışlar)
├── drivers/        → Aygıt sürücüler (net, usb, gpu, vb.)
├── fs/             → Dosya sistemi sürücüler (ext4, btrfs, vb.)
├── include/        → Global başlık dosyaları (headers)
├── io_uring/       → 5.1 çekirdeği ile eklenen asenkron io_uring sistem fonksiyonları
├── init/           → Kernelin ilk başlatma kodları
├── ipc/            → IPC mekanizmaları (semaphore, message queue, vb.)
├── kernel/         → Temel kernel işlevleri (zamanlayıcı, process, vb.)
├── lib/            → Genel amaçlı yardımcı fonksiyonlar
├── mm/             → Bellek yönetimi
├── net/            → Ağ yığını (TCP/IP, socket, protokoller)
├── rust/           → Rust dili ile yazılmış çekirdek kodları (6.x ile)
├── samples/        → Örnek kodlar (eBPF, modül kodları, vb.)
├── scripts/        → Derleme sürecine yardımcı betikler
├── security/       → Güvenlik altyapısı (LSM, SELinux, AppArmor, vb.)
├── sound/          → Ses sürücüler (ALSA, vb.)
├── tools/          → Kullanıcı uzayı araçları (perf, bpftool, vb.)
├── usr/            → Yerleşik initramfs oluşturmak için
├── virt/           → Sanallaştırma (KVM, Xen, vb.)
├── MAINTAINERS     → Dosyaların kim tarafından korunduğu bilgisi
├── Makefile        → Ana derleme talimatı
└── Kconfig         → Kernel yapılandırma seçenekleri
```

Dizinlerin açıklamaları aşağıda verilmiştir.

arch

Dizin adı *architecture* sözcüğünden kısaltılmıştır. Bu dizinde işlemci mimarisine göre değişebilen çekirdek kodları, her bir işlemci ailesi ayrı bir dizinde olacak biçimde bulundurulmaktadır.

block

Eskiden bu dizin *drivers* dizini içerisindeydi. Burada işletim sisteminin blok düzeyinde işlemler yapan kodları bulundurulmuştur.

certs

Bu dizinde çekirdek modüllerine ilişkin sertifikasyonlarla ilgili kodlar bulunmaktadır.

crypto

Çekirdeğin içerisinde kullanılan şifreleme işlemlerine yönelik kaynak kodlar bulunmaktadır.

Documentation

Burada çeşitli alt sistemlere ilişkin dokümanlar bulunmaktadır.

drivers

Burada aygıt sürücülere ilişkin çekirdek kodları bulunmaktadır. Tabii bu aygıt sürücüler isteğe göre seçilerek yalnızca bir grubu derlenmektedir.

fs

Bu dizinde dosya sistemlerine yönelik çekirdek kodları bulundurulmaktadır.

include

Çekirdek kodlarında include edilen tüm başlık dosyaları bu dizinin içerisinde bulundurulmaktadır. Tabii bu dizinde de alt dizinler vardır.

init

Çekirdeğin çalışabilir hale gelmesi için yapılan ilk işlemlere ilişkin kodlar burada bulundurulmaktadır.

io_uring

5.1 çekirdekleri ile eklenen yüksek performanslı asenkron IO işlemleri için kullanılan sistem fonksiyonlarının gerçekleştirimine ilişkin kodlar bu dizinde bulunmaktadır.

ipc

Borular, mesaj kuyrukları gibi IPC nesnelere ilişkin kaynak kodlar bu dizin içerisinde.

kernel

Bu dizin çekirdeğin en temel işlevlerine ilişkin kaynak kodları barındırmaktadır. (Çekirdek kodlarının içerisinde ayrıca bir **kernel** dizinin bulunması biraz tuhaf olsa da bu aslında adeta *çekirdeğin çekirdeği* gibi bir anlama gelmektedir.)

lib

Bu dizinde çekirdeğin değişik yerlerinde kullanılan genel amaçlı utility fonksiyonların kodları bulundurulmaktadır. Burada *lib* sözcüğünün statik ve dinamik kütüphane ile bir ilgisi yoktur. Zaten Linux derlenirken bu biçimde kütüphane dosyaları oluşturulmamaktadır.

mm

Çekirdeğin tüm ana bellek yönetim kodları bu dizinde bulunmaktadır.

net

Çekirdeğin tüm ağ (network) alt sistemine ilişkin kodları bu dizinde bulunmaktadır.

rust

Linux çekirdeğinde 6'lı versiyonlarla Rust Programlama Dili de kullanılmaya başlanmıştır. Bu dizinde çekirdeğe ilişkin Rust kodları bulunmaktadır. Kursun yapıldığı tarihteki Linux çekirdeklerinde Rust kodları çok az yer kaplamaktadır.

samples

Burada alt sistemlere ilişkin örnek kodlar bulunmaktadır.

scripts

Bu dizinde çekirdek derlemesi sırasında faydalanan script dosyaları bulundurulmuştur.

security

Çekirdeğin sistem güvenliği ile ilgili kodları bu dizinde bulunmaktadır.

sound

Ses işlemlerine yönelik çekirdek kodları bu dizinde bulunmaktadır.

tools

Bu dizinde çekirdek geliştirmesinde ve analizinde kullanılan yardımcı programlar bulundurulmaktadır. Aslında bu dizindeki programların çekirdekle bir ilgisi yoktur. Bunlar yardımcı araçlardır; çekirdek kodlarının içerisinde yer almazlar.

usr

Bu dizinde *geçici kök dosya sistemine (initial ramdisk)* ilişkin kodlar bulunmaktadır. Bunların bazı sistemlerde konfigürasyon yoluyla çekirdek kodlarına aktarılmaktadır.

virt

Burada çekirdeğin sanallaştırmaya hizmet eden kodları bulundurulmaktadır.

1.15 Linux Sistemlerinin Boot Edilmesi

Kursumuzun bu bölümünde Linux sistemlerinin boot edilmesi süreci ele alınacaktır. İşletim sisteminin otomatik olarak yüklenerek çalışır hale getirilmesi sürecine *boot* işlemi denilmektedir. (*Boot* terimi İngilizce *askeri bottan (çizmeden)* gelmektedir.) Biz bilgisayar sistemine güç verdiğimizde bir süre sonra işletim sisteminin otomatik yüklendiğini görmekteyiz. Aslında işletim sisteminin yüklenmesi biraz karmaşık bir süreçle gerçekleşmektedir.

1.15.1 Reset Vektörü ve BIOS/Firmware

Mikroişlemciler güç uygulandığında (reset edildiğinde) belli bir adresten itibaren çalışacak biçimde tasarlanmıştır. Buna işlemcilerin *reset vektörü* denilmektedir. İşlemcilerin reset vektörleri genellikle fiziksel belleğin başında ya da sonunda bulunmaktadır. Örneğin Intel işlemcilerinde reset vektörü belleğin sonunda, ARM işlemcilerinde ise belleğin başında bulunmaktadır. İşlemci reset edildiğinde belli bir adresten çalışmaya başladığına göre orada çalışmaya hazır bir kodun bulunuyor olması gerekir. Tabii bu kod RAM (DRAM) bellekte bulunamaz. Çünkü sisteme güç uygulandığında RAM belleğin de içeriği sıfırlanmaktadır, ayrıca bugünkü DRAM belleklerin kullanılabilmesi için de onların donanımsal olarak programlanması gerekmektedir. İşte bilgisayar sistemlerinde işlemcinin reset vektöründe tipik olarak ROM bellekler bulundurulur. Eskiden bu amaçla EPROM bellekler kullanılıyordu. Artık uzunca bir süredir EEPROM (flash EPROM) bellekler kullanılmaktadır.

Bugün kullandığımız DRAM bellekler güç kaynağı verilir verilmez kullanıma hazır hale getirilememektedir. Onların da kullanıma hazır hale getirilmesi (initialize edilmesi) gerekir. Benzer biçimde yine bazı donanım aygıtlarının kullanılmadan önce kullanıma hazır hale getirilmesi de gerekmektedir. İşte reset vektörüne yerleştirilmiş olan kodlar bilgisayar sistemini kullanıma hazır hale getirecek kodları barındırmaktadır. Tabii burada donanımdan donanıma değişen bazı ayrıntılar da söz konusu olabilmektedir. Reset vektöründeki kodlar genellikle donanımı üreten firmalar tarafından yazılmaktadır. Bunlar çok temel kodlar olduğu için bunlara *BIOS* ya da *firmware* gibi isimler verilmiştir. Bu tür temel BIOS ya da firmware kodlarının donanımı kullanıma hazır hale getirmek için yaptığı işlemlerden bazıları şunlardır:

- DRAM bellekleri (bildiğimiz RAM) kullanıma hazır getirilmesi
- Çok çekirdekli sistemlerde çekirdeklerin kullanıma hazır hale getirilmesi
- Donanım aygıtlarının ve çevre birimlerinin tespit edilmesi ve bunlar hakkındaki bilgilerin oluşturulması (örneğin ACPI tablosunun oluşturulması)
- Çevre birimlerinin (yani yardımcı işlemcilerin) kullanıma hazır hale getirilmesi
- SSD ve HDD gibi depolama birimlerinin kullanıma hazır hale getirilmesi
- Klavye, fare gibi giriş çıkış aygıtlarının kullanıma hazır hale getirilmesi
- Ekran kartı gibi görüntü aygıtlarının kullanıma hazır hale getirilmesi

Reset vektöründeki kodlar çalıştırılınca artık donanım genel olarak çalışmaya hazır bir duruma getirilmiştir. Bundan sonra akışın bir biçimde sistem programcısına devredilmesi gerekir. Akışın devredilmesi tipik olarak ikincil bellekteki belli bir disk bloğunun RAM'e yüklenmesi ve oradaki kodun çalıştırılması yoluyla yapılmaktadır. Tersten gidersek sistem programcısı programını ikincil belleğin (diskin) bir bloğuna yerleştirir ve bu süreçler sonucunda bu program çalışır hale gelir. Peki işletim sistemi nasıl yüklenmektedir? ROM'daki BIOS ya da firmware kodunun işletim sistemini yüklemesi genel olarak mümkün değildir. Bunun temel nedenleri şunlardır:

- İşletim sistemleri çok büyük olabilir. ROM'daki kod bunu yapabilecek yeterlilikte olmayabilir.
- İşletim sistemlerinin yüklenmesi sistemden sisteme değişebilen ve nispeten karmaşık bir süreçtir. ROM'daki küçük kodun bunu yapması genellikle mümkün olamamaktadır.
- İkincil bellekte birden fazla işletim sistemi bulunuyor olabilir. Bu durumda bunların hangisinin nasıl yükleneceğine karar verilmesi ve karar verilen işletim sisteminin yüklenmesi ROM'daki küçük program tarafından genellikle yapılamamaktadır.

O halde tek çıkar yol ROM'daki programın diskteki bir önyükleyiciyi yüklemesi ve işletim sisteminin yüklenmesinin bu program tarafından yapılmasıdır.

1.15.2 Önyükleyici (Bootloader) Programlar

İşletim sistemlerini yükleyen bu tür programlara *önyükleyici* (*bootloader*) denilmektedir. (Biz *bootloader* terimi yerine bunun Türkçesi olan *önyükleyici* terimini de kullanacağız.) Yani yukarıda da belirttiğimiz gibi sistem programcısı diskteki önceden tespit edilmiş alana kendi kodunu yerleştirir. (Genellikle BIOS ya da firmware kodları küçük bir disk bloğunu yükleyip çalıştırmaktadır.) İşte sistem programcısının özel disk bloğuna yerleştirdiği bu program da önyükleyici (bootloader) programını yüklemektedir.

Bugün değişik platformlarda kullanılan değişik *önyükleyici* programlar bulunmaktadır. Örneğin Microsoft Windows sistemleri için kendi *önyükleyici* programını kullanmaktadır. Buna *Windows Boot Manager* (*bootmgr*) denilmektedir. UNIX/Linux sistemlerinde değişik proje grupları tarafından yazılmış olan değişik önyükleyici programlar kullanılabilir. Örneğin Linux sistemlerinde eskiden *LILLO* isimli önyükleyici yoğun biçimde kullanılıyordu. Daha sonra *GRUB* isimli önyükleyici yaygın biçimde kullanılmaya başlandı. Son yıllarda *SYSLINUX* isimli bootloader paketi de belli bir ölçüde kullanım bulmuştur. *SYSLINUX* önyükleyicisi minimalist bir tasarıma sahiptir ve daha çok küçük sistemlerde tercih edilmektedir. Gömülü Linux sistemlerinde ise bugün en çok tercih edilen *Das U-Boot* ya da kısaca *U-Boot* (*Universal Bootloader*) denilen önyükleyici programdır.

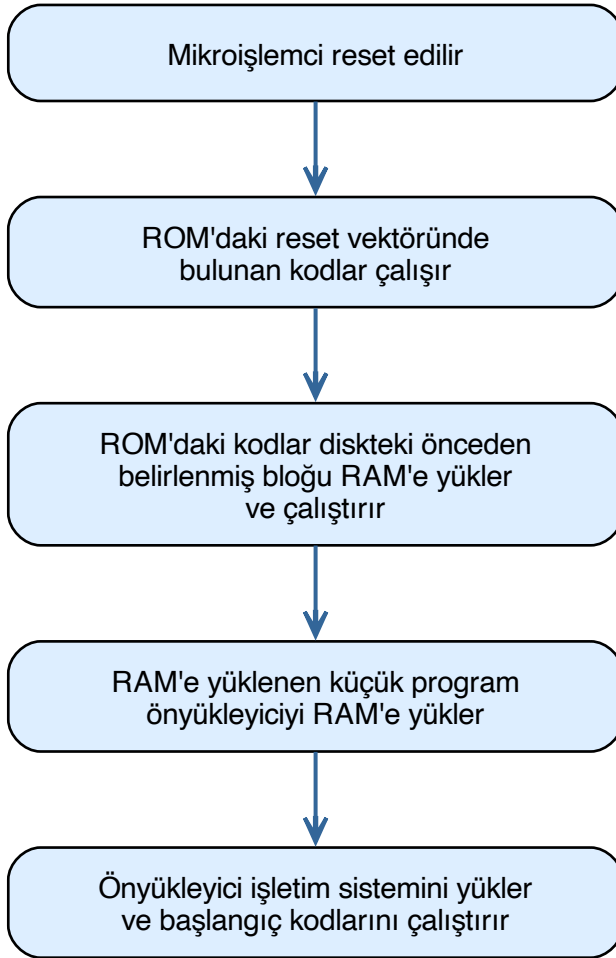
Peki ROM'daki program önyükleyici (bootloader) programını ikincil bellekte nasıl arayıp bulmaktadır? Bunun için birkaç teknik kullanılabilir. Birincisi ROM'daki programın doğrudan ikincil belleğin önceden belirlenmiş bloklarını yüklemesidir. Örneğin klasik PC sistemlerinde ROM'daki (BIOS'taki) program diskin 0'ıncı bloğunu (yani ilk 512 byte'ı içeren bloğu) yüklemektedir. (Ancak daha sonra PC sistemlerinde *UEFI BIOS* ismi altında eski klasik BIOS kodları oldukça geliştirilmiştir. Artık bu UEFI BIOS kodları bazı dosya sistemlerini de tanımaktadır.) Örneğin Raspberry Pi'da ROM'daki kodlar FAT dosya sistemini tanıyabilmektedir. FAT dosya sistemi Microsoft'un DOS sistemlerinde kullandığı karmaşık olmayan sade bir dosya sistemidir. İşte ROM'daki kodlar FAT gibi bir dosya sistemini tanıyorsa FAT dosya sisteminin kök dizininde belli isimdeki dosyayı bulup RAM'e de yükleyebilmektedir. Bazı sistemlerde (örneğin Beaglebone Black'lerde) ROM'daki kod önce küçük bir önyükleyiciyi yüklemekte, bu küçük önyükleyici de asıl önyükleyiciyi yükleyebilmektedir.

Burada birkaç soru aklınıza gelebilir. Bunlardan biri ROM'daki reset vektöründe bulunan kodların oraya kimin tarafından yerleştirilmiş olduğudur. ROM'daki reset vektöründe bulunan kodlar çok aşağı seviyeli kodlar olduğu için bunlar genellikle donanımı tasarlayan

kurumlar tarafından yazılıp oraya yerleştirilmektedir. Örneğin bugün kullandığımız PC sistemlerinde ROM'daki bu kodlara BIOS (Basic Input Output System) denilmektedir. BIOS kodları PC donanımını tasarlayan IBM tarafından yazılmıştı. Örneğin Beaglebone Black kartlarındaki ROM programı Texas Instruments (TI) firması tarafından, Raspberry Pi kartlarındaki ROM programı ise Broadcom firması tarafından yazılmıştır. Özetle ROM'da bulunan bu aşağı seviyeli kodlar ilgili donanımı tasarlayan kurumlardaki sistem programcıları tarafından yazılmaktadır. Belli bir süredir bu tür BIOS alanlarında EEPROM teknolojisi kullanıldığı için BIOS güncellemeleri de yapılabilmektedir.

1.15.3 Genel Boot Akışı

İşletim sisteminin yüklenmesi pek çok donanım ve platformda aşağıdaki aşamalardan geçilerek yapılmaktadır:



1.15.4 Boot Aşamaları Terminolojisi

Sistem reset edildiğinde tüm boot işlemi bir bütünün parçaları gibi düşünülürse burada devreye giren programlara birer aşama numarası da verilebilmektedir. Örneğin boot işleminin reset vektöründe bulunan kısmına *birinci aşama önyükleyici (stage-1 / first stage bootloader)*, bu programın yüklediği programa *ikinci aşama önyükleyici (stage-2 / second stage bootloader)* ve bunun da yüklediği programa (yani işletim sistemini yükleyen GRUB gibi programlara da) *üçüncü aşama önyükleyici (stage-3 / third stage bootloader)* denilebilmektedir. Ancak bazı kaynaklar bu ROM'daki kodu boot sürecinin bir parçası olarak ele almamaktadır. Dolayısıyla bu kaynaklar ROM kodunun kendisine değil, onun yüklediği programa *birinci aşama önyükleyici (stage-1 / first stage bootloader)*, işletim sistemini yükleyen programa (yani GRUB gibi) ise *ikinci aşama önyükleyici (stage-2 / second stage bootloader)* demektedir. Örneğin Wikipedia boot işleminde

donanım bileşenlerini hazır hale getiren reset vektöründeki programı *birinci aşama önyükleyici*, işletim sistemini yükleyen programı ise (GRUB gibi) *ikinci aşama önyükleyici* olarak isimlendirmiştir. Biz kursumuzda izleyen paragraflarda bu terminolojiyi kullanacağız.

Bugün pek çok masaüstü Linux dağıtımında GRUB isimli önyükleyici program kullanılmaktadır. Bu önyükleyici program Linux çekirdek imajını belleğe yükleyerek çalıştırmaktadır. Linux'un çekirdek parametreleri de bu önyükleyici tarafından kullanıcıdan alınıp Linux'a verilmektedir. Biz kursumuzda çekirdek derlemesi yaparken ve sistemi çekirdekle boot ederken önyükleyicinin GRUB olduğunu varsayacağız.

1.15.5 PC Sistemlerinde Boot Süreci (Klasik BIOS)

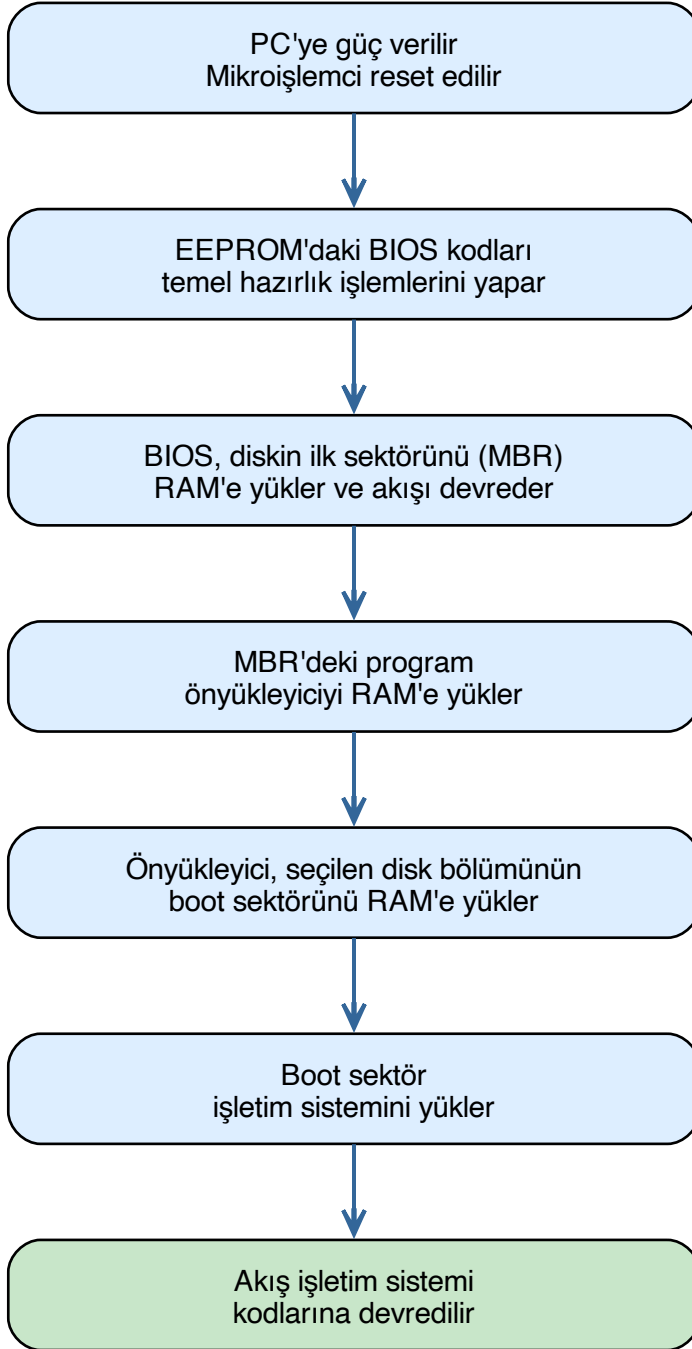
Kursumuzdaki örnekler x86 tabanlı masaüstü bilgisayar kullanılarak verilmektedir. (Burada *masaüstü (desktop)* terimi yalnızca kasalı bilgisayarlar için değil notebook'lar için de kullanılan genel bir terimdir.) x86 tabanlı masaüstü bilgisayarlara tarihsel bakımdan *PC (Personal Computer)* de denilmektedir. (2000'lerin başında Apple Intel tabanlı PC mimarisine geçtiyse de kullandığı PC donanımında bazı farklılıklar da oluşturmuştur. Sonra Apple'ın Intel mimarisini de bırakıp ARM mimarisine geçtiğini biliyorsunuz. Ancak PC denildiğinde Intel ve ARM tabanlı macOS sistemleri kastedilmemektedir.)

Bugün kullandığımız PC sistemlerinin temel donanımları 70'li yılların sonlarında ve 80'li yılların başlarında IBM tarafından tasarlanmıştır. Bu bilgisayarlar 1980'in Aralık ayında IBM'in tasarladığı donanım ve Microsoft'un tasarladığı DOS işletim sistemiyle piyasaya sürüldü. Zaman ilerledikçe bu PC donanımlarında bazı iyileştirmeler yapıldıysa da temel mimari büyük ölçüde aynı kalmıştır.

Eskiden PC'lerde klasik BIOS (*legacy BIOS*) kullanılıyordu. Ancak 2010'lardan sonra modern UEFI BIOS'lar yaygınlaştı. Artık bugün ağırlıklı olarak UEFI BIOS'lar kullanılıyor. (Ancak bu modern UEFI BIOS'lar eski klasik BIOS (*legacy BIOS*) gibi de çalışabilecek özelliklere sahip olabilmektedir.) Biz burada klasik eski BIOS'a sahip PC'lerin boot sürecinden bahsedeceğiz.

Eski klasik BIOS'lara (*legacy BIOS*) sahip PC'ler reset edildiğinde BIOS kodları DRAM belleği ve pek çok donanım birimini programladıktan sonra CMOS setup'ta belirtilen *boot sırasına (boot sequence)* göre ilgili medyanın 0'ıncı sektörünü belleğe yükleyip akışı oraya devretmektedir. (PC mimarisinde diskten okunabilecek ya da diske yazılabilecek en küçük birime *sektör (sector)* denilmektedir.) İkincil belleklerdeki ilk sektöre *MBR (Master Boot Record)* denilmektedir. MBR'de toplam 512 byte'lık bir program bulunur. MBR'nin sonunda 64 byte'lık *Disk Bölümleme Tablosu (Disk Partition Table)* vardır. MBR'deki program duruma göre ya bir önyükleyici programını yüklemekte ya da default durumda aktif disk bölümünün 0'ıncı sektörünü RAM'e yükleyip akışı oraya devretmektedir. (PC sistemlerinde her disk bölümünün ilk sektörüne (0'ıncı sektörüne) *boot sektör* denilmektedir. Boot sektör ilgili işletim sisteminin yüklenmesinden sorumludur.) Tabii MBR'deki program daha büyük bir önyükleyici programını da yükleyebilmektedir. Bu durumda hangi işletim sisteminin yükleneceği önyükleyici program tarafından bir menü yoluyla kullanıcıya da sorulabilmektedir.

UEFI BIOS öncesindeki eski BIOS sistemini kullanan (*legacy BIOS*) PC sistemlerindeki boot sürecini aşağıdaki gibi özetleyebiliriz:



Kursumuzda UEFI BIOS sistemlerinin işlevi ve temel çalışma mekanizması başka bir bölümde ele alınacaktır.

Yukarıda da belirttiğimiz gibi bugünkü Linux yüklü olan Intel x86 tabanlı PC sistemlerinde genellikle önyükleyici olarak GRUB tercih edilmektedir.

Çekirdeğin Derlenmesi

Bu bölümde ayrıntılı bir biçimde Linux çekirdeğinin derlenmesi sürecini ele alacağız.

2.1 Çekirdek Davranışını Değiştirme Yöntemleri

Linux çekirdeğinin kodlarına dokunmadan onunla ilgili bazı davranış değişikliklerinin yapılması temelde beş biçimde sağlanabilmektedir:

1. Çekirdek modülleri ve aygıt sürücüler yoluyla

Çekirdek modunda çalışan *çekirdek modülleri* (*kernel modules*) ve *aygıt sürücüler* (*device drivers*) yoluyla davranış değişikliği oluşturulabilmektedir. İşletim sistemlerinin çoğu çekirdeğin bir parçası gibi işlev görecektir kodların yüklenmesine ve çalıştırılmasına olanak sağlamaktadır. Bu yöntemde çekirdeğin yeniden derlenmesine gerek yoktur. Zaten çekirdek modülleri ve aygıt sürücüler çalışmakta olan çekirdek içerisine yüklenmektedir. Çekirdek modüllerini ve aygıt sürücülerini *kasalı bilgisayarlardaki genişleme yuvalarına takılan kartlara benzetebiliriz*. Nasıl takılan bu kartlar donanımın bir parçası haline geliyorsa çekirdek modülleri ve aygıt sürücüler de çekirdeğin bir parçası haline gelmektedir. Linux'taki çekirdek modüllerinin ve aygıt sürücülerinin yazımı kursumuzun konularına dahil değildir.

2. Çekirdek komut satırı parametreleri yoluyla

Çekirdek yüklenip başlatılırken (initialize ederken) ismine *çekirdek komut satırı parametreleri* (*kernel command line parameters*) denilen parametreler yoluyla çekirdeğin davranışı değiştirilebilmektedir. Çekirdek komut satırı parametreleri *önyükleyici tarafından* çekirdeğe iletilmektedir. Linux çekirdeğinin pek çok komut satırı parametresi vardır. Bu parametreler yoluyla çekirdekte bazı davranış değişiklikleri oluşturulabilmektedir. Bunun için de çekirdeğin yeniden derlenmesi gerekmez.

3. Konfigürasyon parametreleri yoluyla

Çekirdek derlenirken çekirdek kodlarına hiç dokunmadan bazı konfigürasyon parametreleri ile oynanarak çekirdekte davranış değişiklikleri oluşturulabilmektedir. Çekirdek konfigüre edilirken bazı alt sistemler çekirdekten çıkartılabilmekte, bazı alt sistemler üzerinde ince ayarlar yapılabilmektedir. Tabii çekirdek konfigüre edilirken her ne kadar biz çekirdek kodlarını değiştirmiyor olsak da aslında sembolik sabitler yoluyla arka planda derleme işlemine sokulan kodlar üzerinde de değişiklikler yapılmış olmaktadır. O halde çekirdeğin konfigüre edilmesinin iki amacı vardır:

- Çekirdek içerisindeki alt sistemlere ilişkin bileşenlerin çekirdeğe eklenmesini ve çekirdekten çıkartılmasını sağlamak.
- Çekirdeğin üzerinde bazı davranış değişikliklerini oluşturmak.

Çekirdek konfigüre edildikten sonra yeniden derlenmelidir. Yani bu yöntem çekirdeğin yeniden derlenmesini gerektirmektedir.

4. Çekirdek kodlarında doğrudan değişiklik yaparak

Çekirdek kodlarında doğrudan değişiklikler yapıp çekirdek yeniden derlenebilir. Bu çekirdekte davranış değişikliği oluşturmak için kullanılan en aşağı seviyeli yöntemdir.

5. Çekirdek çalışırken komutlar ve mekanizmalar yoluyla

Nihayet çekirdek çalışırken de çekirdeğin davranışı bazı komutlarla (bu komutlar bazı mekanizmaları ve sistem fonksiyonlarını kullanmaktadır), konfigürasyon dosyalarıyla ve bazı mekanizmalarla da (örneğin *proc* ve *sys* dosya sistemi yoluyla) çekirdek davranışları değiştirilebilmektedir. Tabii bunun için de çekirdeğin yeniden derlenmesi gerekmez. Örneğin sistem çalışırken bir prosesin açabileceği dosya sayısını *proc* dosya sistemi yoluyla şöyle değiştirebiliriz:

```
$ echo 2048 | sudo tee /proc/sys/fs/file-max
```

2.2 Çekirdek Komut Satırı Parametreleri

Linux'ta çekirdek komut satırı parametreleri birbirinden boşluklarla ayrılmış yazılardan oluşmaktadır. Bazı parametrelerin argümanları yoktur, bazılarının vardır. Eğer parametrenin bir argümanı varsa *parametre=değer* biçiminde (= karakteri boşluksuz olarak kullanılmalıdır), yoksa yalnızca *parametre* biçiminde belirtilmektedir. Çekirdek komut satırı parametreleri tek bir yazı biçiminde çekirdeğe aktarılmaktadır. Çekirdek bu komut satırı parametrelerini kendini kullanıma hazır hale getirmenin (kendini initialize etmenin) ön aşamalarında parse eder ve bu değerleri yapılandırma amacıyla kullanır. Tabii çekirdeğin komut satırı parametreleri tipik olarak önyükleyici (bootloader) tarafından çekirdeğe iletilmektedir. Örneğin çekirdek komut satırı parametreleri aşağıdaki gibi bir görünümde olabilir:

```
console=serial0,115200 console=tty1 root=PARTUUID=382d6f16-02 rootfstype=ext4 fsck.repair=yes
rootwait quiet splash plymouth.ignore-serial-consoles cfg80211.ieee80211_regdom=TR
```

Linux çekirdeğinin onlarca farklı komut satırı parametresi vardır. Bunların çoğu spesifik konulara ilişkindir ve ancak çekirdeğin yapısını iyi bilen kişiler tarafından anlamlandırılabilir. Tabii bazı parametrelerin anlamları herkes tarafından anlaşılacak kadar açıktır. Çekirdek parametrelerinin dokümantasyonuna aşağıdaki bağlantıdan erişebilirsiniz:

<https://www.kernel.org/doc/html/v4.14/admin-guide/kernel-parameters.html>

Linux çekirdeği komut satırı parametrelerini parse ederken gerçekte olmayan (yani çekirdek tarafından kullanılmayan) bir parametre gördüğünde onu dikkate almamaktadır. Ancak biz kendi parametrelerimizin de çekirdek tarafından saklanması sağlayabiliriz. Bu konunun bazı ayrıntıları vardır.

Biz kursumuzda çeşitli bölümlerde çekirdeğin bazı komut satırı parametreleri hakkında bazı açıklamalarda bulunacağız.

2.3 Çekirdeği Yeniden Derlemenin Gerekliği Durumlar

Bazı durumlarda çekirdeğin sıfırdan derlenmesi gerekebilmektedir. Çekirdeğin yeniden derlenmesinin gerektiği tipik durumlar şunlardır:

- Bazı çekirdek modüllerinin ve aygıt sürücülerin çekirdek imajından çıkartılması ve dolayısıyla çekirdeğin küçültülmesi için.
- Yeni birtakım modüllerin ve aygıt sürücülerin çekirdek imajına eklenmesi için.
- Çekirdeğe tamamen başka birtakım özelliklerin ve alt sistemlerin eklenmesi için.
- Çekirdek üzerinde çekirdek komut satırı parametreleriyle sağlanamayacak bazı konfigürasyon değişikliklerinin yapılabilmesi için.
- Çekirdek kodlarında yapılan değişikliklerin etkin hale getirilmesi için.
- Çekirdeğe yama (patch) yapılması için.
- Yeni çıkan çekirdek kodlarının kullanılabilir hale getirilmesi için ya da eski çekirdeklerin kullanımını sağlamak için.

2.4 Çekirdek Derlemesi Hakkında

Bu bölümde çekirdek kodlarının nasıl derleneceği ve derlenmiş olan çekirdekle sistemin nasıl boot edileceği üzerinde duracağız.

2.4.1 Çapraz Derleme

Çekirdek derlemesi o anda çalışılan platform için yapılabileceği gibi başka bir platform için de yapılabilmektedir. Aşağı seviyeli yazılım terminolojisinde başka bir sistem için yapılan derleme işlemlerine *çapraz derleme* (*cross compile*) denilmektedir. Örneğin Intel tabanlı PC'lerde ARM işlemcilerinin bulunduğu gömülü aygıtlar için Linux çekirdeğini derlemek isteyebiliriz. Bu bir çapraz derleme faaliyetidir. Çapraz derleme işlemleri için *çapraz araç zincirlerinin* (*cross toolchains*) kullanılması gerekmektedir. Linux çekirdeğinin derlenmesinde yalnızca C derleyicisi değil pek çok yardımcı programlara da gereksinim duyulmaktadır.

2.4.2 Çekirdek Kaynak Kodlarının İndirilmesi

Çekirdeğin derlenmesi için öncelikle çekirdek kaynak kodlarının derleme yapılacak bilgisayara indirilmesi gerekir. Bazı dağıtımlar default durumda çekirdeğin kaynak kodlarını da kurulum sırasında makineye çekmektedir. Çekirdek kodları resmi olarak kernel.org sitesinde bulundurulmaktadır. Tarayıcıdan kernel.org sitesine girip `pub/linux/kernel` dizinine geçtiğinizde tüm yayınlanmış çekirdek kodlarını göreceksiniz. İndirmeyi tarayıcıdan doğrudan yapabilirsiniz. Eğer indirmeyi komut satırından *wget* programıyla yapmak istiyorsanız aşağıdaki URL'yi kullanabilirsiniz:

```
https://cdn.kernel.org/pub/linux/kernel/[MAJOR_VERSION].x/linux-[VERSION].tar.xz
```

Buradaki MAJOR_VERSION 3, 4, 5, 6 gibi tek bir sayıyı belirtmektedir. VERSION ise çekirdeğin büyük ve küçük numaralarını belirtmektedir. Örneğin biz çekirdeğin 6.9.2 versiyonunu komut satırından şöyle indirebiliriz:

```
$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.9.2.tar.xz
```

Sıkıştırma Formatları ve tar Komutu

Çekirdek kodları indirildikten sonra açılması gerekir. Açma işlemi *tar* komutuyla aşağıdaki gibi yapılabilir:

```
$ tar -xvJf linux-5.15.12.tar.xz
```

UNIX/Linux sistemlerinde kullanılan *tar* programı sıkıştırma yapmamaktadır. Yalnızca dosyaları uç uca ekleyip onları tek bir dosya haline getirmektedir. Bu sayede dosyalar dosya sisteminde daha az yer kaplar hale getirilmektedir. Aynı zamanda onların iletilmesi ve kopyalanması da kolaylaştırılmaktadır. Sıkıştırma programları ise aslında tek bir dosyayı sıkıştırmaktadır. O halde UNIX/Linux sistemlerinde kullanıcılar bir grup dosyayı sıkıştırmak için önce onları *tar*'layıp tek dosya haline getirirler, sonra bu dosyayı sıkıştırırlar. Bu işlemler sonucunda dosya uzantısı *.tar.gz* gibi, *.tar.xz* gibi dosyanın hem *tar*'lanmış hem de sıkıştırılmış olduğunu belirten biçimde olur. Açım sırasında da önce sıkıştırılan dosyalar açılır, buradan *.tar* dosyası elde edilir; sonra da bu *.tar* dosyasının içerisindeki dosyalar dışarı çıkartılır.

Linux'ta çeşitli sıkıştırma formatları kullanılmaktadır:

- **gzip** — uzantı: *.gz*
- **bz2** — uzantı: *.bz2*
- **xz** — uzantı: *.xz*
- **zip** — uzantı: *.zip* ya da *.z*

Bu formatlar sıkıştırma bakımından farklı performans göstermektedir. Ancak formatın sıkıştırma performansı ne kadar yüksekse işlem yapma süresi de o kadar uzun olmaktadır. Tipik sıkıştırma performans sıralaması şöyledir:

```
xz > bz2 > gzip ≈ zip
```

Yani en iyi sıkıştırma *xz* formatında, daha sonra *bz2* formatında, daha sonra da *gzip* formatındadır. *gzip* ile *zip* aynı algoritmaları kullandığından performansları birbirine benzerdir.

Her format için ayrı araçlar mevcuttur:

Format	Sıkıştırma	Açma
gzip	gzip	gunzip
bz2	bzip2	bunzip2
xz	xz	unxz
zip	zip	unzip

tar programının en çok kullanılan seçenekleri şunlardır: *-c* (*tar*'lamak için), *-x* (açmak için), *-v* (ayrıntılı çıktı), *-f* (*.tar* dosyasını belirtmek için; seçenek listesinin sonunda olmalıdır).

Uyarı

-f seçeneğinin seçenek listesinde en sonda yer alması gerektiğine dikkat ediniz.

gzip ile örnekler:

```
# Yalnızca tar'lama:
$ tar -cf test.tar x.txt y.txt z.txt

# Tek adımda tar'la + gzip sıkıştır (-z seçeneği):
$ tar -cvzf test.tar.gz x.txt y.txt

# Aç:
$ tar -xvzf test.tar.gz
```

Not

gzip ve *gunzip* programlarının kaynak dosyayı sildiğine dikkat ediniz.

bzip2 ile örnekler:

```
# Tek adımda tar'la + bzip2 sıkıştır (-j seçeneği):
$ tar -cvjf test.tar.bz2 x.txt y.txt

# Aç:
$ tar -xvjf test.tar.bz2
```

xz ile örnekler:

```
# Tek adımda tar'la + xz sıkıştır (-J seçeneği, büyük harf):
$ tar -cvJf test.tar.xz x.txt y.txt

# Aç:
$ tar -xvJf test.tar.xz
```

Not

tar seçeneklerinde *-z* gzip için, *-j* bzip2 için, *-J* (büyük harf) xz için kullanılmaktadır.

2.4.3 Dağıtıma Özgü Çekirdek Kaynak Kodları

Dağıtımlar (Ubuntu, Mint, Fedora gibi) çekirdek kodlarında küçük değişiklikler ve kendilerine özgü özelleştirmeler ve yamamalar da yapabilmektedir. Debian tabanlı sistemlerde o anda makinede yüklü olan mevcut çekirdeğin ilgili dağıtıma ilişkin kaynak kodlarını indirmek için aşağıdaki komutu kullanabilirsiniz:

```
$ sudo apt-get install linux-source
```

Burada yükleme `/usr/src` dizinine yapılacaktır. Ancak bu komut doğrudan sıkıştırılmış dosyayı indirmektedir; açım yapmamaktadır. Ayrıca belirli bir versiyona ilişkin kaynak kodlar da indirilebilir. Bunun için komutta `linux-source` argümanına istenilen versiyonun majör, minör ve patch numarası `-majör.minör.patch` biçiminde eklenir. Örneğin:

```
$ sudo apt-get install linux-source-6.8.0
```

Not

Bu biçimde indirilen çekirdek kaynak kodları Debian ya da Ubuntu depolarından indirilmektedir. Bunlar bu dağıtımlar tarafından yamanmış kodlardır. Örneğin Mint'te çalışıyorsanız indirdiğiniz kodlar Ubuntu için yamanmış kodlar olacaktır.

BBB (BeagleBone Black) için:

BBB için bazı özelleştirmelerin de yapılmış olduğu kaynak kodların indirilip derlenmesi birtakım kolaylıklar sağlamaktadır:

```
$ git clone https://github.com/beagleboard/linux.git
```

Raspberry Pi için:

Raspberry Pi'a özgü daha güncel aygıt sürücüler ve aygıt ağacı dosyalarını içeren Linux kaynak kodlarının projenin kendi sitesinden indirilmesi daha uygun olur:

```
$ git clone --depth=1 https://github.com/raspberrypi/linux.git
```

2.5 Çekirdek Versiyon Numaralandırması

Linux kaynak kodlarının versiyonlanması eskiden daha farklıydı. Çekirdeğin 2.6 versiyonlarından sonra versiyon numaralandırma sistemi değiştirildi. Eskiden (2.6 ve öncesinde) versiyon numaraları çok yavaş ilerletiliyordu. 2.6 sonrasındaki yeni versiyonlamada versiyon numaraları daha hızlı ilerletilmeye başlanmıştır. Bugün kullanılan Linux versiyonları nokta ile ayrılmış üç sayıdan oluşmaktadır:

Majör.Minör.Patch

Buradaki *majör numara* büyük ilerlemeleri, *minör numara* ise küçük ilerlemeleri belirtmektedir. Eskiden (2.6 ve öncesinde) tek sayı olan minör numaralar *geliştirme versiyonlarını* (ya da *beta versiyonlarını*), çift olanlar ise stabil hale getirilmiş dağıtılan versiyonları belirtiyordu. Ancak 2.6 sonrasında artık tek ve çift minör numaralar arasında böyle bir anlam farklılığı kalmamıştır. *Patch numarası* birtakım böceklerin giderildiği ya da çok küçük yeniliklerin çekirdeğe dahil edildiği versiyonları belirtmektedir.

Linux kaynak kodları konfigüre edilip derlendiğinde çekirdek imajının ismine bir alan daha eklenebilmektedir. Bu alana biz *Extra* alanı diyeceğiz. Bu durumda çekirdek imaj ismi şu hale gelecektir:

Majör.Minör.Patch-Extra

Extra için `-rcX`, `-stable`, `-custom`, `-generic` gibi sözcükler kullanılabilir. Bu *extra* alanı tamamen derleme işlemini yapan kişi ya da kurumun isteğine göre belirlenmiş bir alandır:

- *rcX* — *release candidate* (yayın adayı) anlamındadır; *X* bir sayı belirtir.
- *stable* — dağıtılan sürümün *kararlı sürüm* olduğunu belirtir.
- *custom* — sistem programcısının çekirdekte birtakım değişiklikler yaptığını belirtir.
- *generic* — çekirdeğin *genel kullanım için konfigüre edilmiş* olduğunu belirtir; dağıtımlar tarafından sıkça kullanılır.
- *realtime* — genellikle gerçek zamanlı bir yapılandırmada kullanılmaktadır.

generic ve *realtime* sözcüklerinin öncesinde `-N-` biçiminde bir sayı da bulunabilmektedir. Bu sayı dağıtıma özgü yama ya da derleme numarasını belirtmektedir. Örneğin:

6.8.0-51-generic

Burada `-51` yapılandırmaya özgü bir numarayı, `-generic` ise yapılandırmanın genel kullanım için olduğunu belirtmektedir.

Çalışmakta olan Linux sistemi hakkında bilgiler "`uname -a`" komutu ile elde edilebilir:

```
$ uname -a
Linux kaan-virtual-machine 5.15.0-91-generic Ubuntu SMP Tue Nov 14 13:30:08 UTC 2023 x86_64 x86_64_
↳x86_64 GNU/Linux
```

Bu bilgi içerisinde yalnızca çekirdek versiyonu görüntülenmek isteniyorsa "`uname -r`" seçeneği kullanılmalıdır:

```
$ uname -r
6.8.0-51-generic
```

Buradan biz çekirdeğin 6.8.0 sürümünün kullanıldığını anlıyoruz. Burada genel yapılandırılmış bir çekirdek söz konusudur. 51 sayısını dağıtıma özgü yama ya da derleme numarasını belirtir.

Daha önceden de belirttiğimiz gibi `uname` komutu bu bilgileri `proc` dosya sisteminin içerisinde almaktadır. Örneğin:

```
$ cat /proc/version
Linux version 5.15.0-91-generic (buildd@lcy02-amd64-045) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0,
GNU ld (GNU Binutils for Ubuntu) 2.38) #101-Ubuntu SMP Tue Nov 14 13:30:08 UTC 2023
```

2.6 Derleme Araçları

Bilindiği gibi büyük projelerin derlenmesi için *build otomasyon araçları* (*build automation tools*) denilen araçlar kullanılmaktadır. Bunların en yaygın kullanılanı *make* denilen araçtır. Linux çekirdeklerinin derlenmesi de *make* aracı ile yapılmaktadır. Ancak Linux çekirdeklerinin derlenmesinde projeye özgü bazı yapılar ve yöntemler de kullanılmıştır. Buna *KConfig sistemi* ya da *KBuild sistemi* denilmektedir. (*KConfig sistemi* ya da *KBuild sistemi* Linux çekirdeğinin dışında aşağı seviyeli pek çok proje tarafından da kullanılmaktadır.) Biz önce çekirdek derleme işleminin hangi adımlardan geçilerek yapılacağını göreceğiz. Sonra çekirdeğin önemli konfigürasyon parametreleri üzerinde biraz duracağız. Sonra da çekirdekte bazı değişiklikler yapıp sistemin değiştirilmiş çekirdekle açılmasını sağlayacağız.

2.7 Çekirdek Derleme Adımları

Linux'ta çekirdek derlemesi tipik olarak aşağıdaki aşamalardan geçilerek gerçekleştirilmektedir.

2.7.1 Adım 1: Gerekli Araçların Kurulması

Derleme öncesinde derlemenin yapılacağı makinede bazı programların yüklenmiş olması gerekmektedir. Çünkü *KBuild sistemi* yalnızca binary araçları değil bazı başka kütüphaneleri ve araçları da kullanmaktadır. Çekirdeğin derlenmesi için gerekebilecek programları şöyle yükleyebilirsiniz (burada Debian türevi bir dağıtımın kullanıldığını varsayacağız):

```
$ sudo apt update
$ sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev dwarves
```

2.7.2 Adım 2: Kaynak Kodların İndirilmesi ve Açılması

Çekirdek kodları indirilir ve açılır. Biz bu konuyu yukarıda ele almıştık. İndirmeyi şöyle yapabiliriz:

```
$ wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.9.2.tar.xz
$ tar -xvJf linux-6.9.2.tar.xz
```

Bu işlemten sonra `linux-6.9.2` isminde bir dizin oluşturulacaktır.

2.7.3 Adım 3: Çekirdeğin Konfigüre Edilmesi

Çekirdek derlenmeden önce konfigüre edilmelidir. Çekirdeğin konfigüre edilmesi birtakım çekirdek özelliklerinin belirlenmesi anlamına gelmektedir. Bu belirmeler konfigürasyon dosyası yoluyla yapılmaktadır. Konfigürasyon dosyası kaynak kod ağacının kök dizininde (örneğimizde `linux-6.9.2` dizini) `.config` ismiyle bulunmalıdır.

Bu `.config` dosyası default durumda kaynak dosyaların kök dizininde yoktur; derlemeyi yapacak kişi tarafından oluşturulması gerekmektedir. Çekirdek konfigürasyon parametreleri oldukça fazladır ve bunların bazılarının anlamlandırılması özel bilgi gerektirmektedir. Çekirdek konfigürasyon parametreleri çok fazla olduğu için bazı genel amaçları karşılayacak biçimde default değerlerle önceden oluşturulmuş konfigürasyon dosyaları kaynak kod ağacında `arch/<mimari>/configs` dizininin içerisinde bulunmaktadır. Örneğin Intel x86 mimarisi için bu default konfigürasyon dosyaları şöyledir:

```
$ ls arch/x86/configs
hardening.config i386_defconfig tiny.config x86_64_defconfig xen.config
```

64 bit Linux sistemleri için `x86_64_defconfig` dosyasını kullanabiliriz. O halde bu dosyayı kaynak dosyaların bulunduğu kök dizine `.config` ismiyle kopyalayabiliriz (bütün işlemlerde çekirdek kaynak kodlarının kök dizininde bulunduğumuzu varsayıyoruz):

```
$ cp arch/x86/configs/x86_64_defconfig .config
```

Not

Linux kaynak kodlarındaki default konfigürasyon dosyaları minimalist biçimde konfigüre edilmiştir. Bu nedenle pek çok modül bu default konfigürasyon dosyalarında işaretlenmiş değildir. Bu tür denemeleri zaten çalışan çekirdeğin derlenmesinde kullanılan konfigürasyon dosyalarından hareketle yaparsanız daha fazla modül dosyası oluşturulabilir ancak daha az zahmet çekebilirsiniz.

Burada bir noktaya dikkatinizi çekmek istiyoruz. Çekirdek kaynak kodlarındaki `arch/<platform>/configs` dizinindeki `x86_64_defconfig` konfigürasyon dosyası `.config` ismiyle kopyalandıktan sonra ayrıca `“make menuconfig”` ya da `“make oldconfig”` gibi bir işlemle onun satırlarına eklenecek çekirdekte bulunan yeni birtakım özelliklere ilişkin bazı default değerlerin de eklenmesi gerekir.

Linux sistemlerinde genel olarak `/boot` dizini içerisinde `config-<çekirdek_sürümü>` ismiyle mevcut çekirdeğe ilişkin konfigürasyon dosyası bulundurulmaktadır.

`.config` dosyasını oluşturma'nın alternatif yolları şöyle özetlenebilir:

make defconfig

Çalışılan sisteme uygun olan konfigürasyon dosyasını temel alarak mevcut donanım bileşenlerini de gözden geçirerek sistemin açılması için gerekli minimal bir konfigürasyon dosyasını `.config` ismiyle oluşturur. Örneğin 64 bit Intel sisteminde `make defconfig` çalıştırıldığında `arch/x86/configs/x86_64_defconfig` dosyası temel alınır.

make oldconfig

Bu seçeneği kullanmak için kaynak kök dizinde `.config` dosyasının bulunuyor olması gerekir. `KConfig` dosyalarındaki ve kaynak dosya ağacındaki diğer değişiklikleri de göz önüne alarak bu eski `.config` dosyasını yeni versiyona uyumlandırmaktadır. Başka bir deyişle `“make oldconfig”` eski bir konfigürasyon dosyasını yeni çekirdekler için uyumlandırmaktadır.

make <platform>_defconfig

Belli bir platformun default konfigürasyon dosyasını `.config` dosyası olarak kaydeder. Örneğin Intel makinelerinde çalışırken BBB için `make bb.org_defconfig` komutu uygulanabilir.

make modules

Yalnızca modülleri (aygıt sürücü dosyaları) derler; çekirdek derlemesi yapmaz. Yalnızca `make` işlemi zaten aynı zamanda bu işlemi de yapmaktadır.

make uninstall

`“make install”` işlemi ile yapılanları geri alır.

Aşağıdaki `make xxxconfig` komutları seyrek kullanılmaktadır:

make allnoconfig

Tüm seçenekleri *hayır (no)* olarak ayarlar (minimal özellikler).

make allyesconfig

Tüm seçenekleri *evet (yes)* olarak ayarlar (maksimum özellikler).

make allmodconfig

Tüm aygıt sürücülerin çekirdeğin dışında modül biçiminde derleneceğini belirtir.

make localmodconfig

Sistemde o anda yüklü modüllere dayalı bir yapılandırma dosyası oluşturur.

make silentoldconfig

Yeni seçenekler için onları görmezden gelir; yeni özellikler `.config` dosyasına yansıtılmaz.

make dtbs

Kaynak kod ağacında `/arch/platform/boot/dts` dizinindeki aygıt ağacı kaynak dosyalarını derler ve `dtb` dosyalarını elde eder. Gömülü sistemlerde bu işlemin yapılması ve her çekirdek versiyonuyla o versiyonun `dtb` dosyasının kullanılması tavsiye edilir.

Pek çok dağıtım o anda yüklü olan çekirdeğe ilişkin konfigürasyon dosyasını `/boot` dizini içerisinde `config-$(uname -r)` ismiyle bulundurmaktadır. Örneğin kursun yapılmakta olduğu Mint dağıtımında `/boot` dizininin içeriği şöyledir:

```
$ ls /boot
config-6.8.0-51-generic      initrd.img.old
System.map-6.8.0-51-generic efi
grub                        vmlinuz
initrd.img                  vmlinuz-6.8.0-51-generic
initrd.img-6.8.0-51-generic
```


Buradaki `config-6.8.0-51-generic` dosyası çalışmakta olduğumuz çekirdekte kullanılan konfigürasyon dosyasıdır. Bu dosya sistem açılırken herhangi bir biçimde kullanılmamaktadır (yani bu dosyayı silseniz hiçbir sorun oluşmaz). Bu dosya o çekirdeği yeniden derleyecek kişiler için kolaylık sağlamak amacıyla bulundurulmaktadır.

Eğer çalışılan sistemdeki konfigürasyon dosyasını temel alacaksanız bu dosyayı Linux kaynak kodlarının bulunduğu kök dizine `.config` ismiyle kopyalayabilirsiniz:

```
$ cp /boot/config-$(uname -r) .config
```

Eski bir konfigürasyon dosyasını yeni bir çekirdekle kullanmak için ayrıca `“make oldconfig”` işleminin de yapılması gerekmektedir. Bir sonraki adımda göreceğimiz `“make menuconfig”` işlemi aynı zamanda `“make oldconfig”` işlemi de kendi içerisinde barındırmaktadır.

2.7.4 Adım 4: Konfigürasyon Değişikliklerinin Yapılması

Elimizde pek çok değerin set edilmiş olduğu `.config` isimli bir konfigürasyon dosyası vardır. Artık bu konfigürasyon dosyasından hareketle yalnızca istediğimiz özellikleri değiştirebiliriz. Bunun için `“make menuconfig”` komutunu kullanabiliriz:

```
$ make menuconfig
```

Bu komutla birlikte text ekranda konfigürasyon seçenekleri listelenecektir. Tabii buradaki seçenekler `.config` dosyasındaki içerikten hareketle oluşturulmuş durumdadır. Bunların üzerinde değişiklikler yaparak `.config` dosyasını yeniden save edebiliriz. Aslında `“make menuconfig”` işlemi hiç `.config` dosyası oluşturulmadan doğrudan da yapılabilir. Bu durumda hangi sistemde çalışılıyorsa o sisteme özgü default konfigürasyon dosyası temel alınmaktadır. Biz en azından General Setup/Local version - append to kernel release seçeneğine `-custom` gibi bir sonek girmenizi, böylece yeni çekirdeğe `-custom` soneki iliştiirmenizi tavsiye ederiz.

Not

`“make menuconfig”` işlemi zaten `“make oldconfig”` işlemi de kendi içerisinde barındırmaktadır. Dolayısıyla `“make menuconfig”` yapıyorsanız ayrıca `“make oldconfig”` işlemi yapmanıza gerek yoktur.

Peki biz hazır bir `.config` dosyasını kaynak kod ağacının kök dizinine kopyaladıktan sonra hiç `“make menuconfig”` ya da `“make oldconfig”` yazmazsak ne olur? Bu durumda sorun çıkmayabilir. Eğer kaynak kod çekirdeği yeniyse `make` işlemi sırasında `“make oldconfig”` gibi bir işlem de yapılmaktadır. Ancak `.config` dosyasını kök dizine kopyaladıktan sonra `“make menuconfig”` ya da `“make oldconfig”` işlemi yapmanızı salık veriyoruz.

`.config` dosyası elde edildiğinde çekirdek imzalamasını ortadan kaldırmak için dosyayı açıp aşağıdaki özellikleri aşağıda gösterildiği gibi değiştirebilirsiniz (bunların bazıları zaten default durumda aşağıdaki gibi olabilir):

```
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_REVOCATION_KEYS=""
CONFIG_SYSTEM_TRUSTED_KEYRING=n
CONFIG_SECONDARY_TRUSTED_KEYRING=n

CONFIG_MODULE_SIG=n
CONFIG_MODULE_SIG_ALL=n
CONFIG_MODULE_SIG_KEY=""
```

Çekirdek imzalaması konusu daha ileride ele alınacaktır.

Derlenecek çekirdeklere yerel bir versiyon belirteci ve numarası da atanabilmektedir. Bu işlem `“make menuconfig”` menüsünde General Setup/Local version - append custom release seçeneği kullanılarak ya da `.config` dosyasında `CONFIG_LOCALVERSION` satırı düzenlenerek yapılabilir. Örneğin:

```
CONFIG_LOCALVERSION="-custom"
```

Bu işlemle artık çekirdek sürümüne `-custom` sonekini eklemiş olduk.

2.7.5 Adım 5: Derleme

Derleme işlemi için `make` komutu kullanılmaktadır:


```
$ make
```

Eğer derleme işleminin birden fazla CPU ya da çekirdek ile yapılmasını istiyorsanız `-j<cpu_sayısı>` seçeneğini komuta dahil edebilirsiniz. Çalışılan sistemdeki CPU sayısının `nproc` komutuyla elde edildiğini anımsayınız. O halde derleme için `make` komutunu şöyle kullanabiliriz:

```
$ make -j$(nproc)
```

Derleme işlemi bittiğinde ürün olarak çekirdek imajı, çekirdek tarafından yüklenecek olan modül dosyaları ve diğer bazı dosyalar elde edilmiş olur. Derleme işleminden sonra oluşturulan dosyalar ve yerleri şöyledir (buradaki `<çekirdek_sürümü>`, “`uname -r`” ile elde edilecek yazıyı belirtiyor):

- **Sıkıştırılmış Çekirdek İmajı:** `arch/<platform>/boot` dizininde `bzImage` ismiyle oluşturulmaktadır. Denemeyi yaptığımız Intel makinede dosyanın yol ifadesi `arch/x86_64/boot/bzImage` biçimindedir. (Ancak buradaki dosya `x86_64` platformu için `arch/x86/boot/bzImage` dosyasına sembolik link de yapılmış olabilir.)
- **Çekirdeğin Sıkıştırılmamış ELF İmajı:** Kaynak kök dizininde `vmlinux` ismiyle oluşturulmaktadır.
- **Çekirdek Modülleri (Aygıt Sürücü Dosyaları):** `drivers`, `fs` ve `net` dizinlerinin altındaki dizinlerde bulunur. Ancak “`make modules_install`” ile bunların hepsi belirli bir dizine çekilebilir.
- **Çekirdek Sembol Tablosu:** Kaynak kök dizininde `System.map` ismiyle bulunur. Çekirdek sembol tablosundan yalnızca çekirdek debug edilirken faydalanılmaktadır. Bu dosya silinse bile sistemin çalışmasında bir sorun oluşmaz.

Not

Derleme süresini uzatan en önemli etken çekirdek konfigüre edilirken seçilen modül (aygıt sürücü) sayısıdır. Pek çok dağıtım “belki ileride lazım olur” gerekçesiyle konfigürasyon dosyalarında çok sayıda modülü dahil etmektedir. Bu nedenle bir dağıtımın konfigürasyon dosyasını kullandığınız zaman çekirdek derlemesi uzayacaktır. Çekirdek kodlarındaki platforma özgü default konfigürasyon dosyaları daha minimalist biçimde oluşturulmuş durumdadır.

Tabii çekirdek bütünsel olarak bir kez derlendikten sonra çekirdek kodlarında değişiklik yapıp çekirdeği yeniden derlemek istediğimizde artık derleme süresi bütünsel derleme kadar uzun olmayacaktır.

2.7.6 Adım 6: Modüllerin Kurulması

Farklı dizinlerde oluşturulmuş olan aygıt sürücü dosyalarını (modülleri) belli bir dizine kopyalamak için “`make modules_install`” komutu kullanılmaktadır. Bu komut seçeneksiz kullanılırsa default olarak `/lib/modules/<çekirdek_sürümü>` dizinine kopyalama yapar:

```
$ sudo make modules_install
```

Not

Eskiden `make` işlemi çekirdek modüllerinin derlenmesini sağlamıyordu. Çekirdek modüllerinin derlenmesi için “`make modules`” işlemi gerekiyordu. Ancak çekirdeğin 2.6 versiyonundan itibaren `make` işlemi zaten çekirdek modüllerinin de derlenmesini sağlamaktadır.

Ayrıca “`make modules_install`” komutunun modül dosyalarını istediğimiz bir dizine kopyalamasını da sağlayabiliriz. Bunun için `INSTALL_MOD_PATH` çevre değişkeni kullanılmaktadır. Örneğin:

```
$ sudo INSTALL_MOD_PATH=modules make modules_install
```

Burada aygıt sürücü dosyaları `/lib/modules/<çekirdek_sürümü>` dizinine değil, bulunulan yerdeki `modules` dizinine kopyalanacaktır.

2.7.7 Adım 7: Aygıt Ağacı Dosyalarının Derlenmesi

Eğer gömülü sistemler için derleme yapıyorsanız kaynak kod ağacında `arch/<platform>/boot/dts` dizini içerisindeki aygıt ağacı kaynak dosyalarını da derlemelisiniz. Aygıt ağacı kaynak dosyalarını derlemek için “`make dtbs`” komutunu kullanabilirsiniz:

```
$ make dtbs
```

Derlenmiş aygıt ağacı dosyaları `arch/<platform>/boot/dts` dizininde ya da bu dizinin altındaki ilgili vendor dizininde oluşturulacaktır.

Not

Aygıt ağaçları gömülü sistemlerde kullanılmaktadır. Intel tabanlı PC’lerde donanım birimlerinin tespit edilmesi otomatik olarak ACPI protokolü yoluyla yapıldığı için aygıt ağacı dosyaları bu platformda kullanılmamaktadır. Ancak ARM platformunu kullanan gömülü sistemlerde ya da BBB ve Raspberry Pi gibi SBC’lerde aygıt ağaçları kullanılmaktadır.

2.7.8 Adım 8: Kurulum

Bizim çekirdek imajını, geçici kök dosya sistemine ilişkin dosyayı ve aygıt ağacı dosyasını `/boot` dizinine kopyalamamız gerekir. Ancak aslında bu işlem de “*make install*” komutuyla otomatik olarak yapılabilir. “*make install*” komutu bu dosyaları `/boot` dizinine kopyalamanın yanı sıra aynı zamanda GRUB önyükleyici programın konfigürasyon dosyalarında da güncellemeler yapıp yeni çekirdeğin GRUB menüsü içerisinde görünmesini de sağlamaktadır:

```
$ sudo make install
```

Geçici Kök Dosya Sistemi (initrd)

Burada *geçici kök dosya sistemi* (“*initial ramdisk*” ya da “*initrd*”) diye bir terim kullandık. Geçici kök dosya sistemi diskteki asıl kök dosya sistemi mount edilene kadar geçici bir süre sanki diskteki dosya sistemiymiş gibi işlev gören bir RAM disk imajıdır. Tipik Linux sistemlerinde önce geçici kök dosya sistemi mount edilerek temel dosyalara oradan erişilir. Sonra bu geçici dosya sistemi RAM’den atılıp diskteki gerçek kök dosya sistemi mount edilmektedir.

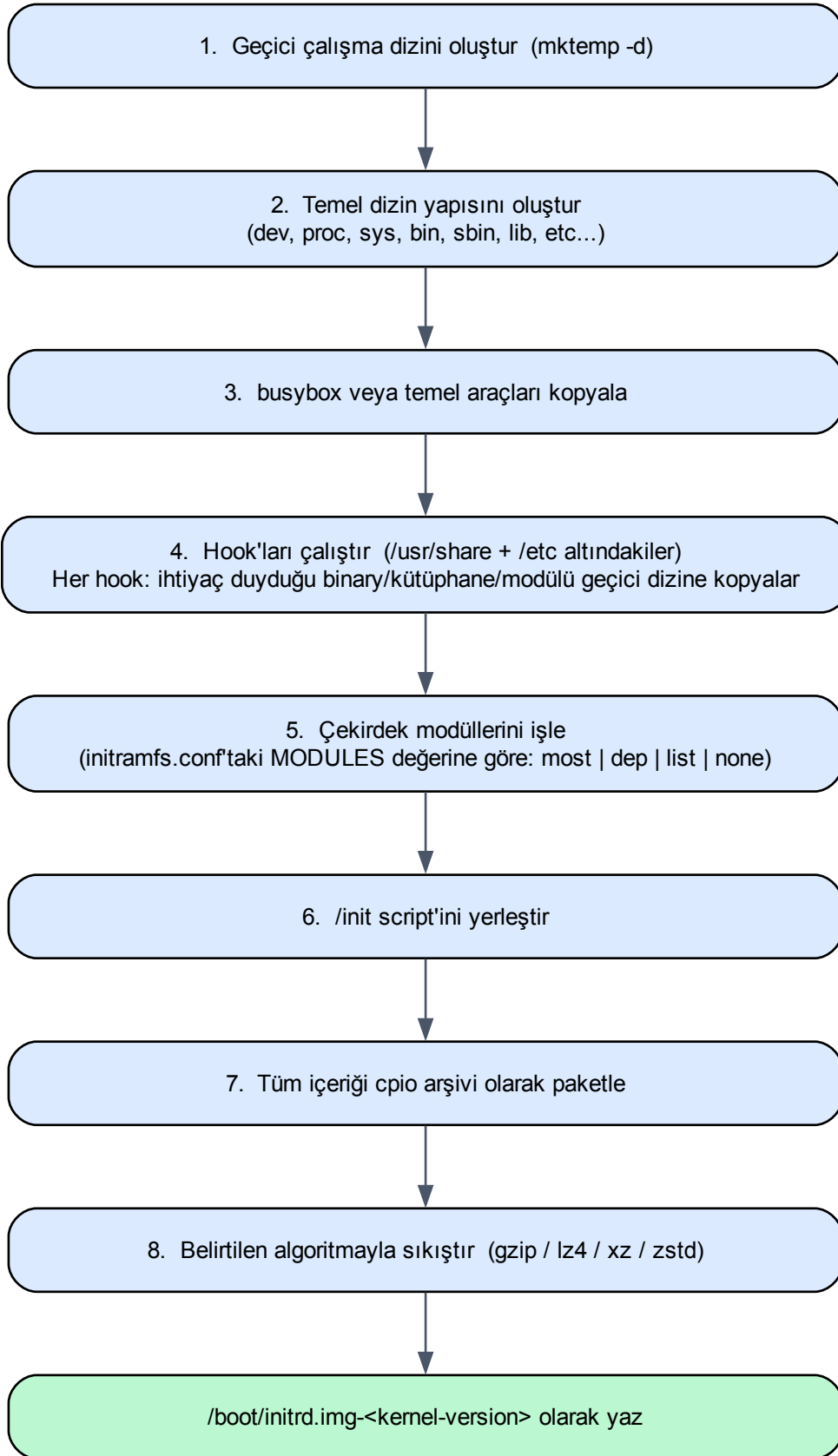
Geçici kök dosya sistemine gereksinimi basit bir örnekle anlayabiliriz. Diyelim ki diske erişmekte kullanılan aygıt sürücüsü çekirdeğin içerisine yerleştirilmemiş olsun, yani dışarıda `lib/modules/$(uname -r)` dizininde bir dosya biçiminde bulunuyor olsun. Şimdi çekirdeğin diske erişebilmesi için bu aygıt sürücüyü ihtiyacı olacaktır. Ancak aygıt sürücüsü de diskte. İşte böyle bir durumda bu aygıt sürücülerini de barındıran bir RAM disk dosya sistemi oluşturulmakta ve önyükleyici tarafından (örneğin GRUB önyükleyicisi) bu dosya da RAM’e yüklenmektedir. Böylece çekirdek artık diske erişebilir hale gelir.

Not

Eğer çekirdeğin gereksinim duyacağı bütün aygıt sürücü dosyaları konfigürasyon aşamasında çekirdeğin içerisine gömülmüşse geçici kök dosya sistemi oluşturmadan da sistem boot edilebilir. Ancak bu sırada çözülmesi gereken problemlerle de karşılaşılabilir. Özellikle masaüstü sistemlerinde geçici kök dosya sistemi olmadan sistemi boot etmek oldukça zahmetlidir.

Geçici kök dosya sistemi aynı zamanda işletim sistemini *güvenli kipte (safe mode)* açmak için ve sistem güncellemelerinde de kullanılmaktadır.

Debian türevi dağıtımlarda geçici kök dosya sistemini oluşturmak için “*update-initramfs*” komutu kullanılmaktadır. Bu komut da *mkinitramfs* komutunu çalıştırmaktadır. Geçici kök dosya sistemi oluşturulurken bu komutlar işlemlerini kabaca aşağıdaki adımlarla gerçekleştirmektedir:



“make install” komutu masaüstü sistemlerde sırasıyla şunları yapmaktadır:

- `arch/<platform>/boot` dizinindeki `bzImage` çekirdek imajı alınarak hedef `/boot` dizinine `vmlinuz-<çekirdek_sürümü>` ismiyle kopyalanır.
- `System.map` dosyası hedef `/boot` dizinine `System.map-<çekirdek_sürümü>` ismiyle kopyalanır.
- `.config` dosyası `/boot` dizinine `config-<çekirdek_sürümü>` ismiyle kopyalanır.
- Geçici kök dosya sistemine ilişkin dosya oluşturulur ve hedef `/boot` dizinine `initrd.img-<çekirdek_sürümü>` ismiyle kopyalanır.
- Eğer GRUB önyükleyicisi kullanılıyorsa GRUB konfigürasyonu güncellenir ve GRUB menüsüne yeni girişler eklenir.

Böylece sistemin otomatik olarak yeni çekirdekle açılması sağlanır.

“*make install*” komutu uygulandığında eğer çekirdeğin versiyon bilgisi aynı ise `/boot` dizinindeki bir önceki kurulumun dosyaları `.old` uzantısıyla saklanmaktadır. Böylece son “*make install*” komutundan önceki kurulumla manuel olarak geri dönebilirsiniz. Örneğin yeniden aynı çekirdek versiyonunu “*make install*” yaptığımızda `/boot` dizininin içeriği şöyle olacaktır:

```
kaan@kaan-Huawei:~/Study/LinuxKernel/linux-6.9.2$ ls -l /boot
total 655480
-rw-r--r-- 1 root root 287375 Haz 7 2024 config-6.8.0-38-generic
-rw-r--r-- 1 root root 287766 Ağu 15 11:57 config-6.9.2-custom
-rw-r--r-- 1 root root 287766 Ağu 15 11:55 config-6.9.2-custom.old
drwx----- 3 root root 4096 Oca 1 1970 efi
drwxr-xr-x 6 root root 4096 Ağu 15 11:57 grub
lrwxrwxrwx 1 root root 23 Ağu 10 11:20 initrd.img -> initrd.img-6.9.2-custom
-rw-r--r-- 1 root root 73052139 Kas 19 2024 initrd.img-6.8.0-38-generic
-rw-r--r-- 1 root root 526888758 Ağu 15 11:57 initrd.img-6.9.2-custom
-rw----- 1 root root 9055262 Haz 7 2024 System.map-6.8.0-38-generic
-rw-r--r-- 1 root root 8365115 Ağu 15 11:57 System.map-6.9.2-custom
-rw-r--r-- 1 root root 8365115 Ağu 15 11:55 System.map-6.9.2-custom.old
lrwxrwxrwx 1 root root 20 Ağu 15 11:57 vmlinuz -> vmlinuz-6.9.2-custom
-rw-r--r-- 1 root root 14944648 Haz 7 2024 vmlinuz-6.8.0-38-generic
-rw-r--r-- 1 root root 14819840 Ağu 15 11:57 vmlinuz-6.9.2-custom
-rw-r--r-- 1 root root 14819840 Ağu 15 11:55 vmlinuz-6.9.2-custom.old
lrwxrwxrwx 1 root root 24 Ağu 15 11:57 vmlinuz.old -> vmlinuz-6.9.2-custom.old
```

Not

“*make install*” komutu masaüstü sistemler için kullanılmaktadır. Gömülü sistemlerde tüm dosyaların toplanıp hedef sisteme aktarılması gerekir. Gömülü sistemlerde “*make install*” tarafından yapılan işlemleri siz manuel bir biçimde yapabilirsiniz. Tabii gömülü sistemlerde GRUB önyükleyicisi yerine ağırlıklı olarak *U-Boot* denilen önyükleyici kullanılmaktadır. Dolayısıyla gömülü sistemlerde sistemin yeni çekirdekle açılmasını sağlamak için *U-Boot* önyükleyicisini de ayarlamamız gerekecektir.

2.8 make modules_install Komutu ile İlgili Ayrıntılar

“*make modules_install*” komutu yalnızca modül dosyalarını hedef dizine kopyalamakla kalmaz; aynı zamanda bazı dosyaları da oluşturup onları da hedef dizine kopyalar. Hedef dizinin default `/lib/modules/<çekirdek_sürümü>` dizini olduğu varsayımıyla komut sırasıyla şunları yapmaktadır:

- Modül dosyalarını `/lib/modules/<çekirdek_sürümü>` dizinine kopyalar.
- `modules.dep` isimli dosyayı oluşturur ve bunu `/lib/modules/<çekirdek_sürümü>` dizinine kopyalar.
- `modules.alias` isimli dosyayı oluşturur ve bunu `/lib/modules/<çekirdek_sürümü>` dizinine kopyalar.
- `modules.order` isimli dosyayı oluşturur ve `/lib/modules/<çekirdek_sürümü>` dizinine kopyalar.
- `modules.builtin` isimli dosyayı `/lib/modules/<çekirdek_sürümü>` dizinine kopyalar.

Aslında burada oluşturulan dosyaların bazıları mutlak anlamda bulundurulmak zorunda değildir. Ancak sistemin öngörüldüğü gibi işlem göstermesi için bu dosyaların ilgili dizinde bulundurulması uygundur.

2.8.1 modules.dep

Bir aygıt sürücü başka aygıt sürücüleri de kullanıyor olabilir. Yani bir aygıt sürücünün çalışabilmesi için başka aygıt sürücülerin de yüklü olması gerekebilmektedir. İşte `modules.dep` dosyası bir aygıt sürücünün yüklenmesi için başka hangi aygıt sürücülerin yüklenmesi gerektiği bilgisini tutmaktadır. `modules.dep` bir text dosyadır. Satırların içeriği şöyledir:

```
<modül_yolu>: <bağımlılık1> <bağımlılık2> ...
```

Dosyanın içeriğine örnek verebiliriz:

```
kernel/arch/x86/crypto/nhpoly1305-sse2.ko.zst: kernel/crypto/nhpoly1305.ko.zst kernel/lib/crypto/
↳libpoly1305.ko.zst
kernel/arch/x86/crypto/nhpoly1305-avx2.ko.zst: kernel/crypto/nhpoly1305.ko.zst kernel/lib/crypto/
↳libpoly1305.ko.zst
kernel/arch/x86/crypto/curve25519-x86_64.ko.zst: kernel/lib/crypto/libcurve25519-generic.ko.zst
```

Eğer bu `modules.dep` dosyası olmazsa `modprobe` komutu çalışmaz ve çekirdek modülleri yüklenirken eksik yükleme yapılabilir. Dolayısıyla sistem düzgün bir biçimde çalışmayabilir. Eğer bu dosya elimizde yoksa ya da bir biçimde silinmişse “`depmod -a`” komutu ile yeniden oluşturulabilir:

```
$ sudo depmod -a
```

Siz yüklü olan başka bir çekirdek sürümü için `modules.dep` dosyasını oluşturmak istiyorsanız çekirdek sürümünü de argüman olarak vermelisiniz:

```
$ sudo depmod -a <çekirdek_sürümü>
```

Not

`depmod` komutunun çalışabilmesi için `/lib/modules/<çekirdek_sürümü>` dizininde modül dosyalarının bulunuyor olması gerekir. Çünkü bu komut bu dizindeki modül dosyalarını tek tek bulup ELF formatının ilgili bölümlerine bakarak modülün hangi modülleri kullandığını tespit ederek `modules.dep` dosyasını oluşturmaktadır.

2.8.2 modules.alias

`modules.alias` dosyası belli bir isim ya da id ile aygıt sürücü dosyasını eşleştiren bir text dosyadır. Bu dosyanın bulunmaması bazı durumlarda sorunlara yol açmayabilir. Ancak örneğin USB porta bir aygıt takıldığında bu aygıtla ilişkin aygıt sürücünün hangisi olduğu bilgisi bu dosyada tutulmaktadır. Bu durumda bu dosyanın olmayışı aygıt sürücünün yüklenememesine neden olabilir. Dosyanın içeriği aşağıdaki formata uygun satırlardan oluşmaktadır:

```
alias <tanımlayıcı> <modül_adı>
```

Örnek bir içerik:

```
alias usb:v05ACp*d*dc*dsc*dp*ic*isc*ip*in* apple_mfi_fastcharge
alias usb:v8086p0B63d*dc*dsc*dp*ic*isc*ip*in* usb_ljca
alias usb:v0681p0010d*dc*dsc*dp*ic*isc*ip*in* idmouse
alias usb:v0681p0005d*dc*dsc*dp*ic*isc*ip*in* idmouse
alias usb:v07C0p1506d*dc*dsc*dp*ic*isc*ip*in* iowarrior
alias usb:v07C0p1505d*dc*dsc*dp*ic*isc*ip*in* iowarrior
```

Bu dosya silinirse yine “`depmod -a`” komutu ile oluşturulabilir.

2.8.3 modules.order

`modules.order` dosyası aygıt sürücü dosyalarının yüklenme sırasını barındıran bir text dosyadır. Bu dosyanın her satırında bir çekirdek aygıt sürücüsünün dosya yol ifadesi bulunur. Daha önce yazılmış aygıt sürücüler daha sonra yazılanlardan daha önce yüklenir. Bu dosyanın olmaması genellikle bir soruna yol açmaz. Ancak modüllerin belli sırada yüklenmemesi bazı durumlarda bozukluklara da neden olabilmektedir. Bu dosyanın silinmesi durumunda yine “`depmod -a`” komutuyla oluşturulabilmektedir.

2.9 Manuel Kurulum

Biz “*make install*” komutu ile yapılan işlemleri manuel olarak da yapabiliriz. Aslında çekirdek imajı ve geçici kök dosya sistemi dosyaları default yerlerin dışında başka yerlerde de bulundurulabilir; önyükleyiciye bu konuda bilgi verilebilir. Ancak yukarıdaki dosyaların hedef sistemde bulundurulduğu default yerler şöyledir:

Dosya	Hedef Dizin
Çekirdek İmajı	/boot
Çekirdek Sembol Tablosu	/boot
Modül Dosyaları	/lib/modules/<çekirdek_sürümü>/kernel
Geçici Kök Dosya Sistemi Dosyası	/boot

İsteğe bağlı olarak aşağıdaki dosyalar da hedef sisteme konuşlandırılabilir:

Dosya	Hedef Dizin
Konfigürasyon Dosyası	/boot
Modüllere İlişkin Bazı Dosyalar	/lib/modules/<çekirdek_sürümü>

Peki yukarıda belirttiğimiz dosyalar hedef sistemdeki ilgili dizinlere hangi isimlerle kopyalanmalıdır? Tipik isimlendirme şöyle olmalıdır (buradaki <çekirdek_sürümü>, “*uname -r*” komutuyla elde edilecek olan yazıdır):

- **Çekirdek İmajı:** /boot/vmlinuz-<çekirdek_sürümü> — örneğin vmlinuz-6.9.2-custom
- **Çekirdek Sembol Tablosu:** /boot/System.map-<çekirdek_sürümü> — örneğin System.map-6.9.2-custom
- **Modüllere İlişkin Dosyalar:** /lib/modules/<çekirdek_sürümü> dizininin içerisine
- **Konfigürasyon Dosyası:** /boot/config-<çekirdek_sürümü> — örneğin config-6.9.2-custom
- **Geçici Kök Dosya Sistemi:** /boot/initrd.img-<çekirdek_sürümü> — örneğin initrd.img-6.9.2-custom; “*update-initramfs*” programıyla oluşturulabilir.

Ayrıca bazı dağıtımlarda /boot dizini içerisindeki vmlinuz dosyası default olan vmlinuz-<çekirdek_sürümü> dosyasına, initrd.img dosyası da /boot/initrd.img-<çekirdek_sürümü> dosyasına sembolik link yapılmış durumda olabilir. Ancak bu sembolik bağlantıları GRUB kullanmamaktadır. Aşağıda Intel sistemindeki 6.8.0 çekirdeğinin yüklü olduğu /boot dizininin default içeriğini görüyorsunuz:

```
$ ls -l /boot
-rw-r--r-- 1 root root 287375 Haz 7 2024 config-6.8.0-38-generic
drwx----- 3 root root 4096 Oca 1 1970 efi
drwxr-xr-x 6 root root 4096 Ağu 15 11:57 grub
lrwxrwxrwx 1 root root 23 Ağu 10 11:20 initrd.img -> initrd.img-6.8.0-38-generic
-rw-r--r-- 1 root root 73052139 Kas 19 2024 initrd.img-6.8.0-38-generic
-rw----- 1 root root 9055262 Haz 7 2024 System.map-6.8.0-38-generic
lrwxrwxrwx 1 root root 20 Ağu 15 11:57 vmlinuz -> vmlinuz-6.8.0-38-generic
-rw-r--r-- 1 root root 14944648 Haz 7 2024 vmlinuz-6.8.0-38-generic
```

Geçici kök dosya sisteminin içerisindeki dosyalar *cpio* denilen bir arşiv formatıyla arşivlenmektedir. *cpio* arşivi tıpkı *tar* arşivinde olduğu gibi yalnızca dosyaların uç uca eklenmesiyle oluşturulmaktadır. İsterseniz geçici kök dosya sistemine ilişkin *initrd-xxx* dosyalarını açabilirsiniz. Ancak bu dosyaların içeriği dağıtımların versiyonlarına göre değişebilmektedir. Yeni dağıtımlarda bu *initrd-xxx* arşiv dosyası birkaç bölümden oluşmaktadır. Eğer biz bu dosyayı *cpio* programıyla açmaya çalışırsak bu program yalnızca ilk bölümü açacaktır. Tüm bölümleri açmak için *unmkinitramfs* programından faydalanabilirsiniz:

```
unmkinitramfs /boot/initrd.img-6.9.2-custom initrd
```

Bu komutla geçici kök dosya sistemine ilişkin arşiv dosyası *initrd* isimli dizinin altına açılacaktır.

2.9.1 Çekirdek Sürüm Yazısının Belirlenmesi

Derleme sonucunda elde ettiğimiz dosyaları manuel isimlendirirken çekirdek sürüm yazısını nasıl bileceğiz? Bunun için “*uname -r*” komutunu kullanamayız. Çünkü bu komut bize o anda çalışmakta olan çekirdeğin sürüm yazısını verir. Sürüm yazısı çekirdek imajının içerisine de yazılmaktadır ve bizim bazı dosyalara verdiğimiz isimlerin çekirdek içerisindeki bu yazıyla uyumlu olması gerekir.

Default olarak `kernel.org` sitesinden indirilen kaynak kodlar derlendiğinde çekirdek sürümü 6.9.2 gibi üç haneli bir sayılardan oluşmaktadır. Yani yazının sonunda `-generic` ya da `-custom` gibi sonekler yoktur. Tabii çekirdeği derlemeden önce `.config` dosyasında `CONFIG_LOCALVERSION` özelliğine bu sürüm numarasından sonra eklenecek bilgiyi girebilirsiniz.

Sürüm yazısını bulmak için sıkıştırılmamış çekirdek dosyası içerisindeki (kaynak kök dizindeki `vmlinux` dosyası) string tablosunda `Linux version` yazısını aramak yeterlidir:

```
$ strings vmlinux | grep "Linux version"
Linux version 6.9.2-custom (kaan@kaan-virtual-machine) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0,
GNU ld (GNU Binutils for Ubuntu) 2.38) # SMP PREEMPT_DYNAMIC
Linux version 6.9.2-custom (kaan@kaan-virtual-machine) (gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0,
GNU ld (GNU Binutils for Ubuntu) 2.38) #2 SMP PREEMPT_DYNAMIC Thu Dec 5 17:55:14 +03 2024
```

Buradan sürüm yazısının 6.9.2-custom olduğu görülmektedir. O halde derleme sonucunda elde ettiğimiz dosyaları manuel biçimde kopyalarken sürüm bilgisi olarak 6.9.2-custom yazısını kullanmamız gerekir. Çekirdek imajının `/boot` dizinine manuel kopyalanması işlemi şöyle yapılabilir (kaynak kök dizinde bulunduğumuzu varsayıyoruz):

```
$ sudo cp arch/x86_64/boot/bzImage /boot/vmlinuz-6.9.2-custom
$ sudo cp .config /boot/config-6.9.2-custom
```

Not

Çekirdek modüllerinin kopyalanması biraz zahmetli bir işlemdir çünkü bunlar derlediğimiz çekirdekte farklı dizinlerde bulunmaktadır. Bu kopyalamanın en etkin yolu “*make modules_install*” komutunu kullanmaktır. Benzer biçimde çekirdek dosyalarının ve gerekli diğer dosyaların uygun yerlere kopyalanması için de en etkin yöntem “*make install*” komutudur.

2.10 GRUB Yapılandırması

Normal olarak “*make install*” yaptığımızda eğer sistemimizde GRUB önyükleyicisi varsa komut GRUB konfigürasyon dosyalarında da güncellemeler yaparak sistemin yeni çekirdekle açılmasını sağlamaktadır. Böylece kullanıcı bir menü yoluyla sistemin kendi istediği çekirdekle açılmasını da sağlayabilmektedir. GRUB menüsü otomatik olarak görüntülenmemektedir. Boot işlemi sırasında ESC tuşuna basılırsa menü görüntülenir. Eğer GRUB menüsünün her zaman görüntülenmesi isteniyorsa `/etc/default/grub` dosyasındaki iki satır aşağıdaki gibi değiştirilmelidir:

```
GRUB_TIMEOUT_STYLE=menu
GRUB_TIMEOUT=5
```

Buradaki `GRUB_TIMEOUT` satırı eğer müdahale yapılmamışsa menünün en fazla 5 saniye görüntüleneceğini belirtmektedir.

Bu işlemten sonra `update-grub` programı da çalıştırılmalıdır:

```
$ sudo update-grub
```

Bu tür denemeler yapılırken GRUB menüleri bozulabilmektedir. Düzeltme işlemleri bazı konfigürasyon dosyalarının düzenlenmesiyle manuel biçimde yapılabilir. Konfigürasyon dosyaları güncellendikten sonra `update-grub` programı mutlaka çalıştırılmalıdır. Ancak eğer GRUB konfigürasyon dosyaları konusunda yeterli bilgiye sahip değilseniz GRUB işlemlerini görsel bir biçimde `grub-customizer` isimli programla da yapabilirsiniz. Bu program Debian depolarında olmadığı için önce aşağıdaki gibi programın bulunduğu yerin `apt` kayıtlarına eklenmesi gerekmektedir:

```
$ sudo add-apt-repository ppa:danielrichter2007/grub-customizer
$ sudo apt-get update
$ sudo apt-get install grub-customizer
```

2.11 Çekirdek Derleme Süreci Özeti

Yukarıdaki adımların özeti şöyledir:

1. Çekirdek derlemesi için gerekli olan araçlar kurulur.
2. Çekirdek kodları indirilir ve açılır.
3. Zaten hazır olan konfigürasyon dosyası `/boot` dizininden alınarak `.config` ismiyle kaynak kök dizine kopyalanır.
4. Konfigürasyon dosyası üzerinde `"make menuconfig"` komutu ile değişiklikler yapılır.
5. Eğer çekirdeğin imzalanması istenmiyorsa `.config` dosyasındaki ilgili satırlar üzerinde değişiklikler yapılır.
6. Çekirdek derlemesi `"make -j$(nproc)"` komutu ile gerçekleştirilir.
7. Modüller ve ilgili dosyalar hedefe `"sudo make modules_install"` komutu ile konuşlandırılır.
8. Çekirdek imajı ve ilgili dosyalar masaüstü sistemlerde `"sudo make install"` komutu ile hedefe konuşlandırılır.

2.12 Çekirdeğin Sistemden Kaldırılması

Yeni çekirdeği derleyip sisteme dahil ettikten sonra onu sistemden tamamen çıkartmak için yapılan işlemlerin tersini yapmak gerekir. Kaldırma işlemi manuel biçimde şöyle yapılabilir:

- `/lib/modules/<çekirdek_sürümü>` dizini tamamen silinir.
- `/boot` dizinindeki çekirdek sürümüne ilişkin dosyalar silinir.
- `/boot` dizininden çekirdek sürümüne ilişkin dosyalar silindikten sonra `update-grub` programı `sudo` ile çalıştırılmalıdır. Bu program `/boot` dizinini inceleyip otomatik olarak ilgili girişleri GRUB menüsünden siler. Yani aslında GRUB konfigürasyon dosyaları üzerinde manuel değişiklik yapmaya gerek yoktur.

Not

`grub-customizer` programı ile de görsel silme yapılabilir. Ancak bu program `/boot` dizini içerisindeki dosyaları ve modül dosyalarını silmez; yalnızca ilgili girişleri GRUB menüsünden çıkartmaktadır.

2.13 Çekirdek Kodları Üzerinde Değişiklik Yapma Yöntemleri

Çekirdeği yeniden derlemenin gerekçelerinden bahsetmiştik. Bunlardan biri de çekirdek kodları üzerinde değişikliklerin yapılmış olmasıydı. Peki çekirdek kodları üzerinde değişiklikler nasıl yapılabilir? Çekirdek kodları üzerinde değişiklikler tipik olarak dört yolla yapılmaktadır:

1. Çekirdek kodlarındaki zaten var olan bir dosya içerisinde bulunan fonksiyon kodlarında değişiklik yapılması.
2. Çekirdek kodlarındaki zaten var olan bir dosya içerisine yeni bir fonksiyon eklenmesi.
3. Çekirdek kodlarındaki bir dizin içerisine yeni bir C kaynak dosyası eklenmesi.
4. Çekirdek kodlarındaki bir dizin içerisine yeni bir dizin ve bu dizinin içerisine de çok sayıda C kaynak dosyalarının eklenmesi.

Eğer biz birinci ve ikinci maddedeki gibi çekirdek kodlarına yeni bir dosya eklemiyorsak çekirdeğin derlenmesini sağlayan `make` dosyalarında bir değişiklik yapmamıza gerek yoktur. Ancak çekirdeğe yeni bir kaynak dosya ya da dizin ekleyeceksek bu eklemeyi yaptığımız dizindeki `make` dosyasında izleyen paragraflarda açıklayacağımız biçimde bazı güncellemelerin yapılması gerekir. Böylece çekirdek yeniden derlendiğinde bu dosyalar da çekirdek imajının içerisine eklenmiş olacaktır. Eğer kaynak kod ağacında bir dizinin altına yeni bir dizin eklemek istiyorsak bu durumda o dizini yine üst dizine ilişkin `make` dosyasında belirtmemiz ve o dizinde ayrı bir `Makefile` dosyası oluşturmamız gerekir.

2.14 KBuild Sistemi ve GNU Make

GNU Make aracı oldukça ayrıntılı özelliklere sahip bir build aracıdır. Bu aracın ayrıntılarını öğrenmek ayrı bir çabayı gerektirmektedir. Make dili aslında oldukça aşağı seviyeli bir build dilidir. Bu nedenle özellikle son yirmi yıldır programcılar doğrudan GNU Make aracını kullanmak yerine daha üst düzey `make` araçlarını kullanmayı tercih etmektedir. Bunlardan en yaygın olanlardan biri `CMake` denilen araçtır. Microsoft ise `MSBuild` isimli kendi tasarladığı build aracını kullanmaktadır.

Make dilinde değişkenler oluşturulabilmektedir. Örneğin:

```
obj-y = a.o
```

Burada `obj-y` isimli değişken `a.o` bilgisini tutmaktadır. Değişkenleri bir çeşit makro gibi düşünebilirsiniz. Bir değişkene ekleme yapmak için Make dilinde `+=` operatörü kullanılmaktadır. Örneğin:

```
obj-y = a.o
obj-y += b.o
obj-y += c.o
```

Burada artık `obj-y` değişkeni `a.o b.o c.o` biçiminde olacaktır.

Linux çekirdeğinde özyinelemeli bir make yöntemi kullanılmaktadır. Her dizinde bir `Makefile` dosyası vardır. Bunun içerisindeki `obj-y` gibi, `obj-m` gibi bazı değişkenler `+=` operatörüyle eklenerek biriktirilmektedir. Bunlar da derleme ve bağlama işlemine sokulmaktadır. Yukarıda da belirttiğimiz gibi Linux'taki bu build sistemine *KBuild* ya da *KConfig* sistemi denilmektedir.

Bizim Linux'ta `Makefile` dosyaları üzerinde gerekli güncellemeleri yapmak için çok fazla bilgiye sahip olmamız gerekmez. Bazı yönergeleri uygun bir biçimde yerine getirirsek hedefimize ulaşabiliriz.

2.15 Makefile Dosyalarının Güncellenmesi

Linux kaynak kod ağacında dizinlerin altında `Makefile` isimli make dosyaları bulunur. Eğer bir dizinin altına yeni bir dosya eklenecekse o dizinin içerisinde bulunan `Makefile` dosyasının içerisine aşağıdaki gibi bir satırın eklenmesi gerekir:

```
obj-y += dosya_ismi.o
```

Buradaki `+=` operatörü `obj-y` isimli hedefe ekleme yapma anlamına gelmektedir. `obj` sözcüğünün yanındaki `-y` harfi ilgili dosyanın çekirdeğin bir parçası biçiminde çekirdek imajının içerisine gömüleceğini belirtmektedir. Make dosyalarının bazı satırlarında `obj-y` yerine `obj-m` de görebilirsiniz. Bu da ilgili dosyanın ayrı bir modül biçiminde derleneceği anlamına gelmektedir. Eklemeler genellikle çekirdek imajının içine yapıldığı için biz de genellikle `obj-y` kullanırız. Eğer bir dosyanın (aygıt sürücüler için bu durum söz konusudur) çekirdek imajının içine gömülmesi yerine ayrı bir çekirdek modülü olarak derlenmesi isteniyorsa bu durumda dosyanın yerleştirildiği dizinin `Makefile` dosyasına aşağıdaki gibi bir eklemenin yapılması gerekir:

```
obj-m += dosya_ismi.o
```

Eğer çekirdek kaynak kodlarına tümünden bir dizinin eklenmesi isteniyorsa bu durumda önce o dizinin oluşturulduğu dizindeki `Makefile` dosyasına aşağıdaki gibi bir satır eklenmelidir (dizin isminden sonra / karakterini unutmayınız):

```
obj-y += dizin_ismi/
```

Tabii bu ekleme bir modül biçiminde de olabilirdi:

```
obj-m += dizin_ismi/
```

Fakat bu ekleme tek başına yetmemektedir. Bu ekleme yapıldıktan sonra ayrıca yaratılan dizinde `Makefile` isimli bir dosyanın oluşturulması ve o dosyanın içerisinde o dizindeki kaynak dosyaların da belirtilmesi gerekmektedir. Örneğin biz `drivers` dizinin altında `mydriver` isimli bir dizin oluşturup onun da içerisine `a.c`, `b.c` ve `c.c` dosyalarını eklemiş olalım. Bu durumda önce `drivers` dizini içerisindeki `Makefile` dosyasına aşağıdaki gibi bir satır ekleriz:

```
obj-y += mydriver/
```

Sonra da `mydriver` dizini içerisinde `Makefile` isimli bir dosya oluşturup bu dosyanın içerisinde de dizin içerisindeki dosyaları belirtiriz. Örneğin:

```
obj-y += a.o
obj-y += b.o
obj-y += c.o
```

2.16 Kconfig Dosyaları

Kaynak kod ağacında Makefile dosyasının dışında build sistemiyle ilgili Kconfig isimli dosyalar da bulunmaktadır. Bu dosyaların içerisinde ilgili dosyaların ya da dizinlerin konfigürasyon dosyasına yansıtılması için gerekli bilgiler bulundurulmaktadır. Örneğin biz eklediğimiz mydriver dizinindeki dosyaların çekirdek kodlarına dahil edilip edilmeyeceğini çekirdeğin derleyenin konfigürasyon aşamasında belirlemesini sağlayabiliriz. Bunun için bu Kconfig dosyasına bir giriş eklememiz gerekir. Böylece bu giriş de “make menuconfig” yapıldığında bir seçenek olarak karşımıza gelecektir. Tabii ekleyeceğimiz dosya ve dizinleri Kconfig dosyasında belirtmek zorunda değiliz.

“make menuconfig” menüsünde seçenekler için birkaç seçme biçimi bulunmaktadır:

Gösterim	Anlamı
[]	Seçilebilir ya da seçilmeyebilir. Seçildiğinde [*] biçiminde gösterilir.
< >	Üç konumlu seçenek: boş (seçilmedi), <M> (modül) ya da <*> (çekirdek içine gömülü).
-*-	Seçilemezlik yapılamaz; ilgili özellik mutlaka çekirdek kodlarında bulunmalıdır.
değer	İlgili özellik için doğrudan bir değer girilmektedir.

Tabii “make menuconfig” menüsünde yapılan her şey aslında .config dosyasına yansıtılmaktadır.

2.17 Konfigürasyon Seçeneklerinin Kaynak Kodlara Yansıtılması

Peki çekirdeğin konfigüre edilmesi aşamasında “make menuconfig” işleminde belirlediğimiz seçenekler kaynak kodlara nasıl yansıtılmaktadır? Örneğin biz “make menuconfig” işleminde bir modülün çekirdek kodlarına dahil edilmesini ilgili girişi * ile seçerek sağlayabilmekteyiz. Benzer biçimde biz konfigürasyon aşamasında bazı çekirdek parametrelerini de değiştirebilmekteyiz. Örneğin timer tick frekansı “make menuconfig” menüsünde bir sayı biçiminde belirlenebilmektedir. Peki buradaki belirlemeler çekirdek kodlarına ve build sistemine nasıl yansıtılmaktadır?

Anımsanacağı gibi “make menuconfig” ve diğer config menülerinde yapılan seçimler .config isimli bir metin dosyaya kaydedilmektedir. Bu .config dosyası *özellik=değer* biçiminde satırlardan oluşmaktadır. Aşağıda dosyanın birkaç satırını görüyorsunuz:

```
CONFIG_CC_HAS_ASM_INLINE=y
CONFIG_CC_HAS_NO_PROFILE_FN_ATTR=y
CONFIG_PAHOLE_VERSION=125
CONFIG_IRQ_WORK=y
CONFIG_BUILDTIME_TABLE_SORT=y
CONFIG_THREAD_INFO_IN_TASK=y
```

Çekirdek derlenirken ilk aşamada bu .config dosyasının içeriği include/generated/autoconf.h dosyasının içerisine #define önişlemci komutları biçiminde aktarılmaktadır. Eğer ilgili konfigürasyon dosyasındaki değer y ya da m ise bu autoconf.h dosyası içerisinde buna ilişkin sembolik sabit 1 olarak görünür. (Başka bir deyişle “make menuconfig” menüsünde [*] ya da <*> ya da <M> seçenekleri için sembolik sabit 1 olur.) Eğer konfigürasyon dosyasında ilgili seçenek gerçekten bir değer belirtiyorsa autoconf.h dosyası içerisinde bu sembolik sabit o değerde olur. Eğer konfigürasyon dosyasında ilgili seçenek n biçiminde seçilmişse (yani “make menuconfig” menüsünde ilgili seçenek [] ya da < > biçiminde seçilmişse) bu durumda ilgili sembolik sabit hiç #define edilmemiş hale gelir.

Özetle aslında .config dosyası içerisindeki satırlardan C dilinde anlamlı olan #define önişlemci komutları oluşturulmaktadır. Aşağıda üretilmiş olan autoconf.h dosyasının birkaç satırını görüyorsunuz:

```
#define CONFIG_IGB_HWMON 1
#define CONFIG_ACPI_HOTPLUG_CPU 1
#define CONFIG_DEV_DAX_KMEM_MODULE 1
#define CONFIG_RIONET_RX_SIZE 128
#define CONFIG_USB_SERIAL_KEYSPAN_PDA_MODULE 1
#define CONFIG_BOOTTIME_TRACING 1
```

Çekirdeğe ilişkin bir C kodu içerisinde “ilgili seçenek seçilmişse” bir kod parçasını derlemeye dahil etmek için #ifdef önişlemci komutundan faydalanabilirsiniz. Örneğin:

```
#ifdef CONFIG_XXX
...
#endif
```

Tabii üretilen bu `autoconf.h` dosyası çekirdek kaynak kodlarındaki çeşitli başlık dosyalarında doğrudan ya da dolaylı bir biçimde eklenmiş durumdadır. Linux kaynak kodları da bu sembolik sabitleri kullanacak biçimde yazılmıştır. Linux'un kaynak kodlarında `autoconf.h` dosyasını bulamazsınız. Çünkü bu dosya `make` işlemi sırasında oluşturulmaktadır.

2.18 Konfigürasyon Seçeneklerinin Makefile Dosyalarına Yansıtılması

Biz yukarıdaki paragrafta `.config` dosyasındaki konfigürasyon parametrelerinin nasıl C'ye sembolik sabitler biçiminde yansıtıldığını gördük. Peki bu konfigürasyon seçenekleri `Makefile` dosyalarına nasıl yansıtılmaktadır? Aşağıda çekirdeğin bir `Makefile` içeriğini görüyorsunuz:

```
obj-$(CONFIG_I8254)      += i8254.o
obj-$(CONFIG_104_QUAD_8) += 104-quad-8.o
obj-$(CONFIG_INTERRUPT_CNT) += interrupt-cnt.o
obj-$(CONFIG_RZ_MTU3_CNT) += rz-mtu3-cnt.o
obj-$(CONFIG_STM32_TIMER_CNT) += stm32-timer-cnt.o
```

Bu satırlar aslında “ilgili konfigürasyon seçenekleri seçilmişse ilgili dosyaların derlemeye dahil edileceğini” belirtmektedir.

İşte `KBuild` sistemi aynı zamanda bu `.config` dosyasından hareketle `make` programı için anlamlı olan değişkenler de oluşturmaktadır. Bu değişkenleri yukarıda açıkladığımız sembolik sabitlerle karıştırmayınız. Bu değişkenler `make` dili için anlamlı olan `make` dilinin değişkenleridir. Örneğin eğer `.config` dosyasında bir seçenek `y` olarak belirtilmişse (başka bir deyişle “`make menuconfig`” menüsünde seçenek `[*]` ya `<*>` biçiminde seçilmişse) bu konfigürasyon seçeneği için `make` dilinde `y` değeri, eğer `m` olarak belirtilmişse de (başka bir deyişle “`make menuconfig`” menüsünde seçenek `<M>` biçiminde seçilmişse) `m` değeri oluşturulmaktadır.

Böylece aslında yukarıdaki `make` satırları ilgili seçenek `y` olarak seçilmişse `obj-y` biçimine, `m` olarak seçilmişse `obj-m` biçimine dönüştürülmektedir. Örneğin:

```
obj-$(CONFIG_I8254) += i8254.o
```

Burada `CONFIG_I8254` seçeneği `y` ise ilgili dosya `obj-y` değişkenine dahil olacaktır. Yani çekirdeğin içerisinde bulunacaktır. Ancak bu `CONFIG_I8254` seçeneği `m` ise ilgili dosya `obj-m` değişkenine dahil olacaktır. Eğer bu seçenek `n` ise (yani hiç seçilmemişse) çekirdek `build` sistemi ya bu `CONFIG_I8254` değişkenini hiç tanımlamamakta ya da bunu `n` olarak tanımlamaktadır. Her iki durumda da artık bu dosya herhangi bir biçimde derlemeye dahil edilmeyecektir.

Çekirdeğe birtakım kodlar ekleyenler eğer eklemeleri `Kconfig` dosyası yoluyla konfigürasyona yansıtmışlarsa bu durumda kendi `Makefile` dosyasına bu eklemeleri yukarıdaki gibi girebilirler. Örneğin biz `mymodule` ile temsil ettiğimiz bir modül dosyası oluşturup bu modül dosyasının çekirdek kodlarına eklenip eklenmeyeceğini konfigürasyonda `Kconfig` dosyası yoluyla belirtebiliriz. Bu durumda `Makefile` içerisindeki girişi aşağıdaki gibi de oluşturabiliriz:

```
obj-$(CONFIG_MYMODULE) += mymodule.o
```

Görüldüğü gibi burada aslında konfigüre eden nasıl seçmişse biz onun seçimini yansıtmış olmaktadır. `.config` dosyasında bir özellik `n` ise bu durumda ilgili `Makefile` satırı şu hale gelecektir:

```
obj-n += mymodule.o
```

`obj-n` biçiminde bir birikim yapılmadığı için zaten bu satır derleme aşamasında dikkate alınmayacaktır. Ancak bazen sistem programcıları `y` durumu için aşağıdaki gibi bir kontrol ile modülü koşullu bir biçimde de derleme sürecine ekleyebilmektedir:

```
ifeq ($(CONFIG_MYSYSCALL), y)
    obj-y += mysyscall.o
endif
```

Burada eğer konfigürasyon yapılırken ilgili seçenek `y` biçiminde (yani `[*]` ya da `<*>` biçiminde) geçilmişse bu durumda biz de ilgili dosyayı derlemeye dahil etmiş olduk.

2.19 Yeni Dizin için Makefile ve Kconfig Dosyaları

Bir C dosyasını ya da dizini çekirdek kodlarına ekledikten sonra onun konfigürasyon sırasında (örneğin “*make menuconfig*” işlemi sırasında) görünebilirliğini sağlamak için Kconfig dosyalarının kullanıldığını belirtmiştik. Yani Kconfig dosyaları yaptığımız değişikliklerin konfigüre edilebilirliğini sağlamak için kullanılmaktadır. Kconfig dosyalarının genel formatı için aşağıdaki bağlantıya başvurabilirsiniz:

<https://docs.kernel.org/kbuild/kconfig-language.html>

Kconfig dosyaları tıpkı Makefile dosyalarında olduğu gibi özyinelemeli biçimde işletilmektedir. Yani biz çekirdek kaynak kod ağacında bir dizin yaratmayıp zaten var olan bir dizinin içerisine bir .c dosyası yerleştiriyorsak Makefile ve Kconfig dosyaları oluşturmamıza gerek yoktur. Gerekli işlemleri zaten dizin içerisinde var olan bir Makefile ve Kconfig dosyaları üzerinde yapabiliriz. Ancak eğer biz bir dizin oluşturup onun içerisine dosyalar yerleştireceksek o dizin için bir tane Makefile ve bir tane de Kconfig dosyası oluşturmamız gerekir.

Bir dizin yaratıp onun içerisine dosyalar yerleştirirken o dizin için Makefile ve Kconfig dosyalarının yazılması gerektiğini belirtmiştik. (Tabii aslında Kconfig dosyasının bulundurulması zorunlu değildir. Ancak eklenen özelliğin konfigüre edilebilirliğinin sağlanması için gerekmektedir.) Bu dosyalar oluşturulduktan sonra dış dizindeki Makefile ve Kconfig dosyalarında aşağıda belirtilen işlemler de yapılmalıdır:

1. Dış dizindeki Makefile dosyasında alt dizinin dikkate alınacağı aşağıdaki gibi bir satırla belirtilmelidir:

```
obj-y += <dizin_ismi>/
```

2. Dış dizinin Kconfig dosyasında iç dizindeki Kconfig dosyasının dikkate alınması aşağıdaki gibi bir satırın eklenmesiyle sağlanmaktadır (buradaki yol ifadesi çekirdek kodlarının kök dizinine göreli olmalıdır):

```
source "drivers/mydriver/Kconfig"
```

Biz Kconfig dosyasına yukarıdaki gibi bir giriş yerleştirdiğimizde artık “*make menuconfig*” gibi konfigürasyon menülerinde eklediğimiz Kconfig elemanı bir menü seçeneği biçiminde karşımıza çıkacaktır.

Örneğin biz bir aygıt sürücümüzün dosyalarını çekirdeğin kaynak kod ağacında drivers dizinin altına mydriver dizinini açarak eklemek isteyelim. Bu durumda şunları yapmamız gerekir:

1. drivers dizini içerisinde mydriver dizinini yaratıp içerisine mydriver.c dosyasını (belki de mydriver.h gibi bir başlık dosyasını da) yerleştirmeliyiz.
2. drivers/mydriver dizininde aşağıdaki gibi bir Kconfig dosyasını oluşturmamız gerekir. Buradaki config MYDRIVER satırı aslında make dilinde CONFIG_MYDRIVER değişkeninin oluşturulmasına yol açmaktadır:

```
config MYDRIVER
    tristate "My Driver"
    default y
    help
        Enable this option to include support for My Device Driver.
        It can either be built as a module or statically linked into the kernel.
```

Eğer ilgili konfigürasyon seçeneği yes/no biçimindeyse (yani [*] biçimindeyse) yukarıdaki config direktifinde tristate yerine bool kullanılmalıdır. Örneğin:

```
config MYDRIVER
    bool "My Driver"
    default y
    help
        Enable this option to include support for My Device Driver.
```

3. Üst dizindeki (drivers dizinindeki) Kconfig dosyasına aşağıdaki satırı yerleştirmeliyiz:

```
source "drivers/mydriver/Kconfig"
```

4. drivers/mydriver dizinindeki Makefile dosyası içerisine aşağıdaki gibi bir satır eklemeliyiz:


```
obj-$(CONFIG_MYDRIVER) += mydriver.o
```

5. Üst dizindeki (yani `drivers` dizinindeki) `Makefile` içerisine aşağıdaki gibi bir satır eklemeliyiz:

```
obj-$(CONFIG_MYDRIVER) += mydriver/
```

Tabii eğer siz bir dizin yaratmayıp zaten çekirdek kaynak kod ağacındaki bir dizine bir dosya yerleştiriyorsanız bu durumda ayrı bir `Kconfig` dosyasını oluşturmanıza gerek yoktur. Bu durumda doğrudan yukarıdaki `config` içeriğini dizinde zaten var olan `Kconfig` dosyasına yerleştirebilirsiniz.

2.20 Örnek: Çekirdeğe Yeni Modül Ekleme

Şimdi çekirdeğe bazı kodlar ekleyip onu yeniden derleyerek bir deneme yapalım. Örneğin çekirdeğe yeni bir çekirdek modülü ekleyelim ve çekirdeğin o modül gömülü olarak başlatılmasını sağlayalım. Aynı zamanda bu çekirdek modülünün “*make menuconfig*” ile seçilebilmesini de sağlayalım. Çekirdek modüllerinin nasıl yazılacağını bilmediğinizi varsayıyoruz. Ancak biz yine de örneğimizde “hiçbir şey yapmayan iskelet bir çekirdek modülü” oluşturacağız. Bu işlem şu adımlardan geçilerek yapılabilir (kaynak kod ağacının kök dizininde bulunduğumuzu varsayıyoruz):

Adım 1: `drivers/mydriver` dizini yaratılır.

Adım 2: İskelet bir çekirdek modülü `mydriver.c` biçiminde `drivers/mydriver` dizininde aşağıdaki gibi oluşturulur:

```
/* mydriver.c */

#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kaan Aslan");
MODULE_DESCRIPTION("General Device Driver");

static int __init mydriver_init(void)
{
    printk(KERN_INFO "Hello World...\n");
    return 0;
}

static void __exit mydriver_exit(void)
{
    printk(KERN_INFO "Goodbye World...\n");
}

module_init(mydriver_init);
module_exit(mydriver_exit);
```

Adım 3: `drivers/mydriver` dizininde `Kconfig` dosyası aşağıdaki gibi oluşturulmalıdır. Burada konfigürasyon makrosunun ismi `CONFIG_MYDRIVER` biçiminde olacaktır:

```
config MYDRIVER
    tristate "My Character Device Driver"
    default y
    help
        Enable this option to include support for My Device Driver.
        It can either be built as a module or statically linked into the kernel.
```

Adım 4: Üst dizinin (yani `drivers` dizininin) `Kconfig` dosyasına aşağıdaki ekleme yapılmalıdır:

```
source "drivers/mydriver/Kconfig"
```

Adım 5: `drivers/mydriver` dizininde aşağıdaki içeriğe sahip bir `Makefile` dosyası oluşturulmalıdır:

```
obj-$(CONFIG_MYDRIVER) += my_driver.o
```

Adım 6: Üst dizindeki (drivers dizinindeki) Makefile dosyasına aşağıdaki satır eklenmelidir:

```
obj-$(CONFIG_MYDRIVER) += mydriver/
```

Artık çekirdeği derleyebiliriz. “*make menuconfig*” menüsünde kendi aygıt sürücümüze ilişkin seçenek de çıkacaktır.

Çekirdek imzalamasını devre dışı bırakmak için konfigürasyon dosyasındaki satırlarda aşağıdaki değişiklikleri yapmayı unutmayınız:

```
CONFIG_SYSTEM_TRUSTED_KEYS=""
CONFIG_SYSTEM_REVOCATION_KEYS=""
CONFIG_SYSTEM_TRUSTED_KEYRING=n
CONFIG_SECONDARY_TRUSTED_KEYRING=n

CONFIG_MODULE_SIG=n
CONFIG_MODULE_SIG_ALL=n
CONFIG_MODULE_SIG_KEY=""
```

Yeni çekirdeğimize -custom ismini de ekleyebiliriz. Daha önceden de belirttiğimiz gibi eğer çekirdeğin eski versiyonundan konfigürasyon dosyası alınacaksa “*make oldconfig*” uygulanıp o versiyondan sonra eklenmiş olan özelliklerin gözden geçirilmesi sağlanmalıdır. Ancak “*make menuconfig*” işlemi zaten “*make oldconfig*” işlemini de içermektedir.

Adım 7: Artık çekirdek derlemesi aşağıdaki gibi yapılabilir:

```
$ make -j$(nproc)
```

Adım 8: Derleme işlemi bittikten sonra önce çekirdek modüllerini “*make modules_install*” ile sonra da çekirdeğin kendisini “*make install*” ile kurabilirsiniz:

```
$ sudo make modules_install
$ sudo make install
```

Anımsanacağı gibi “*make install*” komutu artık sistemin yeni çekirdekle açılmasını sağlayacaktır. “*make install*” aynı zamanda geçici kök dosya sistemini *update-initramfs* komutu ile oluşturup /boot dizinine yerleştirmektedir. Tabii *update-initramfs* programını siz de gerektiğinde kullanabilirsiniz. Programın tipik kullanımı şöyledir:

```
$ sudo update-initramfs -c -k <çekirdek_sürümü>
```

Buradaki *çekirdek_sürümü* yalnızca çekirdeğin numarasını değil ona verdiğiniz ekleri de içermelidir. (Örneğin 6.9.2-custom gibi.) Bu komut geçici kök dosya sistemini o anda çalışmakta olan sistemin konfigürasyonunu da dikkate alarak oluşturur ve /boot dizinine kopyalar. Yukarıda da belirttiğimiz gibi “*make install*” zaten bu programı çalıştırarak geçici kök dosya sistemini /boot dizininde oluşturmaktadır.

Peki çekirdeğin kaynak kodlarına yaptığımız eklemenin gerçekten yapılmış olduğunu nasıl anlayabiliriz? Bizim yazdığımız iskelet aygıt sürücü kodlarında çekirdek aygıt sürücümüz yüklendiğinde *mydriver_init* fonksiyonu çağrılacaktır. Bu fonksiyonun içinde de *printk* isimli çekirdek fonksiyonu ile biz bir log mesajı yazdırdık. Bu log mesajları *kernel ring buffer* denilen bir kuyruk sistemine yazılmaktadır. *dmesg* komutuyla bu kuyruk sistemi görüntülenebilir. Eğer *dmesg* yaptığımızda biz bu mesajları görürsek aygıt sürücümüzün yüklenmiş olduğu sonucunu çıkartabiliriz. Örneğin:

```
$ dmesg | grep "Hello"
Hello World...
```

Çekirdeğin içerisine gömülmüş olan modüller /proc/modules dosyasında görünmezler, dolayısıyla da *lsmod* komutu ile de bunları göremeyiz. Bunlar için /sys/module dizininde de bir giriş oluşturulmamaktadır. *modinfo* komutu ise çekirdeğe ilişkin bazı dosyalara da baktığı için bize bu konuda bilgi verebilmektedir.

2.21 Yeniden Derleme Sonrası Güncellemeler

Peki çekirdek kodlarında küçük değişiklikler yaptıktan sonra yeniden “*make modules_install*” ve “*make install*” işlemlerine gerek var mı? Aslında küçük değişiklikler için bu işlemler yapılmazsa genellikle bir sorun ortaya çıkmaz. Yeni oluşturulan çekirdek imajı doğrudan

eskinin üzerine kopyalanabilir. Ancak değişikliğin yerine ve kapsamına göre çekirdeğin sembol tabloları değişebileceği için genel olarak her derlemeden sonra “*make install*” yapabilirsiniz.

`drivers` dizininde `obj-m` biçiminde değişiklikler yapılmışsa “*make modules_install*” yapılmalıdır. Yukarıdaki örnekte biz `drivers` dizininin içerisine `obj-y` ile eklemeler yaptık; bu durumda aslında “*make modules_install*” yapmaya gerek yoktur. Ancak aygıt sürücüler `obj-m` biçiminde ekleniyorsa “*make modules_install*” komutu uygulanmalıdır. Çekirdeğin modüllerle ilgili olmayan kısımlarında yapılan değişiklikler için “*make modules_install*” yapılmasına gerek olmadığını bir kez daha belirtmek istiyoruz. “*make modules_install*” işleminden önce eski `/lib/modules/<çekirdek_sürümü>` dizinini “*rm -r*” komutu ile silmek daha güvenli bir yaklaşımdır.

2.22 Çekirdek ve Modül İmzalama

Biz yukarıdaki çekirdek derlemesi sürecinde imzalama (signing) işlemlerini devre dışı bırakmıştık. Çekirdek kodları ve özellikle de aygıt sürücüler belli imzalara sahip olacak biçimde derlenebilmektedir. Böylece onlar üzerinde birtakım istenmeyen değişikliklerin yapılmış olduğu değiştirilmiş çekirdeklerin ya da aygıt sürücülerin yüklenmesi engellenmiş olur. Yukarıda da gördüğünüz gibi çekirdek kodları ve aygıt sürücülerde bu imzalama işlemi devre dışı da bırakılabilmektedir. Ancak imzalama süreci sistem güvenliğini artırmaktadır. Bu tür imzalama işlemleri yalnızca Linux sistemlerinde değil diğer UNIX türevi sistemlerde, Windows ve macOS sistemlerinde de bulunmaktadır.

Çekirdeğin imza kontrolü temel olarak UEFI BIOS (eğer *secure boot* seçeneği aktif ise) ve önyükleyiciler (örneğin GRUB gibi, U-Boot gibi önyükleyiciler) tarafından yapılmaktadır. Ancak Linux çekirdeği de aygıt sürücüler ve modüller yüklenirken imza kontrolü uygulayabilmektedir. Biz burada önce modül imzalama işleminin ve sonra da çekirdek imzalama işleminin nasıl yapılacağı üzerinde duracağız.

İmzalama işlemi tipik olarak şu adımlardan geçilerek yapılmaktadır:

Adım 1: İmzalama işlemi için öncelikle *openssl* kütüphanesinin yüklenmiş olması gerekir. Yükleme işlemi aşağıdaki gibi yapılabilir:

```
$ sudo apt-get install openssl
```

Daha sonra *openssl* programı ile aşağıdaki gibi bir anahtar çifti ve sertifika dosyası üretilir. Buradaki `.key` dosyası özel anahtarı (private key), `.crt` dosyası ise sertifika dosyasını belirtmektedir. Bu dosyaların uzantıları `.key`, `.crt`, `.pem` olsa da içeriği PEM (Privacy Enhanced Mail) formatındadır. Dolayısıyla aslında burada verdiğiniz dosyaların uzantısı herhangi bir biçimde olabilir:

```
$ openssl req -new -x509 -newkey rsa:2048 -sha256 \
-keyout signing_key.key -out certs/signing_key.crt \
-nodes -days 36500 -subj "/CN=Local Kernel Module Key/"
```

Bu iki dosyanın kaynak kod ağacına aşağıdaki gibi kopyalanması gerekir:

```
$ cp signing_key.key certs/
$ cp signing_key.crt certs/
```

Daha sonra konfigürasyon dosyasında imzalama için aşağıdaki değişiklikler yapılmalıdır:

```
CONFIG_MODULE_SIG=y
CONFIG_MODULE_SIG_ALL=y
CONFIG_MODULE_SIG_SHA256=y
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_SYSTEM_TRUSTED_KEYS="certs/signing_key.crt"
```

Aslında `.key` ve `.crt` dosyaları tek bir dosyada da birleştirilebilir:

```
$ cat certs/signing_key.key certs/signing_key.crt > certs/signing_key.pem
```

Tabii artık `.config` dosyasındaki isimleri de şöyle değiştirmeliyiz:

```
CONFIG_MODULE_SIG=y
CONFIG_MODULE_SIG_ALL=y
CONFIG_MODULE_SIG_SHA256=y
CONFIG_SYSTEM_TRUSTED_KEYRING=y
CONFIG_MODULE_SIG_KEY="certs/signing_key.pem"
CONFIG_SYSTEM_TRUSTED_KEYS="certs/signing_key.pem"
```

Biz bu işlemlerle aygıt sürücülerini imzalamış olduk. Ancak eğer `.config` dosyasında `CONFIG_MODULE_SIG_FORCE=n` ise (default durumda genellikle böyledir) bu durum çekirdek log amaçlı bir uyarı oluştursa da imzalanmamış modülleri yine de yükler. Eğer imzalanmamış modülleri yüklemek istemiyorsanız `CONFIG_MODULE_SIG_FORCE=y` yapmalısınız. (Bu işlem “*make menuconfig*” menüsünde *Enable loadable module support / Module signature verification / Require modules to be validly signed* seçeneğinden de yapılabilir.) Bu durumda çekirdek kendi oluşturduğumuz imzayla imzalanmamış olan modülleri artık yüklemeyecektir.

Eğer biz başkalarının yazdığı bir aygıt sürücüyü yüklerken çekirdeğin uyarı vermesini ya da yüklemeyi reddetmesini istemiyorsak o aygıt sürücüyü de kendi ürettiğimiz anahtarla (ya da dağıtımın public anahtarıyla) imzalamalıyız. Bu işlem çekirdek kodlarındaki `scripts` dizini içerisinde bulunan `sign-file` betiği ile yapılmaktadır. Bu betiğin tipik kullanımı şöyledir:

```
$ scripts/sign-file <hash_alg> <private_key.pem> <public_cert.pem> <module.ko>
```

Örneğin:

```
$ scripts/sign-file sha256 signing_key.pem signing_key.pem mydriver.ko
```

Peki Ubuntu, Mint gibi dağıtımlar çekirdek imzalaması uygulamakta mıdır? Evet, genel olarak dağıtımlar kendi özel anahtarlarıyla (private keys) çekirdeği ve aygıt sürücülerini imzalamaktadır. Ancak aygıt sürücülerin yüklenmesinde imza kontrolünü zorunlu hale getirmemektedir. İmzalama için kullanılacak public anahtarlar `/proc/keys` dosyasında belirtilmektedir.

2.23 Çekirdek İmjasının İmzalanması

Biz yukarıdaki işlemleri yaptığımızda yalnızca aygıt sürücülerini imzalamış oluruz. Çekirdeğin kendisinin imzalanması ayrıca yapılmalıdır. Yukarıda da belirttiğimiz gibi UEFI BIOS’lar ve GRUB gibi önyükleyiciler çekirdek imajını yüklemekten önce ayarlar uygun biçime getirildiyse çekirdek imzasına bakmaktadır. Eğer çekirdek imzası yanlışsa (çekirdek dışarıdan kasti ya da yanlışlıkla bozulmuş olabilir) çekirdeği hiç yüklememektedir.

Çekirdeğin imzalanması için önce anahtar ve sertifikasyon dosyaları aşağıdaki gibi oluşturulur:

```
$ openssl req -new -x509 -newkey rsa:2048 -sha256 \
-keyout certs/sb-signing.key \
-out certs/sb-signing.crt \
-nodes -days 36500 \
-subj "/CN=Secure Boot Signing Key/"
```

Sonra da `sbsign` programı ile imzalama aşağıdaki gibi yapılır:

```
$ sbsign --key db.key --cert signing_key.pem --output bzImage.signed arch/x86/boot/bzImage
```

Eğer bu işlemten sonra “*make install*” yaparsanız imzalanmış çekirdeği eski ismiyle bulundurmalısınız (`mv` komutu hedef dosya varsa onu ezerek işlemini yapmaktadır):

```
$ mv arch/x86/boot/bzImage.signed arch/x86/boot/bzImage
```

Genellikle bu biçimde bir çekirdek imzalaması seyrek olarak yapılmaktadır. Önceki paragrafta biz aygıt sürücü dosyalarının imzalandığını belirtmiştik. Aynı makinede aygıt sürücüyü derlerken (build ederken) oluşturulan imza bilgisi de kullanılmaktadır. Yani biz aynı makinede bir aygıt sürücü derlediğimizde aygıt sürücümüz de zaten imzalanmış olacaktır.

2.24 Kök Dosya Sisteminin Oluşturulması

Bir Linux sisteminin düzgün bir biçimde açılması için belli dizinlerin kök dosya sisteminde bulunuyor olması gerekir. Biz bir dağıtımı kurduğumuzda zaten bu kök dosya sistemi de oluşturulmaktadır. Peki sıfırdan dağıtımı tamamen kurmadan kök dosya sistemini nasıl oluşturulabiliriz? Bu işlem tamamen manuel biçimde yapılabilir. Yani uygulamacı kök dizin içerisindeki gerekli dizinleri elle yaratır. Sonra gerekli programları kaynak kodlarından hareketle hedef makine için derler ve onları konuşturur. Sonra yine gerekli birtakım konfigürasyon dosyalarını elle oluşturur. Ancak bu manuel yöntem oldukça zahmetlidir.

Bunun yerine bu işlemi pratik bir biçimde yapan araçlar geliştirilmiştir. Örneğin gömülü sistemlerde *BusyBox* denilen araç bu amaçla sıkça kullanılmaktadır. Kullanımı da oldukça kolaydır. Gömülü sistemler için *Buildroot* ve *Yocto* gibi projeler daha genel amaçlar için gerçekleştirilmiştir ancak bunlarla kök dosya sistemi de oluşturulabilmektedir. Bazı dağıtımların bu işi yapan özel yardımcı programları da vardır. Örneğin *debootstrap* programı Debian tabanlı kök dosya sistemini İnternet’ten indirerek yerel makinede oluşturabilmektedir. Ancak bu araçların bazıları esnek değildir. Özellikle gömülü sistemlerde düşük bir sistem kaynağının olduğu dikkate alındığında bu araçların bazıları minimalist bir kurulum sağlayamamaktadır.

2.24.1 debootstrap ile Kök Dosya Sistemi Oluşturma

debootstrap programı default olarak sisteminizde yüklü değildir. Bunu aşağıdaki gibi kurabilirsiniz:

```
$ sudo apt-get install debootstrap
```

debootstrap programının pek çok komut satırı argümanı vardır. Biz burada en önemli birkaç argüman üzerinde duracağız:

- `--arch`: Hedef CPU mimarisini belirtmektedir. Bu argüman girilmezse o andaki platform temel alınır. 64 bit Intel platformu için `amd64`, BBB gibi 32 bit ARM platformu için `armhf`, 64 bit ARM platformu için `arm64` girilmelidir.
- İlk seçeneksiz argüman Debian sisteminin varyantını belirtmektedir (örneğin `bullseye`, `buster`).
- İkinci komut satırı argümanı hedef kök dosya sisteminin oluşturulacağı dizini belirtmektedir.
- Üçüncü komut satırı argümanı ise paketlerin indirileceği depoyu (repository) belirtmektedir.

Örneğin:

```
$ sudo debootstrap --arch=amd64 --include=systemd bullseye myrootfs http://deb.debian.org/debian/
```

`--arch` seçeneği girilmemişse programın çalıştırıldığı makine için kök dosya sistemi indirilip kurulmaktadır. Default durumda *debootstrap* pek çok paketi kök dosya sistemine dahil ettiği için paketlerin indirilmesi ve kök dosya sisteminin oluşturulması biraz zaman alacaktır.

Uygulamacı isterse `--include` ve `--exclude` komut satırı seçenekleriyle birtakım paketleri dahil edebilir ya da dışlayabilir. Örneğin biz *systemd* dışında *sudo* ve *gcc* paketlerini de aşağıdaki gibi kurulumla dahil edebiliriz:

```
$ sudo debootstrap --arch=amd64 --include=systemd,sudo,gcc bullseye myrootfs http://deb.debian.org/debian/
```

debootstrap programı ile eğer host makineyle aynı platform için indirme işlemi yapılıyorsa *debootstrap* önce İnternet'ten gerekli paketleri indirip yerel makinede bir dizin içerisinde kök dosya sistemini oluşturur. Eğer host makineden farklı bir sistem için indirme yapılıyorsa (örneğin host sistem Intel tabanlı bir makineyse ve ARM tabanlı bir Debian kök dosya sistemi oluşturulmak isteniyorsa) *debootstrap* yüklemesini yapmadan önce aşağıdaki gibi *qemu* emülatör paketinin statik versiyonu ve *binfmt* destek paketi kurulmalıdır:

```
$ sudo apt install qemu-user-static binfmt-support
```

Bu işlemden sonra *debootstrap* programı yukarıda belirttiğimiz biçimde çalıştırılabilir:

```
$ sudo debootstrap --include=systemd --arch armhf buster myrootfs http://deb.debian.org/debian/
```

Bu komut hem birinci aşama hem de ikinci aşama işlemleri yapıp bitirecektir. Artık istenildiği zaman *chroot* işlemi de yapılabilir:

```
$ sudo chroot myrootfs
```

2.24.2 Geçici Kök Dosya Sisteminin Oluşturulması

debootstrap programı ile biz Debian kök dosya sistemi için geçici kök dosya sistemi de oluşturabiliriz. Bunun en pratik yolu kök dosya sistemini kurduktan sonra *chroot* yapıp *update-initramfs* programı ile geçici kök dosya sistemini oluşturmaktır. Ancak bunun için `/boot` ve `/lib/modules` dizinlerinin uygun biçimde oluşturulmuş olması gerekir. *update-initramfs* programı bu dizinlerdeki içerikten faydalanmaktadır. *update-initramfs* programı *initramfs-tools* isimli pakettir. *chroot* yaptıktan sonra öncelikle bu paketi aşağıdaki gibi kurmalısınız:

```
$ sudo apt-get install initramfs-tools
```

Bundan sonra geçici kök dosya sistemini aşağıdaki gibi oluşturabilirsiniz (Debian kök dosya sisteminin kökünde olduğumuzu varsayıyoruz):

```
$ update-initramfs -c -k 6.9.2-custom -b .
```

Burada `6.9.2-custom` çekirdeğin sürüm ismidir. Geçici kök dosya sistemi `initrd.img-6.9.2-custom` ismiyle bulunan dizinde oluşturulacaktır. `-b .` seçeneği oluşturulacak dosyanın dizinini belirtmektedir.

Not

Bu tür konuların ayrıntılarına bu kursta girmeyeceğiz. Bu konular daha çok *Gömülü Linux Sistemleri - Geliştirme ve Uygulama* kursunun konularını oluşturmaktadır.

2.25 Çekirdek Başlık Dosyalarının Kurulumu

Aygıt sürücü geliştirirken çekirdeğin kaynak kodlarına gereksinim duyulmaz. Ancak çekirdeğin başlık dosyalarının geliştirmenin yapıldığı bilgisayarda yüklü olması gerekir. Çekirdek kodlarının kendisini değil de yalnızca başlık dosyalarını indirmek için aşağıdaki komut kullanılabilir:

```
$ sudo apt install linux-headers-$(uname -r)
```

Buradaki `$(uname -r)` çalışmakta olan makinedeki çekirdek sürümünü belirtmektedir. Tabii biz istediğimiz çekirdek sürümünün başlık dosyalarını da indirebiliriz. İndirilen dosyalar `/usr/src` dizinin altına `$(uname -r)` isimli dizin içerisine yerleştirilmektedir.

Bağlı Listelerin ve Hash Tablolarının Çekirdek Gerçekleştirimleri

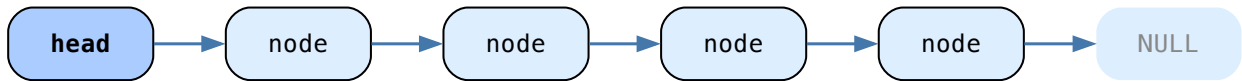
Bu bölümde Linux çekirdeğindeki bağlı listelerin ve hash tablolarının gerçekleştirimleri üzerinde duracağız.

3.1 Bağlı Listeler

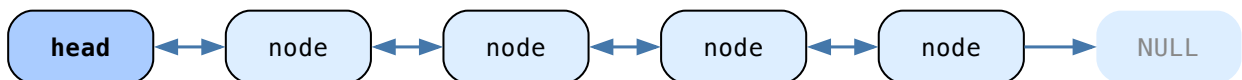
Çekirdeğin en önemli veri yapılarından biri bağlı listelerdir. Genel olarak çekirdekteki neredeyse tüm bağlı listeler *çift bağlı (doubly linked)* biçimde kullanılmaktadır. Önce bağlı listelerin Linux çekirdeğindeki gerçekleştirimleri üzerinde duracağız.

Aralarında öncelik-sonralık ilişkisi olan veri yapılarına *liste (list)* tarzı veri yapıları denilmektedir. Örneğin bu tanıma göre diziler de “liste tarzı” veri yapılarıdır. Liste tarzı veri yapılarının en yaygın kullanılanlarından biri *bağlı liste (linked list)* denilen veri yapısıdır. Önceki elemanın sonraki elemanın yerini gösterdiği dolayısıyla elemanların ardışıl olma zorunluluğunun ortadan kaldırıldığı listelere “bağlı liste” denilmektedir. Dizi elemanlarının bellekte fiziksel olarak ardışıl biçimde bulunduğunu anımsayınız. Bağlı listeler adeta “elemanları bellekte ardışıl olmak zorunda olmayan diziler” gibidir.

Bağlı listelerin her elemanına *düğüm (node)* denilmektedir. Bağlı listelerde her düğüm sonraki düğümün yerini tuttuğuna göre ilk elemanın yeri biliniyorsa liste elemanlarının hepsine erişilebilmektedir:



Her düğümün yalnızca sonraki düğümün değil aynı zamanda önceki düğümün yerini de tuttuğu bağlı listelere *çift bağlı listeler (doubly linked lists)* denilmektedir. Çift bağlı listelerde belli bir düğümün adresini biliyorsak yalnızca ileriye doğru değil, geriye doğru da gidebiliriz:



Çift bağlı listelere ilişkin bir düğümün bellekte daha fazla yer kaplayacağına dikkat ediniz. Çift bağlı listelerin tek bağlı listelere göre en önemli özelliği “adresli bilinen bir düğümün” silinebilmesidir. Tek bağlı listelerde bu durum mümkün değildir. Uygulamalarda ve özellikle çekirdek kodlarında buna çok sık gereksinim duyulmaktadır.

Eğer bir bağlı listede son eleman da ilk elemanı gösteriyorsa bu tür bağlı listelere *döngüsel bağlı listeler (circular linked lists)* denilmektedir.

3.2 Bağlı Listelere Neden Gereksinim Duyulur?

Peki bağlı listelere neden gereksinim duyulmaktadır? Diziler varken bağlı listelere gerek var mıdır? Dizilerle bağlı listeler arasındaki farklılıkları, benzerlikleri ve bağlı listelere neden gereksinim duyulduğunu birkaç maddede açıklayabiliriz:

- Bellek parçalanması sorunu:** Diziler ardışıl alana gereksinim duymaktadır. Ancak belleğin bölündüğü (fragmente olduğu) durumlarda bellekte yeteri kadar küçük boş alanlar olduğu halde bunlar ardışıl olmadığı için dizi tahsisatı mümkün olamamaktadır. Bu tür durumlarda ardışıklık gereksinimi olmayan bağlı listeler kullanılabilir. Özellikle heap gibi bir alanda çok sayıda dinamik dizi bellek kullanımı bakımından verimsizliğe yol açabilmektedir. Bu dinamik diziler zamanla büyüdükçe birbirini engeller hale gelebilmektedir. İşte uzunluğu baştan belli olmayan çok sayıda dizinin oluşturulacağı durumlarda dinamik diziler yerine bağlı listeler bellek kullanımı bakımından toplamda daha iyi performans gösterebilmektedir. Dinamik dizilerde büyütme sırasında bloklar yer değiştirebileceği için bu işlem yavaştır.
- Araya ekleme ve silme verimliliği:** Dizilerde araya eleman ekleme (insert etme) ve aradaki bir elemanı silme dizinin kaydırılmasına yol açacağından yavaş bir işlemdir. Teknik olarak dizilerde eleman insert etme ve eleman silme $O(N)$ karmaşıklıkta bir işlemdir. Halbuki bağlı listelerde eğer düğümün yeri biliniyorsa bu işlem $O(1)$ karmaşıklıkta (yani döngü olmadan tekil işlemlerle) yapılabilmektedir. O halde araya eleman eklemenin ve aradan eleman silmenin çok yapıldığı sistemlerde diziler yerine bağlı listeler tercih edilebilmektedir. Çekirdek veri yapılarında araya eleman ekleme ve aradan eleman silme gibi işlemler çok yoğun yapılmaktadır.
- İndeksle erişim yavaşlığı:** Bağlı listelerde belli bir indeksteki elemana erişmek $O(N)$ karmaşıklıkta bir işlemdir. Halbuki dizilerde elemana erişim $O(1)$ karmaşıklıkta yani çok hızlıdır. O halde belli bir indeks değeri ile elemana erişimin yoğun yapıldığı durumlarda bağlı listeler yerine diziler tercih edilmelidir.
- Ek bellek maliyeti:** Bağlı listeler toplamda bellekte daha fazla yer kaplama eğilimindedir. Çünkü bağlı listenin her düğümü sonraki (ve duruma göre önceki) elemanın yerini de tutmaktadır.

O halde bağlı listeler tipik olarak şu durumlarda dizilere tercih edilmelidir:

- Eleman insert etmenin ve eleman silmenin sık yapıldığı durumlarda.
- Uzunluğu baştan belli olmayan çok sayıda dizinin kullanıldığı durumlarda.
- İndeks yoluyla erişimin az yapıldığı durumlarda.
- Toplam bellek miktarının yeteri kadar fazla olduğu sistemlerde.

İşte tüm yukarıdaki nedenlerden dolayı bağlı listeler çekirdek için en önemli veri yapılarından biridir.

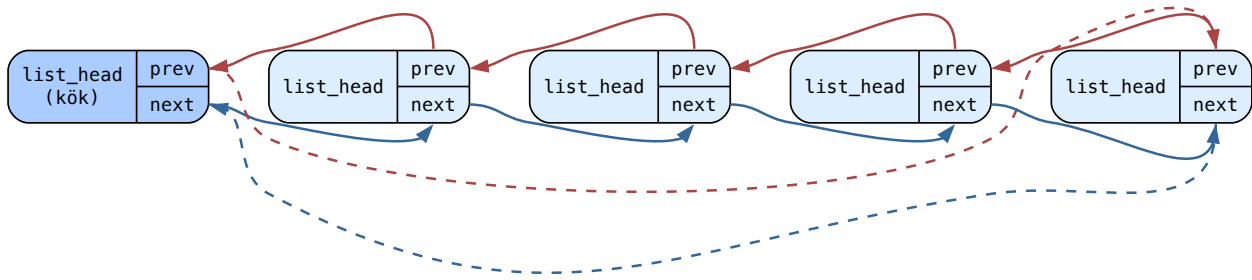
3.3 Linux Çekirdeğindeki Bağlı Liste Gerçekleştirimi

Linux çekirdeğinde bağlı liste gerçekleştirimi `include/linux/list.h` dosyasında yapılmıştır. Bu dosya içerisindeki fonksiyonlar `static inline` biçiminde tanımlanmıştır. Çekirdek derlemesi sırasında derleyicinin optimizasyon seçeneği ayarlandığı için derleyici buradaki inline fonksiyonları sanki bir makro gibi koda açmaktadır.

Linux çekirdeğindeki bağlı listelerde düğümler `list_head` isimli bir yapıyla temsil edilmektedir:

```
struct list_head {
    struct list_head *next;
    struct list_head *prev;
};
```

Aslında belli yapılar değil, bu `list_head` yapıları bağlı liste içerisinde birbirine bağlanmaktadır:



Tabii eğer bu `list_head` yapıları başka bir yapının (buna asıl yapı diyelim) içerisindeyse bu durumda aslında bir `list_head` yapısının adresi asıl yapının bir elemanının adresi haline gelmektedir. Biz C’de bir yapının bir elemanının adresini biliyorsak kolaylıkla o yapının başlangıç adresini elde edebiliriz. Çünkü yapı elemanları ardışıldır ve standart `offsetof` makrosuyla belli bir yapı elemanının yapının başlangıcından itibaren hangi offset’te bulunduğu bilgisi elde edilebilmektedir. `offsetof` makrosu `<stddef.h>` içerisinde aşağıdakine benzer biçimde tanımlanmıştır:

```
#define offsetof(type, member) (((size_t)&(((type *)0)->member)))
```

`offsetof` makrosunun birinci parametresi yapının tür ismini, ikinci parametresi ilgili yapı elemanının ismini almaktadır. Tabii `offsetof` makrosu yalnızca `size_t` türünden bir byte offset’i vermektedir. Elemanın adresinin makroyla elde edilen bu değerden çıkartılarak tür dönüştürmesinin yapılması gerekir. İşte bunun için Linux çekirdeğinde `container_of` isimli bir makro bulundurulmuştur. Bu makronun basit yazımı şöyle yapılabilir:

```
#define container_of(ptr, type, member) ((type *)((char *)ptr - myoffsetof(type, member)))
```

`container_of` makrosunun birinci parametresi yapı elemanının adresini, ikinci parametresi yapının tür ismini ve üçüncü parametresi de ilgili yapı elemanının ismini almaktadır. Bu makro ikinci parametresine girilmiş olan yapı türünden bir adres vermektedir.

`container_of` makrosunu `list_entry` ismiyle de kullanabilirsiniz:

```
#define list_entry(ptr, type, member) container_of(ptr, type, member)
```

3.3.1 Bağlı Liste Başlatma

Linux çekirdeğindeki bağlı listeler kullanılırken önce bir başlangıç düğümünün oluşturulması gerekir. Başlangıç düğümünde `next` ve `prev` göstericilerinin başlangıç düğümünün kendisini göstermesi gerekir. Örneğin:

```
struct list_head head = {&head, &head};
```

Bu ilkdeğer verme C’de geçerlidir. Çünkü C’de bir değişken faaliyet alanına = atomu ile ilkdeğer verme kısmından önce sokulmaktadır. `list.h` dosyasında bu işlemi yapan aşağıdaki gibi bir makro da bulundurulmuştur:

```
#define LIST_HEAD_INIT(name) {&(name), &(name)}
```

Bu durumda başlangıç düğümü bu makroyla şöyle de oluşturulabilir:

```
struct list_head head = LIST_HEAD_INIT(head);
```

Aslında bu tanımlamanın tamamını yapan aşağıdaki gibi bir makro da bulundurulmuştur:

```
#define LIST_HEAD(name) \
    struct list_head name = LIST_HEAD_INIT(name)
```

O halde biz başlangıç düğümünü basit bir biçimde şöyle oluşturabiliriz:

```
LIST_HEAD(head);
```

Linux'un bağlı liste gerçekleştirmesinde `next` ve `prev` göstericilerinde `NULL` adres kullanılmamıştır. Son düğümün `next` göstericisi `NULL` yerine başlangıç düğümünü, ilk düğümün `prev` göstericisi de `NULL` yerine son düğümü göstermektedir. Yani aslında bağlı liste gerçekleştirmesi dögüsel (circular) gibidir. Herhangi bir düğümünden başlanarak başlangıç düğümü de atlanırsa tam dolaşım sağlanabilir. Bağlı listenin başlangıç düğümünün `prev` göstericisi de son düğümü göstermektedir.

3.3.2 Düğüm Ekleme

Bağlı listenin başına `list_head` düğümünü eklemek için `list_add` fonksiyonu kullanılmaktadır:

```
static inline void list_add(struct list_head *new, struct list_head *head)
{
    __list_add(new, head, head->next);
}
```

Fonksiyonun birinci parametresi eklenecek düğümün adresini, ikinci parametresi başlangıç düğümünün adresini almaktadır.

Çekirdek kodlarında iki alt tire ile başlayan fonksiyonlar aşağı seviyeli fonksiyonlardır. Yani doğrudan çağrılmayan ancak başka fonksiyonların içerisinden dolaylı biçimde çağrılan fonksiyonlardır. Buradaki `__list_add` fonksiyonu şöyle yazılmıştır:

```
static inline void __list_add(struct list_head *new,
                             struct list_head *prev,
                             struct list_head *next)
{
    if (!__list_add_valid(new, prev, next))
        return;

    next->prev = new;
    new->next = next;
    new->prev = prev;
    WRITE_ONCE(prev->next, new);
}
```

Buradaki `WRITE_ONCE` makrosu atama işlemini volatile erişimle yapmaktadır. `WRITE_ONCE(a, b)` çağrısını `a = b` gibi düşünebilirsiniz.

Bağlı listenin sonuna düğüm eklemek için `list_add_tail` fonksiyonu kullanılmaktadır:

```
static inline void list_add_tail(struct list_head *new, struct list_head *head)
{
    __list_add(new, head->prev, head);
}
```

3.3.3 Liste Dolaşımı

Peki biz bağlı listeyi nasıl dolaşabiliriz? Başlangıç düğümünü geçerek sürekli ileriye gidersek son düğüm de başlangıç düğümünü gösterdiğine göre dolaşımı aşağıdaki gibi bir for dögüsüyle yapabiliriz:

```
struct list_head *lh;

for (lh = head.next; lh != &head; lh = lh->next) {
    /* ... */
}
```

Tabii böyle bir dögüde dolaşım yapılırken aslında biz her yinelemede ilgili yapının başlangıç adresini değil `list_head` yapısının başlangıç adresini elde ederiz. `container_of` makrosu ile bu adresin yapı adresine dönüştürülmesi gerekir. Örneğin biz `struct SAMPLE` türünden yapı nesnelerini birbirine bağlamış olalım:

```
LIST_HEAD(head);

struct SAMPLE {
    int a;
    struct list_head link;
};
```

Eklemleri şöyle yapmış olalım:

```
struct SAMPLE *ps;

for (int i = 0; i < 10; ++i) {
    if ((ps = (struct SAMPLE *)malloc(sizeof(struct SAMPLE))) == NULL) {
        fprintf(stderr, "cannot allocate memory!...\n");
        exit(EXIT_FAILURE);
    }
    ps->a = i;
    list_add_tail(&ps->link, &head);
}
```

Not

Çekirdek kodlarında *malloc* gibi kullanıcı modu fonksiyonları kullanılamaz. Ancak burada yalnızca anlaşılır bir test örneği verilmiştir.

`struct SAMPLE` yapımızdaki `link` elemanı `list_head` yapılarını birbirine bağlamak için bulundurulmuştur. Örneğimizdeki `list_add_tail` fonksiyonunda ekleme yapılacak düğümün `SAMPLE` yapısının içerisindeki `link` elemanının adresi olduğuna dikkat ediniz. Şimdi dolaşımı şöyle yapabiliriz:

```
struct SAMPLE *ps;
struct list_head *lh;

for (lh = head.next; lh != &head; lh = lh->next) {
    ps = container_of(lh, struct SAMPLE, link);
    /* ... */
}
```

Burada artık `ps` göstericisi `list_head` nesnesini değil `struct SAMPLE` nesnesini göstermektedir. Aslında `list.h` dosyası içerisinde buradaki döngüyü oluşturan `list_for_each` isimli bir makro da bulundurulmuştur:

```
static inline int list_is_head(const struct list_head *list,
                              const struct list_head *head)
{
    return list == head;
}

#define list_for_each(pos, head) \
    for (pos = (head)->next; !list_is_head(pos, (head)); pos = pos->next)
```

Makronun birinci parametresi `list_head` türünden bir göstericiyi, ikinci parametresi başlangıç düğümünün adresini almaktadır. Döngünün her yinelenmesinde bu göstericiye sonraki düğümün `list_head` adresi atanmaktadır. Örneğin:

```
struct list_head *lh;

list_for_each(lh, &head) {
    ps = container_of(lh, struct SAMPLE, link);
    printf("%d\n", ps->a);
}
```

Aslında çekirdekteki `list.h` dosyası içerisinde yukarıdaki dolaşımı tek hamlede yapan `list_for_each_entry` isimli bir makro da bulunmaktadır:

```
#define list_for_each_entry(pos, head, member) \
    for (pos = list_first_entry(head, typeof(*pos), member); \
         !list_entry_is_head(pos, head, member); \
         pos = list_next_entry(pos, member))
```

Makronun birinci parametresi asıl yapı türünden bir göstericidir; makro her yinelemede bu göstericiye sonraki düğümün adresini yerleştirmektedir. Makronun ikinci parametresi başlangıç düğümünün adresini, üçüncü parametresi ise yapı içerisindeki `list_head` elemanın ismini belirtmektedir. Bu makronun eşdeğeri bazı ayrıntılar ihmal edilerek şöyle de yazılabilir:

```
#define my_list_for_each_entry(pos, head, member) \
    for (pos = container_of((head)->next, typeof(*pos), member); &(pos)->member != head; \
         pos = container_of((pos)->member.next, typeof(*pos), member))
```

Not

C standartlarında bir ifadenin türünü veren bir tür belirleyicisi yoktu. C++'a C++11 ile birlikte `decltype` ismi ile böyle bir belirleyici eklendi. gcc derleyicileri uzun süredir bu işi yapan `typeof` isimli belirleyiciyi desteklemektedir. Nihayet C'ye de C23 ile birlikte resmi olarak `typeof` belirleyicisi eklenmiştir. Makroda türün `typeof(*pos)` ifadesiyle elde edildiğine dikkat ediniz.

Bu makro ile dolaşım şöyle sağlanabilir:

```
struct SAMPLE *ps;

list_for_each_entry(ps, &head, link) {
    /* ... */
}
```

regular]lightbulb Tüyo

Bağlı liste makrolarının ve fonksiyonlarının isimlerinin sonunda `entry` sözcüğü varsa bu makro ya da fonksiyon asıl yapıya ilişkin adres, yoksa `list_head` türünden adres verip almaktadır.

Ayrıca `list.h` içerisinde `list_for_each_entry_safe` isimli bir döngü makrosu da bulundurulmuştur. Bu makro eğer liste boşsa hiç dolaşım yapmamaktadır. Makro aşağıdaki gibi yazılmıştır:

```
#define list_for_each_entry_safe(pos, n, head, member) \
    for (pos = list_first_entry(head, typeof(*pos), member), \
         n = list_next_entry(pos, member); \
         !list_entry_is_head(pos, head, member); \
         pos = n, n = list_next_entry(n, member))
```

Makronun birinci elemanı dolaşımda kullanılacak asıl yapı türünden göstericiyi almaktadır. Makronun ikinci parametresi de asıl yapı göstericidir. Üçüncü ve dördüncü parametreler sırasıyla kök düğümün adresi ve bağ düğümünün ismini almaktadır.

3.3.4 Düğüm Silme

Bağlı listeden bir düğümü silmek için `list_del` fonksiyonu kullanılmaktadır. Fonksiyon şöyle tanımlanmıştır:

```
static inline void __list_del(struct list_head *prev, struct list_head *next)
{
    next->prev = prev;
    WRITE_ONCE(prev->next, next);
}

static inline void __list_del_entry(struct list_head *entry)
{
    if (!__list_del_entry_valid(entry))
        return;

    __list_del(entry->prev, entry->next);
}

static inline void list_del(struct list_head *entry)
```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```
{
    __list_del_entry(entry);
    entry->next = LIST_POISON1;
    entry->prev = LIST_POISON2;
}
```

Fonksiyonun parametresi silinecek düğüme ilişkin `list_head` nesnesinin adresini almaktadır. Burada silinen düğümdaki `next` ve `prev` göstericilerine özel bazı değerlerin atandığını görüyorsunuz. Bu değerler silinmiş düğümün kullanılması durumunda *page fault* oluşmasına yol açmaktadır; dolayısıyla bir çeşit debug mekanizması oluşturmak amacıyla atandıklarını söyleyebiliriz.

3.3.5 Araya Düğüm Ekleme

Bağlı listenin arasına düğüm ekleyen ayrı bir `insert` fonksiyonu yoktur. Zaten `list_add` fonksiyonu araya ekleme işlemini de yapmaktadır. Örneğin biz araya şöyle ekleme yapabiliriz:

```
list_add(&ps->link, &ps_insert->link);
```

Burada `ps` eklenecek düğümün asıl yapı adresini, `ps_insert` ise önüne eklemenin yapılacağı asıl yapı adresini belirtmektedir.

Bir bağlı listeyi tümünden serbest bırakmak için düğümlerin tek tek serbest bırakılması gerekmektedir. Buradaki düğümler aslında başka yapıların içerisinde olduğuna göre liste içerisindeki o yapı nesnelerinin serbest bırakılması gerekir.

3.4 Tam Örnek: Kullanıcı Alanında Bağlı Liste Kullanımı

Aşağıda kullanıcı alanında çekirdekteki bağlı liste kullanımına bir örnek verilmiştir.

```
#include <stdio.h>
#include <stdlib.h>
#include <stddef.h>

struct list_head {
    struct list_head *next, *prev;
};

#define LIST_HEAD_INIT(name) { &(name), &(name) }

#define LIST_HEAD(name) \
    struct list_head name = LIST_HEAD_INIT(name)

#define container_of(ptr, type, member) ({ \
    void *__mptr = (void *) (ptr); \
    ((type *) (__mptr - offsetof(type, member))); })

#define list_entry(ptr, type, member) \
    container_of(ptr, type, member)

#define list_entry_is_head(pos, head, member) \
    (&pos->member == (head))

#define list_first_entry(ptr, type, member) \
    list_entry((ptr)->next, type, member)

#define list_next_entry(pos, member) \
    list_entry((pos)->member.next, typeof(*(pos)), member)

#define list_for_each(pos, head) \
    for (pos = (head)->next; !list_is_head(pos, (head)); pos = pos->next)

#define list_for_each_entry(pos, head, member) \
```

(sonraki sayfaya devam)

```

    for (pos = list_first_entry(head, typeof(*pos), member); \
        !list_entry_is_head(pos, head, member); \
        pos = list_next_entry(pos, member))

#define list_for_each_entry_safe(pos, n, head, member) \
    for (pos = list_first_entry(head, typeof(*pos), member), \
        n = list_next_entry(pos, member); \
        !list_entry_is_head(pos, head, member); \
        pos = n, n = list_next_entry(n, member))

#define my_list_for_each_entry(pos, head, member) \
    for (pos = container_of((head)->next, typeof(*pos), member); &(pos)->member != head; \
        pos = container_of((pos)->member.next, typeof(*pos), member))

static inline int list_is_head(const struct list_head *list,
                              const struct list_head *head)
{
    return list == head;
}

static inline void __list_add(struct list_head *new,
                             struct list_head *prev,
                             struct list_head *next)
{
    next->prev = new;
    new->next = next;
    new->prev = prev;
    prev->next = new;
}

static inline void list_add(struct list_head *new, struct list_head *head)
{
    __list_add(new, head, head->next);
}

static inline void list_add_tail(struct list_head *new, struct list_head *head)
{
    __list_add(new, head->prev, head);
}

static inline void __list_del(struct list_head *prev, struct list_head *next)
{
    next->prev = prev;
    prev->next = next;
}

static inline void __list_del_entry(struct list_head *entry)
{
    __list_del(entry->prev, entry->next);
}

static inline void list_del(struct list_head *entry)
{
    __list_del_entry(entry);
}

LIST_HEAD(head);

struct SAMPLE {

```

(önceki sayfadan devam)

```

int a;
struct list_head link;
};

int main(void)
{
    struct SAMPLE *ps, *ps_node, *ps_temp, *ps_del, *ps_insert;
    struct list_head *lh;

    for (int i = 0; i < 10; ++i) {
        if ((ps = (struct SAMPLE *)malloc(sizeof(struct SAMPLE))) == NULL) {
            fprintf(stderr, "cannot allocate memory!...\n");
            exit(EXIT_FAILURE);
        }
        ps->a = i;
        list_add_tail(&ps->link, &head);

        if (i == 5)
            ps_del = ps;

        if (i == 7)
            ps_insert = ps;
    }

    list_for_each(lh, &head) {
        ps = container_of(lh, struct SAMPLE, link);
        printf("%d ", ps->a);
    }
    printf("\n");

    list_del(&ps_del->link);

    list_for_each(lh, &head) {
        ps = container_of(lh, struct SAMPLE, link);
        printf("%d ", ps->a);
    }
    printf("\n");

    if ((ps = (struct SAMPLE *)malloc(sizeof(struct SAMPLE))) == NULL) {
        fprintf(stderr, "cannot allocate memory!...\n");
        exit(EXIT_FAILURE);
    }
    ps->a = 100;

    list_add(&ps->link, &ps_insert->link);

    list_for_each(lh, &head) {
        ps = container_of(lh, struct SAMPLE, link);
        printf("%d ", ps->a);
    }
    printf("\n");

    ps_temp = NULL;
    list_for_each_entry_safe(ps, ps_node, &head, link) {
        free(ps_temp);
        ps_temp = ps;
    }

    return 0;
}

```

(sonraki sayfaya devam)

}

3.5 RCU Desteği

Belli bir süreden sonra `list.h` dosyasına RCU (Read-Copy-Update) mekanizmasını destekleyecek biçimde dolaşım yapan `list_for_each_rcu` isimli makro da eklenmiştir. Bu makro bir yazıcı varsa okuyucuları bekletmeden işlem yapabilmeyi sağlamaktadır. *Read-Copy-Update* mekanizması ileride ele alınacaktır.

3.6 Eski Çekirdek Sürümlerindeki `list.h`

Çekirdeğin eski sürümlerinde `list.h` dosyası oldukça sade idi. Sonra bu dosyanın içeriğinde de bazı faydalı değişiklikler yapıldı. Ancak bu değişiklikler veri yapısının anlaşılmasını da biraz zorlaştırmıştır. Örneğin 2.2.26 çekirdeğindeki `list.h` dosyası oldukça sade bir biçimde aşağıdaki gibi oluşturulmuştur:

```
/* 2.2.26 <list.h> dosyasının içeriği */

#ifndef _LINUX_LIST_H
#define _LINUX_LIST_H

#ifdef __KERNEL__

/*
 * Simple doubly linked list implementation.
 *
 * Some of the internal functions ("__xxx") are useful when
 * manipulating whole lists rather than single entries, as
 * sometimes we already know the next/prev entries and we can
 * generate better code by using them directly rather than
 * using the generic single-entry routines.
 */

struct list_head {
    struct list_head *next, *prev;
};

#define LIST_HEAD_INIT(name) { &(name), &(name) }

#define LIST_HEAD(name) \
    struct list_head name = { &name, &name }

#define INIT_LIST_HEAD(ptr) do { \
    (ptr)->next = (ptr); (ptr)->prev = (ptr); \
} while (0)

static __inline__ void __list_add(struct list_head *new,
    struct list_head *prev,
    struct list_head *next)
{
    next->prev = new;
    new->next = next;
    new->prev = prev;
    prev->next = new;
}

static __inline__ void list_add(struct list_head *new, struct list_head *head)
{
    __list_add(new, head, head->next);
}
```

(sonraki sayfaya devam)

```

}

static __inline__ void list_add_tail(struct list_head *new, struct list_head *head)
{
    __list_add(new, head->prev, head);
}

static __inline__ void __list_del(struct list_head *prev,
                                struct list_head *next)
{
    next->prev = prev;
    prev->next = next;
}

static __inline__ void list_del(struct list_head *entry)
{
    __list_del(entry->prev, entry->next);
}

static __inline__ int list_empty(struct list_head *head)
{
    return head->next == head;
}

static __inline__ void list_splice(struct list_head *list,
                                struct list_head *head)
{
    struct list_head *first = list->next;

    if (first != list) {
        struct list_head *last = list->prev;
        struct list_head *at = head->next;

        first->prev = head;
        head->next = first;

        last->next = at;
        at->prev = last;
    }
}

#define list_entry(ptr, type, member) \
    ((type *)((char *)(ptr)-(unsigned long)(&((type *)0)->member)))

#define list_for_each(pos, head) \
    for (pos = (head)->next; pos != (head); pos = pos->next)

#endif /* __KERNEL__ */

#endif

```

3.7 Arama Algoritmaları ve Sözlük Tarzı Veri Yapıları

Arama işlemleri bir anahtar (key) eşliğinde yapılmaktadır. Anahtar bulunması istenen varlığı temsil etmektedir. Örneğin pek çok kişinin bilgileri bir veri yapısında tutuluyor olabilir. Biz de ismini bildiğimiz bir kişiyi bu veri yapısında arayabiliriz. Burada anahtar isimdir. Veri yapıları dünyasında anahtar-değer çiftlerini tutan ve anahtar verildiğinde ona karşı gelen değerini elde edilmesini sağlayan veri yapılarına genel olarak *sözlük* (*dictionary*) tarzı veri yapıları denilmektedir.

Elemanların sıralı olmadığı liste tarzı veri yapılarında elemanların tek tek gözden geçirilmesi yoluyla yapılan aramalara *sıralı arama* (*se-*

quential search) denilmektedir. Sıralı arama oldukça yavaş bir yöntemdir. Sıralı aramada listedeki eleman sayısı N olmak üzere anahtarın bulunması için ortalama $N/2$ kez karşılaştırmanın yapılması gerekmektedir. Bu tür algoritmaların karmaşıklığına *Big O* notasyonunda $O(N)$ *karmaşıklık* denildiğini anımsayınız. Ancak ne olursa olsun eleman sayısının makul olduğu bir durumda sıralı arama en iyi yöntem haline de gelebilmektedir. Örneğin en fazla 20 civarında elemanın bulunduğu bir durumda bu elemanları diziye yerleştirip sıralı bir biçimde aramak en etkin yöntem haline gelebilmektedir.

Eğer elemanlar sıralıysa ve herhangi bir elemana erişim çok hızlı (buna *rastgele erişim* de denilmektedir) yapılabiliriyorsa *ikili arama* (*binary search*) en iyi yöntemdir. İkili aramanın algoritmik karmaşıklığı $O(\log N)$ biçimindedir. Aslında ikili aramanın daha genel bir biçimine *enterpolasyon araması* (*interpolation search*) denilmektedir. Enterpolasyon aramasında bölme ortadan değil daha uygun yerlerden yapılmaktadır. Ancak enterpolasyon araması dizi dağılımının bilindiği ve özellikle de dizinin düzgün dağıldığı durumlarda faydalı bir etki oluşturmaktadır. Diziyi önce sıraya dizip sonra ikili arama uygulamak ise genellikle iyi bir fikir değildir. Çünkü sırayı korumak için araya eleman ekleme ve aradan eleman silme gibi işlemlerde $O(N)$ karmaşıklıkta kaydırmaların yapılması gerekmektedir.

3.7.1 İndeksli Arama

Peki ideal bir anahtar-değer araması nasıl olabilir? Şüphesiz ideal durumda aramanın $O(1)$ karmaşıklıkta yapılması istenir. Anahtarın bir `int` değer olduğunu ve kişinin numarasını belirttiğini düşünelim. Biz de numarasını bildiğimiz kişinin bilgilerini elde etmek isteyelim. Örneğimizdeki kişilerin bilgileri `PERSON` isimli bir yapıyla temsil edilmiş olsun:

```
struct PERSON {
    ...
};
```

`struct PERSON` türünden büyük bir dizi açabiliriz:

```
struct PERSON people[MAX_SIZE];
```

Sonra da kişilerin numaralarını indeks yaparak bu diziye yerleştirebiliriz. Örneğin numarası 123 olan kişinin bilgilerini diziye şöyle yerleştirebiliriz:

```
people[123] = person_info;
```

Artık numarası 123 olan bu kişinin bilgilerini $O(1)$ karmaşıklıkta aşağıdaki gibi elde edebiliriz:

```
person_info = people[123];
```

Bu yöntem ilk bakışta çok iyi bir yöntem gibi gözükse de genellikle kullanılabilir bir yöntem değildir. Çünkü burada anahtar `int` türündendir. Ancak uygulamalarda anahtarlar farklı türlerden olabilmektedir. Örneğin anahtar kişinin adı soyadı olabilir. Ad ve soyad gibi yazısal bilgiler indeks belirtmemektedir. Bu yöntemin diğer bir sakıncası da anahtarların yüksek basamaklı sayılardan oluşabildiği durumlarda dizilerin çok fazla yer kaplamasıdır. Örneğin TC kimlik numarasının anahtar yapılarak kişilerin bilgilerinin elde edilmesinin istendiği bir durumu düşünelim. TC kimlik numarası 11 basamaklı bir sayıdır. Yani skalası 100 milyarlık sınırdadır. 100 milyarlık bir yapı dizisini bu amaçla oluşturmak mümkün olmayabilir, mümkün olsa da etkin olmayabilir. Bu yöntem *indeksli arama* (*index search*) denilmektedir. Ancak çok özel durumlarda bu yöntem uygun bir yöntem olarak kullanılabilir.

3.8 Hash Tabloları

Algoritmik aramalarda en çok kullanılan yöntemlerden biri *hash tabloları* (*hash tables*) denilen yöntemdir. Hash tabloları aslında yukarıda belirttiğimiz indeksli arama ile sıralı aramanın hibrit bir biçimi gibidir. Yöntemde ismine *hash tablosu* (*hash table*) denilen makul uzunlukta bir dizi oluşturulur. Sonra anahtarlar ismine *hash fonksiyonu* (*hash function*) denilen bir fonksiyona sokularak dizi indeksine dönüştürülür. Sonra da dizinin o indeksteki elemanına başvurulur. Örneğin kişilerin bilgilerini TC kimlik numaralarına göre saklayıp geri almak isteyelim. Hash tablomuzun uzunluğu da 1000 olsun. Hash fonksiyonumuzun da “1000’e bölümden elde edilen kalan” değerini veren fonksiyon olduğunu varsayalım. Bu durumda örneğin 2566198712 TC kimlik numarasına sahip kişinin bilgileri hash tablosunun 712’nci indeksteki elemanında saklanacaktır. 72484926820 TC kimlik numarasına sahip kişinin bilgileri de dizinin 820’nci indeksteki elemanında saklanacaktır. Ancak farklı kişilerin TC kimlik numaraları hash fonksiyonuna sokulduğunda aynı indeks değerleri de elde edilebilecektir. Örneğin 6238517712 TC numarasına sahip kişi de dizinin 712’nci indeksteki elemanına yerleşmek isteyecektir. İşte hash tablosu yönteminde bu duruma *çakışma* (*collision*) denilmektedir. Hash tablosu yöntemi çakışma durumunda izlenecek stratejiye göre çeşitli alt kollara ayrılmaktadır.

3.8.1 Çakışma Çözümleme Stratejileri

Hash tabloları yönteminde çakışma durumunda bu sorunu çözmek için temel olarak iki alt yöntem grubu kullanılmaktadır: *ayrı zincir oluşturma* (*separate chaining*) yöntemi ve *açık adresleme* (*open addressing*) yöntemi. Açık adresleme yöntemi de kendi aralarında *doğrusal yoklama* (*linear probing*), *karesel yoklama* (*quadratic probing*), *çift hash'leme* (*double hashing*) gibi alt yöntemlere ayrılmaktadır. Ayrı zincir oluşturma ve açık adresleme yöntemlerinin dışında başka çakışma çözümleme stratejileri de vardır. Ancak ağırlıklı olarak bu stratejiler tercih edilmektedir.

Not

Linux çekirdeklerindeki tüm hash tablolarında *ayrı zincir oluşturma* (*separate chaining*) alt yöntemi tercih edilmiştir.

Ayrı Zincir Oluşturma (Separate Chaining)

Ayrı zincir oluşturma (*separate chaining*) yönteminde hash tablosu aslında bir bağlı liste dizisi gibi oluşturulur. Yani hash tablosunun her elemanı bağlı listenin ilk elemanını (*head pointer*) gösteren bir gösterici durumundadır. Anahtar hash fonksiyonuna sokulur, bir indeks elde edilir ve o indeksteki bağlı listenin hemen önüne (ya da duruma göre arkasına) eklenir. Eleman aranırken anahtar yine aynı hash fonksiyonuna sokulur ve dizinin ilgili indeksteki bağlı listede sıralı arama yapılır.

Hash tablolarına eleman insert etmek $O(1)$ karmaşıklığındadır. Tabii burada kullanılacak hash fonksiyonu da önemlidir. Küçük döngüler içeren hash fonksiyonları $O(1)$ karmaşıklığı yükseltmemektedir. Elemanın silinmesi de $O(1)$ karmaşıklıkta yapılabilir. Eleman aramanın $O(1)$ karmaşıklıkta yapılabilmesi için bağlı listelerdeki zincir uzunluklarının uzun olmaması gerekir. Örneğin yukarıda 10 kadar eleman için en hızlı arama yönteminin sıralı arama olduğunu söylemiştik. Bu koşul sağlandığında sıralı aramanın $O(1)$ karmaşıklıkta olduğu söylenebilir. O halde eğer zincirlerdeki ortalama eleman makul bir düzeyde tutulursa arama işlemi de $O(1)$ karmaşıklıkta yapılabilir. Ancak hash tablosu küçük fakat tabloya eklenecek eleman fazla ise bu durumda hash tablosu yöntemi artık sıralı arama yöntemine benzer hale gelir. Yani bu yöntemin *en kötü durumdaki* (*worst case*) karmaşıklığının $O(N)$ olduğu söylenebilir.

Tabii hash tablosu yöntemini kullanan kişiler sistem hakkında bazı ön bilgilere sahip olursa tabloyu uygun bir büyüklükte oluşturabilirler. İşletim sistemi gibi yüksek performans isteyen sistemlerde hash tablolarının zincirlerinin ortalama 1 civarında tutulması uygun olabilmektedir. Hash tabloları da duruma göre büyütülebilmektedir. Ancak büyütmenin önemli bir zaman maliyeti vardır. Yeni bir hash tablosunun tahsis edilmesi; eski tablodaki elemanların yeniden hash'lenerek yeni tabloya yerleştirilmesi uzun zaman alan bir işlemdir. İşletim sistemlerinin çekirdeklerinde bu biçimde uzun zaman alacak işlemler tercih edilmez. Bu nedenle Linux çekirdeğindeki hash tabloları büyütülmektedir.

Hash tablolarında tablo elemanlarına (zincirlere değil) İngilizce *bucket* (*kova*), tablodaki toplam eleman sayısının bucket sayısına bölümüne de *yükleme faktörü* (*load factor*) denilmektedir. İdeal yükleme faktörünün ≤ 1 olduğunu söyleyebiliriz.

Sözlük tarzı veri yapılarında genel olarak aynı anahtara ilişkin birden fazla anahtar-değer çifti veri yapısına yerleştirilememektedir. (Bazı kütüphanelerde buna izin verilebilmektedir.) Eğer aynı anahtara ilişkin yeni bir değer insert edilmeye çalışılırsa eski değer yeni değerle yer değiştirilmektedir. Bazı tasarımlar ise aynı anahtara ilişkin insert yapmayı engellemektedir. Yani bu tasarımlarda yalnızca olmayan elemanlar tabloya insert edilebilmektedir. Linux çekirdeğindeki hash tablolarında anahtarlar birbirinden farklı olmak zorundadır.

Hash Fonksiyonu Tasarımı

Peki hash tablolarında kullanılacak iyi bir hash fonksiyonu nasıl olmalıdır? İyi bir hash fonksiyonunun *hızlı* olması gerekir. Çünkü insert gibi arama gibi işlemlerde hash fonksiyonu kullanılacaktır. İyi bir hash fonksiyonunun *anahtarlar yanlı bile olsa* tabloya onları iyi bir biçimde yaydırabilmesi gerekir. Örneğin aslında sayısal anahtarlar için “bölümden elde edilen kalan” iyi bir hash fonksiyonu değildir.

Hash tablolarında tablonun asal sayı uzunluğunda olması hash fonksiyonlarının daha iyi yaydırmasına yardımcı olmaktadır. (Örneğin tablo uzunluğu için 100 yerine 101 değeri tercih edilebilmektedir.) Hash fonksiyonları *sayıyı indekse dönüştüren* ve *yazıyı indekse dönüştüren* fonksiyonlar biçiminde oluşturulabilir. Hash tablolarının 2'nin kuvveti uzunluğunda alınması hash fonksiyonlarının daha hızlı çalışmasına da yol açabilmektedir. (Örneğin bölümden elde edilen kalan yerine bit düzeyinde öteleme işlemleri hız kazancı sağlayabilmektedir.) Linux çekirdeklerinde hash tabloları için genellikle sayfa katlarında (yani 4096'nın katlarında) alanlar tahsis edilmektedir.

Açık Adresleme ve Doğrusal Yoklama

Her ne kadar Linux çekirdeğindeki hash tablolarında *ayrı zincir oluşturma* (*separate chaining*) stratejisi kullanılıyorsa da burada *açık adresleme* (*open addressing*) stratejisi hakkında da küçük bir açıklama yapmak istiyoruz.

Açık adresleme yöntemi de kendi arasında *yoklama* (*probing*) biçimine göre çeşitli alt yöntemlere ayrılmaktadır. Açık adreslemenin en yaygın ve basit biçimi *doğrusal yoklama* (*linear probing*) denilen biçimdir.

Doğrusal yoklama oldukça basit bir fikre dayanmaktadır. Bu yöntemde yine bir hash tablosu oluşturulur. Ancak hash tablosunda bağlı listelerin adresleri tutulmaz; bizzat değerlerin kendisi tutulur. Tabloya eleman ekleneceği zaman yine anahtardan bir hash değeri elde

edilir. Doğrudan değer tablonun hash ile elde edilen indeksine yerleştirilir. Başka bir anahtar aynı hash değerini verdiğinde (yani çakışma durumu olduğunda) o indeksten itibaren boş yer bulunana kadar yan yana indekslere sırasıyla bakılır. Örneğin hash olarak 123 değerini elde etmiş olalım. Tablonun 123'üncü elemanının dolu olduğunu düşünelim. Bu durumda 124'üncü elemanına bakarız. O da doluysa 125'inci elemanına bakarız. Ta ki boş bir indeks bulunana kadar. Değeri ilk boş indekse yerleştiririz.

Bu yöntemde arama işlemi de benzer biçimde yapılmaktadır. Yani aranacak elemanın hash değeri elde edilir. O indekse başvurulur. Anahtar o indekste değilse anahtar bulunana kadar ya da boş bir kova (bucket) görülene kadar yan yana diğer indekslere bakılır.

Anahtara dayalı eleman silme de benzer biçimde yapılmaktadır. Ancak eleman silindiğinde ilgili kovanın (bucket) boşaltılması arama işlemlerinde sorunlara yol açabilecektir. Burada yöntemlerden biri silinen elemana ilişkin kovanın boş yapılmayıp silinmenin özel bir değerle belirtilmesidir. Örneğin her kova için bir durum bayrağı tutulabilir. Bu durum bayrağı ilgili kovanın *dolu* olduğunu, *boş* olduğunu ya da *silinmiş* olduğunu belirtebilir. Böylece arama sırasında *silinmiş* kovalar görüldüğünde durulmaz. İlk boş kova görüldüğünde durulur. Tabii silinmiş kovalara yeni elemanlar eklenebilir.

3.9 Linux Çekirdeğinde Hash Tablosu Gerçekleştirimi

Şimdi de Linux çekirdeğindeki hash tablosu gerçekleştirimi üzerinde duralım. Linux kaynak kodlarında hash tablosuna ilişkin yapılar ve fonksiyonlar bağlı listelere ilişkin yapıların ve fonksiyonların bulunduğu başlık dosyasında tanımlanmıştır. Yani hash tabloları için ayrı bir başlık dosyası oluşturulmamıştır. RCU'suz hash tablolarının gerçekleştirimi `include/linux/list.h` dosyası içerisinde, RCU'lu hash tablolarının gerçekleştirimi ise `include/linux/rculist.h` dosyası içerisinde bulunmaktadır. Ancak her iki gerçekleştirim de aynı yapıları kullanmaktadır.

3.9.1 hlist_head ve hlist_node Yapıları

Ayrı zincir oluşturma yönteminde hash tablosundaki kovaların (buckets) aslında bağlı listelerin ilk düğümünün yerini tutan göstericilerden oluştuğunu belirtmiştik. Linux çekirdeklerinde bağlı listenin kök düğümü de eleman düğümleri de `list_head` yapısıyla temsil ediliyordu. Ancak ayrı zincir oluşturmali hash tablolarındaki kovalarda (buckets) bağlı listenin son düğümünün yerinin tutulmasına gerek yoktur. Elemanlar hemen listenin başına eklenebilir. Arama da listenin başından itibaren yapılabilir. Tabii performans bakımından bağlı liste düğümlerinin yine çift bağlı (doubly linked) olması gerekir. Çünkü adresini bildiğimiz bir düğümü hash tablosundan kolaylıkla çıkartabiliriz. O halde çekirdek tablolarında *kök düğümde tek gösterici*, *eleman düğümlerinde ise çift gösterici* bulunmalıdır.

Çekirdekte kullanılan hash tablosu gerçekleştiriminde kök düğüm `hlist_head` yapısıyla temsil edilmiştir:

```
struct hlist_head {
    struct hlist_node *first;
};
```

Görüldüğü gibi yapıda tek bir gösterici vardır. O da zincirdeki ilk elemanı göstermektedir. Zincirdeki bağlı listenin düğümleri de `hlist_node` yapısıyla temsil edilmiştir:

```
struct hlist_node {
    struct hlist_node *next, **pprev;
};
```

Buradaki düğüm yapısını listelerdeki düğüm yapısı ile karşılaştırınız:

```
struct list_head {
    struct list_head *next, *prev;
};
```

Hash tablosundaki düğümlerin `pprev` elemanı önceki düğümün adresini değil önceki düğümdeki `next` elemanının adresini göstermektedir. Bu nedenle `pprev` göstericiyi gösteren göstericidir. Hash tablolarındaki düğümlerde geri gitmek için bir neden yoktur. Ancak eleman silme durumunda bu tasarım önceki elemanın `next` göstericisinin daha kolay güncellenmesine yol açmaktadır. Örneğin `p` göstericisi bir `hlist_node` düğümünü gösteriyor olsun. Biz de bu düğümü silecek olalım. Burada önceki düğümün `next` elemanının sileceğimiz düğümün `next` elemanındaki düğümü göstermesi sağlanmalıdır. Bu işlem pratik olarak şöyle yapılabilir:

```
*p->pprev = p->next;
```

Halbuki düğümler `list_head` yapısıyla temsil ediliyor olsaydı bu işlem ancak şöyle yapılabilirdi:

```
p->prev->next = p->next;
```

Görüldüğü gibi bu güncellemede fazladan bir işlem yapılmaktadır. `hlist_node` tasarımının diğer önemli bir faydası da bu tasarımda kök düğüm için özel bir işlemin yapılmasına gerek kalmamasıdır. Yani listeye ilk kez eleman eklerken `*p->pprev` kök düğümün `next` göstericisi haline gelecektir. Heterojen yapılarda bu tür güncellemelerin yapılması daha fazla çabayı gerektirmektedir.

Peki neden bütün çift bağlı listelerde bu teknik kullanılmıyor? `hlist_node` yapısında olduğu gibi `prev` göstericisi önceki düğümün başlangıç adresini göstermek yerine neden önceki düğümün `next` göstericisini göstermiyor? İşte geriye doğru ilerlemenin gerekli olabileceği durumlarda bu tasarımda geriye gidiş zorlaşmaktadır. Ancak yukarıda da belirttiğimiz gibi hash tablolarındaki zincirlerde zaten geriye gitmenin bir anlamı yoktur.

3.9.2 Hash Tablosunun Oluşturulması

Peki biz yukarıdaki `hlist_head` ve `hlist_node` yapılarını kullanarak hash tablosunu nasıl oluşturabiliriz? İşte yapılacak ilk şey `hlist_head` yapısı türünden bir dizi tahsis etmektir. Örneğin biz bu işlemi kullanıcı modunda aşağıdaki gibi simüle edebiliriz:

```
#include <stdio.h>
#include <stdlib.h>

#define TABLE_SIZE    4096

struct hlist_head {
    struct hlist_node *first;
};

struct hlist_node {
    struct hlist_node *next, **pprev;
};

int main(void)
{
    struct hlist_head *hash_table;

    if ((hash_table = (struct hlist_head *)malloc(
        sizeof(struct hlist_head) * TABLE_SIZE)) == NULL) {
        fprintf(stderr, "cannot allocate memory!...\n");
        exit(EXIT_FAILURE);
    }

    /* ... */

    free(hash_table);

    return 0;
}
```

Çekirdekteki `list.h` dosyası içerisinde hash tablosuna eleman eklemek için çeşitli basit fonksiyonlar bulundurulmuştur. Tabii `hlist_node` düğümleri de aslında başka yapıların elemanı durumunda olacaktır. Yine asıl yapı nesnesinin adresi `container_of` makrosuyla elde edilecektir.

Başlangıçta `hlist_head` yapısındaki `first` elemanı `NULL` değerinde olmalıdır. Bunun için `list.h` içerisinde makrolar bulundurulmuştur:

```
#define HLIST_HEAD_INIT { .first = NULL }
#define HLIST_HEAD(name) struct hlist_head name = { .first = NULL }
#define INIT_HLIST_HEAD(ptr) ((ptr)->first = NULL)
```

Örneğin:

```
for (int i = 0; i < TABLE_SIZE; ++i)
    INIT_HLIST_HEAD(&hash_table[i]);
```

Zincirlerdeki son düğümün `next` elemanı da `NULL` değerindedir. Böylece arama `NULL` görmeyene kadar ilerlenerek yapılabilir. Tablodaki zincirlerin önüne eleman eklemek için `hlist_add_head` fonksiyonu bulundurulmuştur:

```
static inline void hlist_add_head(struct hlist_node *n, struct hlist_head *h)
{
    struct hlist_node *first = h->first;
    WRITE_ONCE(n->next, first);
    if (first)
        WRITE_ONCE(first->pprev, &n->next);
    WRITE_ONCE(h->first, n);
    WRITE_ONCE(n->pprev, &h->first);
}
```

Fonksiyonun birinci parametresi eklenecek yeni düğümün adresini, ikinci parametresi ise kök düğüm nesnesini belirtmektedir. Buradaki WRITE_ONCE atamayı volatile erişimle yapmaktadır. WRITE_ONCE(a, b) çağrısını a = b gibi düşünebilirsiniz.

3.9.3 Düğüm Silme

Hash tablosundan bir düğüm silmek için hlist_del fonksiyonu kullanılmaktadır. Fonksiyon list.h içerisinde şöyle tanımlanmıştır:

```
static inline void hlist_del(struct hlist_node *n)
{
    __hlist_del(n);
    n->next = LIST_POISON1;
    n->pprev = LIST_POISON2;
}
```

Burada asıl silme işlemini yapan fonksiyon __hlist_del fonksiyonudur. Düğüm silindikten sonra silinen düğümün next ve pprev göstericilerine güvenlik amacıyla özel değerler atandığını görüyorsunuz. Bu özel değerler geçerli adresler belirtmemektedir. Eğer silinen düğüm yanlışlıkla kullanılırsa *page fault* oluşmasına yol açacaktır. __hlist_del fonksiyonu da şöyle tanımlanmıştır:

```
static inline void __hlist_del(struct hlist_node *n)
{
    struct hlist_node *next = n->next;
    struct hlist_node **pprev = n->pprev;

    WRITE_ONCE(*pprev, next);
    if (next)
        WRITE_ONCE(next->pprev, pprev);
}
```

Zincirdeki son düğümün next göstericisinde NULL adres bulunmalıdır. Fonksiyonun içerisinde bu durumun kontrol edildiğine ve ilk düğümün silinmesinin özel bir durum oluşturmadığına dikkat ediniz.

3.9.4 Ek Insert Fonksiyonları

Çok fazla gereksinim olmasa da zincirdeki belli bir düğümün önüne ve arkasına düğüm insert eden hlist_add_before ve hlist_add_behind fonksiyonları da bulundurulmuştur:

```
static inline void hlist_add_before(struct hlist_node *n,
                                   struct hlist_node *next)
{
    WRITE_ONCE(n->pprev, next->pprev);
    WRITE_ONCE(n->next, next);
    WRITE_ONCE(next->pprev, &n->next);
    WRITE_ONCE(*(&n->pprev), n);
}

static inline void hlist_add_behind(struct hlist_node *n,
                                   struct hlist_node *prev)
{
    WRITE_ONCE(n->next, prev->next);
    WRITE_ONCE(prev->next, n);
}
```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```
WRITE_ONCE(n->pprev, &prev->next);

if (n->next)
    WRITE_ONCE(n->next->pprev, &n->next);
}
```

3.9.5 Döngü Makroları

Hash tablosundaki bir `hlist_node` adresi bilindiğinde onun içinde bulunduğu asıl yapının adresinin `container_of` makrosu ile elde edilebildiğini biliyorsunuz. Ancak tıpkı bağlı listelerde olduğu gibi hash tablolarında da bunun için `container_of` makrosu ile aynı işlemi yapan bir `entry` makrosu bulundurulmuştur:

```
#define hlist_entry(ptr, type, member) container_of(ptr, type, member)
```

Hash tablosundaki bir zinciri dolaşmak için `hlist_for_each` döngü makrosu kullanılmaktadır:

```
#define hlist_for_each(pos, head) \
    for (pos = (head)->first; pos ; pos = pos->next)
```

Makronun ilk parametresi `hlist_node` türünden bir göstericiyi, ikinci parametresi ise bağlı liste zincirinin başlangıç düğümüne ilişkin `hlist_head` nesnesinin adresini almaktadır. Bu döngü makrosunun `next` elemanı `NULL` olmayana kadar ilerleme sağladığına dikkat ediniz. Döngü her yinelenirken birinci parametreye girilen göstericinin içerisine zincirdeki sonraki düğümün adresi yerleştirilmektedir. Bu döngü makrosuyla zincir dolaşılırken döngünün her yinelenmesinde asıl yapı nesnesinin değil onun içerisindeki `hlist_node` nesnesinin adresinin elde edildiğine de dikkat ediniz. Bu adresten hareketle `container_of` ya da `hlist_entry` makrolarıyla asıl nesnenin başlangıç adresinin elde edilmesi gerekir. İşte bu iki işlemi aynı anda yapan `hlist_for_each_entry` isimli bir döngü makrosu da bulundurulmuştur:

```
#define hlist_for_each_entry(pos, head, member) \
for (pos = hlist_entry((head)->first, typeof(*(pos)), member); \
    pos; \
    pos = hlist_entry((pos)->member.next, typeof(*(pos)), member))
```

Bu makronun artık birinci parametresi doğrudan asıl yapı türünden bir göstericiyi, ikinci ve üçüncü parametreler sırasıyla bağlı listenin `hlist_head` adresini ve asıl yapıdaki bağ elemanının ismini almaktadır.

Yukarıdaki makroda bir noktaya dikkatinizi çekmek istiyoruz. Hash tablosundaki `hlist_head` kök düğümünün `first` elemanında `NULL` adres varsa yukarıdaki `hlist_for_each_entry` makrosu tanımsız davranışa yol açar. İşte eğer `first` elemanı `NULL` ise bu durumu ele alıp dolaşımı sonlandıran `hlist_for_each_entry_safe` döngü makrosu da bulundurulmuştur:

```
#define hlist_for_each_entry_safe(pos, n, head, member) \
for (pos = hlist_entry_safe((head)->first, typeof(*(pos)), member); \
    pos && ({ n = pos->member.next; 1; }); \
    pos = hlist_entry_safe(n, typeof(*(pos)), member))
```

Makronun birinci parametresi asıl yapı nesnesi türünden göstericiyi, ikinci parametresi `hlist_node` türünden göstericiyi, üçüncü parametresi `hlist_head` türünden kök düğümün adresini, dördüncü parametresi de asıl yapıdaki bağ düğümünün ismini almaktadır. Bu makronun `hlist_for_each_entry` makrosundan tek farkı zincir boşsa bir soruna yol açmadan döngünün sonlanmasını sağlamasıdır. Döngü makrosunun içerisinde `hlist_entry_safe` makrosunun kullanıldığına dikkat ediniz. Bu makro zincirin sonundaki `NULL` adresi de dikkate almaktadır:

```
#define hlist_entry_safe(ptr, type, member) \
({ typeof(ptr) ____ptr = (ptr); \
    ____ptr ? hlist_entry(____ptr, type, member) : NULL; \
})
```

`list.h` dosyası içerisinde hash tablolarına ilişkin başka yararlı fonksiyonlar da vardır. Bu fonksiyonları ileride gerektiğinde açıklayacağız. Bunları siz de inceleyebilirsiniz.

Bir hash tablosunun tamamen yok edilmesi için yalnızca hash tablosunun değil onun tüm zincirlerdeki elemanlarının da serbest bırakılması gerekir. Çekirdekte genel olarak böyle bir gereksinim yoktur.

3.9.6 Örnek Uygulama

Aşağıda `list.h` içerisindeki hash tablosu işlemleri için kullanıcı modunda çalıştırılabilecek tam bir örnek verilmiştir.

```
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define TABLE_SIZE    4096

struct hlist_head {
    struct hlist_node *first;
};

struct hlist_node {
    struct hlist_node *next, **pprev;
};

#define HLIST_HEAD_INIT { .first = NULL }
#define HLIST_HEAD(name) struct hlist_head name = { .first = NULL }
#define INIT_HLIST_HEAD(ptr) ((ptr)->first = NULL)

#define WRITE_ONCE(a, b)    ((a) = (b))    /* bize özgü */
#define LIST_POISON1      (struct hlist_node *)0x00100100
#define LIST_POISON2      (struct hlist_node **)0x00200200

static inline void hlist_add_head(struct hlist_node *n,
                                struct hlist_head *h)
{
    struct hlist_node *first = h->first;
    WRITE_ONCE(n->next, first);
    if (first)
        WRITE_ONCE(first->pprev, &n->next);
    WRITE_ONCE(h->first, n);
    WRITE_ONCE(n->pprev, &h->first);
}

static inline void __hlist_del(struct hlist_node *n)
{
    struct hlist_node *next = n->next;
    struct hlist_node **pprev = n->pprev;
    WRITE_ONCE(*pprev, next);
    if (next)
        WRITE_ONCE(next->pprev, pprev);
}

static inline void hlist_del(struct hlist_node *n)
{
    __hlist_del(n);
    n->next = LIST_POISON1;
    n->pprev = LIST_POISON2;
}

#define container_of(ptr, type, member) ({          \
    void *__mptr = (void *) (ptr);                  \
    ((type *) (__mptr - offsetof(type, member))); })

#define hlist_entry(ptr, type, member)              \
    container_of(ptr, type, member)
```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```

    container_of(ptr, type, member)

#define hlist_entry_safe(ptr, type, member) \
    ({ typeof(ptr) ____ptr = (ptr); \
      ____ptr ? hlist_entry(____ptr, type, member) : NULL; \
    })

#define hlist_for_each(pos, head) \
    for (pos = (head)->first; pos ; pos = pos->next)

#define hlist_for_each_safe(pos, n, head) \
    for (pos = (head)->first; pos && ({ n = pos->next; 1; }); \
         pos = n)

#define hlist_for_each_entry(pos, head, member) \
    for (pos = hlist_entry((head)->first, typeof(*(pos)), member); \
         pos; \
         pos = hlist_entry((pos)->member.next, typeof(*(pos)), member))

#define hlist_for_each_entry_safe(pos, n, head, member) \
    for (pos = hlist_entry_safe((head)->first, typeof(*(pos)), member); \
         pos && ({ n = pos->member.next; 1; }); \
         pos = hlist_entry_safe(n, typeof(*(pos)), member))

/* Test code */

struct PERSON {
    char name[32];
    int no;
    struct hlist_node hlink;
};

void set_random_person(struct PERSON *per);
unsigned int hash_func(unsigned int key);

int main(void)
{
    struct hlist_head *hash_table;
    struct PERSON *per;
    unsigned int hash;
    int no;

    srand(time(NULL));

    if ((hash_table = (struct hlist_head *)malloc(
        sizeof(struct hlist_head) * TABLE_SIZE)) == NULL) {
        fprintf(stderr, "cannot allocate memory!...\n");
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < TABLE_SIZE; ++i)
        INIT_HLIST_HEAD(&hash_table[i]);

    for (int i = 0; i < 100; ++i) {
        if ((per = (struct PERSON *)malloc(sizeof(struct PERSON))) == NULL) {
            fprintf(stderr, "cannot allocate memory!...\n");
            exit(EXIT_FAILURE);
        }
        if (i == 50) {

```

(sonraki sayfaya devam)

```

        strcpy(per->name, "ALI SERCE");
        per->no = 12345678;
    }
    else
        set_random_person(per);
    hash = hash_func(per->no);
    hlist_add_head(&per->hlink, &hash_table[hash]);
}

/* hlist_for_each ile arama */
{
    struct hlist_node *node;
    struct PERSON *per_find;

    printf("Person no:");
    scanf("%d", &no);

    hash = hash_func(no);

    hlist_for_each(node, &hash_table[hash]) {
        per_find = hlist_entry(node, struct PERSON, hlink);
        if (per_find->no == no) {
            printf("Found: %s, %d\n", per_find->name, per_find->no);
            break;
        }
    }
    if (node == NULL)
        printf("cannot find...\n");
}

/* hlist_for_each_entry_safe ile arama */
{
    struct PERSON *per_find;
    struct hlist_node *node;

    hash = hash_func(no);

    hlist_for_each_entry_safe(per_find, node, &hash_table[hash], hlink) {
        if (per_find->no == no) {
            printf("Found: %s, %d\n", per_find->name, per_find->no);
            break;
        }
    }
    if (per_find == NULL)
        printf("cannot find...\n");
}

/* Tüm tabloyu serbest bırakma */
{
    struct PERSON *per, *temp;
    struct hlist_node *node;

    for (int i = 0; i < TABLE_SIZE; ++i) {
        temp = NULL;
        hlist_for_each_entry_safe(per, node, &hash_table[i], hlink) {
            free(temp);
            temp = per;
        }
        free(temp);
    }
}

```

(önceki sayfadan devam)

```

    }
    free(hash_table);
}

return 0;
}

void set_random_person(struct PERSON *per)
{
    int i;

    for (i = 0; i < 31; ++i)
        per->name[i] = rand() % 26 + 'A';
    per->name[i] = '\0';

    per->no = rand() % 10000000;
}

unsigned int hash_func(unsigned int key)
{
    key = (key ^ 61) ^ (key >> 16);
    key = key + (key << 3);
    key = key ^ (key >> 4);
    key = key * 0x27d4eb2d;
    key = key ^ (key >> 15);

    return key % TABLE_SIZE;
}

```

Örnekte TABLE_SIZE uzunluğunda, her bir elemanı hlist_head türünden olan bir dizi oluşturulmuştur. Sonra bu hlist_head elemanlarına ilk değer verilmiştir (yani onların first göstericilerine NULL adres yerleştirilmiştir). Ondan sonra hash tablosuna rastgele 100 eleman eklenmiştir; ancak bunun yanı sıra belli bir eleman (numarası 12345678 olan “ALI SERCE”) da listeye ayrıca eklenmiştir. Program biterken tüm hash tablosu zincirleriyle birlikte serbest bırakılmıştır.

Not

Bağlı liste düğümleri serbest bırakılırken sonraki düğümün adresini saklamak gerekir. NULL adrese free uygulamanın bir soruna yol açmayacağını anımsayınız.

3.10 RCU’lu Hash Tablosu Fonksiyonları

Linux çekirdeğine kilitlessiz (lock-free) RCU mekanizması eklendiğinde tıpkı bağlı listelerde olduğu gibi hash tablolarına da yukarıdaki fonksiyonların sonu _rcu ile biten RCU uyumlu versiyonları eklenmiştir. Bu fonksiyonlar include/linux/rculist.h dosyası içerisindedir. Bu dosyada yukarıda gördüğümüz hash tablosu fonksiyonlarının RCU mekanizmalı versiyonları bulunmaktadır. Biz buradaki fonksiyonların yalnızca isimlerini vereceğiz. RCU mekanizması başka bir başlık altında ele alınacaktır:

```

hlist_add_head_rcu
hlist_del_rcu
hlist_add_before_rcu
hlist_add_behind_rcu
hlist_for_each_entry_rcu
hlist_for_each_entry_srcu    /* safe version */

hlist_first_rcu(head)
hlist_next_rcu(node)
hlist_pprev_rcu(node)

```

Bu RCU’lu fonksiyonlar tıpkı bağlı listelerde olduğu gibi birden fazla okuyan ancak tek bir yazan taraf varsa beklemeye yol açmamak-

tadır. Tabii birden fazla yazan tarafın ayrıca bir senkronizasyon nesnesiyle korunması gerekir. Bu makrolarla dolaşım yapılırken yine bağlı listelerde olduğu gibi ilgili kod bloğunun başına ve sonuna `rcu_read_lock` ve `rcu_read_unlock` çağrılarının yerleştirilmesi gerekmektedir.

Çekirdek tıpkı listelerde olduğu gibi pek çok yerde (ancak her yerde değil) hash tablolarıyla RCU mekanizması eşliğinde işlem yapmaktadır. Çekirdeğin RCU mekanizması eşliğinde işlem yaptığı yerlerde sizin de bu RCU'lu fonksiyonları kullanmanız gerekir.

Proses Kontrol Bloğu ve task_struct Yapısı

Bu bölümde proses kavramı ve proses kontrol bloğu üzerinde duracağız ve Linux çekirdeğindeki proses kontrol bloğunu temsil eden task_struct yapısını ele alacağız.

4.1 Proses Kavramı

İşletim sistemlerinde *program* terimi çoğu kez “çalıştırılabilen bir dosyayı” ya da “bir kaynak dosyayı” belirtmektedir. Çalışmakta olan programlara ise *proses* denilmektedir. Bir program çalıştırıldığında artık o bir proses haline gelmektedir. Aynı programı birden fazla kez de çalıştırabiliriz. Bu durumda birbirinden bağımsız birden fazla proses oluşacaktır.

4.2 Proses Kontrol Bloğu

İşletim sistemlerinin proses yönetimlerindeki en önemli veri yapısı *proses kontrol bloğu* (*process control block*) denilen veri yapısıdır. İşletim sistemleri her proses için kavramsal olarak ismine “proses kontrol bloğu” denilen bir yapı türünden nesne oluşturmaktadır. Proseslerin yönetimi proses kontrol bloğu denilen bu veri yapısına başvurularak yapılmaktadır. Proses kontrol bloğu terimi yerine bazı kaynaklar *proses betimleyicisi* (*process descriptor*) terimini de kullanmaktadır.

Proses kontrol blokları içerisinde prosese ilişkin gerekli bütün bilgiler bulundurulmaktadır. Bu bilgilerden bazıları şunlardır:

- Prosesin kullanıcı id’si, grup id’si gibi hesap (credential) bilgileri
- Proses id’si
- Prosesin üst prosesinin, alt proseslerinin, kardeş proseslerinin hangi prosesler olduğu bilgisi
- Prosesin durumsal bilgisi
- Prosesin bağlamsal geçişini (context switch) sağlama için gereksinim duyulan alanlar
- Prosesin bellekte nereye yüklü olduğu gibi bellekle ilgili bilgileri
- Prosesin çizelgeleyici ile ilgili olan bilgileri (örneğin çizelgeleme politikası, önceliği gibi)
- Prosesin CPU kullanım istatistiği
- Prosesin sinyal bilgileri
- Prosesin proseslerarası haberleşme için gerekli olan bilgileri
- Prosesin çalışma dizini (current working directory)
- Prosesin açmış olduğu dosyalara ilişkin bilgiler

4.3 task_struct Yapısı

Linux çekirdeğinde proses kontrol bloğu `<include/linux/sched.h>` dosyası içerisinde bulunan `task_struct` isimli yapıyla temsil edilmiştir. Bu `task_struct` nesnesi eskiden daha az eleman içeriyordu. Sonra çekirdek gittikçe geliştirilince dev bir yapı haline geldi. Örneğin bu `task_struct` nesnesi Linus projeye ilk başladığında (Linux 0.01) şu biçimdeydi:

```
struct task_struct {
/* these are hardcoded - don't touch */
    long state;      /* -1 unrunnable, 0 runnable, >0 stopped */
    long counter;
    long priority;
    long signal;
    fn_ptr sig_restorer;
    fn_ptr sig_fn[32];
/* various fields */
    int exit_code;
    unsigned long end_code, end_data, brk, start_stack;
    long pid, father, pgrp, session, leader;
    unsigned short uid, euid, suid;
    unsigned short gid, egid, sgid;
    long alarm;
    long utime, stime, cutime, cstime, start_time;
    unsigned short used_math;
/* file system info */
    int tty;          /* -1 if no tty, so it must be signed */
    unsigned short umask;
    struct m_inode * pwd;
    struct m_inode * root;
    unsigned long close_on_exec;
    struct file * filp[NR_OPEN];
/* ldt for this task 0 - zero 1 - cs 2 - ds&ss */
    struct desc_struct ldt[3];
/* tss for this task */
    struct tss_struct tss;
};
```

Çekirdeğin 2.4'lü versiyonunda bu yapı oldukça büyümüş, bellek, sinyal, dosya sistemi ve kimlik bilgileri gibi pek çok alanı kapsayan kapsamlı bir yapı haline gelmiştir. Bugünkü 6'lı çekirdeklerde bu yapı birkaç sayfa uzunluğundadır. Eskiden yapının her elemanı çekirdek derlemesine dahil ediliyordu. Ancak 2.6'lı versiyonlardan başlanarak artık yapı elemanlarının bazıları konfigürasyona bağlı olarak yapıya dahil edilmektedir. Bu nedenle gelişmiş çekirdeklerde yapıda aşağıdaki gibi `#ifdef` blokları görürseniz şaşırmayınız:

```
struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    struct thread_info    thread_info;
#endif
    unsigned int          __state;

    /* saved state for "spinlock sleepers" */
    unsigned int          saved_state;

    randomized_struct_fields_start

    void                  *stack;
    refcount_t            usage;
    unsigned int          flags;
    ...
};
```

Bilindiği gibi Linux sistemlerinde yeni bir proses `fork` isimli POSIX fonksiyonuyla yaratılmaktadır. `fork` POSIX fonksiyonu çekirdek içerisindeki `sys_fork` isimli sistem fonksiyonunu çağırır. Yeni çekirdeklerde bu fonksiyon da `kernel_clone` isimli çekirdek fonksiyonunu çağırır. İşte `task_struct` nesnesi bu fonksiyonlar tarafından çekirdeğin heap alanında yaratılmaktadır.

4.4 Proseslerin ID Değerleri (PID)

Bilindiği gibi UNIX/Linux sistemlerinde her prosesin (yani çalışmakta olan her programın) o anda sistem genelinde tek (unique) olan bir *proses id* (*process id*) değeri vardır. Proses id değeri hem çekirdek tarafından hem de kullanıcı modundan çağrılabilen sistem fonksiyonları tarafından kullanılan bir kavramdır. Proses id değeri bir prosesi temsil etmekte kullanılan tamsayısal bir değerdir. *ps* komutuyla proseslere ilişkin proses id değerlerinin görüntülendiğini anımsayınız. Kullanıcı modunda *getpid* POSIX fonksiyonu çalışmakta olan programın proses id değerini, *getppid* POSIX fonksiyonu ise üst prosesin proses id değerini vermektedir. Tabii prosesin proses id değeri ve üst prosesin proses id değeri proses kontrol bloğunda yani *task_struct* nesnesi içerisinde saklanmaktadır.

4.5 Thread Kavramı

İşletim sistemlerine thread kavramı 90'lı yıllarda sokulmuştur. Ancak ilk thread denemeleri çeşitli çalışmalar eşliğinde daha önceleri yapılmıştır. Linux işletim sistemine thread'ler ilk kez çekirdeğin 2.0 versiyonuyla eklenmiş olsa da daha sonra çeşitli düzeltmelerle thread yapısı biraz değiştirilmiştir.

Proses kavramı çalışmakta olan programın tüm bilgilerini temsil etmektedir. Halbuki thread yalnızca bir akış belirtmektedir. Bir proses tek thread'le çalışmaya başlar. Prosesin çalışmaya başladığı thread'e *ana thread* (*main thread*) de denilmektedir. Thread'ler de kullanıcı modundan sistem fonksiyonları (*sys_clone* sistem fonksiyonu) ile yaratılmaktadır.

UNIX türevi sistemlere thread'ler ilk kez sokulduğunda farklı varyantlar farklı thread kütüphaneleri kullanıyordu. Sonra thread kavramı POSIX standartlarına da POSIX.1c ile 1995 yılında sokuldu. Böylece POSIX taşınabilir thread fonksiyonları oluşturuldu. POSIX'in taşınabilir thread fonksiyonlarının gerçekleştirimi için Linux ve diğer UNIX varyantları çekirdek içerisinde de değişiklikler yapmak zorunda kalmıştır.

Thread'lerin yalnızca bir akış belirttiğini söylemiştik. Proses kavramı ise tüm thread'lerle birlikte çalışmakta olan programın tüm bilgilerini temsil etmektedir. Pek çok bilgi thread'e özgü değildir, prosese özgüdür. Örneğin bir thread'in "çalışma dizini (current working directory)" diye bir kavram yoktur. Prosesin çalışma dizini diye bir kavram vardır. Örneğin bir thread'in kullanıcı id'si, grup id'si diye bir kavram yoktur. Prosesin kullanıcı id'si ve grup id'si diye bir kavram vardır. Yani çalışmakta olan programa ilişkin pek çok bilgi programın tüm thread'leri için de geçerlidir.

Aslında thread'ler aynı bellek alanını ve ortak bilgileri kullanan prosesler gibi de düşünülebilir. Zaten Linux işletim sisteminde thread yaratmakla proses yaratmak aslında aynı çekirdek fonksiyonuyla yapılmaktadır. Yani bir prosesin thread'lerini biz aynı bellek alanını kullanan prosesler gibi de düşünebiliriz.

Linux işletim sisteminde thread'ler de adeta aynı bellek alanı üzerinde çalışan birer proses gibi düşünülmüştür. Bu nedenle yalnızca prosesler için değil thread'ler için de *task_struct* nesneleri oluşturulmaktadır. Tabii işletim sistemi bir prosesin thread'lerini izleyen paragraflarda göreceğimiz biçimde bağlı listelerde tutmaktadır.

4.6 Prosesler Arasındaki Altlık-Üstlük İlişkisi

UNIX/Linux sistemlerinde prosesler arasında kuvvetli bir *altlık-üstlük* (*parent-child*) ilişkisi vardır. Bir proses yaratıldığında prosesin *task_struct* bilgilerinin çoğu üst prosesin (parent process) alınmaktadır. Örneğin bir program başka bir programı *fork/exec* ile çalıştırdığında çalıştırılan program çalıştıran programın pek çok özelliğini almaktadır. Örneğin *fork* işlemi sırasında *fork* yapan programın kullanıcı id'si ve çalışma dizini neyse yeni yaratılan alt prosesin (child process) kullanıcı id'si ve çalışma dizini de aynı olur. Daha teknik olarak ifade edersek *fork* işlemi sırasında alt proses için yaratılan *task_struct* nesnesinin pek çok elemanı üst prosesin *task_struct* nesnesinden kopyalanmaktadır.

POSIX thread sisteminde thread'ler arasında önemli bir altlık-üstlük ilişkisi yoktur. Yani birkaç özel durum dışında bir thread'i hangi thread'in yarattığının önemi yoktur.

4.7 task_struct Yapısının Dalı Budaklı Tasarımı

task_struct yapısının pek çok gösterici elemanı vardır. Bu gösterici elemanları başka yapıları göstermektedir. Hatta o gösterici elemanların gösterdiği yapıların elemanları da başka yapıları göstermektedir. O halde *task_struct* aslında dalı budaklı bir yapıdır. Biz bir bilginin proses kontrol bloğunda olduğunu söylediğimiz zaman o bilginin hemen *task_struct* içerisindeki bir elemanda olduğunu kastetmeyeceğiz. O bilgi *task_struct* içerisindeki bir elemanın gösterdiği başka bir yapının içerisinde de olabilir. Önemli olan o bilgiye *task_struct* yapısından hareketle erişilebilmesidir.

Bir *task_struct* yapı nesnesinin ana elemanları diğer bir *task_struct* nesnesine kopyalanırsa aslında asıl yapı nesnesi ile kopyalanmış olan yapı nesnesinin gösterici elemanları aynı yerleri gösterir duruma gelir. Programlamada bu tür kopyalamaya *sığ kopyalama* (*shallow copy*) denildiğini anımsayınız. Aslında *task_struct* yapısının bu biçimdeki dalı budaklı tasarımı üst proses-alt proses ilişkisinde alt

prosesin üst procesten birtakım özellikleri alması (inherit etmesi) sürecinde faydalar sağlamaktadır. Böylece alt prosesin `task_struct` nesnesine daha az eleman kopyalanmaktadır. Örneğin güncel çekirdeklerdeki `task_struct` yapısının aşağıdaki kısmına dikkat ediniz:

```
struct task_struct {
    /* ... */

    /* Filesystem information: */
    struct fs_struct      *fs;

    /* Open file information: */
    struct files_struct   *files;

    /* ... */
};
```

Burada aslında `fs` göstericisi aşağıdaki gibi bir yapıyı göstermektedir:

```
struct fs_struct {
    int users;
    spinlock_t lock;
    seqcount_spinlock_t seq;
    int umask;
    int in_exec;
    struct path root, pwd;
} __randomize_layout;
```

Prosesin kök dizininin ve çalışma dizinin burada tutulduğuna dikkat ediniz. `files` göstericisi ise aşağıdaki gibi bir yapıyı göstermektedir:

```
struct files_struct {
    atomic_t count;
    bool resize_in_progress;
    wait_queue_head_t resize_wait;

    struct fdtable __rcu *fdt;
    struct fdtable fdtab;

    spinlock_t file_lock ____cacheline_aligned_in_smp;
    unsigned int next_fd;
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};
```

`fork` işlemi sırasında sığ kopyalama yapıldığından dolayı `fork` işleminden sonra üst ve alt prosesin `task_struct` nesnelerinin `fs` ve `files` göstericileri aynı `fs_struct` ve `files_struct` nesnelerini gösteriyor olacaktır.

4.8 current Makrosu

Thread'lerin de tıpkı prosesler gibi `task_struct` nesnelere sahip olduğunu belirtmiştik. Peki işletim sisteminin çizelgeleyici (scheduler) alt sistemi neleri çizelgelemektedir? İşte çizelgeleyici alt sistem aslında yalnızca thread'leri yani thread'lere ilişkin `task_struct` nesnelerini çizelgelemektedir. Yani çizelgeleyici alt sistemin *çalışma kuyruğunda* (*run queue*) yalnızca `task_struct` nesnelerinin olduğunu varsayabilirsiniz. Bir proses yaratıldığında zaten o proses için yaratılan `task_struct` nesnesi aynı zamanda prosesin ana thread'inin `task_struct` nesnesi gibidir. Başka bir deyişle aslında bütün `task_struct` nesneleri birer thread belirtmektedir.

Eskiden Linux sistemleri yalnızca tek işlemciyi destekliyordu. Sonra 2.0 versiyonuyla birlikte çekirdek zamanla birden fazla işlemciyi ya da çekirdeği (core) destekler hale geldi. İşte bir thread çalışırken `task_struct` * türünden `current` isimli global değişken o anda çalışmakta olan thread'e ilişkin `task_struct` nesnesini gösterecek biçimde ayarlanmaktadır. Yani bir çekirdek kodunda `current` göstericisini görürseniz bu `current` göstericisi o anda çalışmakta olan thread'e ilişkin `task_struct` nesnesini gösteriyor durumda olacaktır. Tabii bu `current` göstericisinin o anda çalışan thread'e ilişkin `task_struct` nesnesini göstermesini bağlamsal geçişi (context switch) gerçekleştiren çizelgeleyici alt sistem sağlamaktadır.

Bir süredir birden fazla işlemciyi ya da çekirdeği destekleyen Linux çekirdeklerinde artık *current* değişkeni bir gösterici değil fonksiyon belirtmektedir. Mevcut çekirdeklerde *current* aşağıdaki gibi bir makro biçiminde tanımlanmıştır:

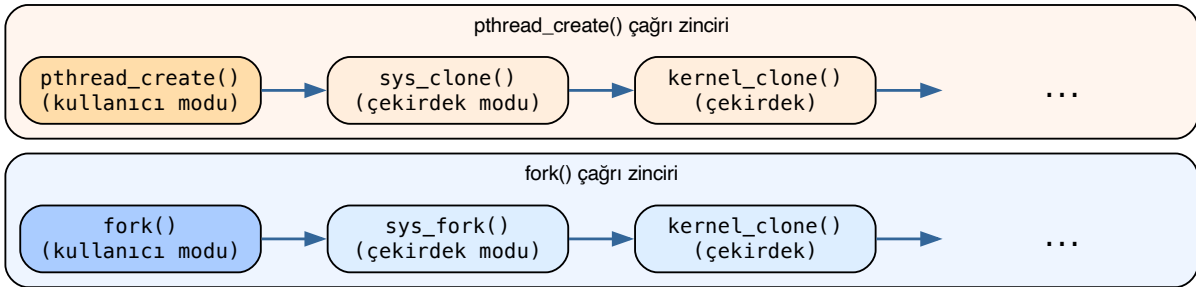
```
#define current get_current()
```

Görüldüğü gibi artık biz *current* değişkenini kullandığımızda aslında *get_current()* fonksiyonunu çağırıp bu fonksiyonun geri dönüş değerini kullanmış olmaktadır. Bu konunun ayrıntıları thread'in *çekirdek stack alanı (kernel stack)* ve *işlemciye özgü global alanlar* konusuyla ilgilidir ve başka bir bölümde ele alınacaktır.

Biz kursumuzda *current* için “current göstericisi” ya da “current makrosu” terimlerini kullanacağız.

4.9 fork ve pthread_create Çağrı Zincirleri

Bilindiği gibi UNIX/Linux sistemlerinde kullanıcı modunda yeni bir proses *fork* POSIX fonksiyonu ile, yeni bir thread de *pthread_create* fonksiyonu ile yaratılmaktadır. Yukarıda da belirttiğimiz gibi aslında bu iki çağrı da belirli bir noktadan sonra çekirdek içerisindeki aynı fonksiyonları çağırılmaktadır:



O halde aslında çekirdek gözüyle bakıldığında bir thread başka bir thread tarafından yaratılmaktadır. Yani bu yaratımda iki thread söz konusudur: Yaratan thread ve yaratılan thread. Yaratan thread'e ilişkin zaten *task_struct* nesnesi mevcuttur. O halde yaratılan thread için de bir *task_struct* nesnesi oluşturulup bir biçimde bu yapılar ilişkilendirilecektir.

4.10 task_struct Nesneleri Arasındaki İlişkiler

Şimdi de *task_struct* nesneleri arasındaki ilişkilerin bağlı listelerle nasıl oluşturulduğu üzerinde duracağız. Çekirdeğin bir prosesin thread'lerine sonra da alt proseslerine nasıl eriştiğini açıklayacağız.

4.10.1 Thread Listesi: thread_head / thread_node

Bir prosesin thread'lerine erişim için eskiden *task_struct* içerisindeki *thread_group* isimli döngüsel bağlı liste bağı kullanılıyordu. 4.2 çekirdeği (2015) ile birlikte bu veri yapısında bir değişiklik yapılmıştır. Yeni sistemde prosesin thread'lerine ilişkin bağlı listenin kök düğümü prosesin sinyal bilgilerinin bulunduğu yerde saklanmaktadır. Prosesin sinyal bilgileri *task_struct* içerisindeki *signal* göstericisinin gösterdiği yerdeki *signal_struct* yapısı içerisinde tutulmaktadır. İşte *signal_struct* yapısının *thread_head* elemanı da prosesin thread'lerine ilişkin bu kök düğümü belirtmektedir. Prosesin thread'lerinin bağlı listesi için ise *task_struct* içerisindeki *thread_node* elemanı kullanılmaktadır. Thread listesine ilişkin ilgili elemanları şöyle betimleyebiliriz:

```

struct signal_struct {
    /* ... */
    struct list_head thread_head;
    /* ... */
};

struct task_struct {
    /* ... */

```

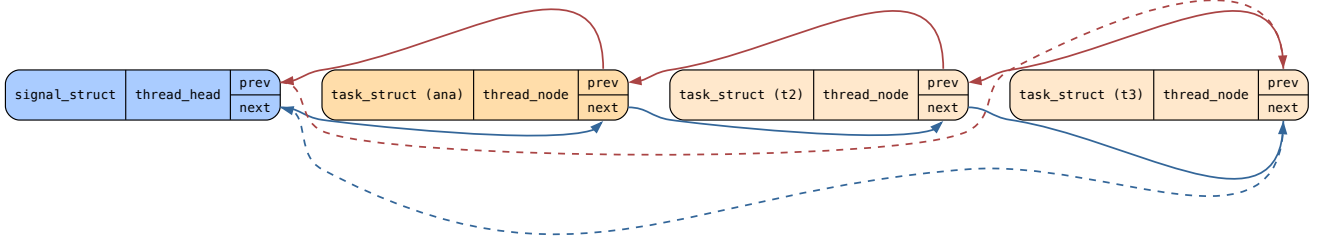
(sonraki sayfaya devam)

```

struct signal_struct *signal;
struct list_head thread_node;
/* ... */
};

```

Aşağıdaki diyagram bu bağlı listenin yapısını göstermektedir. Mavi renkteki `thread_head` kök düğümü `signal_struct` içerisinde; turuncu renkteki `thread_node` düğümleri ise her bir thread'e ait `task_struct` içerisinde:



`thread_node` bağlı listesi çekirdeğe ilk eklendiğinde bir süre eski `thread_group` listesi de muhafaza edilmişti. Sonra tamamen eski `thread_group` listesi `task_struct` içerisinden kaldırıldı. Artık yeni çekirdeklerde prosesin thread'lerinin `task_struct` nesnelerini elde etmek için prosesin `signal_struct` nesnesindeki `thread_head` kök düğümünden başlanarak bağlı listenin `thread_node` düğümlerinin dolaşılması gerekmektedir.

Aslında Linux çekirdeklerinde prosesin tüm thread'lerine ilişkin `task_struct` içerisinde bulunan `signal` göstericisi aynı `signal_struct` nesnesini göstermektedir. Yani `thread_head` kök düğümüne biz prosesin ana thread'inden erişmek zorunda değiliz. Prosesin herhangi bir thread'ine ilişkin `task_struct` nesnesindeki `signal` göstericisi yoluyla bu kök düğüme erişebiliriz.

Çekirdek içerisinde bir `task_struct` yapısının prosesin ana thread'ine ilişkin olup olmadığını belirleyen `thread_group_leader` isimli bir makro da vardır:

```

#include <linux/sched/signal.h>

static inline bool thread_group_leader(struct task_struct *p)
{
    return p->exit_signal >= 0;
}

```

Tabii aslında bir `task_struct` nesnesinin ana thread'e ilişkin olup olmadığı `p == p->group_leader` ya da `p->pid == p->tgid` işlemiyle anlaşılabilir. Ancak özel bir bilgi olarak `task_struct` içerisindeki `exit_signal` elemanı da bu bilgiyi verebilmektedir. Bu eleman eğer thread ana thread değilse -1 değerinde, ana thread ise ≥ 0 değerinde olmaktadır.

Linux çekirdeğinde terminoloji bağlamında “prosesin ana thread'i” yerine *thread grup lideri* (*thread group leader*) terimi tercih edilmektedir.

4.10.2 pid ve tgid Elemanları

Bilindiği gibi UNIX/Linux sistemlerinde her prosesin sistem genelinde tek (unique) olan bir *proses id* (*pid*) değeri vardır. Linux çekirdeği prosesin id değerini prosesin ana thread'ine ilişkin `task_struct` içerisindeki `pid` elemanında saklamaktadır:

```

struct task_struct {
    /* ... */
    pid_t pid;
    /* ... */
};

```

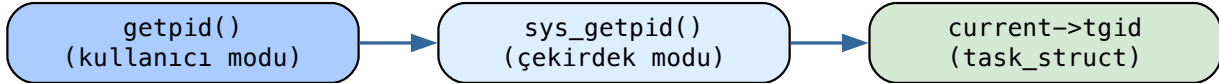
Linux'ta her thread'in `task_struct` nesnesinde ayrı bir `pid` değeri vardır, fakat prosesin `pid` değeri denildiğinde prosesin ana thread'inin `pid` değeri anlaşılmalıdır. Ana thread'in `pid` değeri aynı zamanda `task_struct` içerisindeki `tgid` isimli bir eleman da tutulmaktadır. `task_struct` nesnesi hangi thread'e ilişkin olursa olsun `tgid` elemanı her zaman prosesin ana thread'inin `pid` değerini tutmaktadır:

```

struct task_struct {
    /* ... */
    pid_t    pid;      /* o thread'e ilişkin pid değeri, POSIX'te böyle bir kavram yok */
    pid_t    tgid;     /* ana thread'e ilişkin pid değeri, getpid bu değeri veriyor */
    /* ... */
};

```

O halde bazı ayrıntıları da göz ardı edersek *getpid* POSIX fonksiyonunun çağırdığı *sys_getpid* sistem fonksiyonu prosesin proses id değerini doğrudan *current* göstericisinin gösterdiği *task_struct* nesnesinin içerisindeki *tgid* değerinden alarak vermektedir:



Şimdi aklınıza “neden her thread’te ayrıca ana thread’in pid değeri tgid ismiyle tutuluyor?” sorusu gelebilir. Bunun iki nedeni vardır: Birincisi prosesin ana thread’i sonlanabilir, bu durumda ana thread’e ilişkin *task_struct* nesnesine erişilemeyebilir. İkincisi ise bu yolla prosesin id değerine daha hızlı erişimin sağlanabilmesidir.

Linux’ta thread’lere özgü pid değeri Linux’a özgü *gettid* fonksiyonu ile elde edilebilmektedir:

```

#define _GNU_SOURCE
#include <unistd.h>

pid_t getpid(void);

```

Not

gettid fonksiyonunun bir POSIX fonksiyonu olmadığına dikkat ediniz.

4.10.3 group_leader Göstericisi

Aslında Linux çekirdeği thread’lere ilişkin *task_struct* nesnelerinde yalnızca prosesin ana thread’ine ilişkin pid değerini değil, aynı zamanda ana thread’in *task_struct* nesnesinin adresini de tutmaktadır. Ana thread’in *task_struct* nesnesinin adresi *task_struct* yapısının *group_leader* elemanında tutulmaktadır:

```

struct task_struct {
    /* ... */
    pid_t    pid;      /* o thread'e ilişkin pid değeri */
    pid_t    tgid;     /* ana thread'e ilişkin pid değeri */
    struct task_struct *group_leader; /* ana thread'e ilişkin task_struct nesnesinin adresi */
    /* ... */
};

```

Peki prosesin ana thread’i sonlandığında *group_leader* göstericisinin ve *signal_struct* yapısı içerisindeki *list_head* düğümünün durumu ne olacaktır? İşte Linux çekirdeği prosesin ana thread’i sonlandığında istisna olarak ana thread’e ilişkin *task_struct* nesnesini yok etmemektedir (başka bir deyişle *hortlak* (*zombie*) olarak tutulmaktadır). Yani bu *group_leader* göstericisi her zaman geçerli bir *task_struct* nesnesini gösteriyor durumda olmaktadır.

4.10.4 real_parent ve parent Göstericileri

Şimdi de Linux çekirdeğinde prosesler arasındaki altlık-üstlük ilişkisinin nasıl sağlandığı üzerinde duralım. Bilindiği gibi UNIX/Linux sistemlerinde her proses başka bir prosesin thread’i tarafından *fork* işlemi ile yaratılmaktadır. Bu durumda prosesi yaratan thread’in ilişkin olduğu prosese *üst proses* (*parent process*), yeni yaratılan prosese de *alt proses* (*child process*) denilmektedir.

Her thread'in `task_struct` nesnesi içerisindeki `real_parent` ve `parent` isimli göstericiler o thread'i yaratan thread'in `task_struct` nesnesinin adresini tutmaktadır:

```
struct task_struct {
    /* ... */
    pid_t    pid;
    pid_t    tgid;
    struct task_struct *group_leader;

    struct task_struct __rcu *real_parent; /* fork yapan üst prosesteki thread */
    struct task_struct __rcu *parent;      /* fork yapan üst prosesteki thread */
    /* ... */
};
```

Burada `real_parent` üst prosese ilişkin thread'in `task_struct` nesnesinin adresini tutmaktadır. Normalde `real_parent` elemanı ile `parent` elemanı aynı `task_struct` nesnesini gösterir. Ancak seyrek durumlarda (örneğin debug işlemlerinde ve `ptrace` işlemlerinde) geçici olarak `parent` başka bir `task_struct` nesnesini de (reparenting işlemi) gösteriyor durumda olabilmektedir.

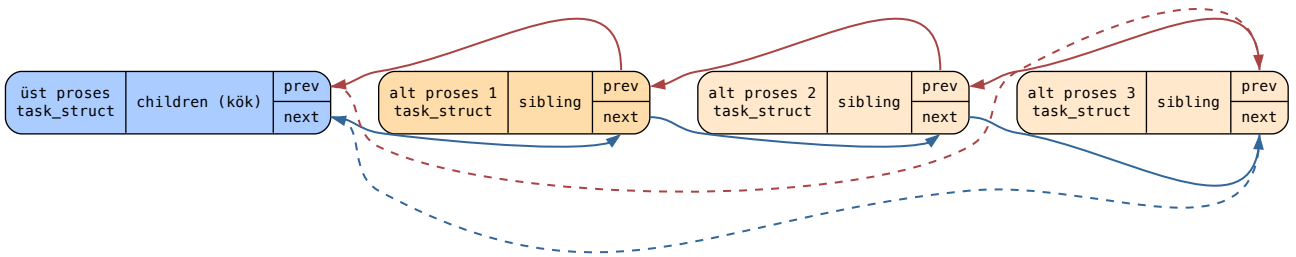
`getppid` POSIX fonksiyonu üst prosesin pid değerini vermektedir. İşte bu fonksiyonun çağırdığı `sys_getppid` sistem fonksiyonu `real_parent` elemanının gösterdiği `task_struct` nesnesi içerisindeki `tgid` değerini geri döndürmektedir.

4.10.5 Alt Proses Listesi: children / sibling

Şimdi de bir prosesin alt proseslerinin nasıl bağlı listelerde tutulduğu üzerinde duralım. Örneğin bir proses üç kez `fork` yapmış olsun. Bu durumda bu prosesin üç tane alt prosesi olacaktır. Üst prosesleri aynı olan proseslere *kardeş prosesler* (*sibling processes*) de denilmektedir. İşte bir prosesin alt prosesleri `children` isimli kök düğümle erişilen `sibling` bağları yoluyla bağlı listede tutulmaktadır:

```
struct task_struct {
    /* ... */
    struct list_head children; /* alt proses listesinin kök düğümü */
    struct list_head sibling;   /* alt proseslerin bağlı liste bağı */
    /* ... */
};
```

Aşağıdaki diyagram bu ilişkiyi görsel olarak ortaya koymaktadır. Üst prosesin `children` düğümü (kök) mavi, alt proseslerin `sibling` düğümleri turuncu renkte gösterilmiştir:



`sibling` bağları her zaman prosesin ana thread'ine ilişkin (grup liderine ilişkin) `task_struct` nesnelerini göstermektedir. Bir thread akışında yeni bir thread yaratıldığı zaman yaratılan thread bu `children/sibling` listesinde bulunmaz; prosesin thread'leri yalnızca `thread_head/thread_node` listesinde tutulmaktadır.

Ayrıca Linux'ta alt proses listesi ana thread'in `children` kök düğümünden hareketle dolaşmak zorundadır. Alt prosesler yaratıldığında yalnızca ana thread'in `children` kök düğümü güncellenmektedir.

4.10.6 Tüm Proseslerin Listesi: tasks

Çekirdek ayrıca proseslerin ana thread'lerine ilişkin `task_struct` nesnelerini de `task_struct` içerisindeki `tasks` isimli bir düğüm ile birbirine bağlamaktadır:

```

struct task_struct {
    /* ... */
    pid_t pid;
    pid_t tgid;
    struct task_struct *group_leader;

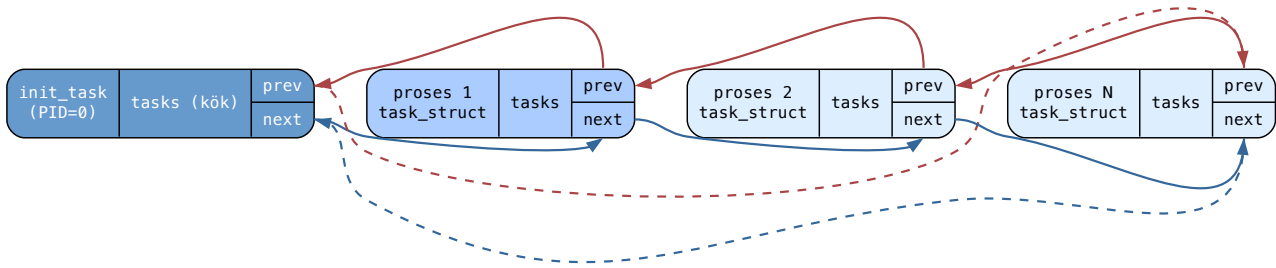
    struct task_struct __rcu *real_parent;
    struct task_struct __rcu *parent;

    struct list_head children;
    struct list_head sibling;
    struct list_head tasks;           /* ana thread'lerin tutulduğu liste bağı */
    /* ... */
};

```

Bu `tasks` düğümlerine ilişkin bağlı listenin kök düğümü `init_task` prosesindedir. Linux'ta boot işlemi sırasında ilk oluşturulan process'e `init_task` denilmektedir. Bu `init_task` prosesine ilişkin `task_struct` nesnesi statik bir biçimde `init/init_task.c` dosyasında bulunmaktadır. `init_task` prosesinin pid değeri 0'dır. `init_task` processi `init` prosesini yarattıktan sonra işlevini sonlandırarak durdurulmaktadır. Ancak bu process'e ilişkin `task_struct` nesnesi gerçek anlamda hiçbir zaman yok edilmemektedir. İşte `init_task.tasks` elemanı proseslerin ana thread'lerine ilişkin `task_struct` nesnelerini dolaşmak için bir kök düğüm olarak kullanılmaktadır.

Aşağıdaki diyagram `tasks` bağlı listesini göstermektedir. Kök düğüm `init_task` (PID=0) koyu mavi, proseslerin ana thread `task_struct` nesneleri açık mavi renktedir:



4.10.7 Özet: task_struct ilişkileri

Yukarıda açıkladığımız konuyu madde madde özetleyelim:

- Her thread'in ayrı bir pid değeri vardır, ancak POSIX'teki `getpid` fonksiyonu ana thread'in (thread grup liderinin) pid değerini vermektedir.
- Bir prosesin tüm thread'lerini dolaşmak için `signal` göstericisinin gösterdiği `signal_struct` yapısı içerisindeki `thread_head` bağlı listesi `thread_node` düğümleriyle dolaşılır.
- `task_struct` içerisindeki `real_parent` ve `parent` göstericileri o processi ya da thread'i yaratan thread'in `task_struct` nesnesini göstermektedir.
- `task_struct` içerisindeki `tgid` prosesin ana thread'inin pid değerini, `group_leader` göstericisi ise prosesin ana thread'inin `task_struct` nesnesinin adresini tutmaktadır. Ana thread sonlansa bile onun `task_struct` nesnesi proses sonlanana kadar muhafaza edilmektedir.
- Bir prosesin bütün alt proseslerine ilişkin ana thread'lerin `task_struct` nesneleri `children/sibling` bağlı listesinde tutulmaktadır.
- Prosesin alt proseslerini elde etmek için prosesin ana thread'inin `children` kök düğümünden yola çıkmak gerekir.
- Çekirdek yaratılmış olan tüm proseslerin ana thread'lerine ilişkin `task_struct` nesnelerini ayrıca `task_struct` içerisindeki `tasks` düğümüyle birbirine bağlamaktadır. Bu düğümün kökü de `init_task.tasks` elemanıdır.

4.10.8 Sık Sorulan Sorular

Aşağıda konuyla ilgili bazı işlemlerin bu bağlı listeler kullanılarak nasıl gerçekleştirileceğine yönelik tipik sorular ve yanıtları verilmektedir:

Soru: Bir prosesin tüm thread'leri nasıl elde edilebilir?

Bir prosesin tüm thread'leri prosesin signal yapısı içerisinde bulunan `thread_head` elemanı kök yapılarak `thread_node` düğümlerinin dolaşılmasıyla elde edilebilir.

Soru: Bir prosesin bütün alt prosesleri nasıl elde edilebilir?

Prosesin ana thread'indeki `children` düğümü kök yapılarak `sibling` düğümlerinin dolaşılmasıyla prosesin tüm alt prosesleri elde edilebilir.

Soru: Bir ``task\_struct`` nesnesinin prosesin ana thread'ine ilişkin olup olmadığı nasıl anlaşılır?

Eğer `task_struct` nesnesinde `pid == tgid` ise bu `task_struct` nesnesi ilgili prosesin ana thread'ine ilişkin `task_struct` nesnesidir. Zaten prosesin tüm thread'lerine ilişkin `task_struct` nesnelerinde `group_leader` göstericisi ana thread'e ilişkin bu `task_struct` nesnesini göstermektedir. O anda çalışmakta olan thread eğer prosesin ana thread'i ise `current == group_leader` koşulunun sağlanacağına da dikkat ediniz.

Soeu: Sistemdeki tüm proseslerin ana thread'lerine ilişkin ``task\_struct`` nesneleri nasıl dolaşılır?

`init_task` prosesindeki `tasks` düğümü kök düğüm yapıp `tasks` düğümleri dolaşılırsa tüm proseslerin ana thread'lerine ilişkin `task_struct` nesneleri elde edilebilir.

4.11 task_struct Yapısına İlişkin Bağlı Listeler Üzerinde Gezinme İşlemleri

Şimdi de görmüş olduğumuz `task_struct` yapısına ilişkin bağlı listeler üzerinde gezinme işlemlerine örnekler verelim. Bu biçimdeki küçük testler için çekirdeği yeniden derlememize gerek yoktur. Basit bir karakter aygıt sürücüsü yazarak da bu işlemleri yapabiliriz.

4.11.1 Geliştirme Ortamının Hazırlanması

Genellikle bir Linux sistemini yüklediğimizde zaten çekirdek modüllerini ve aygıt sürücülerini oluşturabilmek için gereken başlık dosyaları ve diğer gerekli öğeler `/usr/src` dizini içerisindeki `linux-headers-$(uname -r)` dizininde yüklü biçimde bulunmaktadır. Ancak bunlar yüklü değilse Debian tabanlı sistemlerde bunları şöyle yükleyebilirsiniz:

```
$ sudo apt-get install linux-headers-$(uname -r)
```

4.11.2 Derleme ve Yükleme

Karakter aygıt sürücülerini bir ya da birden fazla kaynak `.c` dosyası oluşturularak yazılabilmektedir. Aygıt sürücülerin derlenmesi oldukça karmaşık bir süreçle gerçekleşmektedir. Bu nedenle derleme işlemi için hazır `Makefile` dosyaları bulundurulmuştur. Programcı kendisi bir `Makefile` dosyası oluşturup asıl `make` dosyalarını buradan devreye sokar. Aygıt sürücüler için en basit `Makefile` dosyası aşağıdaki gibi oluşturulabilir:

```
obj-m += ${file}.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=${PWD} modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=${PWD} clean
```

Bu `Makefile` dışarıdan `file` isimli bir argüman almaktadır. Aygıt sürücünün derlenmesi şöyle yapılır (kaynak dosya için uzantı belirtilmediğine dikkat ediniz):

```
$ make file=mydriver
```

Aygıt sürücüler derlendikten sonra `.ko` uzantılı bir dosya elde edilecektir. Bu dosya çekirdeğe yüklenmelidir. Aygıt sürücü dosyalarını (`.ko` dosyalarını) çekirdeğe yüklemek için `insmod` ya da `modprobe` komutları kullanılmaktadır. `insmod` komutu herhangi bir bağımlılığa bakmadan doğrudan yükleme yapar. Biz kursumuzda aygıt sürücülerini `insmod` komutuyla yükleyeceğiz. Tabii aygıt sürücülerini yükleyebilmek için `root` önceliğine sahip olmak gerekir. Örneğin:


```
$ sudo insmod mydriver.ko
```

insmod ile yüklenmiş olan aygıt sürücüsü çekirdekten *rmmmod* komutuyla çıkartılır:

```
$ sudo rmmmod mydriver.ko
```

Aygıt sürücünün yüklenip yüklenmediğini anlayabilmek için `/proc/devices` dosyasına başvurabilirsiniz.

4.11.3 Çekirdek Modülü ve Aygıt Sürücüsü Kavramları

Aygıt sürücüler çekirdek modunda çekirdeğin bir parçasıymış gibi çalışan modüllerdir. Aygıt sürücüler içerisinde kullanıcı modundaki standart C fonksiyonları ya da POSIX fonksiyonları kullanılamaz. Aygıt sürücüler yalnızca çekirdek içerisinde *export* edilmiş fonksiyonları kullanabilirler. Bir fonksiyon ya da nesne çekirdek kodlarında `EXPORT_SYMBOL` makrosu ile *export* edilmektedir. Örneğin:

```
void clear_nlink(struct inode *inode)
{
    if (inode->i_nlink) {
        inode->__i_nlink = 0;
        atomic_long_inc(&inode->i_sb->s_remove_count);
    }
}
EXPORT_SYMBOL(clear_nlink);
```

Aslında Linux sistemlerinde bu bağlamda birbirleriyle ilişkili iki kavram vardır: *Çekirdek modülleri* (*kernel modules*) ve *aygıt sürücüler* (*device drivers*). Çekirdeğin içerisine yüklenebilen modüllere “çekirdek modülleri” denilmektedir. Eğer bir çekirdek modülüne kullanıcı modundan bir dosya (buna *aygıt dosyası* (*device file*) denilmektedir) yoluyla erişilip aygıt sürücüyü işlemler yaptırılabilirse bu tür çekirdek modüllerine *aygıt sürücüsü* (*device driver*) de denilmektedir. Yani her aygıt sürücüsü bir çekirdek modülü belirtir ancak her çekirdek modülü bir aygıt sürücüsü belirtmek zorunda değildir. Sistemdeki yüklü olan çekirdek modülleri `/proc/modules` dosyası yoluyla, yüklü olan aygıt sürücüler ise `/proc/devices` dosyası yoluyla görüntülenebilmektedir.

Aşağıda iskelet bir çekirdek modülü görüyorsunuz:

```
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");

static int helloworld_init(void);
static void helloworld_exit(void);

static int helloworld_init(void)
{
    printk(KERN_INFO "Hello World...\n");

    return 0;
}

static void helloworld_exit(void)
{
    printk(KERN_INFO "Goodbye World...\n");
}

module_init(helloworld_init);
module_exit(helloworld_exit);
```

Bir çekirdek modülü *insmod* komutuyla yüklendiğinde onun `module_init` makrosuyla belirtilen fonksiyonu çağrılmaktadır. Çekirdek modülü *rmmmod* ile sistemden çıkartılırken de `module_exit` makrosunda belirtilen fonksiyon çağrılmaktadır.

Çekirdek modülleri içerisinde birtakım mesajlar iletilmek isteniyorsa bu mesajlar *kernel ring buffer* denilen log sistemine yazdırılmaktadır. Bu log sistemine yazdırılan yazılar *dmesg* komutuyla ya da `/var/log/syslog` dosyası ile görüntülenebilmektedir. Çekirdek modülü içerisinde bu log sistemine mesajlar *printk* isimli çekirdek fonksiyonuyla yazılmaktadır. Örneğin:

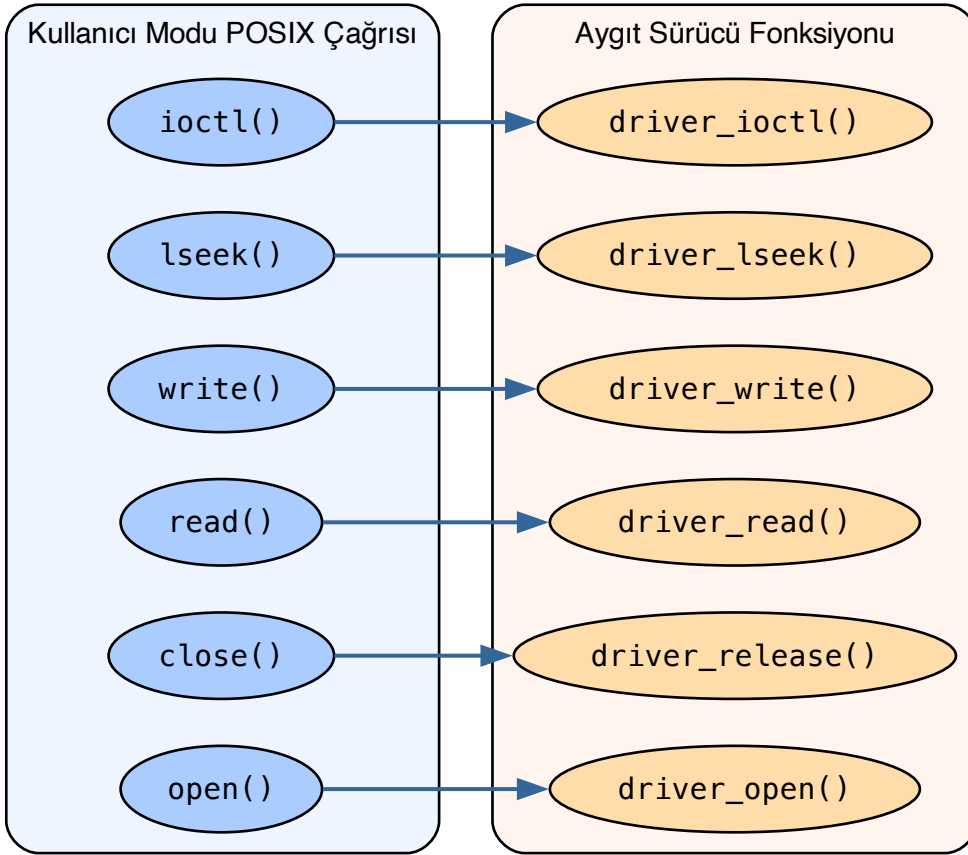
```
printk(KERN_INFO "Goodbye World...\n");
```

KERN_INFO mesajın türünü belirtmektedir. KERN_INFO ile diğer argüman arasında virgül atomunun bulunmadığına dikkat ediniz. KERN_INFO aslında bir sembolik sabittir ve bir string açımı yapmaktadır. (C'de yana yana iki string'in birleştirildiğini anımsayınız.) KERN_INFO mesaj türü ile mesaj yazdırmanın daha basit yolu `pr_info` makrosunu kullanmaktadır:

```
pr_info("Goodbye World...\n");
```

4.11.4 Aygıt Dosyaları

Aygıt sürücülere erişmekte kullanılan özel dosyalara *aygıt dosyaları* (*device files*) denilmektedir. Aygıt dosyası *open* POSIX fonksiyonuyla açıldığında çekirdek diske değil yüklü olan aygıt sürücüyü referans etmektedir. Kullanıcı modundaki programcı aygıt dosyası üzerinde dosya işlemlerini yaptığına aslında programın akışı çekirdek moduna geçerek aygıt sürücü içerisindeki ilgili fonksiyonlar çalıştırılmaktadır:



Linux sistemlerinde aygıt sürücüler (ve dolayısıyla aygıt dosyaları) iki kısma ayrılmaktadır:

- *Karakter Aygıt Sürücülere (Character Device Drivers)*
- *Blok Aygıt Sürücülere (Block Device Drivers)*

Blok aygıt sürücülere *hard disk*, *SSD*, *CDROM*, *ramdisk* gibi blok blok transferlerin yapıldığı aygıtlar söz konusu olduğunda kullanılmaktadır. Karakter aygıt sürücülere ilişkin aygıt dosyaları UNIX/Linux sistemlerinde c dosya türü ile, blok aygıt sürücülere ilişkin aygıt dosyaları ise b dosya türü ile temsil edilmektedir.

UNIX/Linux sistemlerinde aygıt dosyaları genel olarak `/dev` dizininde bulunmaktadır. Linux sistemlerinde *udev* denilen bir servis (demon) ile bu dizinde aygıt dosyalarının yaratılması mümkün hale getirilmiştir.

4.11.5 Aygıt Sürücülerin Majör ve Minör Numaraları

Her aygıt sürücünün *majör* ve *minör* numarası vardır. Bu majör ve minör numaralar çekirdeğin aygıt sürücüyü erişebilmesi için bir adres görevi görmektedir. Majör numara aygıt sürücünün ana kodunu temsil eder. Minör numara ise onun örneklerini (instances) temsil etmektedir.

Aygıt dosyaları komut satırında *mknod* isimli programla yaratılmaktadır:

```
$ sudo mknod -m 666 mydriver c 250 0
```

Tipik olarak sistem programcısı bir *kabuk betiği* (shell script) yazarak bu işlemi otomatize eder. Bu betik önce *insmod* ile aygıt sürücüyü yükler, `/proc/devices` dosyasına başvurup onun majör numarasını elde eder ve aygıt dosyasını *mknod* komutuyla dinamik bir biçimde oluşturur. Biz de denemelerimizde böyle yapacağız. Bunun için aşağıdaki gibi bir *load* betiği kullanacağız:

```
#!/bin/bash
# load (bu satırı dosyaya kopyalamayınız)

module=$1
mode=666

/sbin/insmod ./${module}.ko ${@:2} || exit 1
major=$(awk "\$2 == \"\$module\" {print \$1}" /proc/devices)
rm -f $module
mknod -m $mode $module c $major 0
```

Bu betik dosyasını oluşturduktan sonra dosyaya `x` hakkını vermeyi unutmayınız:

```
$ chmod +x load
$ sudo ./load mydriver
```

Aygıt sürücüyü *rmmmod* ile çıkartıp ilgili aygıt dosyasını silen *unload* betiği de aşağıdaki gibi yazılabilir:

```
#!/bin/bash
# unload (bu satırı dosyaya kopyalamayınız)

module=$1

/sbin/rmmmod ./${module}.ko || exit 1
rm -f $module
```

```
$ chmod +x unload
$ sudo ./unload mydriver
```

4.11.6 İskelet Karakter Aygıt Sürücüsü Örneği

Aşağıda iskelet bir karakter aygıt sürücüsü oluşturulmuştur. Bu karakter aygıt sürücüsü için bir `ioctl` kodu da hazırlanmıştır.

test-driver.h:

```
#ifndef TEST_DRIVER_H_
#define TEST_DRIVER_H_

#include <linux/stddef.h>
#include <linux/ioctl.h>

#define TEST_DRIVER_MAGIC 't'
#define IOC_TEST _IO(TEST_DRIVER_MAGIC, 0)

#endif
```

test-driver.c:

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include "test-driver.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kaan Aslan");
MODULE_DESCRIPTION("test-driver");

static int test_driver_open(struct inode *inodep, struct file *filp);
static int test_driver_release(struct inode *inodep, struct file *filp);
static ssize_t test_driver_read(struct file *filp, char *buf, size_t size, loff_t *off);
static ssize_t test_driver_write(struct file *filp, const char *buf, size_t size, loff_t *off);
static long test_driver_ioctl(struct file *filp, unsigned int cmd, unsigned long arg);
static long ioctl_test(struct file *filp, unsigned long arg);

static dev_t g_dev;
static struct cdev g_cdev;
static struct file_operations g_fops = {
    .owner          = THIS_MODULE,
    .open           = test_driver_open,
    .read           = test_driver_read,
    .write          = test_driver_write,
    .release        = test_driver_release,
    .unlocked_ioctl = test_driver_ioctl
};

static int __init test_driver_init(void)
{
    int result;

    printk(KERN_INFO "test-driver module initialization...\n");

    if ((result = alloc_chrdev_region(&g_dev, 0, 1, "test-driver")) < 0) {
        printk(KERN_INFO "cannot alloc char driver!...\n");
        return result;
    }
    cdev_init(&g_cdev, &g_fops);
    if ((result = cdev_add(&g_cdev, g_dev, 1)) < 0) {
        unregister_chrdev_region(g_dev, 1);
        printk(KERN_ERR "cannot add device!...\n");
        return result;
    }

    return 0;
}

static void __exit test_driver_exit(void)
{
    cdev_del(&g_cdev);
    unregister_chrdev_region(g_dev, 1);
    printk(KERN_INFO "test-driver module exit...\n");
}

static int test_driver_open(struct inode *inodep, struct file *filp)
{
    printk(KERN_INFO "test-driver opened...\n");

```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```

    return 0;
}

static int test_driver_release(struct inode *inodep, struct file *filp)
{
    printk(KERN_INFO "test-driver closed...\n");

    return 0;
}

static ssize_t test_driver_read(struct file *filp, char *buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "test-driver read...\n");

    return 0;
}

static ssize_t test_driver_write(struct file *filp, const char *buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "test-driver write...\n");

    return 0;
}

static long test_driver_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    long result;

    printk(KERN_INFO "test_driver_ioctl...\n");

    switch (cmd) {
        case IOC_TEST:
            result = ioctl_test(filp, arg);
            break;
        default:
            result = -ENOTTY;
    }

    return result;
}

long ioctl_test(struct file *filp, unsigned long arg)
{
    printk(KERN_INFO "IOC_test_driver_TEST1\n");

    return 0;
}

module_init(test_driver_init);
module_exit(test_driver_exit);

```

Aygıt sürücüyü şöyle derleyebilirsiniz:

```

$ make file=test-driver
$ sudo ./load test-driver

```

Aşağıdaki kullanıcı modu programı aygıt sürücüyü açıp ioctl kodunu çalıştırmaktadır:

app.c:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include "test-driver.h"

void exit_sys(const char *msg);

int main(void)
{
    int fd;

    if ((fd = open("test-driver", O_RDONLY)) == -1)
        exit_sys("open");

    if (ioctl(fd, IOC_TEST) == -1)
        exit_sys("ioctl");

    close(fd);

    return 0;
}

void exit_sys(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}

```

```

$ gcc -Wall -o app app.c
$ ./app

```

app programını çalıştırdıktan sonra *dmesg* komutunu kullandığımızda log mesajları aşağıdaki gibi görülmelidir:

```

[ 431.787375] test-driver module initialization...
[ 450.631940] test-driver opened...
[ 450.631953] test_driver_ioctl...
[ 450.631958] IOC_test_driver_TEST1
[ 450.631967] test-driver closed...

```

4.11.7 RCU Mekanizması ile task_struct Listelerinin Dolaşılması

Biz *task_struct* listelerini dolaşırken çekirdek bu listelere eleman eklerse ya da bu listelerden eleman silerse bizim dolaşım yapan kodumuzda *tanımsız davranışlar* (*undefined behaviours*) oluşabilir. Bu da tüm sistemin çökmesine yol açabilir. Artık belli bir süredir çekirdek *task_struct* listeleri üzerinde kilitsiz bir biçimde *RCU* (*Read-Copy-Update*) mekanizmasıyla işlemler yapmaktadır. Dolayısıyla bizim de kendimizi çekirdeğin kullandığı bu RCU mekanizmasına uydurarak dolaşım yapmamız gerekir.

RCU mekanizmasının ayrıntılarını ileride ayrı başlıkta ele alacağız. Ancak bu noktada RCU mekanizması ile *task_struct* listelerinin dolaşılması için bazı temel bilgileri de vereceğiz.

RCU mekanizmasıyla bağlı listeleri dolaşırken işlemin başında *rcu_read_lock* fonksiyonunun, işlemin sonunda da *rcu_read_unlock* fonksiyonunun çağırılması gerekmektedir:

```

#include <linux/rcupdate.h>

void rcu_read_lock(void);
void rcu_read_unlock(void);

```

Bizim dolaşım kodunu bu iki çağrı arasına yerleştirmemiz gerekir. RCU mekanizması altında bağlı listelerin bağlarına erişilirken *rcu_dereference* makrosu kullanılmalıdır. Örneğin sistemdeki bütün proseslerin ana thread'lerine ilişkin *task_struct* nesneleri

aşağıdaki gibi güvenli bir biçimde dolaşılabilir:

```
static void walk_processes(void)
{
    struct task_struct *ts;
    struct list_head *lh;

    rcu_read_lock();

    lh = rcu_dereference(init_task.tasks.next);
    while (lh != &init_task.tasks) {
        ts = container_of(lh, struct task_struct, tasks);
        printk(KERN_INFO "PID = %d, COMM = %s", ts->pid, ts->comm);
        lh = rcu_dereference(ts->tasks.next);
    }

    rcu_read_unlock();
}
```

container_of makrosu ile list_entry makrosunun tamamen eşdeğer olduğunu anımsayınız. task_struct içerisindeki comm elemanında en fazla 16 karakterlik task_struct nesnesini betimleyen bir yazı bulunmaktadır. fork işlemi ya da thread yaratımı sırasında bu yazı üst prosesin comm elemanından kopyalanmaktadır. exec işlemleri sırasında ise bu yazı çalıştırılan programın dosya isminden hareketle değiştirilmektedir.

<linux/sched/signal.h> dosyası içerisinde next_task isimli bir makro da vardır. Bu makro bir task_struct nesnesinin adresini parametre olarak alıp RCU mekanizmasına uygun olarak onun tasks.next göstericisinin gösterdiği yerdeki task_struct nesnesinin adresini vermektedir:

```
#include <linux/sched/signal.h>

next_task(p)
```

Bu makro kullanılarak walk_process fonksiyonu şöyle de yazılabilir:

```
static void walk_processes(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    ts = next_task(&init_task);
    while (ts != &init_task) {
        printk(KERN_INFO "PID = %d, COMM = %s", ts->pid, ts->comm);
        ts = next_task(ts);
    }

    rcu_read_unlock();
}
```

4.11.8 Tüm Prosesleri Dolaşan Çekirdek Modülü

Aşağıda tüm proseslerin ana thread'lerini dolaşan örnek bir çekirdek modülü verilmiştir.

test-module.c (next_task ile):

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/rcupdate.h>

MODULE_LICENSE("GPL");
```

(sonraki sayfaya devam)

```

static int test_module_init(void);
static void test_module_exit(void);
static void walk_processes(void);

static int test_module_init(void)
{
    printk(KERN_INFO "test_module_init...\n");
    walk_processes();

    return 0;
}

static void walk_processes(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    ts = next_task(&init_task);
    while (ts != &init_task) {
        printk(KERN_INFO "PID = %d, COMM = %s", ts->pid, ts->comm);
        ts = next_task(ts);
    }

    rcu_read_unlock();
}

static void test_module_exit(void)
{
    printk(KERN_INFO "test_module_exit...\n");
}

module_init(test_module_init);
module_exit(test_module_exit);

```

<linux/sched/signal.h> başlık dosyasında bulunan `for_each_process` isimli döngü makrosu da `tasks` listesini daha kolay dolaşmak için kullanılabilir. Bu makro kendi içerisinde `rcu_dereference` işlemini de yapmaktadır:

```

rcu_read_lock();

for_each_process(ts) {
    /* ... */
}

rcu_read_unlock();

```

test-module.c (for_each_process ile):

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/sched/signal.h>

MODULE_LICENSE("GPL");

static int test_module_init(void);
static void test_module_exit(void);
static void walk_processes(void);

static int test_module_init(void)

```


(önceki sayfadan devam)

```

{
    printk(KERN_INFO "test_module_init...\n");
    walk_processes();

    return 0;
}

static void walk_processes(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    for_each_process(ts) {
        printk(KERN_INFO "PID = %d, COMM = %s", ts->pid, ts->comm);
    }

    rcu_read_unlock();
}

static void test_module_exit(void)
{
    printk(KERN_INFO "test_module_exit...\n");
}

module_init(test_module_init);
module_exit(test_module_exit);

```

Ayrıca <linux/rculist.h> dosyasında bulunan `list_for_each_entry_rcu` isimli döngü makrosu da RCU mekanizmasına uygun biçimde bağlı listeleri dolaşmak için kullanılabilir:

```

#include <linux/rculist.h>

list_for_each_entry_rcu(pos, head, member)

```

Bu makroyu kullanarak tüm proseslerin ana thread'lerini dolaşan fonksiyon şöyle yazılabilir:

```

static void walk_processes(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    list_for_each_entry_rcu(ts, &init_task.tasks, tasks) {
        printk(KERN_INFO "PID = %d, COMM = %s", ts->pid, ts->comm);
    }

    rcu_read_unlock();
}

```

Aynı başlık dosyasındaki `list_entry_rcu` makrosu bir bağın adresini alarak RCU mekanizmasına uygun bir biçimde bağın bulunduğu nesnenin adresini vermektedir. Aslında önceki paragraflarda gördüğümüz `next_task` makrosu da bu makro kullanılarak yazılmıştır:

```

#define next_task(p) \
    list_entry_rcu((p)->tasks.next, struct task_struct, tasks)

```

4.11.9 Thread Listesini Dolaşan Aygıt Sürücü Örneği

Şimdi de thread'lere ilişkin `task_struct` nesnelerinin dolaşılmasına örnekler verelim. Anımsanacağı gibi yeni çekirdeklerde belli bir prosesin thread'lerine ilişkin `task_struct` nesneleri `signal_struct` nesnesindeki `thread_head` düğümü kök alınarak `thread_node` düğümlerinin dolaşılmasıyla elde ediliyordu.

Belli bir prosesin thread'lerinin dolaşılabilmesi için çekirdek modülü yetersiz kalmaktadır. Çünkü çekirdek modüllerindeki fonksiyonlar dışarıdan prosesler tarafından çağrılmamaktadır. Bu nedenle thread dolaşım testlerini ancak bir aygıt sürücü yazarak oluşturabiliriz. Aygıt sürücülerdeki fonksiyonlar kullanıcı modundaki thread'ler tarafından çağrılabilir. Bu fonksiyonlarda `current` makrosu kullanıldığında ilgili fonksiyonu çağırarak thread'in `task_struct` nesnesinin adresi elde edilecektir.

Prosesin thread'leri aşağıdaki gibi dolaşılabilir:

```
static void walk_process_thread(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    list_for_each_entry_rcu(ts, &current->signal->thread_head, thread_node) {
        printk(KERN_INFO "Thread PID = %d, COMM = %s", ts->pid, ts->comm);
    }

    rcu_read_unlock();
}
```

Bir `task_struct` adresini alıp onun `thread_node.next` göstericisinin gösterdiği yerdeki `task_struct` adresini veren `next_thread` isimli bir fonksiyon da vardır:

```
#include <linux/sched/signal.h>

static inline struct task_struct *next_thread(struct task_struct *p)
{
    return __next_thread(p) ?: p->group_leader;
}
```

Bu fonksiyon ile prosesin herhangi bir thread'inden başlayarak tüm thread'leri dolaşılabilir:

```
static void walk_process_thread(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    ts = current;
    do {
        printk(KERN_INFO "Thread PID = %d, COMM = %s", ts->pid, ts->comm);
        ts = next_thread(ts);
    } while (ts != current);

    rcu_read_unlock();
}
```

Aslında prosesin thread'lerini basit bir biçimde dolaşmak için `for_each_thread` döngü makrosu tercih edilmektedir. Makronun birinci parametresi prosesin herhangi bir thread'inin `task_struct` adresini almaktadır. Döngü her yinelendikçe yeni bir thread'e ilişkin `task_struct` yapının adresi ikinci parametreyle belirtilen göstericinin içerisine yerleştirilmektedir:

```
#include <linux/sched/signal.h>

for_each_thread(p, t)
```

```
static void walk_process_thread(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    for_each_thread(current, ts) {
        printk(KERN_INFO "Thread PID = %d, COMM = %s", ts->pid, ts->comm);
    }

    rcu_read_unlock();
}
```

Aşağıda prosesin thread'lerini dolaşan tam aygıt sürücü kodları ve test programı verilmiştir. Aygıt sürücüyü yükledikten sonra *app* programı çalıştırılmalıdır. Bu program önce 10 tane thread yaratıp sonra aygıt sürücüdeki ioctl kodunu çalıştırmaktadır.

test-driver.c (thread dolaşımı):

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include "test-driver.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kaan Aslan");
MODULE_DESCRIPTION("test-driver");

/* ... (open/release/read/write fonksiyonları önceki örnekle aynıdır) ... */

static void walk_process_thread(void);

long ioctl_test(struct file *filp, unsigned long arg)
{
    walk_process_thread();

    return 0;
}

static void walk_process_thread(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    for_each_thread(current, ts) {
        printk(KERN_INFO "Thread PID = %d, COMM = %s", ts->pid, ts->comm);
    }

    rcu_read_unlock();
}

module_init(test_driver_init);
module_exit(test_driver_exit);
```

app.c (thread dolaşımı için test programı):

```
#define _GNU_SOURCE

#include <stdio.h>
```

(sonraki sayfaya devam)

```
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/ioctl.h>
#include "test-driver.h"

#define NTHREADS 10

void exit_sys(const char *msg);
void *thread_proc(void *param);

int main(void)
{
    int fd;
    int result;
    pthread_t tids[NTHREADS];

    for (int i = 0; i < NTHREADS; ++i) {
        if ((result = pthread_create(&tids[i], NULL, thread_proc, NULL)) != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(result));
            exit(EXIT_FAILURE);
        }
    }

    if ((fd = open("test-driver", O_RDONLY)) == -1)
        exit_sys("open");

    if (ioctl(fd, IOC_TEST) == -1)
        exit_sys("ioctl");

    close(fd);

    for (int i = 0; i < NTHREADS; ++i)
        pthread_join(tids[i], NULL);

    return 0;
}

void *thread_proc(void *param)
{
    printf("Thread PID: %jd\n", (intmax_t)gettid());
    sleep(120);

    return NULL;
}

void exit_sys(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}
```

4.12 Tüm task_struct Nesnelerini Dolaşma

Peki sistemdeki bütün task_struct nesnelerini dolaşabilir miyiz? Bunu yapmanın en pratik yolu init_task nesnesinden hareketle tüm proseslerin ana thread'lerine ilişkin task_struct nesnelerini elde edip o ana thread'lerden faydalanarak onların diğer thread'lerine ilişkin task_struct nesnelerini elde etmektir. Aslında bunu iç içe iki döngü oluşturarak RCU mekanizması eşliğinde yapan for_each_process_thread isimli bir makro vardır:

```
#include <linux/sched/signal.h>

#define for_each_process_thread(p, t) \
    for_each_process(p) for_each_thread(p, t)
```

Makroya biz iki task_struct göstericisi veririz. Makro her yinelemede prosesin ana thread'ine ilişkin task_struct nesnesinin adresini birinci parametreyle verilen göstericinin içerisine, o prosesin thread'lerine ilişkin task_struct nesnelerinin adresleri de ikinci parametreyle verilen göstericinin içerisine yerleştirmektedir:

```
static void walk_all_threads(void)
{
    struct task_struct *tsp;
    struct task_struct *tst;

    rcu_read_lock();

    for_each_process_thread(tsp, tst) {
        if (tsp == tst) {
            printk(KERN_INFO "Process Main Thread PID = %d, COMM = %s\n",
                    tsp->pid, tst->comm);
        }
        else {
            printk(KERN_INFO "    Thread PID == %d COMM = %s\n",
                    tst->pid, tst->comm);
        }
    }

    rcu_read_unlock();
}
```

dmesg çıktısı aşağıdakine benzer biçimde elde edilecektir:

```
[ 6022.928044] Process Main Thread PID = 4271, COMM = kworker/0:0
[ 6022.928045] Process Main Thread PID = 4279, COMM = kworker/u259:0
[ 6022.928048] Process Main Thread PID = 4543, COMM = kworker/3:1
[ 6022.928051] Process Main Thread PID = 4565, COMM = bash
[ 6022.928053] Process Main Thread PID = 4599, COMM = app
[ 6022.928054]     Thread PID == 4600 COMM = app
[ 6022.928056]     Thread PID == 4601 COMM = app
[ 6022.928057]     Thread PID == 4602 COMM = app
[ 6022.928059]     Thread PID == 4603 COMM = app
[ 6022.928060]     Thread PID == 4604 COMM = app
[ 6022.928087] Process Main Thread PID = 4610, COMM = sudo
[ 6022.928090] Process Main Thread PID = 4612, COMM = insmod
```

4.12.1 Alt Proses Listesini Dolaşma

Anımsanacağı gibi prosesin alt proses listesinin kök düğümü ana prosesin ana thread'ine ilişkin task_struct nesnesinin children elemanında tutuluyordu. children/sibling listesinde yalnızca alt proseslerin ana thread'lerine ilişkin task_struct nesnelerinin bulunduğunu da anımsayınız.

Alt prosesleri dolaşan hazır bir döngü makrosu yoktur. Ancak biz yukarıdaki tekniklerle dolaşımı yapabiliriz:

```
static void walk_child_processes(void)
{
    struct task_struct *ts;

    rcu_read_lock();

    list_for_each_entry_rcu(ts, &current->children, sibling) {
        printk(KERN_INFO "Child Process Main Thread PID = %d, COMM = %s\n",
            ts->pid, ts->comm);
    }

    rcu_read_unlock();
}
```

Bu kodu bir karakter aygıt sürücüsünün ioctl kodu içerisinde çağırabilirsiniz. Kodu önce çeşitli alt prosesler yarattıktan sonra aygıt sürücüsü üzerinde ioctl çağırısı yapan bir programla test edebilirsiniz. Aşağıdaki test programı 10 tane alt proses ve her alt proseste 3 tane thread yaratmaktadır:

app.c (alt proses dolaşımı için test programı):

```
#define _GNU_SOURCE

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
#include <pthread.h>
#include <sys/ioctl.h>
#include "test-driver.h"

#define NCHILDS 10
#define NTHREADS 3

void exit_sys(const char *msg);
void *thread_proc(void *param);

int main(void)
{
    int fd;
    int result;
    pid_t pid;

    for (int i = 0; i < NCHILDS; ++i) {
        if ((pid = fork()) == -1) {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (pid == 0) {
            pthread_t tids[NTHREADS];

            for (int k = 0; k < NTHREADS; ++k) {
                if ((result = pthread_create(&tids[k], NULL, thread_proc, NULL)) != 0) {
                    fprintf(stderr, "pthread_create: %s\n", strerror(result));
                    exit(EXIT_FAILURE);
                }
            }
        }
    }
}
```

(sonraki sayfaya devam)

```

    for (int k = 0; k < NTHREADS; ++k)
        pthread_join(tids[k], NULL);

    _exit(0);
}
else {
    printf("Child PID = %jd\n", (intmax_t)pid);
}
}

if ((fd = open("test-driver", O_RDONLY)) == -1)
    exit_sys("open");

if (ioctl(fd, IOC_TEST) == -1)
    exit_sys("ioctl");

close(fd);

for (int i = 0; i < NCHILDS; ++i)
    if (wait(NULL) == -1) {
        perror("wait");
        exit(EXIT_FAILURE);
    }

return 0;
}

void *thread_proc(void *param)
{
    sleep(60);

    return NULL;
}

void exit_sys(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}

```

Not

Çekirdeğin uyguladığı RCU mekanizması tek bir yazan ve birden fazla okuyan akışın bulunduğu durumda beklemeyi ortadan kaldırmaktadır. Ancak veri yapısına birden fazla yazanın olması durumunda yazan tarafların bir kilit mekanizmasıyla senkronize edilmesi gerekir. İşte güncel çekirdekler hala `task_struct` bağlı listelerine yazma için `tasklist_lock` kilidini kullanmaktadır. `tasklist_lock` okuma yazma kilidi artık çekirdek modülleri ve aygıt sürücüler için export edilmemektedir.

4.13 PID Tahsisatı

Şimdi de pid değerleriyle `task_struct` nesneleri arasındaki ilişkiyi ele alalım. Bilindiği gibi kullanıcı modunda prosesler pid değerleriyle temsil edilmektedir. Örneğin pid parametresi alan tipik POSIX fonksiyonları şunlardır: `kill`, `waitpid`, `getpgid`, `setpgid`, `getsid`, `getpriority`, `setpriority`.

O halde çekirdeğin pid değerinden hareketle o pid değerine ilişkin `task_struct` nesnesini hızlı bir biçimde elde etmesi gerekmektedir. Bu bağlamda çekirdek geliştiricisinin aşağıdaki iki sorunun yanıtını biliyor olması gerekir:

1. Yeni bir proses ya da thread yaratıldığında çekirdek o proses ya da thread'e ilişkin pid değerini nasıl üretmektedir?

2. Çekirdek belli bir pid değerine ilişkin proses ya da thread'in `task_struct` nesnesine nasıl ulaşmaktadır?

4.13.1 PID Üretim Mekanizmasının Tarihsel Gelişimi

Bir `task_struct` nesnesi çekirdek tarafından yaratıldığında ona atanacak pid değeri eskiden oldukça basit bir biçimde belirleniyordu. Linux 0.01 versiyonundaki `find_empty_process` fonksiyonu bu bakımdan çok ilkeldi:

```
int find_empty_process(void)
{
    int i;

    repeat:
        if ((++last_pid) < 0) last_pid = 1;
        for (i = 0; i < NR_TASKS; i++)
            if (task[i] && task[i]->pid == last_pid) goto repeat;
        for (i = 1; i < NR_TASKS; i++)
            if (!task[i])
                return i;

    return -EAGAIN;
}
```

Bu ilk versiyonda gördüğünüz gibi sistemde tahsis edilebilecek en fazla `task_struct` nesnesi 64 kadardı (`NR_TASKS = 64`). Zaten bu versiyonda 64 tane `task_struct` yapısı baştan statik olarak tahsis edilmişti:

```
struct task_struct *task[NR_TASKS] = {&(init_task.task), };
```

2.4 çekirdeğinde pid numarasının belirlenmesi işlemi belli bir değerden itibaren aday olan pid değerlerinin herhangi bir `task_struct` nesnesi tarafından kullanılıp kullanılmadığına bakılarak yapılmıştır. Bu versiyondaki `get_pid` fonksiyonu tüm `task_struct` nesnelerini dolaşarak boş bir pid değerini doğrusal aramayla tespit etmeye çalışmaktadır.

pid araması 2.6 çekirdekleriyle birlikte bitmap veri yapısı kullanılarak elde edilmeye başlanmıştır. Bitmap veri yapısı bitleri tutan bir veri yapısıdır. Modern işlemcilerde belli bir adresten itibaren 0 olan ilk bitin yerini veren makine komutları bulunmaktadır. Bu veri yapısı kullanılarak boş bir pid değeri elde eden fonksiyondaki kilit satır şudur:

```
offset = find_next_offset(map, offset);
```

`find_next_offset` fonksiyonu bitmap'teki ilk boş olan 0 bitinin offset değerini bulan özel makine komutu kullanılarak yazılmıştır. Kursumuzda bitmap veri yapısını ileride ayrı bir başlık altında inceleyeceğiz.

Çekirdeğin 4.15 versiyonu ile pid tahsisatında bitmap kullanımı bırakılarak radix ağaçları kullanılmaya başlanmıştır. Güncel çekirdekler ise boş pid değerinin üretimi için artık radix ağaçlarının özel bir biçimi olan `XArray` veri yapısını kullanmaktadır. Güncel çekirdeklerde boş pid değerini bu karmaşık yöntemle elde eden `alloc_pid` isimli yüksek seviyeli bir fonksiyon bulunmaktadır:

```
struct pid *alloc_pid(struct pid_namespace *ns, pid_t *set_tid, size_t set_tid_size);
```

4.13.2 Maksimum PID Değeri

Linux çekirdekleri maksimum pid değeri için bir üst limit kullanmaktadır. 2.6 ve günümüze kadarki çekirdeklerde `PID_MAX_LIMIT` sembolik sabiti şöyle bildirilmiştir:

```
#define PID_MAX_LIMIT (IS_ENABLED(CONFIG_BASE_SMALL) ? PAGE_SIZE * 8 : \
    (sizeof(long) > 4 ? 4 * 1024 * 1024 : PID_MAX_DEFAULT))
```

Görüldüğü gibi `CONFIG_BASE_SMALL` konfigürasyon parametresi n yapılmış ve 64 bit sistemlerde bu limit $4 \times 1024 \times 1024 = 4.194.304$ değerini belirtmektedir. 32 bit sistemlerde ise bu değer `PID_MAX_DEFAULT` yani 32.768'dir.

Bugün kullandığımız 64 bitlik işlemcilerdeki Linux sistemlerinde maksimum pid değeri 4.194.304, 32 bit sistemlerde ise 32.767 biçimindedir. Bu değer `proc` dosya sisteminde `/proc/sys/kernel/pid_max` dosyasında da belirtilmektedir. Programcı isterse `sysctl` komutu yoluyla sistem çalışırken bu değeri düşürebilir ancak yükseltemez.

Sisteminizde tahsis edilebilecek maksimum `task_struct` nesnelerinin sayısını (yani maksimum thread sayısını) `/proc/sys/kernel/threads-max` dosyasından öğrenebilirsiniz. Bu değer sistem çalışırken de değiştirilebilir:


```
$ echo 1000000 | sudo tee /proc/sys/kernel/threads-max
```

4.14 PID'den task_struct Nesnesine Hızlı Erişim

“Çekirdek belli bir pid değerine ilişkin `task_struct` adresini nasıl hızlı bir biçimde elde etmektedir?” sorusuna yanıt verelim. Düz mantıkla sistemdeki bütün `task_struct` nesneleri dolaşarak elde edilebilir. Ancak böyle bir arama çekirdek için çok yavaştır. Bu tür hızlı aramalar için Linux çekirdeğinde genel olarak hash tabloları, bazen de arama ağaçları kullanılmaktadır.

Linux'un 2.6.24 versiyonuna kadar bunun için hash tabloları kullanılıyordu. Ancak bu versiyondan itibaren bu amaçla *radix ağaçları* da işin içine sokulmuştur. Güncel çekirdeklerdeki arama sistemi radix ağaçlarının özel bir biçimi olan *XArray* veri yapısı ve hash tablolarının hibrit bir biçimine benzemektedir. Biz kursumuzun bu noktasında Linux çekirdeğinde hash tablolarının gerçekleştirimi üzerinde duracağız.

Çekirdeğin ilk 0.01 versiyonunda zaten en fazla 64 tane `task_struct` nesnesi oluşturulabiliyordu. Bu versiyonda pid değerine ilişkin `task_struct` nesnesinin bulunması için bu `task_struct` nesnelerinin tutulduğu `task` isimli global dizide sıralı arama yapılmıştır.

Çekirdeğin 2.2 versiyonlarında pid değerinden hareketle `task_struct` nesnesinin bulunması için hash tablosu kullanılmıştır. Bu versiyonlarda bu işi yapan `find_task_by_pid` fonksiyonu aşağıdaki gibi yazılmıştır:

```
extern __inline__ struct task_struct *find_task_by_pid(int pid)
{
    struct task_struct *p, **htable = &pidhash[pid_hashfn(pid)];

    for (p = *htable; p && p->pid != pid; p = p->pidhash_next)
        ;

    return p;
}
```

Bu versiyonlarda henüz daha sonra açıkladığımız `hlist` hash tablosu fonksiyonları çekirdekte bulunmuyordu. `find_task_by_pid` fonksiyonunda önce pid değeri `pid_hashfn` isimli bir hash fonksiyonuna sokularak bir indeks değeri elde edilmiş, sonra da `pidhash` dizisinin bu indeksteki bağlı liste zincirinde arama yapılmıştır. `pidhash` dizisi şöyle tanımlanmıştır:

```
struct task_struct *pidhash[PIDHASH_SZ];
```

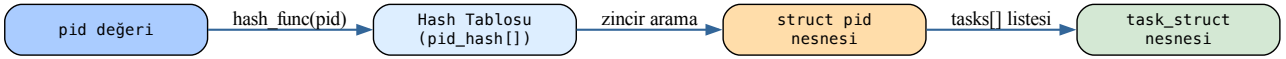
Görüldüğü gibi hash tablosundaki zincirler doğrudan `task_struct` nesnelerini tutmaktadır. Bu versiyonlarda bu zincirler için `task_struct` içerisinde iki link elemanı bulunduruluyordu:

```
struct task_struct {
    /* ... */
    struct task_struct *pidhash_next;
    struct task_struct **pidhash_pprev;
    /* ... */
};
```

Çekirdeğin 2.4 versiyonunda da algoritmada ve yukarıdaki fonksiyonda bir değişiklik yapılmamıştır. Yani yine bir hash tablosu eşliğinde arama yapılmaktadır.

4.14.1 2.6 Versiyonu: pid Nesnesi ve Bağlı Listeler

Çekirdeğin 2.6'lı versiyonlarında artık her farklı pid değeri için o pid değerine ilişkin bir pid nesnesi (`struct pid` nesnesi) oluşturulmaya başlanmıştır. Bu versiyonlarda çekirdek her farklı pid değeri için bir pid nesnesi oluşturup bu pid nesnelerini de hash tablolarında saklamaktadır. Her pid nesnesi de aşağıda açıklayacağımız gibi bir grup bağlı listenin kök düğümlerini tutmaktadır. Bu versiyonlarda bir pid değerine ilişkin `task_struct` nesnesini bulmak için çekirdek önce hash tablosundan o pid değerine ilişkin pid nesnesini elde etmekte, sonra o pid nesnesinin içerisindeki bağlı listelerde arama yapmaktadır:



2.6'lı versiyonlardaki pid yapısı şöyleydi:

```

struct pid {
    atomic_t    count;
    unsigned int level;
    /* lists of tasks that use this pid */
    struct hlist_head tasks[PIDTYPE_MAX];
    struct rcu_head rcu;
    struct upid numbers[1];
};
  
```

Bu versiyonlardaki durumu şöyle özetleyebiliriz:

1. Çekirdek her pid için bir pid nesnesi (struct pid nesnesi) oluşturup onu bir hash tablosunda saklamaktadır. Yani pid nesneleri için bir hash tablosu kullanılmıştır.
2. pid yapısının içerisindeki tasks elemanı o pid değerine ilişkin bağlı liste zincirlerinin kök düğümlerini tutmaktadır.
3. Sistem bir pid değerine ilişkin task_struct nesnesini elde etmek için önce o pid değerine ilişkin pid nesnesini hash tablosundan bulmakta, sonra onun içerisindeki ilgili bağlı listede arama yapmaktadır.

2.6 çekirdekleriyle birlikte Linux'a *isim alanları* (name spaces) kavramı da sokulmaya başlanmıştır. Bu versiyonlar artık pid değerleri için bir isim alanı da kullanmaktadır. Pid isim alanı sayesinde sanki çekirdek birden fazla pid dünyasına sahip gibi bir etki oluşturulmaktadır. Pid isim alanları iç içe de oluşturulabilmektedir. Artık bu versiyonlarla birlikte pid değerleri sistem genelinde tek değil isim alanı genelinde taktır. Linux çekirdeğine eklenen bu isim alanları özelliği *docker* gibi container teknolojilerinin gelişmesine katkı sağlamıştır.

4.14.2 pid_type Enum Değerleri ve Bağlı Listeler

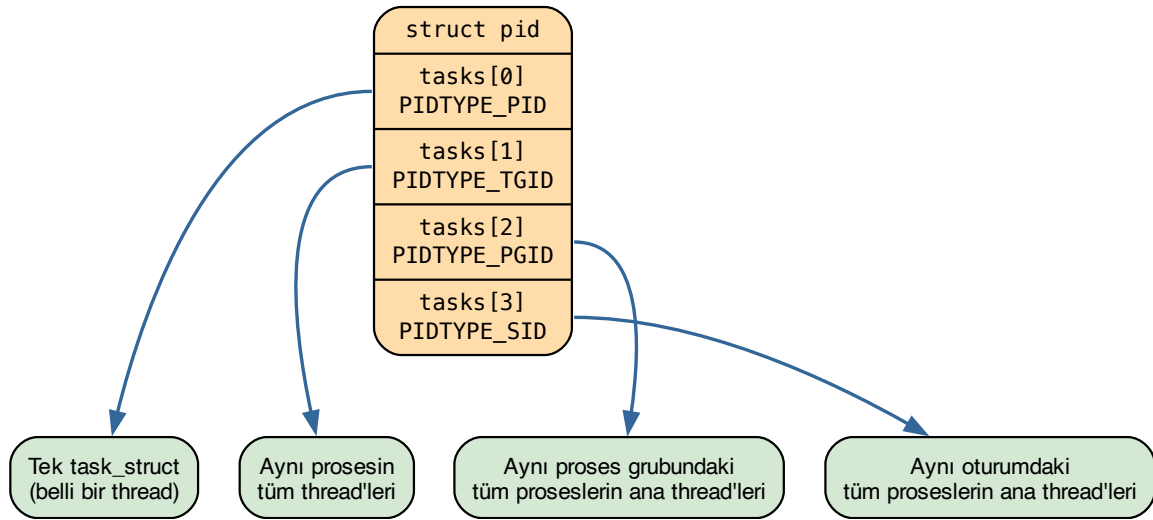
pid yapısındaki tasks dizisinin PIDTYPE_MAX kadar elemana sahip olduğunu görüyorsunuz. PIDTYPE_MAX aşağıdaki gibi bir enum türünün elemanıdır:

```

/* 2.6'lı versiyonlar */
enum pid_type {
    PIDTYPE_PID,
    PIDTYPE_PGID,
    PIDTYPE_SID,
    PIDTYPE_MAX    /* = 3 */
};

/* Güncel versiyonlar */
enum pid_type {
    PIDTYPE_PID,
    PIDTYPE_TGID,
    PIDTYPE_PGID,
    PIDTYPE_SID,
    PIDTYPE_MAX    /* = 4 */
};
  
```

Daha sonra PIDTYPE_TGID ile temsil edilen bir bağlı listenin daha eklendiğine dikkat ediniz. Peki pid değerini saklamak için neden birden fazla bağlı liste kullanılmaktadır? İşte 2.6'lı çekirdeklerle birlikte bu tarz aramalarda hız kazancı sağlamak için tasarım değiştirilmiştir. Aşağıdaki diyagram her listenin hangi task_struct nesnelerini tuttuğunu göstermektedir:

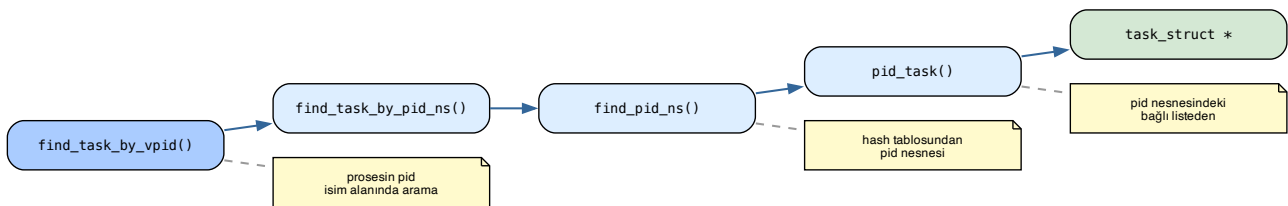


Bu bağlı listeleri daha ayrıntılı açıklayalım:

- **PIDTYPE_PID:** Bu bağlı liste zincirinde aslında tek bir eleman vardır. Amacımız yalnızca belli bir pid değerine ilişkin `task_struct` nesnesinin adresinin bulunmasıysa hemen bu listenin ilk elemanını alabiliriz.
- **PIDTYPE_TGID:** Bu bağlı liste belli bir prosesin thread'lerini dolaşmak için kullanılmaktadır. Bu bağlı listedeki tüm `task_struct` nesneleri aynı prosesin thread'lerini oluşturmaktadır. Yani biz çekirdeğe bir prosesin ana thread'ine ilişkin bir pid değeri verdiğimizde çekirdek bu bağlı liste zincirini dolaşarak o prosesin tüm thread'lerinin `task_struct` adreslerini elde edebilmektedir.
- **PIDTYPE_PGID:** Aynı proses grubuna ilişkin proseslerin ana thread'lerinin `task_struct` nesnelerini tutmaktadır. Yani aranan pid değeri bir proses grub liderine ilişkin pid belirtiyorsa bu zincirde o proses grubundaki tüm proseslerin ana thread'lerinin `task_struct` nesnelerinin adresleri bulunmaktadır.
- **PIDTYPE_SID:** Belli bir oturuma (session) ilişkin tüm proseslerin (onların ana thread'lerinin) `task_struct` nesnelerinin adreslerini tutmaktadır.

2.6 versiyonlarından önce tek bir hash tablosu vardı. Bu nedenle proses grubundaki proseslerin `task_struct` nesneleri ancak tüm `task_struct` nesneleri dolaşarak elde edilebiliyordu. Halbuki 2.6'daki bu tasarımla bu biçimdeki dolaşımlar bu hash tablosu ve bağlı listeler yoluyla çok daha hızlı yapılabilmektedir.

Aşağıda 2.6 versiyonlarında pid değerinden `task_struct` nesnesine ulaşım çağrı zinciri görülmektedir:



Hash tablosunda pid nesnesini arayan `find_pid_ns` fonksiyonu, `upid` yapı nesnelerini dolaşmakta ve hem pid değerine hem de pid isim alanına birlikte bakmaktadır:

```

struct upid {
    int nr;
    struct pid_namespace *ns;
    struct hlist_node pid_chain;
};

static struct hlist_head *pid_hash;    /* global hash tablosu */

struct pid *find_pid_ns(int nr, struct pid_namespace *ns)
{
    struct hlist_node *elem;
    struct upid *pnr;

    hlist_for_each_entry_rcu(pnr, elem,
        &pid_hash[pid_hashfn(nr, ns)], pid_chain)
        if (pnr->nr == nr && pnr->ns == ns)
            return container_of(pnr, struct pid,
                numbers[ns->level]);

    return NULL;
}

```

Burada hlist_node bağının upid nesnesinin içerisinde olduğuna, upid nesnesinin de pid nesnesinin içerisinde olduğuna dikkat ediniz.

4.14.3 4.20 Versiyonu: Hash Tablosundan XArray'e Geçiş

Çekirdek versiyonu 4.20'lere geldiğinde yukarıdaki mekanizmada önemli değişiklikler yapılmıştır. En önemli değişiklik pid ve isim alanından hareketle pid nesnesinin bulunması işleminde hash tablosu yerine *radix ağacının* kullanılmaya başlanmasıdır. Aslında 2.5 ile birlikte çekirdekte başka alt sistemlerde radix ağacı kullanılmaya başlanmıştı. Sonra bu radix ağaçlarının 4.20 versiyonuyla birlikte XArray isimli yeni bir gerçekleştirimi yapılmıştır.

4.20 ve sonrasında pid yapısındaki bağlı listelerde bir farklılık yoktur. Yalnızca pid ve isim alanı bilgisinden hareketle pid yapı nesnesinin elde edilmesinde hash tablosu yerine radix ağacı (XArray gerçekleştirimi) kullanılmaktadır.

Bu bağlamda pid mekanizmasında radix ağacının hash tablosuna tercih edilmesinin tipik nedenleri şunlardır:

- Radix ağacındaki arama hash tablolarından yavaş gözüксе bile aslında pratikte durum tam böyle değildir. Buradaki radix ağacı seyrek tutulmaktadır. Bu yüzden arama hızlı yapılır.
- pid'leri artan sırayla gezmek, boş pid bulmak (en küçük kullanılmayan pid'i seçmek) çok daha kolaydır.
- Ağaçta yalnızca kullanılan düğümler tutulmaktadır. Bu da bellek kazancı sağlamaktadır.
- pid sayısı arttıkça hash tablosunun performansı düşme eğilimi gösterdiği halde radix ağacının performansı hash tablosuna kıyasla düşmez.
- Radix ağacı yalnızca pid araması için değil, aynı zamanda yeni yaratılan task_struct nesneleri için pid tahsis edilmesinde de kullanılabilir.

Güncel çekirdeklerde hem pid aramaları hem de boş pid değerlerinin tespit edilmesi işlemleri bu XArray gerçekleştirimi ile sağlanmaktadır.

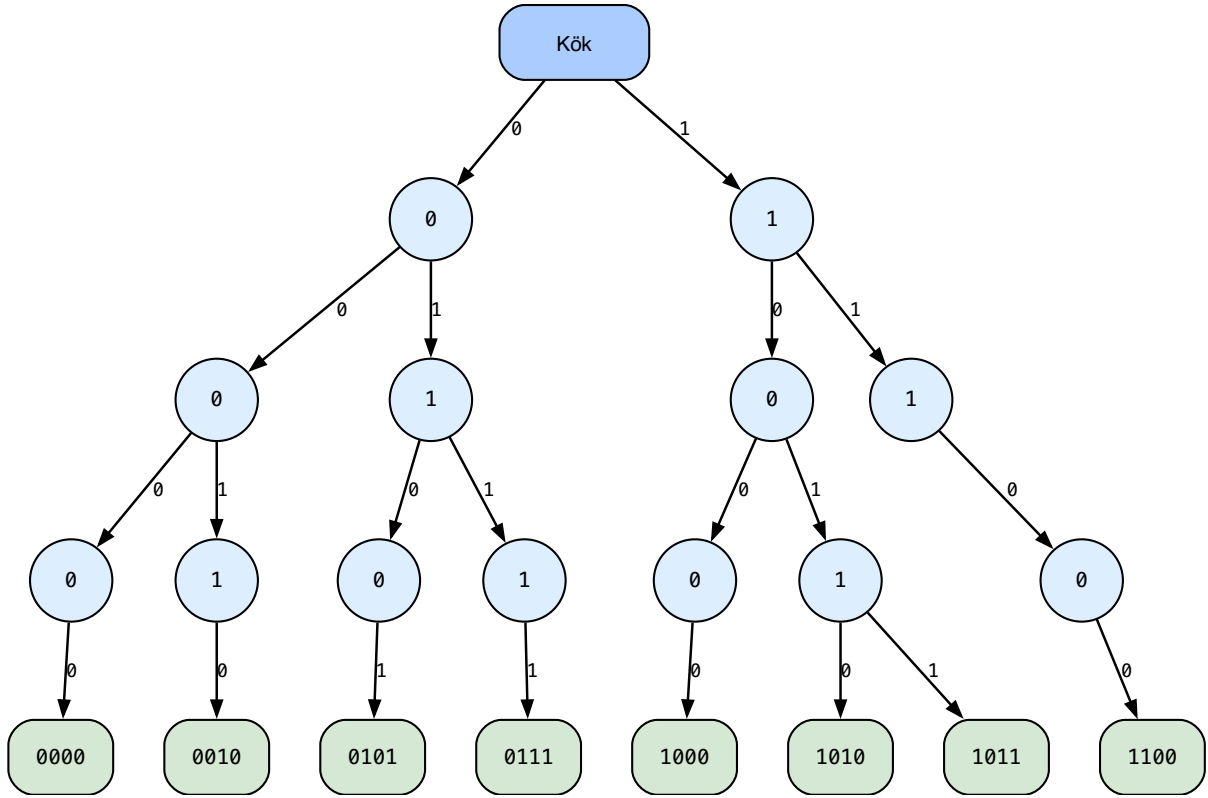
4.14.4 Radix Ağaçlarının Kullanılması

Biz radix arama ağacını ve XArray gerçekleştirimini daha sonra başka bir bölümde ele alacağız. Burada yalnızca radix arama ağacının temel mekanizması üzerinde açıklamalar yapacağız. Radix arama ağaçları için *dijital arama ağaçları* (*digital search tree*), *önek arama ağacı* (*prefix search tree*), *sıkıştırılmış arama ağaçları* (*compressed search trie*) gibi isimler de kullanılmaktadır.

Bu arama ağaçlarında düğümlerde anahtar olarak anahtarın tamamı değil onun ön kısımları kullanılmaktadır. Böylece ağaç bir leksikografik sıralama ağacı gibi de kullanılabilir. Tipik anlatım sayısal değerlerden oluşan ve öneklerin bit belirttiği anahtarlar üzerinde yapılmaktadır. Ağacın kökünden girilir. Her kademede bir bit daha anahtara eklenir.

Örneğin 4 bitlik şu sayıların radix ağacına yerleştirilmek istendiğini düşünelim: 1100, 1010, 0101, 1011, 0010, 0111, 0000, 1000

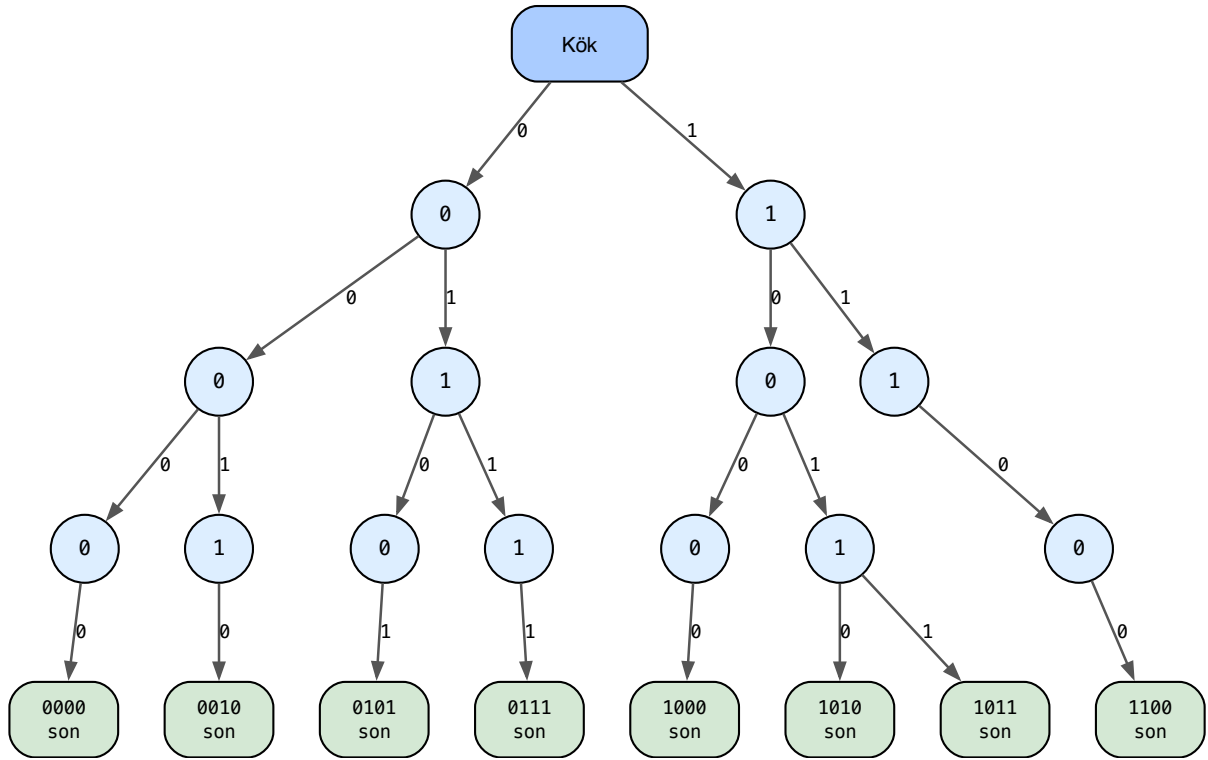
Ağacın her adımında yüksek anlamlı bitten başlayarak solda 0, sağda 1 dallanması yapılmaktadır. Aşağıdaki diyagram bu sekiz anahtarın ağaca yerleşimi sonucunu göstermektedir:



Peki bu ağaçta arama nasıl yapılır? İşte arama için kökten girilir. Her bit pozisyonu için bir aşağıya inilir. Örneğin 0000 değerini arayacak olalım: ilk bit 0 olduğu için kökten sola ineriz, ikinci bit de 0 olduğu için sola devam ederiz, üçüncü bit de 0 olduğu için sola, dördüncü bit de 0 olduğu için sola gideriz. Artık değeri buluruz. Radix ağaçlarını enlemesine (breadth-first gibi) dolaştığımızda anahtarların sıralı bir biçimde elde edilebildiğine dikkat ediniz.

4.14.5 Yapraklarda Değer Saklama Yöntemi

Biz yukarıdaki anlatımda düğümlerde anahtarların tutulduğunu varsaydık. Aslında düğümlerde anahtarların tutulmasına da gerek yoktur. Zaten her kademedeki bir bit ilerlendiğine göre arama yaprağa gelindiğinde sonlandırılabilir. Aşağıdaki ağaçta bit-bit dallanma yapılmış ve yalnızca yapraklarda değerler tutulmuştur:



Görüldüğü gibi bu tasarımda hiç anahtar saklanmadan yaprak görülene kadar ilerlenirse ya ilgili anahtar bulunmuş olur ya da bulunmamış olur. Peki her düğümde anahtarı tutmakla yalnızca yapraklarda değeri tutmanın hangisi daha iyi bir yöntemdir?

Yöntem	Avantajları	Dezavantajları
Her düğümde anahtar saklama	Erken durabilme; ara düğümde tam eşleşme bulunca yaprağa kadar inmek gerekmez. Prefix aramaları daha kolay.	Veri yapısı karmaşıklaşır; bellek kullanımı artar.
Yalnızca yapraklarda değer saklama	Basit tasarım: iç düğümler yalnızca yönlendirme bilgisi tutar. Bellek daha düzenli kullanılır. Ekleme/silme algoritmaları daha etkin.	Her zaman yaprağa kadar inmek gerekir. Prefix tabanlı aramalarda ek iş yapılması gerekir.

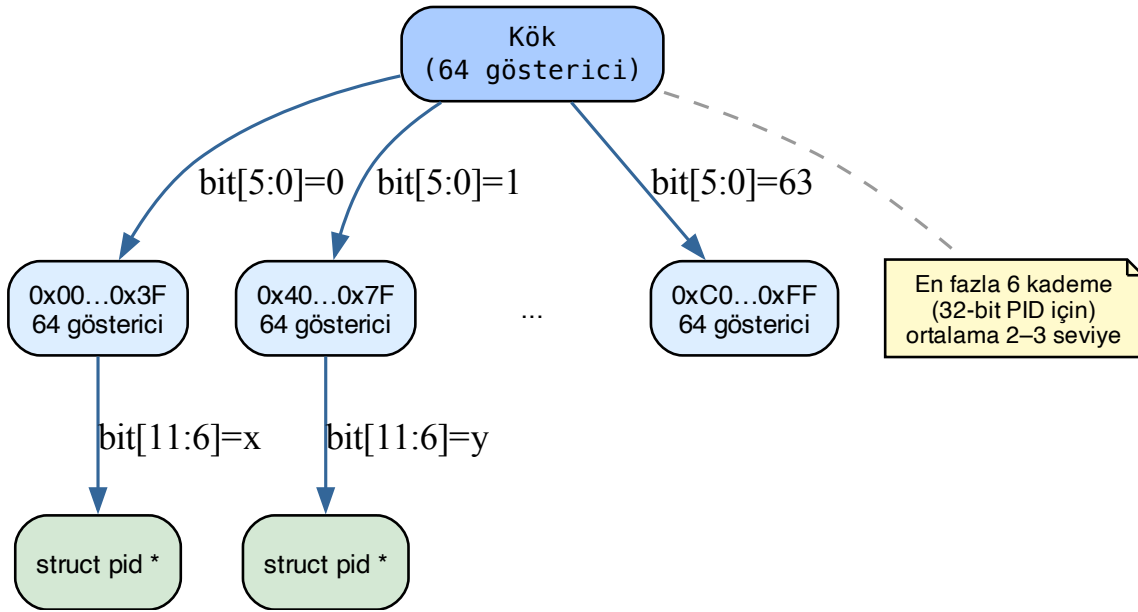
Not

Linux'un klasik radix ve XArray gerçekleştirmelerinde değerler her zaman yapraklarda tutulmaktadır. Ara düğümlerde anahtarlar tutulmamaktadır.

4.14.6 Dallanma Faktörü: n-li Ağaç

Radix ağacının ikili ağaç olması zorunlu değildir. Dallanma bit bit yapıldığında yukarıdaki örnekte olduğu gibi radix ağacı ikili ağaç durumunda olur. Ancak dallanma n bit n bit de yapılabilir. Bu durumda ağaç da 2^n -li ağaç olacaktır.

Örneğin 32 bitlik pid değerlerini böyle bir ağaçta tutmak isteyelim ve dallanmayı 6 bit 6 bit yapalım. 6 bit $\rightarrow 2^6 = 64$ farklı dallanmaya yol açacaktır. Bu durumda ağacımızın her düğümünde 64 gösterici bulunacaktır ve yapraklara varabilmek için en fazla 6 kademe ilerlenecektir:



Güncel sürümlerde Linux'un pid araması için kullandığı XArray ağacı bu örnekteki gibi altışar bitlidir.

Hash tablosu ile XArray/Radix ağacı arasındaki avantaj ve dezavantajları karşılaştıralım:

Kriter	Hash Tablosu	XArray / Radix Ağacı
Arama karmaşıklığı	Ortalama $O(1)$; çakışmada en kötü $O(N)$	$O(\log_k N)$, tipik derinlik 2–3
Bellek kullanımı	Tablo önceden sabit boyutta ayrılır; seyrek pid'lerde boşa gider	Yalnızca kullanılan düğümler açılır; seyrek alanda çok verimli
Çakışma	Mümkün; performansı düşürebilir	Yok; her pid tek bir yolu izler
Ölçeklenebilirlik	Tablo boyutunu baştan iyi seçmek gerekir	pid alanı büyüse bile uyum sağlar
Sıralı gezinme	Doğal sıralama yok; yavaş	Doğası gereği sıralı; çok hızlı
Boş pid bulma	Ayrı veri yapısı gerektirir	Aynı ağaç hem arama hem tahsisat için kullanılır

Özetle: tekil arama için hash tablosu genellikle biraz daha hızlıdır; ancak büyük ve seyrek pid isim alanlarında XArray çok daha verimli bellek kullanır. Sıralı gezinme ve boş pid bulma gerektirdiğinde XArray üstün gelir. Güncel çekirdeklerde hem pid aramaları hem de boş pid değerlerinin tespit edilmesi işlemleri bu XArray gerçekleştirimi ile sağlanmaktadır.

4.14.7 Güncel Çekirdek: pid Araması (4.20+)

Çekirdeğin 4.20 versiyonundan itibaren her pid isim alanı için ayrı bir XArray ağacı oluşturulmaya başlanmıştır. Anımsanacağı gibi 2.6'lı versiyonlarda hash tablolarına geçildiğinde tüm pid isim alanları aynı hash tablosunda bulunuyordu. Halbuki güncel çekirdeklerde bir pid nesnesi aranacaksa o nesne belli bir pid isim alanındaki XArray ağacında aranmaktadır.

4.20 ve sonrasında task_struct yapısında thread_pid elemanı da eklenmiştir:

```

struct task_struct {
    /* ... */
    pid_t    pid;
    struct pid *thread_pid; /* pid değerine ilişkin pid nesnesinin adresi */
}
  
```

(sonraki sayfaya devam)

```
/* ... */
};
```

Güncel pid yapısı şöyledir:

```
struct pid {
    refcount_t    count;
    unsigned int  level;
    spinlock_t    lock;
    struct {
        u64        ino;
        struct rb_node pidfs_node;
        struct dentry *stashed;
        struct pidfs_attr *attr;
    };
    struct hlist_head tasks[PIDTYPE_MAX];
    struct hlist_head inodes;
    wait_queue_head_t wait_pidfd;
    struct rcu_head rcu;
    struct upid      numbers[]; /* isim alanı bilgisi */
};
```

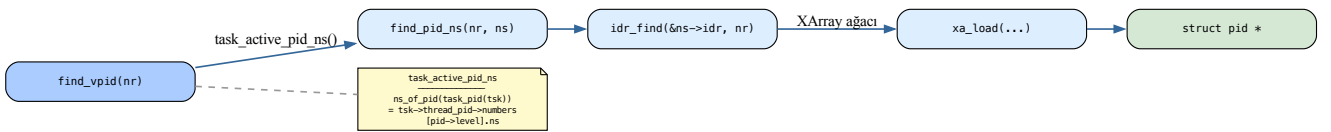
Güncel versiyonlarda upid yapısı şöyledir (hlist_node bağı kaldırılmış, ns bilgisi korunmuştur):

```
struct upid {
    int        nr;
    struct pid_namespace *ns;
};
```

Güncel çekirdeklerde pid araması yapan yüksek seviyeli fonksiyonlardan biri *find_vpid* fonksiyonudur:

```
struct pid *find_vpid(int nr)
{
    return find_pid_ns(nr, task_active_pid_ns(current));
}
EXPORT_SYMBOL_GPL(find_vpid);
```

Bu fonksiyon belli bir pid değerine ilişkin pid nesnesini çağırıp yapan prosesin bulunduğu pid isim alanı içerisindeki XArray ağacında aramaktadır. Aşağıdaki diyagram güncel çekirdekteki çağrı zincirini göstermektedir:



Prosesin içinde bulunduğu pid isim alanının bilgisine ulaşım çağrı zinciri şöyledir:

```
struct pid_namespace *task_active_pid_ns(struct task_struct *tsk)
{
    return ns_of_pid(task_pid(tsk));
}

static inline struct pid *task_pid(struct task_struct *task)
{
    return task->thread_pid;
}
```


(önceki sayfadan devam)

```
static inline struct pid_namespace *ns_of_pid(struct pid *pid)
{
    struct pid_namespace *ns = NULL;
    if (pid)
        ns = pid->numbers[pid->level].ns;
    return ns;
}
```

XArray ağacında arama yapan *find_pid_ns* fonksiyonu artık son derece sade bir görünümündedir:

```
struct pid *find_pid_ns(int nr, struct pid_namespace *ns)
{
    return idr_find(&ns->idr, nr);
}
EXPORT_SYMBOL_GPL(find_pid_ns);
```

Ağaçta arama yapan asıl fonksiyon *idr_find* isimli fonksiyondur.

4.14.8 XArray Gerçekleştirimi

XArray veri yapısının aşağı seviyeli gerçekleştirimi `include/linux/xarray.h` dosyası içerisinde yapılmıştır. XArray ağacı bu dosya-daki *xarray* isimli yapıyla temsil edilmiştir:

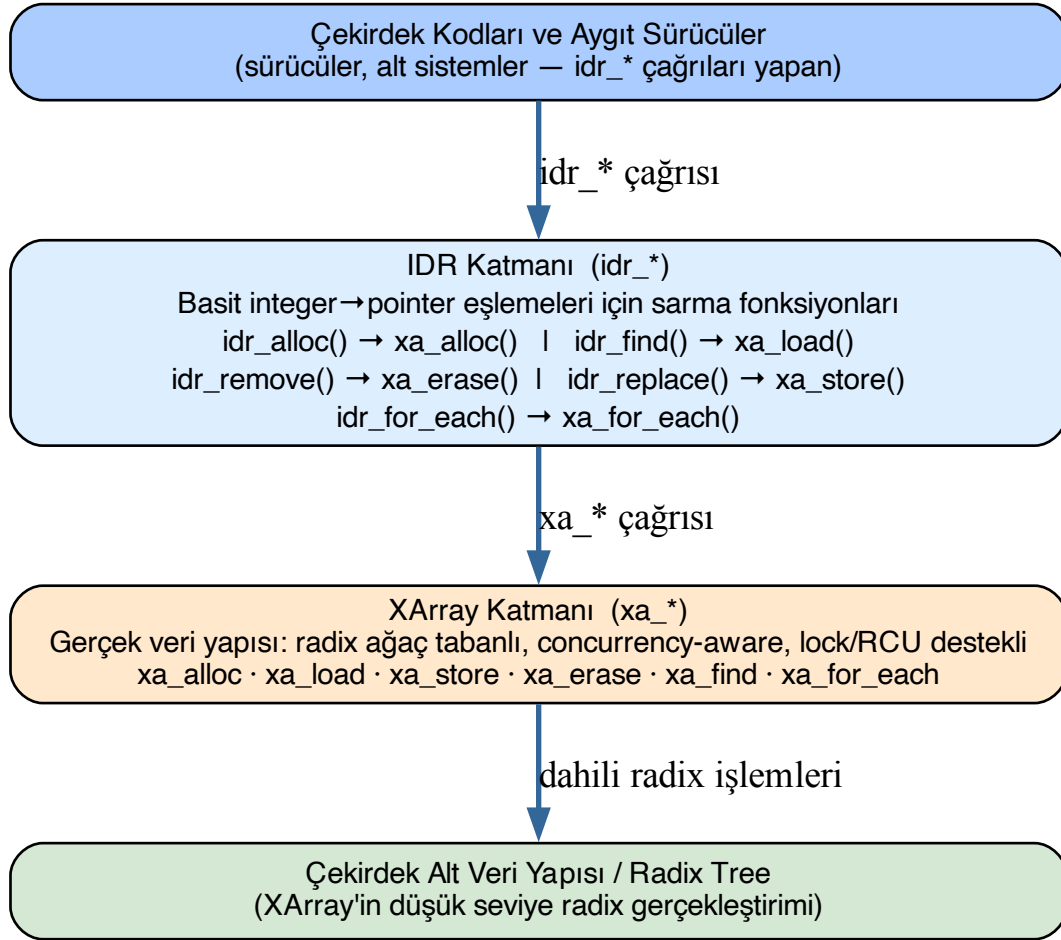
```
struct xarray {
    spinlock_t    xa_lock;
    /* private: */
    gfp_t         xa_flags;
    void __rcu    *xa_head;
};
```

Bu ağaç üzerinde işlem yapan *xa_* öneki ile başlayan bir grup fonksiyon vardır:

```
void *xa_load(struct xarray *, unsigned long index);
void *xa_store(struct xarray *, unsigned long index, void *entry, gfp_t);
void *xa_erase(struct xarray *, unsigned long index);
void *xa_store_range(struct xarray *, unsigned long first,
                    unsigned long last, void *entry, gfp_t);
bool xa_get_mark(struct xarray *, unsigned long index, xa_mark_t);
void xa_set_mark(struct xarray *, unsigned long index, xa_mark_t);
void xa_clear_mark(struct xarray *, unsigned long index, xa_mark_t);
void *xa_find(struct xarray *, unsigned long *index,
             unsigned long max, xa_mark_t);
void *xa_find_after(struct xarray *, unsigned long *index,
                  unsigned long max, xa_mark_t);
unsigned int xa_extract(struct xarray *, void **dst,
                     unsigned long start, unsigned long max,
                     unsigned int n, xa_mark_t);
void xa_destroy(struct xarray *);
```

Bu fonksiyonların tanımlamaları `lib/xarray.c` dosyasında yapılmıştır.

Güncel çekirdeklerin yukarıda belirttiğimiz aşağı seviyeli fonksiyonlarının yanı sıra aynı zamanda bunları kullanan yüksek seviyeli *IDR* fonksiyonları da bulunmaktadır. Çekirdek kodları incelendiğinde genellikle bu yüksek seviyeli *idr_* önekli fonksiyonların kullanıldığı görülmektedir. Aşağıdaki diyagram üç katman arasındaki ilişkiyi göstermektedir:



Boş pid değerinin elde edilmesi de aynı XArray altyapısı üzerinden yapılmaktadır:

```
/* Boş pid tahsisatı çağrı zinciri */
alloc_pid --> idr_alloc_cyclic --> idr_alloc_u32 --> idr_get_free
```

XArray gerçekleştirimi hakkında ileride başka konularda ek bilgiler vereceğiz.

Linux Çekirdeğinde Dosya Sistemi - I. Bölüm

Bu bölümde dikkatimizi dosya sistemine ilişkin çekirdek veri yapıları üzerine yönelteceğiz. Bilindiği gibi UNIX/Linux sistemlerinde pek çok kavram kullanıcıya bir dosya gibi gösterilmektedir. Biz bu birinci bölümde belli bir derinliğe kadar çekirdeğin dosya işlemleri için oluşturduğu organizasyon üzerinde duracağız. Daha sonra ikinci bölümde dosya sistemine ilişkin aşağı seviyeli ayrıntıları ele alacağız.

5.1 Giriş

İşletim sistemlerinin *dosya sistemi* (*file system*) denilen alt sistemlerinin iki tarafı vardır: **Disk tarafı** ve **bellek tarafı**. Dosya bilgileri disk üzerindeki bloklarda tutulmaktadır. (Bu bloklara Microsoft dünyasında *cluster* da denilmektedir.) Hangi dosyaların diskin hangi bloklarında tutulduğu, dosyaların metadata bilgilerinin diskte nasıl saklandığı gibi belirlemeler dosya sisteminin disk tarafını; diskteki dosya sisteminin çekirdekteki temsilinin oluşturulması ve işletim sisteminin açılan dosyalar için yaptığı düzenlemeler ise dosya sisteminin bellek tarafını oluşturmaktadır.

5.2 Disk Aktarımına İlişkin Temel Bilgiler

Biz kursumuzda “disk” terimini ikinci bellekleri belirten genel bir terim olarak kullanacağız. Bir süre önceye kadar disk olarak ağırlıklı biçimde *hard disk* denilen elektromekanik birimler kullanılıyordu. Ancak bir süredir artık disk olarak yarı iletken teknolojiler kullanılarak oluşturulmuş *SSD* (*Solid State Disk*) denilen diskler kullanılmaktadır. Bugün ağırlıklı olarak kullandığımız SSD disklerin herhangi bir mekanik parçası yoktur. SSD’ler *NAND Flash* denilen bellek teknolojisini kullanmaktadır. SSD’ler hard disklere göre oldukça hızlıdır. Ancak onların en önemli handikapları belli bir yazma ömrünün olmasıdır. SSD’lerde aynı bölgeye belli sayıdan daha fazla yazma yapıldığında artık SSD’nin o bölgesi bozulabilmektedir. Tabii teknoloji bu bakımdan da ilerleme içerisinde. SSD teknolojisi ile USB yuvalarına taktığımız flash belleklerin teknolojisi birbirine benzemektedir.

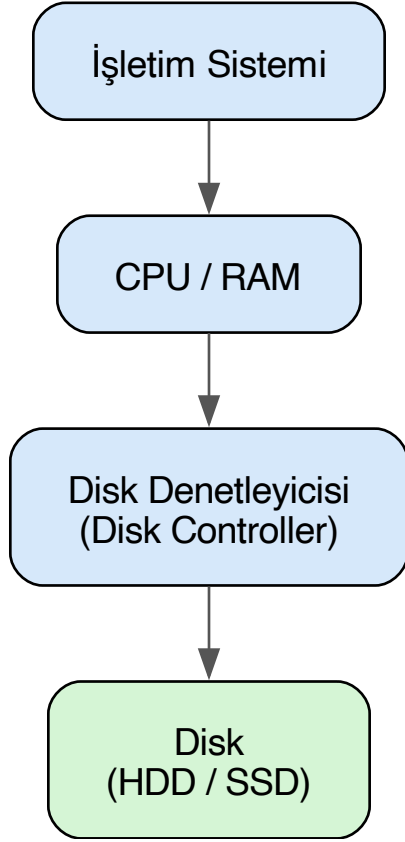
5.2.1 Sektörler ve Disk Denetleyicisi

Kullandığımız disk birimi ister *hard disk* olsun isterse SSD olsun disk ile bilgisayarımızın RAM’i arasındaki transferler *sektör* (*sector*) denilen byte blokları düzeyinde yapılmaktadır. Sektör bir diskten okunabilecek ya da bir diske yazılabilecek en küçük birimdir. Bir sektör tipik olarak 512 byte’tır. Diskte byte düzeyinde erişim yoktur. Sektörel erişim vardır. Örneğin diskteki bir sektörde bulunan bir byte üzerinde değişiklik ancak şöyle yapılabilir: Önce o byte’ın içinde bulunduğu sektör RAM’e okunur. Sonra byte RAM üzerinde değiştirilir. Sonra aynı sektör yeniden diske yazılır.

Diskteki her sektörün ilk sektör 0 olmak üzere bir mantıksal numarası vardır. Hard disklerde ardışıl numaralı mantıksal sektörler disk üzerinde de fiziksel olarak peşi sıra bulunmaktadır. Mekanik hard disklerde bilgiler *track* denilen yollara yazılmaktadır. Ardışıl sektörler aynı track’te bulunurlar. Dolayısıyla hard disklerde diskin kafası bir kez konumlandırıldığında ardışıl sektörlerle daha hızlı okuma yazma yapılabilir. SSD’ler mekanik öge barındırmadığı için rastgele erişimlidir. Yani her sektörden okuma aynı hızda yapılmakta ve her sektöre yazma da aynı hızda yapılmaktadır.

Modern bilgisayar sistemlerinde disk birimine doğrudan erişilmez. Disk erişimlerinde bu işleme aracılık eden ismine *disk denetleyicisi* (*disk controller*) denen yerel bir işlemciden faydalanılmaktadır. Yani sistem programcıları ya da işletim sistemlerini yazarlar disk de-

netleyicisini programlar, disk denetleyicisi isteği elektriksel olarak disk birimine iletir, okuma yazma işlemleri de disk birimi tarafından yapılır:



Bugünkü masaüstü bilgisayarlarımızda *SATA* ve *NVMe* en çok kullanılan disk denetleyicileridir.

5.2.2 DMA (Direct Memory Access)

Peki işletim sistemi tarafından disk denetleyicisi “falanca sektörleri oku” ya da “falanca sektörlerle yaz” biçiminde programlandıktan sonra aktarım nasıl yapılmaktadır? Aktarım CPU tarafından tek tek byte’ların denetleyiciden alınarak RAM’e yerleştirilmesi yoluyla yapılmamaktadır. (Çok eskiden ilk PC mimarilerinde aktarım böyle de yapılabilirdi.) Çünkü CPU’nun bu işle meşgul olması önemli bir zaman kaybı oluşturmaktadır. Bu tür disk ile RAM arasındaki aktarımlar için *DMA (Direct Memory Access)* denilen yardımcı denetleyiciler kullanılmaktadır.

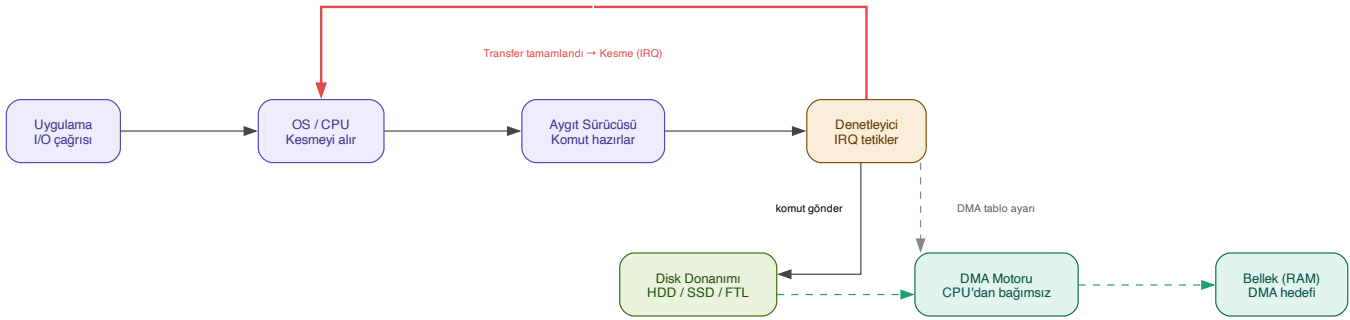
Tipik olarak CPU’da çalışan kod (yani işletim sistemi) disk denetleyicisine transfer isteğini ve aktarımda kullanılacak bellek alanlarının adresini bildirir. Disk denetleyicisi de disk birimini ve DMA’yı elektriksel düzeyde programlayarak aktarımın DMA üzerinden doğrudan RAM’e yapılmasını sağlar. Aktarım sırasında artık CPU bu işle meşgul olmaz, işletim sistemi de CPU’yu başka bir thread’i çalıştırması için *bağlamsal geçişe (task switch)* sokar. Tabii aktarım işlemi bittiğinde disk denetleyicisi CPU’yu bir donanım kesmesi yoluyla durumdan haberdar etmektedir.

Yani disk ile RAM arasındaki aktarım işlemleri tipik olarak şöyle yapılmaktadır:

1. İşletim sistemi disk denetleyicisine aktarılabacak sektörlerle ilişkin bilgileri ve transfer adreslerini CPU yoluyla elektriksel olarak iletir.
2. Okuma söz konusuysa disk denetleyicisi disk birimine elektriksel düzeyde komutlar göndererek sektörlerin okunmasını ve DMA yoluyla bunların RAM’de uygun yerlere aktarılmasını sağlar. Eğer yazma söz konusuysa RAM’de belirtilen adresteki bilgiler yine DMA yoluyla disk birimine iletilerek yazma gerçekleştirilir.
3. Aktarım işlemi bittiğinde disk denetleyicisi bir donanım kesmesi yoluyla CPU’yu durumdan haberdar eder.

4. İşletim sistemi aktarım için gereken kodları çalıştırdıktan sonra aktarım bitene kadar meşgul bir döngüde beklemes. Başka thread'ler çalıştırılabilirse *bağlamsal geçiş (context switch)* oluşturarak CPU'nun boş biçimde beklemesinin önüne geçer.

Bu süreci aşağıdaki diyagram özetlemektedir:



Eskiden Intel tabanlı PC mimarisinde ISA bus kullanıldığı zamanlarda tek bir merkezi DMA denetleyicisi (Intel 8237) vardı. Ancak daha sonra PCI bus kullanılmaya başlanmasıyla birlikte artık transfer yapabilen her donanım birimi kendi DMA denetleyicisini de içermeye başladı. Bugün Intel tabanlı ve Apple Silicon tabanlı bilgisayar mimarilerinde disk denetleyicisi kendi içerisindeki DMA denetleyicisini programlayarak transferi gerçekleştirmektedir. Disk denetleyicilerinin programlanması ise artık uzunca bir süredir *bellekten tabanlı IO (memory-mapped IO)* tekniği ile yapılmaktadır.

Hard disklerde disk birimi içerisinde önbellekler (cache) de bulundurulmaktadır. Böylece disk denetleyicisi aynı sektörleri disk biriminden istediği zaman disk birimi eğer ilgili sektörler önbellek içerisindeyse hiç kafa hareketleri yapmadan onları doğrudan önbellekten verebilmektedir. Bugünlerde örneğin 1 TB'lık hard disklerde 64MB, 128, 256 MB civarında önbellekler kullanılmaktadır. Yalnızca hard disklerde değil SSD'lerde de bir önbellek sistemi vardır. SSD'lerdeki önbellek sistemi özellikle yazma işlemlerinde hız kazancı sağlamakta ve aynı sektörlerle sürekli yazım yapıldığında o bölgenin yıpranmasını (wear leveling) engellemektedir. Tabii bu önbellek sistemleri tamamen disk birimleri tarafından içsel olarak (built-in) işletilmektedir. Bu önbellek sistemleri işletim sistemleri tarafından erişilebilir değildir.

5.2.3 Blok Kavramı

Yukarıda da belirttiğimiz gibi bir disk biriminde transfer edilecek en küçük birime sektör denilmektedir. Bir sektör tipik olarak 512 byte uzunluğundadır. Ancak aslında sektör uzunlukları da disk üreticilerine bağlı olarak değişebilmektedir. 512 byte sektör uzunlukları bugün için standart bir uzunluktur. Tabii zaman geçtikçe diskler büyüdüğü için sektör uzunluklarının da büyüyebileceğini söylemek istiyoruz. Nitekim 4K uzunluğunda sektörlerle sahip olan diskler özellikle büyük sistemlerde gittikçe yaygınlaşmaktadır. Disk birimi her sektöre ilk sektör 0 olmak üzere mantıksal bir numara vermektedir. Yani adeta disk üzerindeki her sektörün bir adresi vardır. Disk denetleyicisi disk birimine transfer edilecek sektörlerin numaralarını elektriksel düzeyde iletmektedir. (Bu biçimde mantıksal sektör numaraları kullanılmadan önce 80'lerde ve 90'ların ilk yarısında sektörlerin yerleri "fiziksel koordinat sistemi" denilen "hangi yüz (head)", "hangi track", "hangi sektör dilimi" biçiminde üç parametreyle belirtiliyordu.)

Sektör kavramı aslında dosya sistemleri için küçük bir depolama birimidir. İşletim sistemleri bir dosyanın parçası olabilecek en küçük disk alanı için sektör yerine *blok (block)* ya da *cluster* denilen daha büyük birimleri kullanmaktadır. Blok terimi daha çok UNIX/Linux sistemlerinde kullanılmaktadır. Microsoft ise blok yerine *cluster* terimini kullanmaktadır. Bir blok ardışıl n tane sektörden oluşmaktadır. Uygulamada bu n değeri 2'nin bir kuvveti olur. Ardışıcılık hard disklerde önemli bir unsurdur. Çünkü hard disklerde en önemli zaman kaybı mekanik bir birim olan disk kafasının track hizasına çekilmesinde yaşanmaktadır. Disk kafası track hizasına çekildiğinde disk dönerken artık ardışıl sektörler hiç kafa hareketi yapılmadan okunup yazılabilmektedir. Peki neden işletim sistemi dosyalar söz konusu olduğunda bir dosyanın parçası olabilecek en küçük birim için sektör değil de ardışıl n tane sektör kullanmaktadır? İşte bunun birkaç nedeni vardır:

1. Dosyaların parçaları disk üzerinde ardışıl yerlerde olmak zorunda değildir. Eğer dosyalar çok fazla parçadan oluşursa hard disklerde (ve kısmen de olsa SSD'lerde de) bu parçalar disk üzerinde daha fazla yayılmış olur, bunlara erişmek için gereken zaman artar.
2. Eğer dosyanın parçaları sektör gibi küçük birimlerden oluşsaydı bu parçaların diskteki yerlerine ilişkin metadata tabloları büyürdü. Bu da hem disk alanını hem de işletim sisteminin bellekte yaptığı düzenlemede alan verimsizliği oluştururdu.
3. CPU'ların kullandığı sayfalama mekanizmasında genellikle 4K uzunluklar kullanılmaktadır. Dosya parçalarının 4K uzunluğun katlarında olması dosya sistemi ile sayfalama sistemi arasında daha iyi bir uyumun ortaya çıkmasına yol açmaktadır.

Peki bu durumda işletim sistemleri blok denilen dosyanın parçası olabilecek en küçük birim için hangi uzunluğu kullanmaktadır? İşte genellikle bu karar disk formatlanırken diskin (disk bölümünün) büyüklüğüne bakılarak verilmektedir. Dosyaların son bloklarında kalan kullanılmayan alanların oluşmasına *içsel bölünme (internal fragmentation)* denilmektedir. Küçük disklerde (disk bölümlerinde) içsel

bölünmenin etkisi daha büyük olacağından blokların 1K gibi küçük uzunluklarda alınması uygun olabilir. Ancak orta büyüklükte disklerde içsel bölünmenin etkisi görece olarak azalacağı için bloklar 4k gibi bir değerde seçilebilmektedir. Büyük disklerde ise 8K, 16K blok büyüklükleri tercih edilmektedir. Aslında blok büyüklükleri ilgili disk bölümü formatlanırken (Linux sistemlerinde `mkfs.xxx` programlarıyla formatlama yapılmaktadır) belirlenmektedir. Yani kullanıcı isterse kendisi bu programda kendi tercih ettiği blok uzunluğunu kullanabilir. Ancak kullanıcılar genellikle böyle bir belirleme yapmazlar. Bu durumda bu programlar disk bölümünün büyüklüğüne bağlı olarak yukarıda açıkladığımız gibi uygun bir blok büyüklüğünü seçerler.

5.2.4 Mount İşlemi

UNIX/Linux sistemlerinde dosya sistemi için tek bir kök vardır. Blok aygıtları (örneğin hard diskler, flash bellekler vb.) belli bir dizine mount edilmektedir. Mount işlemi bir dizin üzerine uygulanır; mount işlemi sonucunda o dizinin içeriği görünmez, artık mount edilen dosya sisteminin kök dizini mount dizininde gözükür. Dolayısıyla bu sistemlerde farklı dizinler farklı blok büyüklüklerine ilişkin dosya sistemlerinin içerisinde olabilmektedir.

Anımsanacağı gibi `stat` POSIX fonksiyonu ya da komut satırından uygulanan `stat` komutu belli bir dosyanın bilgilerini verirken o dosyanın içinde bulunduğu dosya sisteminin blok uzunluğunu da vermektedir. Linux sistemlerinde bu bilgi doğrudan dosya sistemine ilişkin blok aygıtı üzerinde `dumpe2fs` programıyla da elde edilebilmektedir.

Windows sistemlerinde de `cluster` adı altında blok sistemi kullanılmaktadır. O sistemlerde blok uzunluklarını `chkdsk` programı ile ya da `fsutil` programı ile komut satırından elde edebilirsiniz.

5.2.5 Blok Numaralandırması

İşletim sistemleri işlemlerini kolaylaştırmak için her bloğa bir numara da vermektedir. Örneğin bir bloğun 4K (tipik olarak 8 sektör) olduğunu düşünelim. İşletim sistemi için ilgili disk (aslında disk bölümü ya da genel olarak blok aygıtı da diyebiliriz) bloklardan oluşmaktadır. Örneğin diskin (disk bölümünün) ilk 8 sektörü artık 0'ıncı bloktur. Sonraki 8 sektör 1'inci bloktur.

İşletim sistemi içsel olarak artık ilgili diski bloklardan oluşan ve her bloğun bir numarasının olduğu mantıksal bir depolama alanı gibi ele almaktadır. Yani işletim sistemi için yalnızca sektörlerin değil aynı zamanda dosya sistemine ilişkin blokların da numaraları vardır.

5.2.6 Sayfa Önbelleği (Page Cache)

İşletim sistemleri son okunan ya da yazılan disk bloklarını RAM'de bir önbellek sisteminde saklamaktadır. Bu önbellek sistemine genel olarak *işletim sisteminin disk önbellek sistemi* denilmektedir. Linux dünyasında eskiden bu önbellek sistemine *buffer cache* deniliyordu. Sonra bu önbellek sistemi iyileştirildi ve ismi *page cache* olarak değiştirildi.

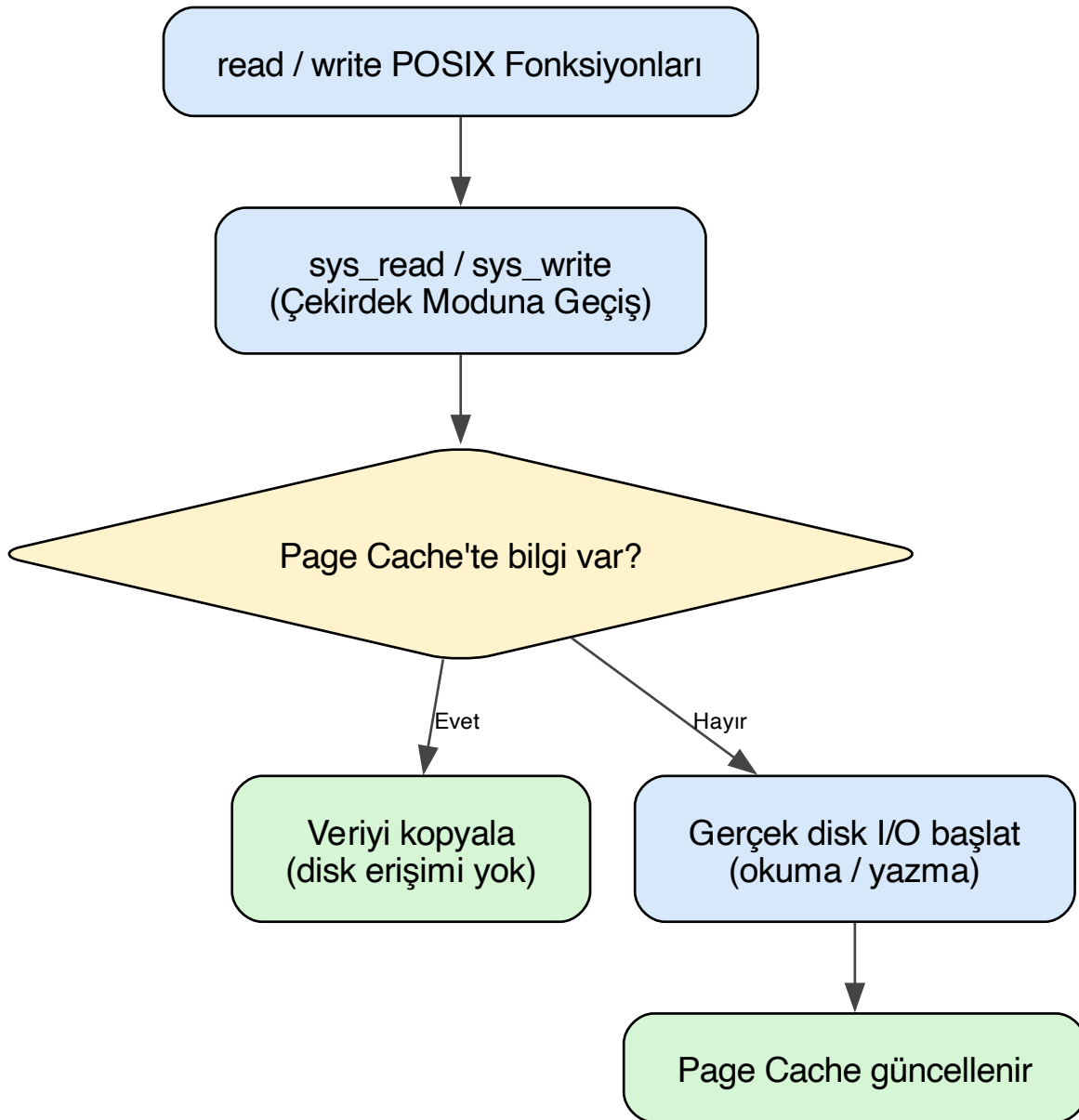
İşletim sistemlerinin bu disk önbellek sistemleri disk erişimini ciddi boyutta azaltmakta ve sistem performansı üzerinde en önemli olumlu etkilerden birini oluşturmaktadır. Eğer işletim sistemlerinde böyle bir disk önbellek sistemi olmasaydı sistemler çok yavaş çalışırdı.

5.3 Dosyalardan Okuma ve Yazma İşlemleri

Biz UNIX/Linux sistemlerinde bir dosyadan okuma yapmak için ya da bir dosyaya yazma yapmak için `read/write` POSIX fonksiyonlarını kullanmış olalım. Bu fonksiyonlar çekirdek içerisindeki `sys_read` ve `sys_write` sistem fonksiyonlarını çalıştırmaktadır.

İşletim sistemi bellek tarafında yaptığı organizasyonla okunacak ya da yazılacak bilginin ilgili blok aygıtının (disk bölümünün) kaç numaralı bloğuna ve sektörüne ilişkin olduğunu belirleyebilmektedir. Ancak `sys_read` ve `sys_write` gibi sistem fonksiyonları hemen diske yönelmez. Bu fonksiyonlar önce dosyanın ilgili bölümünün RAM'de oluşturulmuş bir sayfa önbelleği (*page cache*) içerisinde olup olmadığına bakmaktadır. Eğer ilgili bölüm bu önbellek sisteminin içerisinde varsa bu fonksiyonlar diske hiç erişmeden dolayısıyla da hiç bloke olmadan bu okuma yazma işlemini gerçekleştirmektedir.

`read/write` POSIX fonksiyonları çağrıldığında yapılan işlemler aşağıdaki diyagramda özetlenmiştir:



Peki dosyadaki okunacak ya da yazılacak kısım sayfa önbelleğinde (*page cache*) yoksa gerçek transfer nasıl yapılmaktadır? Linux sistemlerinde bu transferlerin yapıldığı birime *blok aygıt sürücüler* (*block device drivers*) denilmektedir.

Bir Linux sistemi kurulduğunda zaten temel disk denetleyicileri üzerinden transfer yapabilen blok aygıt sürücüler çekirdeğin içerisine gömülmüş durumda olur. Ancak sistem programcısının kendisi de blok aygıt sürücüler yazabilir. Örneğin bir gömülü Linux sisteminde yeni bir SD kart birimi için bir blok aygıt sürücüsü yazmak zorunda kalabilirsiniz.

İşletim sistemi bu IO isteklerini hemen blok aygıt sürücüsüne göndermez. Çünkü çok sayıda farklı proses aynı disk sektörlerini okuyacak ya da o sektörleri yazacak olabilir. İşletim sistemi önce istekleri sıraya dizer, mümkünse birleştirir, bu biçimdeki iyileştirme işleminden sonra istekleri blok aygıt sürücüsüne gönderir. Bu sürece *IO çizelgeleme* (*IO scheduling*) denilmektedir.

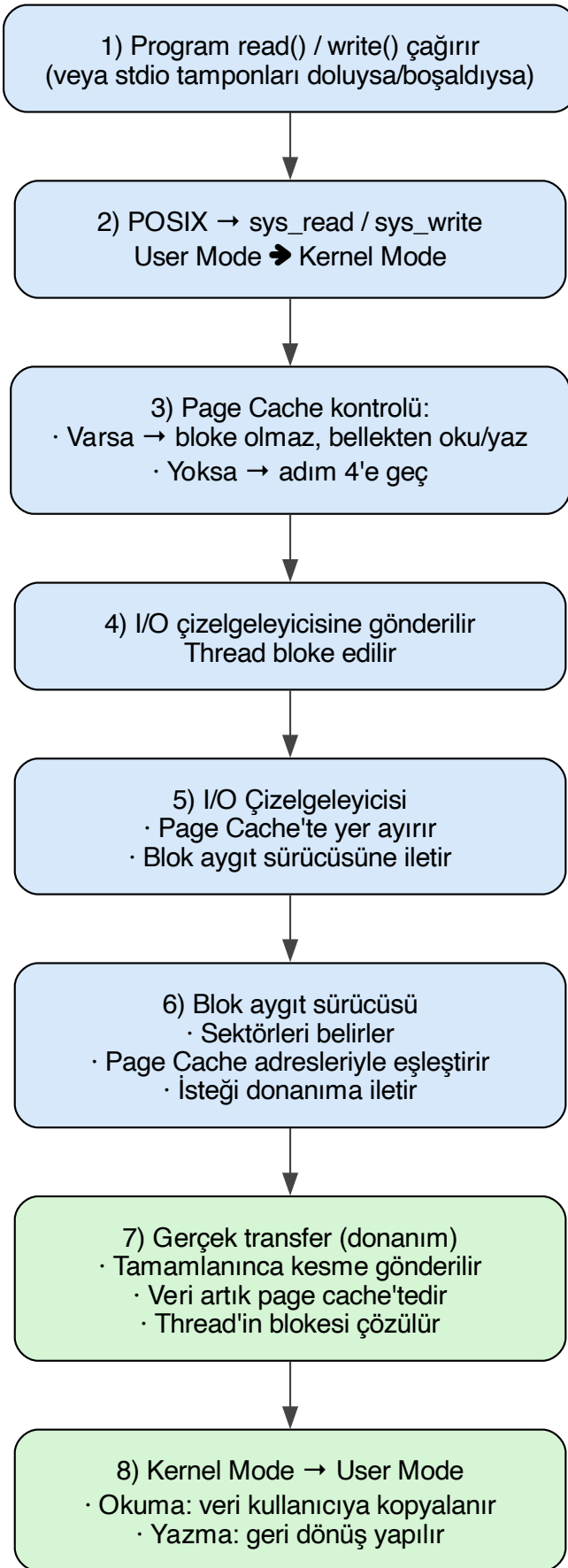
O halde bir dosya okuması ya da yazması sonucunda gelişen olayları şöyle özetleyebiliriz:

1. Kullanıcı modunda çalışan program (yani proses) `read` ya da `write` POSIX fonksiyonlarını çağırır. (UNIX/Linux sistemlerindeki C derleyicilerinin standart dosya fonksiyonları da eğer okunacak ya da yazılacak kısım kendi tamponlarında yoksa zaten bu POSIX fonksiyonlarını çağırır.)
2. `read` ve `write` POSIX fonksiyonları Linux'ta `sys_read` ve `sys_write` isimli sistem fonksiyonlarını çağırır. Artık akış kullanıcı

modundan (user mode) çekirdek moduna (kernel mode) geçmiştir.

3. `sys_read` ve `sys_write` fonksiyonları önce okunacak ya da yazılacak yerin Linux'un disk önbellek sistemi olan sayfa önbelleğinde (*page cache*, eski ismiyle *buffer cache*) olup olmadığına bakar. Eğer ilgili disk blokları sayfa önbelleğinde varsa akış hiç bloke olmadan sayfa önbelleği içerisinden karşılanır. Aksi hâlde Linux çekirdeği isteği *IO çizelgeleyicisi (IO scheduler)* denilen çekirdek birimine iletir; `read/write` çağrısını yapan thread bloke edilir.
4. IO çizelgeleyicisi istekleri çizelgeler. Okuma işlemi söz konusuysa sayfa önbelleğinde transfer edilecek önbellek bloklarını tahsis eder. Yazma işlemi söz konusuysa diske transfer edilecek önbellek bloklarını belirler. Blok aygıt sürücüsüne transfer edilecek sektörleri ve transfere ilişkin bellek adreslerini iletir.
5. Gerçek transfer işlemi blok aygıt sürücüsü tarafından yapılmaktadır. İşletim sistemi blok aygıt sürücüsüne hangi sektörlerin sayfa önbelleğindeki hangi adreslere (ya da tersi yönde) transfer edileceğini bir kuyruk sistemi yardımıyla iletmektedir.
6. Blok aygıt sürücüsü diskten istenen sektörleri sayfa önbelleği içerisinde belirtilen adrese ya da sayfa önbelleğindeki bilgileri diskin belirtilen sektörlerine transfer eder.
7. Artık okuma söz konusuysa okunan bilgi sayfa önbelleği içerisindeydir. İşlemi başlatan thread'in blokesi çözülür. `sys_read` sistem fonksiyonu bunu sayfa önbelleği içerisinden programcının kullanıcı modundaki adresine kopyalar.

Bu süreci aşağıdaki diyagram özetlemektedir:



Linux sistemlerinde yukarıda özetlediğimiz olaylar silsilesi zaman içerisinde değişikliklere uğratılarak ve sürekli geliştirilerek bugünkü durumuna getirilmiştir.

5.3.1 Yazma İşleminin Ayrıntıları

Buradaki süreçte yazma olayı söz konusu olduğunda bazı ayrıntılar da devreye girmektedir. Kullanıcı modundaki program `write` POSIX fonksiyonunu çağırıp bu fonksiyon da `sys_write` fonksiyonunu çağırdığında bu sistem fonksiyonu yazılmak istenen bilgiler diske yazılana kadar `write` işlemini yapan thread'i bloke etmez. Yazma işlemi her zaman Linux'un RAM'deki sayfa önbelleğine yapılmaktadır. `sys_write` fonksiyonu yazmayı sayfa önbelleği içerisine yaptıktan sonra hemen *başarılı* olarak geri dönmektedir.

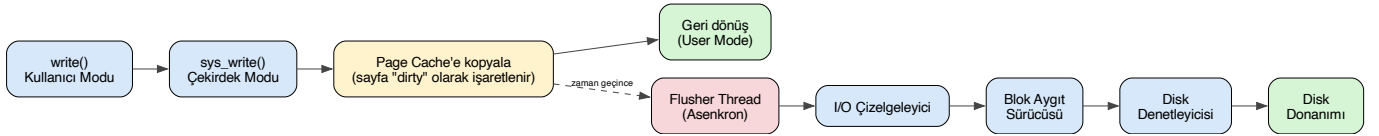
Sistem programlama terminolojisinde IO işlemlerinde *senkron* terimi *fonksiyon geri döndüğünde tüm işlemlerin yapılp bitmiş olması* anlamına gelmektedir. *Asenkron* terimi ise *işlemin başlatılması, fonksiyonun geri dönmesi ancak işlemin aslında arka planda devam etmesi* anlamına gelmektedir. Görüldüğü gibi modern işletim sistemlerinde diske yazma işlemi aslında *senkron* bir işlem değildir.

Ancak bunun bir istisnası vardır. Eğer bir dosya `O_DSYNC` ya da `O_SYNC` bayraklarıyla açılmışsa o dosyaya yapılan yazma işlemleri aygıt aktarılanaya kadar thread `write` fonksiyonunda bloke edilmektedir. Yani bu bayraklar yazma işlemlerinin senkron yapılmasını sağlamaktadır.

5.3.2 Gecikmeli Yazım ve Flush Thread'leri

Peki yazma işleminde sayfa önbelleğine yazılan bilgiler çekirdek tarafından ne zaman gerçek aygıt aktarılmaktadır? İşletim sistemi kasten bu tür durumlarda araya belli bir gecikme koymaktadır. Böylece peşi sıra yapılan `write` işlemlerinin tek tek gereksiz biçimde aygıt yapılması engellenir, bunlar biriktirilerek ve çizelgelenerek blok aygıt sürücüsüne aktarılır. Bu biçimdeki aktarmaya *gecikmeli yazım* (*delayed write*) da denilmektedir.

Buradaki süreci aşağıdaki şekille özetleyebiliriz:



Modern Linux sistemlerinde kirlenmiş sayfaların *flush* edilmesi *çekirdek thread'leri* (*kernel threads*) tarafından yapılmaktadır. Örneğin Linux sistemlerinde bu işlemlerden *flush* isimli çekirdek thread'leri sorumludur. Eskiden Linux çekirdeklerinin 2.6.32 versiyonuna kadar bu işlemler *pdflush* isimli tek bir çekirdek thread tarafından yapılyordu. Bu versiyondan sonra artık her blok aygıt sürücüsü için ayrı bir flush thread'i oluşturulmaya başlandı. Bu thread'leri komut satırında şu şekilde görüntüleyebilirsiniz:

```
$ ps -aux | grep flush
```

flush thread'leri arka planda sürekli olarak sayfa önbelleğini izler. Orada *kirlenmiş* (*dirty*) olan sektörleri ilgili blok aygıt sürücüsüne gönderir.

5.3.3 Flush Thread Parametreleri

Modern Linux sistemleri için yazma gecikmesi ortalama 5 saniye civarındadır. Ancak bu değerler değiştirilebilmektedir. Flush thread'lerinin parametreleri aşağıdaki tabloda özetlenmiştir:

Tablo 1: Flush Thread Parametreleri

Parametre	Varsayılan Değer	Anlamı
<code>dirty_writeback_centisecs</code>	500	Flusher thread'in periyodik olarak çalıştığı aralık (santi saniye cinsinden). 500 cs = 5 saniye. Bu aralıktaki çekirdek <i>dirty</i> sayfaları kontrol eder.
<code>dirty_expire_centisecs</code>	3000	Bir <i>dirty</i> sayfa en fazla bu kadar süre (santi saniye cinsinden) RAM'de kalabilir. 3000 cs = 30 saniye sonra <i>süresi dolmuş</i> sayılır ve flush edilir.
<code>dirty_ratio</code> / <code>dirty_background_ratio</code>	%20 / %10 civarı	RAM'in ne kadarı <i>dirty</i> sayfalarla dolarsa flush işleminin başlatılacağını belirler (bellek baskısı durumunda zaman beklenmez).

Bu değerler `proc` dosya sisteminden görüntülenebilmektedir:

```
$ cat /proc/sys/vm/dirty_writeback_centisecs
$ cat /proc/sys/vm/dirty_expire_centisecs
```

`sysctl` komutu ile de bu değerler değiştirilebilmektedir:

```
$ sudo sysctl -w vm.dirty_writeback_centisecs=100
```

`sysctl` komutu zaten kendi içerisinde `/proc/sys` dizinindeki dosyalar üzerinde güncelleme işlemleri yapmaktadır.

flush thread'lerinin çalışması daha ayrıntılı olarak *sayfa önbelleği (page cache)* konusunun ele alındığı bölümde açıklanacaktır.

5.3.4 Gecikmeli Yazımın Gerekçeleri

Gecikmeli yazım (delayed write) işleminin gerekçeleri nelerdir? En önemli gerekçe peşi sıra yapılan yazma işlemlerinin tek hamlede ağıta yansıtılmasıdır. Bu sayede yazma işlemini yapan thread bloke olmaz ve toplamda bu işlemler paralel yürütüldüğü için sistem performansı yükselir. Aynı zamanda flash belleklerde ve SSD'lerde bu gecikme sürekli yazım sonucunda oluşan belleğin eskimesini de kısmen engellemektedir.

5.3.5 Kirlenmiş Sayfaların Erken Temizlenmesi

Sayfa önbelleğinde kirlenen sayfalar bazı durumlarda işletim sisteminin *tazeleyici (flusher)* thread'lerini beklemeden de diske aktarılabilir. Örneğin bir dosya kapatıldığında artık bu işlem arka planda dosyanın kirlenmiş sayfalarının da diske yazılmasına yol açmaktadır.

5.3.6 sync, fsync ve İlgili Açış Bayrakları

UNIX/Linux sistemlerinin çoğunda bazı özel sistem fonksiyonları yoluyla ya da `open` fonksiyonundaki bayraklarla da bu duruma müdahale edilebilmektedir.

`sync` POSIX fonksiyonu çağrıldığında o anda dosya sistemine ilişkin *kirlenmiş (dirty olan)* sayfaların hepsi flush edilmektedir:

```
#include <unistd.h>

void sync(void);
```

`sync` fonksiyonu asenkron biçimde çalışmaktadır. Yani fonksiyon geri döndüğünde tüm blokların flush edilmiş olma garantisi yoktur. Aynı zamanda Linux sistemlerde `sync` isimli bir kabuk komutu da bulunmaktadır. Bu komut `sync` fonksiyonunu çağırır.

`fsync` POSIX fonksiyonu ise belli bir dosyaya ilişkin kirlenmiş sayfaların flush edilmesi için kullanılmaktadır:

```
#include <unistd.h>

int fsync(int fildes);
```

`fsync` fonksiyonu senkronize (synchronous) çalışmaktadır. Yani fonksiyon geri döndüğünde sayfa önbelleğindeki kirlenmiş sayfaların flush edilmiş olması garanti edilmektedir.

Bir dosya açılırken `open` POSIX fonksiyonunda kullanılan konuyla ilgili üç bayrak vardır:

O_DSYNC Bayrağı

Bu bayrak POSIX'in "Base Definitions" bölümündeki "Synchronized I/O Data Integrity Completion" başlığında açıklanan yazma koşullarının sağlanacağını belirtmektedir. Bu bayrak kullanıldığında aşağıdaki iki durumun çekirdek tarafından sağlanması garanti edilmektedir:

- Dosyaya yazdırılan bilgilerin `write` fonksiyonu geri döndüğünde hedefe transfer edilmiş olması.
- Yazılan bilginin dosyadan okunabilmesi için gereken metadatanın hedefe transfer edilmiş olması. (Tüm metadatanın bilgilerin hedefe transfer edilmiş olması gerekmemektedir.)

O_SYNC Bayrağı

Bu bayrak POSIX'in "Base Definitions" bölümündeki "Synchronized I/O File Integrity Completion" başlığında açıklanan koşulların sağlanacağını belirtmektedir. `O_SYNC` bayrağı `O_DSYNC` bayrağını kapsamaktadır. Fakat bu bayrak `write` fonksiyonu geri dönmeye önce tüm metadatanın bilgilerin hedefe transfer edilmiş olmasını zorunlu tutmaktadır.

O_RSYNC

Bu okuma işlemi ile ilgilidir. Tek başına değil O_DSYNC ya da O_SYNC bayraklarıyla birlikte kullanılır. Eğer O_RSYNC bayrağı O_DSYNC bayrağı ile birlikte kullanılırsa read işlemini etkileyecek olan daha önce yapılmış write işlemleri varsa read fonksiyonu geri dönmeye önce bu write işlemleri için O_DSYNC bayrağında belirtilen semantik uygulanmaktadır. Eğer bu bayrak O_SYNC ile birlikte kullanılırsa read işlemini etkileyecek olan daha önce yapılmış write işlemleri varsa read fonksiyonu geri dönmeye önce bu write işlemleri için O_SYNC bayrağında belirtilen semantik uygulanmaktadır.

5.3.7 C Standart Kütüphanesinin Rolü

Peki C'nin standart dosya fonksiyonları bu süreçte nerede yer almaktadır? C'nin dosya fonksiyonları aslında neticede POSIX fonksiyonlarını çağırır. Ancak C'nin standart dosya fonksiyonları işletim sisteminin okuma yazma fonksiyonlarını daha az çağırmak için *kullanıcı alanında (user space)* her dosya için bir önbellek de oluşturmaktadır. Bu önbellek sistemine genellikle önbellek yerine *tamponlama (buffering)* sistemi, burada kullanılan önbelleğe de *tampon (buffer)* denilmektedir.

Örneğin Linux'ta biz C'nin getc gibi dosya fonksiyonunu çağırmış olalım. Standart C kütüphanesi fgetc ile 1 byte okumak istediğimizde read POSIX fonksiyonu ile 1 byte okumamaktadır. read POSIX fonksiyonu ile <stdio.h> dosyasında belirtilen BUFSIZ kadar byte'ı bir tampona okumakta ve oradan 1 byte'ı programcıya vermektedir. Bu nedenle C'nin standart fonksiyonlarına *tamponlu IO (buffered IO)* fonksiyonları da denilmektedir.

O hâlde C'nin standart dosya fonksiyonlarıyla yapılan tipik bir okuma işlemi şöyle gerçekleşmektedir:

```
C'deki okuma fonksiyonu
↓
read POSIX fonksiyonu
↓
sys_read sistem fonksiyonu
↓
...
```

C'deki bu tamponlama yani önbellek mekanizması C'ye özgü değildir. Diğer programlama dillerinin de standart kütüphanelerinde benzer biçimde tamponlamalar yapılmaktadır. Örneğin C++'taki <iostream> kütüphanesi, C#'ta kullanılan .NET kütüphaneleri, Java'da kullanılan temel kütüphanelerde hep kullanıcı alanında C'de olduğu gibi tamponlama yapmaktadır. Ancak bunların hepsi neticede Linux sistemlerinde POSIX fonksiyonlarını, onlar da sistem fonksiyonlarını çağırır.

POSIX dosya fonksiyonlarının Linux'taki işletim sisteminin sistem fonksiyonlarını çağırdığını belirtmiştik. İşletim sisteminin sistem fonksiyonları bilgi önbellekte olsa bile belli bir yavaşlık oluşturmaktadır. Programın akışının kullanıcı modundan çekirdek moduna geçirilmesi ve akışın ilgili sistem fonksiyonuna aktarılması görece bir zaman kaybına yol açmaktadır. Bu nedenle bu kütüphanelerin kullanıcı modunda tamponlama yapması önemli olmaktadır.

5.4 Çekirdekteki Dosya İşlemi Veri Yapıları

Şimdiye kadar blok ve sektör düzeyinde okuma yazmaların kabaca nasıl gerçekleştirildiğini açıkladık. Ancak çekirdeğin açık dosyalar için oluşturduğu organizasyon hakkında bilgi vermedik. Şimdi sürecin bu yönü üzerinde duracağız.

Anımsanacağı gibi UNIX/Linux sistemlerinde dosyalar open isimli POSIX fonksiyonuyla açılmaktadır. open POSIX fonksiyonu başarı durumunda ismine *dosya betimleyicisi (file descriptor)* denilen bir handle değeri vermektedir. read, write, lseek, close gibi POSIX'in diğer dosya fonksiyonları bu dosya betimleyicisini parametre olarak alıp hangi dosya üzerinde işlem yapılacağını bu betimleyiciden hareketle belirlemektedir.

open, read, write, lseek, close gibi POSIX'in temel dosya fonksiyonları Linux sistemlerinde aslında neredeyse doğrudan Linux'un ilgili sistem fonksiyonlarını çağırır:

```
open ---> sys_open
read ---> sys_read
write ---> sys_write
lseek ---> sys_lseek
close ---> sys_close
```

Bu nedenle birtakım ayrıntıları da göz ardı edersek biz Linux sistemlerinde dosya işlemleri yapan temel POSIX fonksiyonlarının aslında doğrudan sys_xxx sistem fonksiyonlarını çağırdığını varsayabiliriz.

UNIX/Linux sistemlerinde kullanılan standart C kütüphaneleri aynı zamanda POSIX fonksiyonlarını da içermektedir. Bilindiği gibi bugün masaüstü Linux sistemlerinde en fazla kullanılan standart C kütüphanesi GNU'nun libc kütüphanesidir. Eğer standart C fonksiyonlarının ve POSIX fonksiyonlarının nasıl yazıldığını merak ediyorsanız gömülü Linux sistemleri için daha minimalist biçimde yazılmış olan

kütüphanelerin kaynak kodlarını inceleyebilirsiniz. Bu iş için iki alternatif *musl* ve *uclibc* kütüphaneleridir. *uclibc* kütüphanesine *Mikro C kütüphanesi* de denilmektedir. Bu kütüphanelerin kaynak kodlarını elixir.bootlin.com sitesinden inceleyebilirsiniz. Bu site yalnızca Linux çekirdekleri için değil başka projeler için de kodlar üzerinde gezinme olanağı sunmaktadır. Sadeliği nedeniyle *musl* kütüphanesini incelemenizi salık veririz. Kütüphanenin kodları üzerinde gezinebilmek için aşağıdaki bağlantıdan faydalanabilirsiniz:

<https://elixir.bootlin.com/musl/v1.2.5/source>

5.5 task_struct İçerisindeki Dosya Sistemine İlişkin Veri Yapıları

task_struct yapısı (proses kontrol bloğu) içerisinde proseslere ilişkin dosya işlemleri için kullanılan iki önemli eleman bulunmaktadır:

```
struct task_struct {
    /* ... */

    /* Filesystem information: */
    struct fs_struct      *fs;

    /* Open file information: */
    struct files_struct   *files;

    /* ... */
};
```

Bu iki eleman çok uzun süredir task_struct yapısı içerisinde bulunmaktadır. Ancak buradaki fs_struct ve files_struct yapılarının içerisinde çekirdeğin versiyonları ilerledikçe çeşitli değişiklikler de yapılmıştır.

5.5.1 fs_struct Yapısı

Yuradaki fs_struct yapısı açık dosyalara ilişkin yapılan organizasyonla ilgili değildir. Prosesin kök dizini ve çalışma dizini gibi dosya sistemine ilişkin proses bilgileri burada tutulmaktadır. Mevcut çekirdeklerde fs_struct yapısı include/linux/fs_struct.h dosyası içerisinde şöyle bildirilmiştir:

```
struct fs_struct {
    int users;
    spinlock_t lock;
    seqcount_spinlock_t seq;
    int umask;
    int in_exec;
    struct path root, pwd;
} __randomize_layout;
```

Buradaki root ve pwd elemanları sırasıyla prosesin kök dizinini ve çalışma dizinini (current working directory) tutmaktadır. umask elemanı ise prosesin umask değerini tutmaktadır. Buradaki path yapısı da şöyle bildirilmiştir:

```
struct path {
    struct vfsmount *mnt;
    struct dentry *dentry;
} __randomize_layout;
```

vfsmount ve dentry yapıları çeşitli başka bölümlerde ele alınacaktır. Ayrıntıları göz ardı edersek fs_struct yapısındaki en önemli elemanlar prosesin kök dizininin yeri, prosesin çalışma dizininin yeri ve prosesin umask değeridir.

Bu yapı zaman içerisinde de geliştirilmiştir. Çekirdeğin 0.01 versiyonunda bu bilgiler doğrudan task_struct içerisinde bulunmaktaydı:

```
struct task_struct {
    /* ... */

    unsigned short umask;
    struct m_inode * pwd;
    struct m_inode * root;
    unsigned long close_on_exec;
```

(sonraki sayfaya devam)

```

    /* ... */
};

```

5.6 files_struct Yapısı

task_struct içerisindeki files isimli gösterici prosesin açmış olduğu dosyalara ilişkin bilgilerin tutulduğu files_struct türünden yapı nesnesini göstermektedir. files_struct yapısı da zaman içerisinde değişikliklere uğratılmıştır. Güncel çekirdeklerde bu yapı include/linux/fdtable.h dosyası içerisinde şöyle bildirilmiştir:

```

struct files_struct {
    /*
     * read mostly part
     */
    atomic_t count;
    bool resize_in_progress;
    wait_queue_head_t resize_wait;

    struct fdtable __rcu *fdt;
    struct fdtable fdtab;

    /*
     * written part on a separate cache line in SMP
     */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    unsigned int next_fd;
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};

```

Bu yapı versiyonlar arasında önemli farklılıklar göstermiştir. 2.4 çekirdeğindeki hali:

```

struct files_struct {
    atomic_t count;
    rwlock_t file_lock;
    int max_fds;
    int max_fdset;
    int next_fd;
    struct file ** fd;          /* current fd array */
    fd_set *close_on_exec;
    fd_set *open_fds;
    fd_set close_on_exec_init;
    fd_set open_fds_init;
    struct file * fd_array[NR_OPEN_DEFAULT];
};

```

Çekirdeğin 0.01 versiyonunda bu bilgiler doğrudan task_struct içerisinde bulunuyordu:

```

struct task_struct {
    /* ... */

    unsigned short umask;
    struct m_inode * pwd;
    struct m_inode * root;
    unsigned long close_on_exec;

    /* ... */
};

```

5.7 Dosya Nesnesi (struct file)

Linux'ta ne zaman `open` POSIX fonksiyonuyla bir dosya açılrsa `sys_open` sistem fonksiyonu açılan dosya için `file` isimli (`struct file` türünden) bir yapı nesnesini tahsis edip dosya işlemleri için gereken bilgileri bu yapı nesnesinin içerisine yerleştirmektedir. `sys_read`, `sys_write`, `sys_lseek`, `sys_close` gibi sistem fonksiyonları da dosya üzerinde işlem yapabilmek için bu `file` yapısındaki bilgileri kullanmaktadır. İşletim sistemlerinde bu amaçla kullanılan nesnelere *dosya nesnesi* (*file object*) de denilmektedir.

Biz aşağıdaki gibi bir dosya açmış olalım:

```
fd = open(...);
```

`sys_open` sistem fonksiyonu açılmak istenen dosyanın diskteki yerini ve metadata bilgilerini bulur, o bilgilerden hareketle bir dosya nesnesi (`file object`) oluşturur. O dosya nesnesinin adresini de `files_struct` nesnesinin içerisine yerleştirir. Böylece `sys_read`, `sys_write`, `sys_lseek`, `sys_close` gibi sistem fonksiyonları `task_struct` nesnesinden hareketle bu dosya nesnesine erişebilmektedir.

Güncel çekirdeklerde `file` yapısı `include/linux/fs.h` dosyasının içerisinde şöyle bildirilmiştir:

```
struct file {
    spinlock_t          f_lock;
    fmode_t             f_mode;
    const struct file_operations *f_op;
    struct address_space *f_mapping;
    void                *private_data;
    struct inode         *f_inode;
    unsigned int         f_flags;
    unsigned int         f_iocb_flags;
    const struct cred    *f_cred;
    struct fown_struct   *f_owner;
    /* --- cacheline 1 boundary (64 bytes) --- */
    struct path          f_path;
    union {
        struct mutex     f_pos_lock;
        u64              f_pipe;
    };
    loff_t              f_pos;
#ifdef CONFIG_SECURITY
    void                *f_security;
#endif
    /* --- cacheline 2 boundary (128 bytes) --- */
    errseq_t            f_wb_err;
    errseq_t            f_sb_err;
#ifdef CONFIG_EPOLL
    struct hlist_head    *f_ep;
#endif
    union {
        struct callback_head f_task_work;
        struct llist_node    f_llist;
        struct file_ra_state f_ra;
        freeptr_t           f_freeptr;
    };
    file_ref_t          f_ref;
    /* --- cacheline 3 boundary (192 bytes) --- */
} __randomize_layout
__attribute__((aligned(4)));
```

5.8 Dosya Betimleyici Tablosu

Şimdi bir dosya `sys_open` sistem fonksiyonuyla açıldığında dosya nesnesinin ve dosya betimleyicisinin nasıl saklandığını açıklayalım. Önce çekirdeğin 0.01 versiyonunu inceleyelim. Bu ilkel versiyonda henüz `files_struct` biçiminde bir yapı yoktu. Açık dosya bilgileri doğrudan `task_struct` içerisinde bulunan aşağıdaki elemanlarda saklanıyordu:

```

struct task_struct {
    /* ... */

    unsigned long close_on_exec;
    struct file *filp[NR_OPEN];

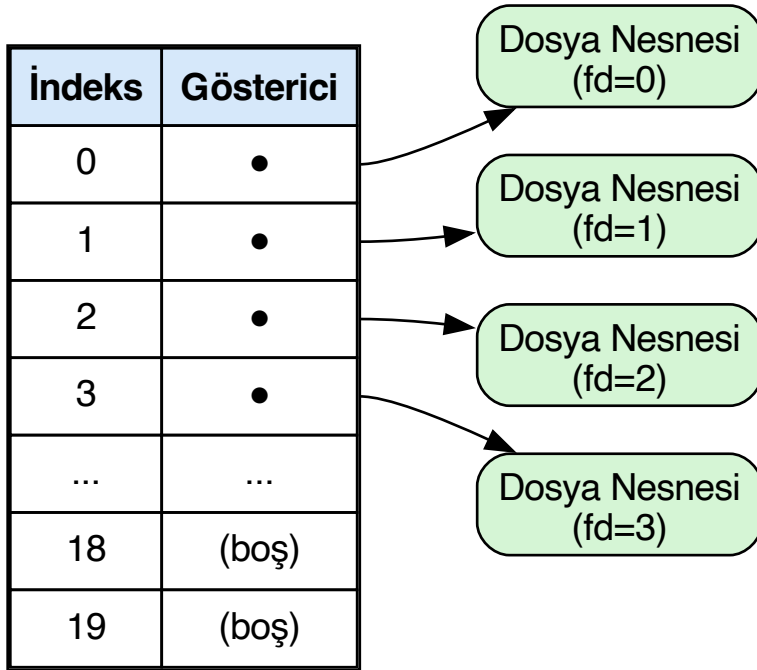
    /* ... */
};

```

Burada `filp` isimli dizinin `struct file *` türünden olduğuna dikkat ediniz. Yani `filp` dizisi `file` nesnelerinin adreslerini tutan bir gösterici dizisidir. Bu versiyonda `NR_OPEN` şöyle tanımlanmıştır:

```
#define NR_OPEN 20
```

Dolayısıyla bu ilkel versiyonda bir proses en fazla 20 dosyayı açık durumda tutabiliyordu. UNIX/Linux dünyasında dosya nesnelerinin adreslerini tutan bu gösterici dizilerine *dosya betimleyici tablosu* (*file descriptor table*) denilmektedir.



`open` POSIX fonksiyonunun (yani `sys_open` sistem fonksiyonunun) verdiği *dosya betimleyicisi* (*file descriptor*) aslında dosya betimleyici tablosundaki dizide bir indeks belirtmektedir. `open` POSIX fonksiyonunun dosya betimleyici tablosundaki en düşük boş indeks vereceği POSIX standartlarında garanti altına alınmıştır.

Dosya betimleyici tablosunun *prosese özgü* olduğuna dikkat ediniz. Bir proseste açılmış olan dosyaya ilişkin dosya nesnesinin adresi o prosesteki dosya betimleyici tablosuna yazılmaktadır. Dosya betimleyicileri sistem genelinde bir değer belirtmemektedir; dosya betimleyici değerleri yalnızca ilgili proses için anlamlıdır.

Yukarıdaki 0.01 versiyonunda konuyla ilgili `unsigned long` türden `close_on_exec` isimli bir elemanın da bulunduğunu görüyorsunuz. Bu elemanın her biti bir betimleyicinin *close-on-exec* durumunu belirtmektedir. Söz konusu bit 1 ise ilgili betimleyici `exec` işlemleri sırasında `close` edilir, 0 ise `close` edilmez. POSIX standartlarında bir dosya açıldığında `close-on-exec` bayrağının varsayılan durumda 0 olduğu belirtilmiştir.

5.8.1 İlk Boş Betimleyicinin Bulunması

`sys_open` sistem fonksiyonu öncelikle dosya betimleyici tablosundaki ilk boş betimleyiciyi bulmaya çalışır. 0.01 versiyonunda boş betimleyici şöyle bulunmuştur:

```
int sys_open(const char * filename, int flag, int mode) {
    /* ... */

    for(fd=0 ; fd<NR_OPEN ; fd++)
        if (!current->filp[fd])
            break;

    /* ... */
}
```

Görüldüğü gibi bu ilkel versiyonda dosya betimleyici tablosu üzerinde tek tek sıralı arama yapılmış, ilk boş betimleyici elde edilmiştir. Ancak uzunca bir süredir proseslerin varsayılan dosya betimleyici tablolarının varsayılan uzunlukları 1024'tür ve bu uzunluk da büyütülebilmektedir.

5.8.2 Bitmap Tabanlı Hızlı Arama

1024 elemanlı bir tabloda sıralı arama ile ilk boş elemanın bulunması yavaş bir işlemdir. Bu nedenle bir süre sonra Linux çekirdeklerinde bu arama işlemi bit düzeyinde aramayla hızlandırılmıştır.

Bit düzeyinde arama yönteminde dosya betimleyici tablosunun uzunluğu kadar bit dizisi oluşturulur. Bu bit dizisindeki 0 olan bitler betimleyici tablosundaki boş elemanları, 1 olan bitler dolu elemanları belirtmektedir.

Güncel çekirdeklerde iki düzey bitmap kullanılmaktadır:

- **Birinci düzey bitmap (`open_fds`):** Tüm dosya betimleyicilerinin dolu/boş olduğunu tutar.
- **İkinci düzey bitmap (`full_fds_bits`):** Birinci düzey bitmap'teki tüm bitleri 0 olmayan (yani tamamen dolu) unsigned long elemanların indeksini hızlıca bulmak için kullanılır.

Bu hızlandırma mantığında:

1. Çekirdek hemen ikinci düzey bitmap'e yönelmez. Önce `open_fds` dizisinde tek bir unsigned long elemanda hızlı bir arama yapar.
2. Eğer bu arama başarısız olursa `full_fds_bits`'te ilk 0 olan bitin indeksi elde edilir; birinci düzey bitmap'te yalnızca bu indeksteki unsigned long dizi elemanında arama yapılır.

Güncel çekirdeklerdeki `sys_open` sistem fonksiyonundan başlanarak ilk boş dosya betimleyicisinin bulunması için yapılan çağrılar şöyledir:

```
sys_open
→ do_sys_open
→ do_sys_openat2
→ __get_unused_fd_flags
→ alloc_fd
→ find_next_fd
→ find_next_zero_bit
```

Güncel çekirdeklerdeki `find_next_fd` fonksiyonu şöyle yazılmıştır:

```
static unsigned int find_next_fd(struct fdtable *fdt, unsigned int start)
{
    unsigned int maxfd = fdt->max_fds;
    unsigned int maxbit = maxfd / BITS_PER_LONG;
    unsigned int bitbit = start / BITS_PER_LONG;
    unsigned int bit;

    /*
     * Try to avoid looking at the second level bitmap
     */
}
```

(sonraki sayfaya devam)

```

bit = find_next_zero_bit(&fdt->open_fds[bitbit], BITS_PER_LONG,
                        start & (BITS_PER_LONG - 1));
if (bit < BITS_PER_LONG)
    return bit + bitbit * BITS_PER_LONG;

bitbit = find_next_zero_bit(fdt->full_fds_bits, maxbit, bitbit) * BITS_PER_LONG;
if (bitbit >= maxfd)
    return maxfd;
if (bitbit > start)
    start = bitbit;
return find_next_zero_bit(fdt->open_fds, maxfd, start);
}

```

Makine dili düzeyinde aramanın ffz fonksiyonunda ve __ffs fonksiyonunda yapıldığına dikkat ediniz.

files_struct yapısının next_fd elemanı aramanın başlatılacağı yeri belirtmektedir. Yani next_fd'nin belirttiği değerden küçük tüm dosya betimleyicileri doludur. Dolayısıyla arama full_fds_bits dizisinin başından başlatılmamaktadır.

5.9 2.6 Çekirdeğinde files_struct ve fdtable

Çekirdeğin 2.6 versiyonlarına gelindiğinde files_struct yapısının içerisi fdtable isimli bir yapı ile biraz daha derli toplu fakat biraz daha karmaşık hale getirilmiştir. 2.6'lı versiyonlardaki files_struct yapısı şöyledir:

```

struct files_struct {
/*
 * read mostly part
 */
    atomic_t count;
    struct fdtable __rcu *fdt;
    struct fdtable fdtab;
/*
 * written part on a separate cache line in SMP
 */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    int next_fd;
    struct embedded_fd_set close_on_exec_init;
    struct embedded_fd_set open_fds_init;
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};

```

fdtable yapısı da şöyledir:

```

struct fdtable {
    unsigned int max_fds;
    struct file __rcu **fd;      /* current fd array */
    fd_set *close_on_exec;
    fd_set *open_fds;
    struct rcu_head rcu;
    struct fdtable *next;
};

```

Bu versiyonlarda çekirdek her zaman fdt göstericisinin gösterdiği yerden işlemine başlamaktadır. fdt göstericisi işin başında yapı içerisindeki fdtab yapı nesnesini göstermektedir. fdtab yapı nesnesinin içerisinde de fd, close_on_exec, open_fds göstericileri vardır. Bu göstericiler de işin başında files_struct içerisindeki fd_array, close_on_exec_init ve open_fds_init elemanlarını göstermektedir.

Bu versiyonlarda current göstericisinden hareketle fdx betimleyicisinin gösterdiği yerdeki dosya nesnesine current->files->fdt->fd[fdx] ifadesiyle erişilebilir.

2.6'lı çekirdeklerde de dosya betimleyicisinden hareketle dosya nesnesine erişimi sağlayan fget ve fput fonksiyonları bulunmaktadır:

```

struct file *fget(unsigned int fd)
{
    return __fget(fd, FMODE_PATH);
}
EXPORT_SYMBOL(fget);

```

```

void fput(struct file *file)
{
    if (atomic_long_dec_and_test(&file->f_count))
        __fput(file);
}

```

fget fonksiyonuyla elde edilen dosya nesnesi fput fonksiyonuyla geri bırakılmaktadır.

5.10 Güncel Çekirdeklerdeki Dosya Veri Yapıları

Belli bir zamandan sonra artık bit dizisi oluşturmak için fd_set yapısının kullanılmasından vazgeçilmiştir. Güncel çekirdeklerdeki files_struct yapısı şöyledir:

```

struct files_struct {
    /*
     * read mostly part
     */
    atomic_t count;
    bool resize_in_progress;
    wait_queue_head_t resize_wait;

    struct fdtable __rcu *fdt;
    struct fdtable fdtab;
    /*
     * written part on a separate cache line in SMP
     */
    spinlock_t file_lock ____cacheline_aligned_in_smp;
    unsigned int next_fd;
    unsigned long close_on_exec_init[1];
    unsigned long open_fds_init[1];
    unsigned long full_fds_bits_init[1];
    struct file __rcu * fd_array[NR_OPEN_DEFAULT];
};

```

fdtable yapısı da şöyledir:

```

struct fdtable {
    unsigned int max_fds;
    struct file __rcu **fd; /* current fd array */
    unsigned long *close_on_exec;
    unsigned long *open_fds;
    unsigned long *full_fds_bits;
    struct rcu_head rcu;
};

```

Görüldüğü gibi artık bit dizileri fd_set yerine doğrudan unsigned long türden bir dizi biçiminde oluşturulmaktadır.

Güncel versiyonlarda fcheck biçiminde bir makro ve fcheck_files isimli bir fonksiyon yoktur. Ancak yine güncel versiyonlarda dosya betimleyicisi yoluyla dosya nesnesine erişimi referans sayacını artırarak yapan fget fonksiyonu ve sayacı eksilterek nesneyi bırakan fput fonksiyonu bulunmaktadır:

```

struct file *fget(unsigned int fd)
{
    return __fget(fd, FMODE_PATH);
}

```

(sonraki sayfaya devam)

```
}
EXPORT_SYMBOL(fget);
```

```
void fput(struct file *file)
{
    if (unlikely(file_ref_put(&file->f_ref)))
        __fput_deferred(file);
}
EXPORT_SYMBOL(fput);
```

5.11 Thread'ler, Fork ve Dosya Betimleyicileri

POSIX sistemlerinde dolayısıyla da Linux çekirdeğinde thread'lerin ayrı dosya betimleyici tabloları yoktur. Dosya betimleyicileri ve dosya betimleyici tablosu prosese özgüdür. Bir thread yaratıldığında thread'e ilişkin `task_struct` nesnesinin `fs` ve `files` gibi elemanları, onu yaratan thread'in `task_struct` nesnesinden sığ kopyalamayla kopyalanmaktadır. Dolayısıyla aslında prosesin bütün thread'leri açık dosyalara ilişkin aynı veri yapısı nesnelerini kullanmaktadır:

```
struct task_struct {
    /* ... */

    /* Filesystem information: */
    struct fs_struct      *fs;

    /* Open file information: */
    struct files_struct   *files;

    /* ... */
};
```

Yeni `task_struct` nesnesi yaratılıp diğer `task_struct` nesnesinden sığ kopyalama yapıldığında `files` göstericisinin de aslında aynı `files_struct` nesnesini göstereceğine dikkat ediniz. Yani toplamda aslında proses için tek bir `files_struct` nesnesi bulunmaktadır.

`fork` işlemi sırasında ise tamamen alt proses için yeni bir `files_struct` nesnesi ve yeni bir dosya betimleyici tablosu (`fd` dizisi) yaratılmaktadır. Ancak üst prosesin dosya betimleyici tablosundaki adresler yeni yaratılan alt prosesteki dosya betimleyici tablosuna kopyalanmaktadır. Böylece üst prosesin dosya betimleyici tablosunun aynı numaralı betimleyicileriyle alt prosesin dosya betimleyici tablosunun aynı numaralı betimleyicileri aynı dosya nesnelerini gösteriyor durumda olur. Tabii artık üst ve alt proseslerin yeni açacağı dosyalar onlara özgü olacaktır. Paylaşılan dosya nesneleri yalnızca `fork` öncesinde açılmış olanlardır.

5.12 Dosya Sistemi Temel Nesneleri

Şimdiye kadar açık dosyalara ilişkin çekirdeğin oluşturduğu veri yapıları hakkında şu bilgileri edindik:

- Açık dosyalara ilişkin `task_struct` içerisindeki veri yapıları.
- Dosya betimleyicilerinin anlamı ve dosya betimleyicisi yoluyla dosya nesnelerine nasıl erişildiği.
- En düşük boş betimleyicinin elde edilme biçimi.

Şimdi dosya sisteminin diğer önemli veri yapıları üzerinde duracağız.

5.12.1 Dosya Nesnesi, inode Nesnesi ve dentry Nesnesi

Çekirdek açık bir dosya üzerinde `read/write` gibi işlemleri yaparken dosyanın son okunma tarihi, son güncelleme tarihi, dosya uzunluğu gibi bilgileri de güncelleyeceğine göre bu bilgilere nasıl erişmektedir? Dosyaların uzunluk, erişim hakları, tarih-zaman bilgisi gibi metadata bilgileri diskte tutulmaktadır. (Ext dosya sistemlerinde bu bilgilerin diskte tutulduğu yere `inode` blok denilmektedir.) Çekirdek dosya açılırken dosyanın bu metadata bilgilerini diskten okuyarak bellekte `inode` isimli bir yapı nesnesinin içerisine yerleştirmektedir. Biz dosyaların metadata bilgilerinin tutulduğu bu *inode nesnesi* türünden nesnelere *inode nesnesi* de diyeceğiz.

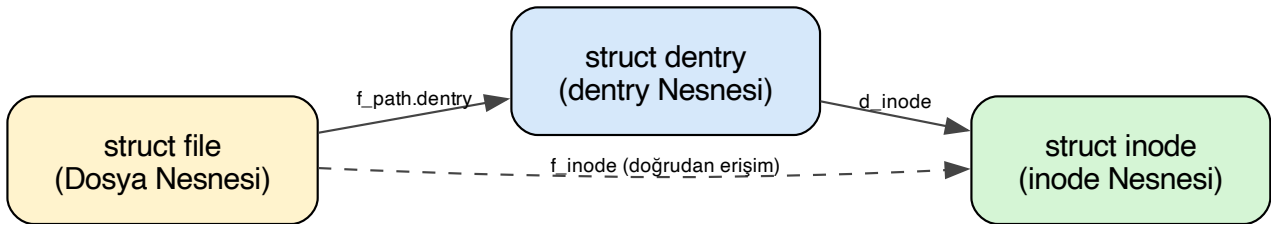
Peki bir dosya açıldığında dosyanın dosya sistemindeki yeri çekirdek tarafından nasıl saklanmaktadır? Çekirdek her dosya açıldığında o dosyanın izin girişi bilgilerini (yani dosya sisteminde nerede olduğu bilgisini ve bazı diğer bilgileri) ismine *dentry* denilen bir yapı nesnesine yerleştirmektedir.

Dosya sistemi nesnelerini şöyle özetleyebiliriz:

Nesne	Açıklama
Dosya Nesnesi (<code>struct file</code>)	Açılmış dosyalar üzerinde işlem yapmak için gereken tüm bilgilerin tutulduğu nesne.
inode Nesnesi (<code>struct inode</code>)	Dosyanın diskteki metadata bilgilerini tutan nesne.
dentry Nesnesi (<code>struct dentry</code>)	Dosyanın dosya sistemi üzerindeki yerini ve buna ilişkin bazı bilgileri tutan nesne.

5.12.2 Nesneler Arası İlişki

Uzunca bir süre (2.6'ya kadar ve 2.6'lı versiyonlar da dahil olmak üzere) dosya nesnesinin (`file` yapısının) içerisinde dentry nesnesinin adresi, dentry nesnesinin içerisinde de o dizin girişinin inode nesnesinin adresi tutuluyordu. Daha sonraları dosya nesnesinin içerisinde de doğrudan inode nesnesinin adresi tutulmaya başlanmıştır:



UNIX/Linux sistemlerinde bir dosya sistemi kök dizinde bir yere mount edilebilmektedir. Çekirdeğin bazı durumlarda bir dosyanın hangi dosya sisteminin içerisinde bulunduğunu anlaması da gerekebilmektedir. 2.6 çekirdeklerinden itibaren de dosyanın dentry nesnesinin adresiyle `vfsmount` bilgileri `path` isimli bir yapıya yerleştirilmiş ve `file` yapısının içerisindeki `f_path` elemanında tutulmaya başlanmıştır:

```

struct file {
    /* ... */

    fmode_t      f_mode;
    unsigned int f_flags;
    loff_t       f_pos;
    atomic_long_t f_count;    /* güncel çekirdeklerde: file_ref_t f_ref */
    struct path  f_path;
    struct inode *f_inode;

    /* ... */
};

struct path {
    struct vfsmount *mnt;
    struct dentry *dentry;
} __randomize_layout;
  
```

5.12.3 Dosya Nesnesi Elemanları

Açık bir dosyanın `open` POSIX fonksiyonunda kullanılan açış bayrakları (`O_` ile başlayan bayraklar) `file` yapısının `f_flags` elemanında saklanmaktadır. Örneğin:

```
fd = open("test.txt", O_RDWR|O_APPEND);
```

Buradaki `O_RDWR` ve `O_APPEND` bayrakları `file` yapısının `f_flags` elemanında saklanmaktadır. Bu `f_flags` elemanının daha çabuk işleme sokulabilecek yeniden düzenlenmiş hali yapının `f_mode` elemanında saklanmaktadır.

Okuma yazma işlemlerinin *dosya göstericisi* (*file pointer*) denilen bir offset'ten itibaren yapıldığını anımsayınız. Dosya göstericisinin konumu da `file` yapısının `f_pos` elemanında tutulmaktadır.

Dosya nesnelerini birden fazla betimleyici gösterebilmektedir. Örneğin `dup` ve `dup2` POSIX fonksiyonları aynı dosya nesnesini gösteren farklı bir betimleyicinin oluşturulmasına yol açmaktadır. Bu durumda `close` fonksiyonu ile dosya kapatıldığında dosya hemen silinmez. Çünkü onu kullanan başka betimleyiciler de bulunuyor olabilir. İşte `file` yapısının içerisindeki referans sayacı (uzun süre `f_count`, güncel çekirdeklerde `f_ref` ismiyle) o dosya nesnesinin kaç betimleyici tarafından gösterildiği bilgisini tutmaktadır. Her betimleyici `close` fonksiyonu ile kapatıldığında bu sayacı 1 eksiltirir; sayacı 0'a düştüğünde dosya nesnesi silinir.

5.13 inode ve dentry Önbellekleri

Her açılan yeni dosya için çekirdek yeni bir dosya nesnesi yaratmaktadır. Ancak aynı dosyalar açıldığında bu dosyalar için tek bir dentry ve inode nesnesi oluşturmaktadır. Bunun için `open` fonksiyonuyla bir dosya açıldığında çekirdeğin bu dosyaya ilişkin dentry nesnesi ve inode nesnesi daha önce yaratılmış mı diye bir araştırma yapması gerekmektedir.

İşte çekirdek dentry ve inode nesneleri için ayrı önbellek (cache) sistemleri oluşturmaktadır. Bunlara Linux sistemlerinde *dentry önbelleği* (*dentry cache*) ve *inode önbelleği* (*inode cache*) denilmektedir. Bir dosya açıldığında çekirdek o dosyaya ilişkin dentry ve inode nesneleri zaten bu önbellek sistemlerinde varsa hiç diske gitmeden doğrudan bu önbelleklerden onları alıp kullanmaktadır.

Bir dosyayı onu açmış olan bütün prosesler kapatmış olsa bile o dosyaya ilişkin dentry ve inode nesneleri bu önbellek sistemlerinde kalmaya devam edebilir. Çünkü dosyalar (özellikle bazı merkezi dosyalar) farklı prosesler tarafından defalarca açılıp kullanılabilir. (Örneğin `/etc/passwd` dosyası pek çok proses tarafından dolaylı bir biçimde açılıp kullanılabilir.) Bu tür durumlarda onların bu önbellekler içerisinde biriktirilmesi sistem performansını oldukça iyileştirmektedir.

Linux'taki bu tür önbellek sistemlerinde önbellekten çıkarma için genel olarak *LRU* (*Least Recently Used*) denilen *önbellek yer değiştirme* (*cache replacement*) algoritması işletilmektedir. Yani önbellekten toplamda en az kullanılanlar değil son zamanlarda en az kullanılanlar çıkartılmaktadır.

Bu noktada *dentry önbelleği* ve *inode önbelleği* ile *sayfa önbelleği* (*page cache*) arasındaki ilişkiye de değinelim. Sayfa önbelleği disk blokları temelinde organize edilen ve her türlü disk bloğunun transfer edilmesinde kullanılan aşağı seviyeli bir önbellek sistemidir. Halbuki dentry ve inode önbellekleri yalnızca dentry ve inode elemanlarından oluşan önbelleklerdir. Bir dosya açıldığında eğer o dosyaya ilişkin dentry ve inode nesneleri dentry ve inode önbelleklerinde yoksa diskten elde edilmektedir. Tabii bu amaçla disk okuması yapılmadan önce bu disk bloklarının sayfa önbelleğinde olup olmadığına da bakılmaktadır.

5.14 open() Fonksiyonunun İşleyişi

Geldiğimiz noktaya kadarki konuları dikkate aldığımızda `open` fonksiyonuyla bir dosyanın açılması durumunda sırasıyla şunların yapıldığını söyleyebiliriz:

1. Prosesin dosya betimleyici tablosunda boş bir betimleyici hızlı bir biçimde bulunur.
2. Bir dosya nesnesi (`struct file` nesnesi) yaratılır ve elemanlarına gerekli ilkdeğerler verilir.
3. Dosyaya ilişkin dentry nesnesi ve inode nesnesi dentry önbelleğinde ve inode önbelleğinde yoksa diskte arama yapılarak yaratılır ve bunların adresleri dosya nesnesine yerleştirilir.
4. Dosya nesnesinin adresi dosya betimleyici tablosuna (fd dizisine) yerleştirilir ve yerleştirilen dizi indeksi dosya betimleyicisi olarak geri döndürülür.

5.15 Prosesin Kök ve Çalışma Dizini

Bir prosesin kök dizininin ve çalışma dizininin proses kontrol bloğunda yani `task_struct` nesnesinde tutulduğunu belirtmiştik. Güncel çekirdeklerde bu bilgi şöyle tutulmaktadır:

```
struct task_struct {
    /* ... */

    struct fs_struct *fs;

    /* ... */
};

struct fs_struct {
    /* ... */
    struct path root, pwd;
```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```

/* ... */
} __randomize_layout;

struct path {
    struct vfsmount *mnt;
    struct dentry *dentry;
} __randomize_layout;

```

Görüldüğü gibi çekirdek prosesin kök dizinin ve çalışma dizininin bilgisini yol ifadesi biçiminde değil dentry nesnesi biçiminde tutmaktadır.

5.16 Dosya Sistemi Veri Yapılarına İlişkin Testler

Şimdi dosya sisteminin temel veri yapıları üzerinde görmüş olduğumuz konulara ilişkin bazı testler yapalım. Bu testleri yapmak için bir aygıt sürücü yazarak `ioctl` yoluyla gerçekleştireceğiz. (Kursumuzda kullandığımız Linux makinedeki çekirdek sürümü 6.9.2'dir.)

5.16.1 `ioctl_test1` ve `ioctl_test2`: Dosya Göstericisini Okuma

İlk örneğimizde bir dosya betimleyicisine ilişkin dosya nesnesini elde ederek onun `f_pos` elemanını yazdırmaya çalışalım. Dosya nesnesindeki `f_pos` elemanının dosya göstericisinin değerini tuttuğunu anımsayınız.

Düşük seviyeli erişim (`ioctl_test1`):

```

static long ioctl_test1(struct file *filp, unsigned long arg)
{
    struct file *f;
    int fd = (int) arg;

    if (fd < 0 || fd >= current->files->fdt->max_fds) {
        printk(KERN_INFO "file descriptor is not valid!..\n");
        return -EBADF;
    }

    f = current->files->fdt->fd[fd];
    if (f == NULL) {
        printk(KERN_INFO "file descriptor is not valid!..\n");
        return -EBADF;
    }

    printk(KERN_INFO "File pointer offset: %lld\n", (long long)f->f_pos);

    return 0;
}

```

Burada dosya betimleyicisinden hareketle dosya nesnesine `current->files->fdt->fd[fd]` ifadesiyle erişilmiştir. Ancak bu erişim aslında güvenli değildir. Çünkü tam bu erişim yapılırken dosya betimleyici tablosu büyütülürse ya da bu betimleyici üzerinde işlem yapılırsa kodumuz kararsız bir durumla karşı karşıya kalır. Genel olarak çekirdeğin uyguladığı senkronizasyon mekanizmasına uygun hareket etmek gerekir.

Çekirdek nesneleri üzerinde işlem yapacak kişi nesnenin referans sayacını artırmazsa çekirdek onu boşaltabilir. Linux çekirdeklerinde sayacı artırarak nesneyi elde eden fonksiyonlar *get* soneki ile, sayacı azaltarak nesneyi bırakan fonksiyonlar ise *put* soneki ile isimlendirilmiştir.

`fget/fput` kullanımı (`ioctl_test2`):

Yukarıdaki testi daha güvenli bir biçimde `fget` ve `fput` fonksiyonlarını kullanarak da yapabiliriz:

```

static long ioctl_test2(struct file *filp, unsigned long arg)
{

```

(sonraki sayfaya devam)

```

struct file *f;
int fd = (int) arg;

if ((f = fget(fd)) == NULL) {
    printk(KERN_INFO "file descriptor is not valid!..\n");
    return -EBADF;
}

printk(KERN_INFO "File pointer offset: %ld\n", (long)f->f_pos);

fput(f);

return 0;
}

```

`fget` isimli yüksek seviyeli çekirdek fonksiyonu dosya betimleyicisinden hareketle bize dosya nesnesini RCU mekanizmasını kullanarak dosya nesnesinin sayacını da artırarak vermektedir. `fput` fonksiyonu da sayacı azaltarak dosya nesnesini geri bırakmaktadır. Bu fonksiyonların prototipleri:

```

struct file *fget(unsigned int fd);
void fput(struct file *);

```

Bu fonksiyonların prototipleri `include/linux/file.h` dosyası içerisinde. Güncel çekirdeklerde `fget` fonksiyonunun tanımlaması `fs/file.c` dosyası içerisinde, `fput` fonksiyonunun tanımlaması ise `fs/file_table.c` içerisinde yapılmıştır. `fget` ve `fput` fonksiyonları export edildiği için aygıt sürücüler içerisinde de kullanılabilir.

Güncel çekirdeklere `fget` ve `fput` işlemlerini daha etkin gerçekleştirmek için `fdget` ve `fdput` isimli fonksiyonlar da eklenmiştir. Her ne kadar yeni çekirdeklerde `fdget` ve `fdput` daha hızlı çalışıyorsa da `fget` ve `fput` basit arayüzü ve kullanım kolaylığı nedeniyle tercih edilebilir.

5.16.2 dup Gerçekleştirimi: `ioctl_test3`

Şimdi de `dup` POSIX fonksiyonunun işlevini yerine getiren (yani `sys_dup` sistem fonksiyonu gibi çalışan) bir fonksiyonu kendimiz yazalım. Bunun için bir `ioctl` kodu oluşturabiliriz. Bu `ioctl` kodu kullanıcı modundan çağrılırken `ioctl` fonksiyonunun üçüncü parametresine aşağıdaki gibi bir yapı nesnesinin adresini geçebiliriz:

```

struct FDDUP {
    int fd;
    int fd_dup;
};

```

Burada yapının `fd` elemanı çiftlenecek dosya betimleyicisini belirtmektedir. `fd_dup` elemanı ise çiftleme sonucunda elde edilecek yeni betimleyiciyi belirtmektedir. Bu elemanların `dup` fonksiyonu ile ilişkisini şöyle temsil edebiliriz:

```
fd_dup = dup(fd);
```

Dosya betimleyicisini çiftleyen `ioctl` kodu test amaçlı olarak şöyle yazılabilir:

```

static long ioctl_test3(struct file *filp, unsigned long arg)
{
    struct file *f;
    struct fdtable *fdt;
    struct file **fd_table;
    struct FDDUP fddup;
    int i;

    f = NULL;

    if (copy_from_user(&fddup, (struct FDDUP *)arg, sizeof(fddup)) != 0)
        return -EFAULT;
}

```


(önceki sayfadan devam)

```

fdt = current->files->fdt;
fd_table = fdt->fd;

if (fddup.fd < 0 || fddup.fd >= fdt->max_fds)
    return -EBADF;
if (fd_table[fddup.fd] == NULL)
    return -EBADF;

for (i = 0; i < fdt->max_fds; ++i)
    if (fd_table[i] == NULL) {
        f = fd_table[fddup.fd];
        fd_table[i] = f;
        ++f->f_count.counter;    /* atomic_long_inc(&f->f_count); */
        fddup.fd_dup = i;
        break;
    }
if (f == NULL)
    return -EMFILE;

if (copy_to_user((struct FDDUP *)arg, &fddup, sizeof(fddup)) != 0)
    return -EFAULT;

return 0;
}

```

Kursumuzun yapıldığı 6.9.2 çekirdeğinde dosya nesnesinin sayacının file yapısının içerisinde şöyle tutulduğunu anımsatalım:

```

struct file {
    /* ... */

    atomic_long_t    f_count;

    /* ... */
};

```

Burada atomic_long_t türü bir yapı biçiminde bildirilmiştir:

```

typedef struct {
    long counter;
} atomic_long_t;

```

Bu atomic_long_t türüyle belirtilen değerler üzerinde işlemler atomik düzeyde özel fonksiyonlarla yapılmaktadır. Bu konuyla ilgili önemli birkaç fonksiyon:

```

void atomic_long_set(atomic_long_t *v, long i);
long atomic_long_read(const atomic_long_t *v);
void atomic_long_inc(atomic_long_t *v);
void atomic_long_dec(atomic_long_t *v);

```

atomic_t ve atomic_long_t türleri hakkında ayrıntıları başka bir başlıkta ele alacağız.

Fonksiyonumuzdaki döngüde aslında mantıksal bir kusur da vardır. Biz döngüde yalnızca o andaki dosya betimleyici tablosunda arama yaptık. Halbuki çekirdek aslında başlangıçta küçük bir dosya betimleyici tablosunu tutarken daha sonra bunu gerektiğinde proses limitinin izin verdiği kadar yükseltebilmektedir.

5.16.3 Yüksek Seviyeli Fonksiyonlarla dup: ioctl_test4

Testi yaptığımız makinedeki Linux çekirdeğinde anımsayacağınız gibi ilk boş betimleyiciyi hızlı bir biçimde bulabilmek için iki düzey bitmap kullanılıyordu. Bu versiyonlarda ilk boş betimleyiciyi bulan `get_unused_fd_flags` isimli yüksek seviyeli bir çekirdek fonksiyonu bulunmaktadır. Bu fonksiyon export edildiği için aygıt sürücülerden de kullanılabilir.

```
int get_unused_fd_flags(unsigned flags)
{
    return __get_unused_fd_flags(flags, rlimit(RLIMIT_NOFILE));
}
EXPORT_SYMBOL(get_unused_fd_flags);
```

`get_unused_fd_flags` fonksiyonu aynı zamanda gerektiğinde dosya betimleyici tablosunu büyütme işlemini de kendisi yapmaktadır. `flags` parametresi ya `O_CLOEXEC` biçiminde ya da `0` biçiminde geçilebilmektedir.

Ayrıca RCU mekanizması eşliğinde dosya nesnesini (`struct file` nesnesini) dosya betimleyici tablosuna yerleştiren export edilmiş `fd_install` isimli yüksek seviyeli bir çekirdek fonksiyonu da bulunmaktadır:

```
void fd_install(unsigned int fd, struct file *file);
```

Böylece yukarıdaki fonksiyonu şu hale getirebiliriz:

```
static long ioctl_test4(struct file *filp, unsigned long arg)
{
    struct file *f;
    struct FDDUP fddup;

    if (copy_from_user(&fddup, (struct FDDUP *)arg, sizeof(fddup)) != 0)
        return -EFAULT;

    if ((f = fget(fddup.fd)) == NULL)
        return -EBADF;

    if ((fddup.fd_dup = get_unused_fd_flags(0)) < 0) {
        fput(f);
        return fddup.fd_dup;
    }

    if (copy_to_user((struct FDDUP *)arg, &fddup, sizeof(fddup)) != 0) {
        fput(f);
        return -EFAULT;
    }

    fd_install(fddup.fd_dup, f);

    return 0;
}
```

5.16.4 Tam Sürücü Kodu

Yukarıdaki test işlemlerine ilişkin tam aygıt sürücü kodu aşağıda verilmiştir.

test-driver.h

```
#ifndef TEST_DRIVER_H_
#define TEST_DRIVER_H_

#include <linux/stddef.h>
#include <linux/ioctl.h>

struct FDDUP {
    int fd;
```

(sonraki sayfaya devam)

(önceki sayfadan devam)

```

    int fd_dup;
};

#define TEST_DRIVER_MAGIC      't'
#define IOC_TEST1             _IOR(TEST_DRIVER_MAGIC, 0, int)
#define IOC_TEST2             _IOR(TEST_DRIVER_MAGIC, 1, int)
#define IOC_TEST3             _IOWR(TEST_DRIVER_MAGIC, struct FDDUP)
#define IOC_TEST4             _IOWR(TEST_DRIVER_MAGIC, 3, struct FDDUP)

#endif

```

test-driver.c

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/fdtable.h>
#include <linux/file.h>
#include "test-driver.h"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Kaan Aslan");
MODULE_DESCRIPTION("test-driver");

static int test_driver_open(struct inode *inodep, struct file *filp);
static int test_driver_release(struct inode *inodep, struct file *filp);
static ssize_t test_driver_read(struct file *filp, char *buf, size_t size, loff_t *off);
static ssize_t test_driver_write(struct file *filp, const char *buf, size_t size, loff_t *off);
static long test_driver_ioctl(struct file *filp, unsigned int cmd, unsigned long arg);

static long ioctl_test1(struct file *filp, unsigned long arg);
static long ioctl_test2(struct file *filp, unsigned long arg);
static long ioctl_test3(struct file *filp, unsigned long arg);
static long ioctl_test4(struct file *filp, unsigned long arg);

static dev_t g_dev;
static struct cdev g_cdev;
static struct file_operations g_fops = {
    .owner          = THIS_MODULE,
    .open           = test_driver_open,
    .read           = test_driver_read,
    .write          = test_driver_write,
    .release        = test_driver_release,
    .unlocked_ioctl = test_driver_ioctl
};

static int __init test_driver_init(void)
{
    int result;

    printk(KERN_INFO "test-driver module initialization...\n");

    if ((result = alloc_chrdev_region(&g_dev, 0, 1, "test-driver")) < 0) {
        printk(KERN_INFO "cannot alloc char driver!...\n");
        return result;
    }
    cdev_init(&g_cdev, &g_fops);
    if ((result = cdev_add(&g_cdev, g_dev, 1)) < 0) {

```

(sonraki sayfaya devam)

```

    unregister_chrdev_region(g_dev, 1);
    printk(KERN_ERR "cannot add device!...\n");
    return result;
}

return 0;
}

static void __exit test_driver_exit(void)
{
    cdev_del(&g_cdev);
    unregister_chrdev_region(g_dev, 1);

    printk(KERN_INFO "test-driver module exit...\n");
}

static int test_driver_open(struct inode *inodep, struct file *filp)
{
    printk(KERN_INFO "test-driver opened...\n");
    return 0;
}

static int test_driver_release(struct inode *inodep, struct file *filp)
{
    printk(KERN_INFO "test-driver closed...\n");
    return 0;
}

static ssize_t test_driver_read(struct file *filp, char *buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "test-driver read...\n");
    return 0;
}

static ssize_t test_driver_write(struct file *filp, const char *buf, size_t size, loff_t *off)
{
    printk(KERN_INFO "test-driver write...\n");
    return 0;
}

static long test_driver_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    long result;

    printk(KERN_INFO "test_driver_ioctl...\n");

    switch (cmd) {
        case IOC_TEST1:
            result = ioctl_test1(filp, arg);
            break;
        case IOC_TEST2:
            result = ioctl_test2(filp, arg);
            break;
        case IOC_TEST3:
            result = ioctl_test3(filp, arg);
            break;
        case IOC_TEST4:
            result = ioctl_test4(filp, arg);
            break;
    }
}

```

(önceki sayfadan devam)

```

        default:
            result = -ENOTTY;
    }

    return result;
}

static long ioctl_test1(struct file *filp, unsigned long arg)
{
    struct file *f;
    int fd = (int)arg;

    if (fd < 0 || fd >= current->files->fdt->max_fds) {
        printk(KERN_INFO "file descriptor is not valid!..\n");
        return -EBADF;
    }

    f = current->files->fdt->fd[fd];
    if (f == NULL) {
        printk(KERN_INFO "file descriptor is not valid!..\n");
        return -EBADF;
    }

    printk(KERN_INFO "File pointer offset: %ld\n", (long)f->f_pos);
    return 0;
}

static long ioctl_test2(struct file *filp, unsigned long arg)
{
    struct file *f;
    int fd = (int)arg;

    if ((f = fget(fd)) == NULL) {
        printk(KERN_INFO "file descriptor is not valid!..\n");
        return -EBADF;
    }

    printk(KERN_INFO "File pointer offset: %ld\n", (long)f->f_pos);
    fput(f);
    return 0;
}

static long ioctl_test3(struct file *filp, unsigned long arg)
{
    struct file *f;
    struct fdtable *fdt;
    struct file **fd_table;
    struct FDDUP fddup;
    int i;

    f = NULL;

    if (copy_from_user(&fddup, (struct FDDUP *)arg, sizeof(fddup)) != 0)
        return -EFAULT;

    fdt = current->files->fdt;
    fd_table = fdt->fd;

    if (fddup.fd < 0 || fddup.fd >= fdt->max_fds)

```

(sonraki sayfaya devam)

```

    return -EBADF;
if (fd_table[fddup.fd] == NULL)
    return -EBADF;

for (i = 0; i < fdt->max_fds; ++i)
    if (fd_table[i] == NULL) {
        f = fd_table[fddup.fd];
        fd_table[i] = f;
        ++f->f_count.counter;    /* atomic_long_inc(&f->f_count); */
        fddup.fd_dup = i;
        break;
    }
if (f == NULL)
    return -EMFILE;

if (copy_to_user((struct FDDUP *)arg, &fddup, sizeof(fddup)) != 0)
    return -EFAULT;

return 0;
}

static long ioctl_test4(struct file *filp, unsigned long arg)
{
    struct file *f;
    struct FDDUP fddup;

    if (copy_from_user(&fddup, (struct FDDUP *)arg, sizeof(fddup)) != 0)
        return -EFAULT;

    if ((f = fget(fddup.fd)) == NULL)
        return -EBADF;

    if ((fddup.fd_dup = get_unused_fd_flags(0)) < 0) {
        fput(f);
        return fddup.fd_dup;
    }

    if (copy_to_user((struct FDDUP *)arg, &fddup, sizeof(fddup)) != 0) {
        fput(f);
        return -EFAULT;
    }

    fd_install(fddup.fd_dup, f);

    return 0;
}

module_init(test_driver_init);
module_exit(test_driver_exit);

```

Makefile

```

obj-m += ${file}.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=${PWD} modules
clean:
    make -C /lib/modules/$(shell uname -r)/build M=${PWD} clean

```

load (yükleyici betik)

```
#!/bin/bash

module=$1
mode=666

/sbin/insmod ./${module}.ko ${@:2} || exit 1
major=$(awk "\$2 == \"\$module\" {print \$1}" /proc/devices)
rm -f $module
mknod -m $mode $module c $major 0
```

unload (kaldırıcı betik)

```
#!/bin/bash

module=$1

/sbin/rmmod ./${module}.ko || exit 1
rm -f $module
```

app.c (test uygulaması)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include "test-driver.h"

void exit_sys(const char *msg);

int main(int argc, char *argv[])
{
    int fd_dev;
    int fd;
    struct FDDUP fddup;
    off_t pos;

    if (argc != 2) {
        fprintf(stderr, "wrong number of arguments!..\n");
        exit(EXIT_FAILURE);
    }

    if ((fd_dev = open("test-driver", O_RDONLY)) == -1)
        exit_sys("open");

    if ((fd = open(argv[1], O_RDONLY)) == -1)
        exit_sys("open");
    lseek(fd, 100, 0);

    fddup.fd = fd;
    if (ioctl(fd_dev, IOC_TEST4, &fddup) == -1)
        exit_sys("ioctl");

    pos = lseek(fddup.fd_dup, 0, 1);
    printf("%jd\n", (intmax_t)pos);

    close(fd);
    close(fddup.fd);
    close(fd_dev);
```

(sonraki sayfaya devam)

```
    return 0;
}

void exit_sys(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}
```

Not

Bu bölümde kullanılan grafiklerin oluşturulabilmesi için Sphinx yapılandırma dosyasında (`conf.py`) `sphinx.ext.graphviz` uzantısının etkin olması gerekmektedir:

```
extensions = [
    ...
    'sphinx.ext.graphviz',
]
```